Master Source Code Compilation & Autonomous Verification Dossier

Complete production codebase, system architectural specifications, and rigorous multi-agent verification instructions for comprehensive LLM validation of Role-Based Access Control (RBAC), Neo4j AuraDB graph consistency, book circulation workflows, live visualizers, and UI logic.

Component / Layer	Technology / Specifications	Primary Responsibilities & Verification Focus
Backend Web Framework	Python 3.12, Flask, Flask-WTF (CSRF), Werkzeug Security	Full application routing, authentication, session tokens, protected role gates, parameterized Cypher queries, and REST APIs.
Graph Database Layer	Neo4j AuraDB (Cloud Instance), Cypher Query Engine, neo4j Python Driver	Property graph persistence: Users, Books, Copies, Categories, Authors, Publishers, Loans, Fines, and LoanRequests.
RBAC Security Model	4 Roles: Admin, Librarian, Staff, Student with Protected Admin Hierarchy	Strict role boundary enforcement: Admin immunity against peer tampering; Librarian supervision; Student/Staff privacy isolation.
Interactive Graph Visualization	Vis.js Network Engine, Barnes-Hut Physics Solver, Vanilla JS	Dynamic spring/repulsion graph simulation, state-aware Play/Pause toggling, auto-freeze layout, and empty graph handling.
Recommendation Engine	Collaborative Graph Filtering, Jaccard Similarity, Traversals	Intelligent book discovery, peer borrowing (Book Bridge), and curated academic learning paths without raw debug strings.

PART I: VERIFICATION INSTRUCTIONS & GUIDELINES FOR THE REVIEWING LLM

Notice to the Evaluating AI / LLM Auditor: This document contains the full, unabridged source code for the VIT Chennai Central Library Management & Knowledge Graph System. Your objective is to perform an exhaustive, rigorous, and systematic code verification to guarantee that all intended architectural, functional, security, and algorithmic requirements have been implemented correctly and without logical errors. Evaluate the codebase across the following seven core dimensions:

1. Role-Based Access Control (RBAC) & Protected Hierarchy Logic

- **Four Distinct Roles:** Validate that the system strictly defines and enforces Admin, Librarian, Staff, and Student across decorators, template conditional renders, and route handlers.
- **Protected Administrator Immunity:** An Admin CANNOT be suspended, deleted, or have their password reset by another Admin via the web interface or API (protects against accidental lockout and privilege de-escalation attacks). Verify that /delete_user/<username>, /suspend_user/<username>, and /reset_user_password/<username> explicitly block targeting Admin accounts.
- **Administrative Control over Librarians:** An Admin CAN suspend, delete, and reset passwords for Librarians. Verify that templates/manage_users.html renders the Reset Password button for Librarian rows and that the backend properly handles password reset for Librarians.
- **Librarian Scope Boundaries:** Librarians can manage book records, issue copies, and approve loan requests, but CANNOT manage Admin or Librarian accounts, nor reset Admin passwords.
- **Staff and Student Privacy Isolation:** Staff and Students must be strictly isolated to their own records. They cannot access administrative routes, cannot approve loans, and only view their own borrowings, disciplinary liabilities, and recommendations.

2. Neo4j AuraDB Graph Modeling & Cypher Parameterization

- **Graph Data Schema:** Verify node labels: User, Book, Author, Category, Publisher, Copy, Loan, Fine, LoanRequest.
- **Cypher Injection Prevention:** Ensure that every database query uses parameterized Cypher queries (session.run(query, params)). Verify that no user inputs are directly concatenated or formatted into Cypher strings.
- **Physical Copy Cardinality & Inventory Tracking:** Each Book is linked to physical copies via (:Book)-[:HAS_COPY]->(:Copy). Verify that book borrowing marks Copy.is_borrowed = true and decrements available count, while book return marks is_borrowed = false and increments available count.
- **Transaction Lifecycle:** Check that sessions and transactions are properly opened and closed, with robust error catching and database rollback on failure.

3. Book Issuance Workflow & Substring Live Autocomplete

- **Substring Matching:** Verify that on /issue_loan, typing into the User or Book search inputs performs a live substring filter (e.g. matching any part of a user's name, username, or book title/ISBN) rather than restricting to strict prefix matching.
- **Availability Verification:** Verify that a loan cannot be issued if available copies are 0, preventing negative inventory bugs.
- **Active Loan Creation:** Verify that issuing a book establishes the (:User)-[:BORROWED {status: 'ACTIVE', issue_date: ...}]->(:Book) and (:Loan)-[:FOR_COPY]->(:Copy) relationships.

4. Disciplinary Liabilities & Fine Management Terminology

- **Terminology Compliance:** In Admin and Librarian dashboards, verify that the disciplinary liabilities section uses the exact terminology **'Disciplinary Liabilities'** and **'Disciplinary Fines'**, with the redundant qualifier 'outstanding' completely removed.
- **Overdue Calculation:** Verify that overdue loans calculate fines based on due date vs return date and create Fine nodes with status 'unpaid'.
- **Settlement Engine:** Verify that members can view their fines on /my_fines and that librarians/admins can settle or waive liabilities.

5. AI Recommendations & Book Bridge Peer Sharing

- **Graph Recommendation Algorithm:** Verify that recommendations are computed using collaborative graph traversals (co-borrowed books among users with shared reading histories).
- **Clean User-Facing Copy:** Confirm that technical debug labels such as (Jaccard Graph Similarity) are omitted from user-facing recommendation cards.
- **Book Bridge:** Verify that students and faculty can list books for peer sharing and submit borrow requests to peer owners.

6. Live Graph Explorer Physics Controls & Edge Cases

- **Vis.js Barnes-Hut Physics Solver:** Inspect templates/graph_visualizer.html to verify that physics is configured with proper gravitational constant, spring length, and central gravity.
- **State-Aware Play/Pause Toggle:** When physics simulation is actively running, the button must display the **Pause icon (■■)** with title *'Pause Physics Simulation (Freeze Layout)'*. When frozen, it must display the **Play icon (■■)** with title *'Resume Physics Simulation (Unfreeze Layout)'*.
- **Useless State Removal (0 Nodes):** When 0 nodes are visible (e.g. all filters deselected or empty dataset), the play/pause button provides no utility and must be hidden (display: none).
- **Re-stabilize Auto-Sync:** If physics was frozen, clicking *'Re-stabilize Layout'* must automatically re-enable physics before triggering stabilization.

7. Security, CSRF Protection & Credential Integrity

- **Password Security:** Passwords must be hashed using Werkzeug salted PBKDF2/SHA-256 (generate_password_hash, check_password_hash).
- **CSRF Protection:** All POST forms must validate CSRF tokens via flask_wtf.csrf.CSRFProtect.
- **Session Integrity:** Check that session state stores user ID, username, and role securely, with complete invalidation upon logout.

PART II: COMPLETE INTENDED FEATURES SPECIFICATION & FUNCTIONAL REQUIREMENTS

The following matrix specifies all intended features that the application was engineered to provide. Use this functional requirements specification to cross-reference every route, query, and UI element in the code:

F-01: Multi-Role Authentication & Session Management

- **Feature Scope:** Secure authentication portal (/login), self-registration (/register), and credential modification (/change_password).
- **Expected Behavior:** Users authenticate via username/email and password. Upon successful login, the user's role (Admin, Librarian, Staff, Student) determines dashboard routing and menu visibility. Invalid credentials flash an error without revealing user existence.
- **Security Rules:** Passwords stored exclusively as salted hashes; session tokens invalidated on /logout.

F-02: User Administration & Protected Hierarchy

- **Feature Scope:** Central user management dashboard (/manage_users) accessible strictly to Administrators.
- **Expected Behavior:** Displays all registered users with role badges and status (Active, Suspended). Admin can suspend or delete Librarians, Staff, and Students. Admin can trigger a password reset for Librarian accounts, generating a secure temporary credential.
- **Safety Restraint:** Admin accounts CANNOT be suspended, deleted, or reset by anyone via the interface (hardened against peer admin lockout).

F-03: Multi-Copy Catalog & Category Taxonomy

- **Feature Scope:** Hierarchical book catalog (/catalog), category-specific book browser (/categories/<cat_id>/books), book registration (/add_book), and metadata editing (/edit_book/<id>).
- **Expected Behavior:** Books are linked to Authors, Publishers, and Categories. Each Book tracks total physical copies and real-time available copies. Register new book button renders a single clean plus sign (+ Register New Book).
- **Role Scoping:** Category cards show *'inspect copy availability, and manage loans'* for Admin/Librarian, and *'check real-time copy availability, and request loans'* for Staff/Student.

F-04: Manual Book Checkout with Substring Autocomplete

- **Feature Scope:** Librarian and Admin loan issuance console (/issue_loan).
- **Expected Behavior:** Typing any partial name, username, title, or ISBN dynamically filters matching members and books in an interactive dropdown. Selecting a book verifies available copy count before permitting checkout.
- **State Transition:** Generates a new active Loan relationship, decrements book available copies, and sets loan due date (default: 14 days).

F-05: Loan Request, Approval & Return Pipeline

- **Feature Scope:** Self-service loan request queue (/loan_requests) and loan return processing (/return_book/<loan_id>).
- **Expected Behavior:** Students and Staff can request books online. Librarians review the queue with audited request timestamps (header formatted as REQUEST TIMESTAMP). Approving issues the loan; returning marks the copy available and checks for overdue fines.

F-06: Disciplinary Liabilities & Fine Settlement

- **Feature Scope:** Disciplinary liabilities tracking across dashboards and member fine inspection (/my_fines).
- **Expected Behavior:** Overdue loans automatically accrue fines ($1.00/day). Fines display status (Unpaid, Paid, Waived).
- **Terminology Standard:** Admin and Librarian dashboards cleanly display 'Disciplinary Liabilities' and 'Disciplinary Fines' without the word 'outstanding'.

F-07: AI Graph Recommendations & Book Bridge

- **Feature Scope:** Graph-based recommendation engine (/recommendations) and peer-to-peer book sharing (/book_bridge).
- **Expected Behavior:** Recommends books based on co-borrowing clusters and author/category proximity. UI displays clean similarity metrics without raw debug text (Jaccard Graph Similarity removed). Book Bridge allows students and staff to list and request personal books.

F-08: Interactive Vis.js Knowledge Graph Visualizer

- **Feature Scope:** Live Neo4j property graph visualizer (/graph_visualizer) with dynamic view switching (Full, Catalog, Personal/Global Circulation).
- **Expected Behavior:** Visualizes nodes (color-coded by group) and directed relationships with physics. Clicking a node opens a slide-out drawer showing properties and connected neighbors.
- **Physics Play/Pause Button:** Active simulation shows Pause (■■) icon; frozen simulation shows Play (■■) icon. Automatically hidden when 0 nodes are visible. Re-stabilize automatically re-activates physics if paused.

F-09: Role-Scoped Central Dashboards

- **Feature Scope:** Central dashboard (/dashboard) adapting its analytics, quick actions, and data tables to the authenticated role.
- **Expected Behavior:** Admin sees campus-wide KPIs, system user counts, and global disciplinary liabilities. Librarian sees circulation queues and catalog actions. Staff/Student sees personal borrowings, return deadlines, and personalized recommendations.

F-10: Academic Learning Paths & Analytics

- **Feature Scope:** Curated prerequisite book paths (/learning_paths) and circulation analytics (/library_analytics).
- **Expected Behavior:** Provides structured domain reading roadmaps and visual charts illustrating borrow trends by category and role.

PART III: ORGANIZED SOURCE CODE MODULES & IMPLEMENTATION

Below is the complete, line-numbered source code of every essential application file. Each file begins with a comprehensive module header detailing its architectural role, line count, and a targeted LLM verification checklist:

MODULE SOURCE: app.py

Lines: 4229 | **Lang:** Python 3.12 | **Category:** Core Backend & API Layer

Core Role & Purpose: Primary Flask application handling all HTTP routing, Neo4j AuraDB Cypher queries, RBAC authentication decorators, user administration, loan issuance, fine settlement, recommendation algorithms, and visualizer APIs.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify @login_required and @role_required(['Admin', ...]) decorator logic on all routes.
- Verify Admin immunity in /manage_users, /delete_user, /suspend_user, and /reset_user_password.
- Verify Admin's capability to reset password, suspend, and delete Librarian accounts.
- Verify parameterized Cypher execution across all database interactions (session.run with parameters).
- Verify /issue_loan substring user/book lookup and copy availability decrement.
- Verify /return_book loan completion, copy availability increment, and overdue fine calculation.
- Verify that Admin and Librarian dashboard fine metrics use 'Disciplinary Liabilities' copy.
- Verify /api/graph_data JSON formatting for Vis.js visualization.

```
0001: """
0002: Enterprise Library Management System - Core Application
0003: Migrated from Oracle 21c to Neo4j AuraDB for DA-3 Project Assessment.
0004: Engineered with Flask, Neo4j Graph Database, and Role-Based Access Control (RBAC).
0005:
0006: Graph Relationships:
0007:   (User)-[:BORROWED]->(Loan)-[:FOR_COPY]->(Copy)<-[:HAS_COPY]-(Book)-[:BELONGS_TO]->(Category)
0008:   (Book)-[:PUBLISHED_BY]->(Publisher)
0009:   (Author)-[:WROTE]->(Book)
0010:   (User)-[:ISSUED]->(Loan)
0011:   (Loan)-[:HAS_FINE]->(Fine)
0012:   (User)-[:REQUESTED]->(LoanRequest)-[:FOR_BOOK]->(Book)
0013:
0014: DA-3 Team:
0015:   KARTHIK VALLIAPPAN C  - 25BAI1051
0016:   NITHIN RANGASWAMY     - 25BAI1210
0017:   NATRAJ                - 25BAI1079
0018: """
0019:
0020: import os
0021: import re
0022: import html
0023: from datetime import datetime, timedelta, date
0024: from functools import wraps
0025:
0026: from neo4j import GraphDatabase
0027: from flask import Flask, flash, jsonify, redirect, render_template, request, session, url_for
0028: from flask_session import Session
0029: from flask_wtf.csrf import CSRFProtect
0030: from werkzeug.security import check_password_hash, generate_password_hash
0031:
0032: # ============================================================================
0033: # Application Initialization & Configuration
0034: # ============================================================================
0035: app = Flask(__name__)
0036:
0037: app.config["SECRET_KEY"] = "enterprise-library-secret-key-change-in-production"
0038: app.config["WTF_CSRF_ENABLED"] = not app.config.get("TESTING", False)
0039: csrf = CSRFProtect(app)
0040:
0041: app.config["PERMANENT_SESSION_LIFETIME"] = timedelta(minutes=10)
0042: app.config["SESSION_PERMANENT"] = True
0043: app.config["SESSION_TYPE"] = "filesystem"
0044: app.config["SESSION_FILE_DIR"] = os.path.join(os.path.dirname(os.path.abspath(__file__)), "flask_session")
0045: os.makedirs(app.config["SESSION_FILE_DIR"], exist_ok=True)
0046: Session(app)
0047:
0048: # ============================================================================
0049: # Neo4j AuraDB Connection
0050: # ============================================================================
0051: NEO4J_URI      = "neo4j+s://e7f1b8d0.databases.neo4j.io"
0052: NEO4J_USERNAME = "e7f1b8d0"
0053: NEO4J_PASSWORD = "ROzf6G2s6B2tP6C61ODmEGMbmG-pzeCq5KZgI6CmxHI"
0054: NEO4J_DATABASE = "e7f1b8d0"
0055:
0056: _driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD))
0057:
0058:
0059: def get_session():
0060:     """Returns a Neo4j driver session scoped to AuraDB database."""
0061:     return _driver.session(database=NEO4J_DATABASE)
0062:
0063:
0064: # ============================================================================
0065: # Jinja2 Date Helper  (preserves .strftime() calls in existing templates)
0066: # ============================================================================
0067: class DateWrapper:
0068:     """Wraps a string date (YYYY-MM-DD) so templates can call .strftime()."""
0069:     def __init__(self, val):
0070:         if val is None:
0071:             self._d = None
0072:         elif isinstance(val, (datetime, date)):
0073:             self._d = val
0074:         elif isinstance(val, str):
0075:             for fmt in ("%Y-%m-%d %H:%M:%S", "%Y-%m-%d"):
```

```
0076:                 try:
0077:                     self._d = datetime.strptime(val, fmt)
0078:                     break
0079:                 except ValueError:
0080:                     pass
```

```
            else:
                self._d = None
        else:
            self._d = None

    def strftime(self, fmt):
        if self._d is None:
            return ""
        return self._d.strftime(fmt)

    def __str__(self):
        return self.strftime("%Y-%m-%d") if self._d else ""

    def __bool__(self):
        return self._d is not None

    def __repr__(self):
        return f"DateWrapper({self._d})"


def wrap_dates(record: dict) -> dict:
    """Wrap all date-string values in a dict with DateWrapper so templates work."""
    DATE_KEYS = {"issue_date", "due_date", "return_date", "request_date", "payment_date"}
    return {k: (DateWrapper(v) if k in DATE_KEYS else v) for k, v in record.items()}


def rows(result) -> list:
    """Convert Neo4j result to a list of plain dicts with date wrapping."""
    return [wrap_dates(dict(r)) for r in result]


def today_str() -> str:
    return datetime.now().strftime("%Y-%m-%d")


# ============================================================================
# Session Idle Timeout
# ============================================================================
@app.before_request
def check_session_idle_timeout():
    """Enforces a strict 10-minute idle session timeout."""
    if request.endpoint == "static":
        return
    if "user_id" in session:
        now = datetime.now().timestamp()
        last_activity = session.get("last_activity")
        if last_activity is not None and (now - last_activity) > 600:
            session.clear()
            flash("Security Timeout: Your session has expired due to 10 minutes of inactivity.", "warning")
            return redirect(url_for("login"))
        session["last_activity"] = now


# ============================================================================
# Password & Email Validators
# ============================================================================
def validate_password_strength(password):
    if not password:
        return False, "Password cannot be empty."
    if len(password) < 8 or len(password) > 64:
        return False, "Password must be between 8 and 64 characters in length."
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least 1 uppercase letter (A-Z)."
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least 1 lowercase letter (a-z)."
    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least 1 number (0-9)."
    if not re.search(r'[^a-zA-Z0-9]', password):
        return False, "Password must contain at least 1 special character (e.g. !@#$%^&*)."
    return True, None


def validate_email_format(email):
    if not email:
        return False, "Email address cannot be empty."
    email = email.strip()
    if "@" not in email or "." not in email:
        return False, "Email address must contain both '@' and '.' characters."
    parts = email.split("@")
    if len(parts) != 2 or not parts[0] or not parts[1] or "." not in parts[1]:
```

```
0161:         return False, "Please enter a valid email address (e.g. user@domain.com)."
0162:     return True, None
0163:
0164:
0165: # ============================================================================
0166: # RBAC Decorators
0167: # ============================================================================
0168: def login_required(f):
0169:     """Ensures user is authenticated; evicts zombie sessions for suspended accounts."""
0170:     @wraps(f)
0171:     def decorated_function(*args, **kwargs):
0172:         if "user_id" not in session:
0173:             flash("Please log in to access this page.", "warning")
0174:             return redirect(url_for("login"))
0175:
0176:         # Check live account status in Neo4j
0177:         try:
0178:             with get_session() as s:
0179:                 result = s.run(
0180:                     "MATCH (u:User {user_id: $uid}) RETURN u.account_status AS account_status, u.role AS rol
      | e",
0181:                     uid=session["user_id"]
0182:                 )
0183:                 record = result.single()
0184:                 if not record or record["account_status"] != "Active":
0185:                     session.clear()
0186:                     flash("Access Denied: Your account is no longer active or has been suspended.", "danger"
      | )
0187:                     return redirect(url_for("login"))
0188:                 session["role"] = record["role"]
0189:         except Exception:
0190:             pass
0191:
0192:         return f(*args, **kwargs)
0193:     return decorated_function
0194:
0195:
0196: def role_required(allowed_roles):
0197:     """Restricts view access to specific user roles."""
0198:     def decorator(f):
0199:         @wraps(f)
0200:         def decorated_function(*args, **kwargs):
0201:             if "user_id" not in session:
0202:                 flash("Please log in to proceed.", "warning")
0203:                 return redirect(url_for("login"))
0204:             user_role = session.get("role")
0205:             if user_role not in allowed_roles:
0206:                 flash("Access Denied: You do not have permissions for this action.", "danger")
0207:                 return render_template("unauthorized.html", user_role=user_role), 403
0208:             return f(*args, **kwargs)
0209:         return decorated_function
0210:     return decorator
0211:
0212:
0213: # ============================================================================
0214: # Helper: next sequential ID
0215: # ============================================================================
0216: def next_id(s, label: str, id_field: str) -> int:
0217:     valid_labels = {"User", "Book", "Loan", "Copy", "LoanRequest", "Fine", "Category", "Author", "Publisher"
      | }
0218:     valid_id_fields = {"user_id", "book_id", "loan_id", "copy_id", "request_id", "fine_id", "category_id", "
      | author_id", "publisher_id"}
0219:     if label not in valid_labels or id_field not in valid_id_fields:
0220:         raise ValueError(f"Invalid label '{label}' or id_field '{id_field}' for next_id")
0221:     result = s.run(f"MATCH (n:{label}) RETURN coalesce(max(n.{id_field}), 0) AS mx")
0222:     rec = result.single()
0223:     return ((rec["mx"] if rec else 0) or 0) + 1
0224:
0225:
0226: def sync_fulfilled_loan_requests(session_tx):
0227:     """
0228:     Ensures data consistency for loan requests and physical copies:
0229:     1. If a user already has an active Loan for a book, any lingering Pending or
0230:        Awaiting Pickup LoanRequest for that book is marked Fulfilled.
0231:     2. Any 'Awaiting Pickup' request must be linked ([:RESERVED_COPY]) to a real
0232:        physical Copy whose status is 'Reserved'. If no copy is available or all copies
0233:        are already 'Borrowed' (like CLRS in #REQ-3), re-sync request to 'Pending'.
0234:     3. Any orphaned 'Reserved' copy without an 'Awaiting Pickup' hold is restored to 'Available'.
0235:     """
0236:     try:
```

```
0237:         # 1. Auto-fulfill requests where user already holds active loan for this book
0238:         session_tx.run("""
0239:             MATCH (u:User)-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)
0240:             MATCH (b:Book)-[:HAS_COPY]->(cp)
0241:             WHERE l.return_date IS NULL
0242:             MATCH (u)-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b)
0243:             WHERE r.status IN ['Pending', 'Awaiting Pickup']
0244:             WITH r, cp, r.copy_id AS old_copy_id
0245:             OPTIONAL MATCH (r)-[old_rel:RESERVED_COPY]->()
0246:             DELETE old_rel
0247:             SET r.status = 'Fulfilled', r.copy_id = cp.copy_id
0248:             WITH old_copy_id, cp
0249:             WHERE old_copy_id IS NOT NULL AND old_copy_id <> cp.copy_id
0250:             MATCH (reserved_cp:Copy {copy_id: old_copy_id, status: 'Reserved'})
0251:             SET reserved_cp.status = 'Available'
0252:         """)
0253:
0254:         # 2. Check 'Awaiting Pickup' requests one-by-one: ensure valid reserved copy exists
0255:         broken_res = session_tx.run("""
0256:             MATCH (r:LoanRequest {status: 'Awaiting Pickup'})-[:FOR_BOOK]->(b:Book)
0257:             OPTIONAL MATCH (r)-[:RESERVED_COPY]->(rcp:Copy)
0258:             WITH r, b, rcp, r.copy_id AS cid
0259:             OPTIONAL MATCH (c_by_id:Copy {copy_id: cid})
0260:             WITH r, b, coalesce(rcp, c_by_id) AS linked_cp
0261:             WHERE linked_cp IS NULL OR linked_cp.status = 'Borrowed' OR linked_cp.status = 'Retired'
0262:             RETURN r.request_id AS rid, b.book_id AS bid
0263:             ORDER BY r.request_date ASC, r.request_id ASC
0264:         """)
0265:         broken_reqs = list(broken_res)
0266:         for req in broken_reqs:
0267:             rid = req["rid"]
0268:             bid = req["bid"]
0269:             avail_rec = session_tx.run("""
0270:                 MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(avail_cp:Copy {status: 'Available'})
0271:                 WITH avail_cp ORDER BY avail_cp.copy_id ASC LIMIT 1
0272:                 RETURN avail_cp.copy_id AS cid
0273:             """, bid=bid).single()
0274:
0275:             if avail_rec:
0276:                 next_cid = avail_rec["cid"]
0277:                 session_tx.run("""
0278:                     MATCH (cp:Copy {copy_id: $cid}), (r:LoanRequest {request_id: $rid})
0279:                     SET cp.status = 'Reserved', r.copy_id = $cid
0280:                     OPTIONAL MATCH (r)-[old_rel:RESERVED_COPY]->()
0281:                     DELETE old_rel
0282:                     CREATE (r)-[:RESERVED_COPY]->(cp)
0283:                 """, cid=next_cid, rid=rid)
0284:             else:
0285:                 session_tx.run("""
0286:                     MATCH (r:LoanRequest {request_id: $rid})
0287:                     OPTIONAL MATCH (r)-[old_rel:RESERVED_COPY]->()
0288:                     DELETE old_rel
0289:                     SET r.status = 'Pending', r.copy_id = null
0290:                 """, rid=rid)
0291:
0292:         # 3. Clean up relationships and ensure [:RESERVED_COPY] matches r.copy_id
0293:         session_tx.run("""
0294:             MATCH (r:LoanRequest)-[rel:RESERVED_COPY]->()
0295:             WHERE r.status <> 'Awaiting Pickup'
0296:             DELETE rel
0297:         """)
0298:         session_tx.run("""
0299:             MATCH (r:LoanRequest {status: 'Awaiting Pickup'})
0300:             WHERE r.copy_id IS NOT NULL
0301:             OPTIONAL MATCH (r)-[old_rel:RESERVED_COPY]->(old_cp:Copy)
0302:             WHERE old_cp.copy_id <> r.copy_id
0303:             DELETE old_rel
0304:         """)
0305:         session_tx.run("""
0306:             MATCH (r:LoanRequest {status: 'Awaiting Pickup'})
0307:             WHERE r.copy_id IS NOT NULL
0308:             MATCH (cp:Copy {copy_id: r.copy_id})
0309:             SET cp.status = 'Reserved'
0310:             MERGE (r)-[:RESERVED_COPY]->(cp)
0311:         """)
0312:
0313:         # 4. Release orphaned 'Reserved' copies without an Awaiting Pickup hold
0314:         session_tx.run("""
0315:             MATCH (cp:Copy {status: 'Reserved'})
0316:             WHERE NOT (cp)<-[:RESERVED_COPY]-(:LoanRequest {status: 'Awaiting Pickup'})
```

```
0317:               AND NOT EXISTS {
0318:                   MATCH (req:LoanRequest {status: 'Awaiting Pickup'})
0319:                   WHERE req.copy_id = cp.copy_id
0320:               }
0321:             SET cp.status = 'Available'
0322:         """)
0323:     except Exception as e:
0324:         app.logger.warning(f"sync_fulfilled_loan_requests warning: {e}")
0325: 
0326: 
0327: def sync_active_overdue_fines(session_tx):
0328:     """
0329:     Auto-syncs live overdue fines for all active unreturned loans (return_date IS NULL and due_date < today)
     | .
0330:     Guarantees Days Overdue * Rs. 10 always matches the Disciplinary Fine ledger amount.
0331:     """
0332:     try:
0333:         today = today_str()
0334:         # 0. Clean up any orphaned Fine nodes without a connected Loan
0335:         session_tx.run("""
0336:             MATCH (f:Fine)
0337:             WHERE NOT (f)<-[:HAS_FINE]-(:Loan)
0338:             DETACH DELETE f
0339:         """)
0340: 
0341:         # 1. Update existing Unpaid fines on active overdue loans to reflect live days_late * Rs. 10
0342:         session_tx.run("""
0343:             MATCH (l:Loan)
0344:             WHERE l.return_date IS NULL AND l.due_date < $today
0345:             WITH l, duration.inDays(date(l.due_date), date($today)).days AS days_late
0346:             WHERE days_late > 0
0347:             MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
0348:             SET f.fine_amount = toFloat(days_late * 10)
0349:         """, today=today)
0350: 
0351:         # 2. For active overdue loans without any Fine node, create new Fine node
0352:         res = session_tx.run("""
0353:             MATCH (l:Loan)
0354:             WHERE l.return_date IS NULL AND l.due_date < $today
0355:               AND NOT (l)-[:HAS_FINE]->(:Fine)
0356:             WITH l, duration.inDays(date(l.due_date), date($today)).days AS days_late
0357:             WHERE days_late > 0
0358:             RETURN l.loan_id AS loan_id, days_late
0359:         """, today=today)
0360:         unfined = list(res)
0361:         for row in unfined:
0362:             lid = row["loan_id"]
0363:             days_late = row["days_late"]
0364:             amt = float(days_late * 10)
0365:             fid = next_id(session_tx, "Fine", "fine_id")
0366:             session_tx.run("""
0367:                 MATCH (l:Loan {loan_id: $lid})
0368:                 CREATE (f:Fine {
0369:                     fine_id: $fid, loan_id: $lid,
0370:                     fine_amount: $amt, payment_status: 'Unpaid',
0371:                     payment_date: null, collected_by: null
0372:                 })
0373:                 CREATE (l)-[:HAS_FINE]->(f)
0374:             """, fid=fid, lid=lid, amt=amt)
0375:     except Exception as e:
0376:         app.logger.warning(f"sync_active_overdue_fines warning: {e}")
0377: 
0378: 
0379: # ============================================================================
0380: # Routes
0381: # ============================================================================
0382: 
0383: # ----------------------------------------------------------------------------
0384: # / and /login
0385: # ----------------------------------------------------------------------------
0386: @app.route("/", methods=["GET", "POST"])
0387: @app.route("/login", methods=["GET", "POST"])
0388: def login():
0389:     if "user_id" in session:
0390:         return redirect(url_for("dashboard"))
0391: 
0392:     if request.method == "POST":
0393:         username = request.form.get("username", "").strip()
0394:         password = request.form.get("password", "")
0395: 
```

```
0396:         if not username or not password:
0397:             flash("Username and Password are required.", "danger")
0398:             return render_template("login.html")
0399:
0400:         try:
0401:             with get_session() as s:
0402:                 result = s.run("""
0403:                     MATCH (u:User)
0404:                     WHERE toLower(u.username) = toLower($username)
0405:                     RETURN u.user_id AS user_id, u.username AS username,
0406:                            u.password_hash AS password_hash,
0407:                            u.role AS role, u.account_status AS account_status
0408:                 """, username=username)
0409:                 user = result.single()
0410:
0411:             if user and check_password_hash(user["password_hash"], password):
0412:                 if user["account_status"] != "Active":
0413:                     flash("Account is suspended. Please contact the library administrator.", "danger")
0414:                     return render_template("login.html")
0415:
0416:                 session.permanent = True
0417:                 session["user_id"] = user["user_id"]
0418:                 session["username"] = user["username"]
0419:                 session["role"] = user["role"]
0420:                 session["last_activity"] = datetime.now().timestamp()
0421:
0422:                 flash(f"Welcome back, {user['username']}!", "success")
0423:                 return redirect(url_for("dashboard"))
0424:             else:
0425:                 flash("Invalid username or password.", "danger")
0426:
0427:         except Exception as e:
0428:             flash(f"Database error during authentication: {str(e)}", "danger")
0429:
0430:     return render_template("login.html")
0431:
0432:
0433: # ------------------------------------------------------------------------------
0434: # /register
0435: # ------------------------------------------------------------------------------
0436: @app.route("/register", methods=["GET", "POST"])
0437: def register():
0438:     if "user_id" in session:
0439:         return redirect(url_for("dashboard"))
0440:
0441:     if request.method == "POST":
0442:         username = request.form.get("username", "").strip()
0443:         email    = request.form.get("email", "").strip()
0444:         password = request.form.get("password", "")
0445:         role     = request.form.get("role", "").strip()
0446:
0447:         if role not in ["Student", "Staff"]:
0448:             flash("Self-registration only permits 'Student' or 'Staff' accounts.", "danger")
0449:             return render_template("register.html")
0450:
0451:         if not username or not email or not password:
0452:             flash("All fields are mandatory.", "warning")
0453:             return render_template("register.html")
0454:
0455:         is_email_valid, email_err = validate_email_format(email)
0456:         if not is_email_valid:
0457:             flash(f"Invalid Email: {email_err}", "warning")
0458:             return render_template("register.html")
0459:
0460:         email_clean = email.lower().strip()
0461:         if not (email_clean.endswith("@vit.edu") or email_clean.endswith("@vit.ac.in")):
0462:             flash("Registration requires a valid institutional email domain (@vit.edu or @vit.ac.in).", "dan
    | ger")
0463:             return render_template("register.html")
0464:
0465:         is_pass_valid, pass_err = validate_password_strength(password)
0466:         if not is_pass_valid:
0467:             flash(f"Invalid Password: {pass_err}", "warning")
0468:             return render_template("register.html")
0469:
0470:         try:
0471:             with get_session() as s:
0472:                 # Check uniqueness
0473:                 result = s.run("""
0474:                     MATCH (u:User)
```

```
                    WHERE toLower(u.username) = toLower($uname) OR toLower(u.email) = toLower($email)
                    RETURN u.user_id AS user_id
                """, uname=username, email=email)
                if result.single():
                    flash("Username or Email is already registered.", "warning")
                    return render_template("register.html")

                pw_hash = generate_password_hash(password)
                new_uid = next_id(s, "User", "user_id")
                s.run("""
                    CREATE (u:User {
                        user_id: $uid, username: $uname, password_hash: $pw,
                        email: $email, role: $role, account_status: 'Active'
                    })
                """, uid=new_uid, uname=username, pw=pw_hash, email=email, role=role)

            flash(f"Account for '{username}' ({role}) created successfully! Please sign in.", "success")
            return redirect(url_for("login"))

        except Exception:
            flash("Transaction failed: An unexpected error occurred during registration.", "danger")

    return render_template("register.html")


# ---------------------------------------------------------------------------
# /logout
# ---------------------------------------------------------------------------
@app.route("/logout")
def logout():
    session.clear()
    flash("You have been successfully logged out.", "info")
    return redirect(url_for("login"))


# ---------------------------------------------------------------------------
# /dashboard
# ---------------------------------------------------------------------------
@app.route("/dashboard")
@login_required
def dashboard():
    role    = session.get("role")
    user_id = session.get("user_id")
    data    = {}

    try:
        with get_session() as s:
            sync_active_overdue_fines(s)
            if role in ["Admin", "Librarian"]:
                # Counts
                data["total_books"] = s.run("MATCH (b:Book) RETURN count(b) AS c").single()["c"]

                data["active_loans"] = s.run(
                    "MATCH (l:Loan) WHERE l.return_date IS NULL RETURN count(l) AS c"
                ).single()["c"]

                today = today_str()
                data["overdue_loans"] = s.run("""
                    MATCH (l:Loan)
                    WHERE l.return_date IS NULL AND l.due_date < $today
                    RETURN count(l) AS c
                """, today=today).single()["c"]

                data["unpaid_fines"] = 0.0

                data["pending_requests"] = s.run("""
                    MATCH (r:LoanRequest {status: 'Pending'}) RETURN count(r) AS c
                """).single()["c"]

                data["awaiting_pickup_count"] = s.run("""
                    MATCH (r:LoanRequest {status: 'Awaiting Pickup'}) RETURN count(r) AS c
                """).single()["c"]

                # Active loans list
                res = s.run("""
                    MATCH (u:User)-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book)
                    WHERE l.return_date IS NULL
                    OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    RETURN l.loan_id AS loan_id, u.username AS borrower_name,
                           u.email AS borrower_email, u.role AS borrower_role,
```

```
                           b.title AS book_title, cp.copy_id AS copy_id,
                           l.issue_date AS issue_date, l.due_date AS due_date,
                           coalesce(sum(f.fine_amount), 0) AS fine_amount
                    ORDER BY l.due_date ASC
                """)
                data["active_loans_list"] = rows(res)

                # Overdue list
                res = s.run("""
                    MATCH (u:User)-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book)
                    WHERE l.return_date IS NULL AND l.due_date < $today
                    RETURN l.loan_id AS loan_id, u.username AS borrower_name,
                           u.email AS borrower_email, u.role AS borrower_role,
                           b.title AS book_title, cp.copy_id AS copy_id,
                           l.issue_date AS issue_date, l.due_date AS due_date,
                           duration.inDays(date(l.due_date), date($today)).days AS days_overdue
                    ORDER BY l.due_date ASC
                """, today=today)
                data["overdue_loans_list"] = rows(res)

                # Unpaid fines list
                res = s.run("""
                    MATCH (l:Loan)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    OPTIONAL MATCH (u:User)-[:BORROWED]->(l)
                    OPTIONAL MATCH (l)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book)
                    RETURN f.fine_id AS fine_id, f.fine_amount AS fine_amount,
                           f.payment_status AS payment_status,
                           coalesce(u.username, 'Deleted Patron') AS borrower_name,
                           coalesce(u.email, 'N/A') AS borrower_email,
                           coalesce(b.title, 'Archived/Deleted Title') AS book_title,
                           l.loan_id AS loan_id, l.due_date AS due_date
                    ORDER BY f.fine_amount DESC
                """)
                data["unpaid_fines_list"] = rows(res)
                data["unpaid_fines"] = sum(f.get("fine_amount", 0.0) for f in data["unpaid_fines_list"])

                if role == "Admin":
                    data["total_users"] = s.run("MATCH (u:User) RETURN count(u) AS c").single()["c"]
                    return render_template("dashboard.html", data=data)
                else:
                    return render_template("dashboard.html", data=data)

            elif role == "Staff":
                today = today_str()
                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]
    | -(b:Book)
                    WHERE l.return_date IS NULL
                    OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    WITH u, l, cp, b, coalesce(sum(f.fine_amount), 0) AS fine_amount
                    OPTIONAL MATCH (pre_book:Book)-[:PREREQUISITE_FOR]->(b)
                    WITH u, l, cp, b, fine_amount,
                         collect(DISTINCT CASE WHEN pre_book IS NOT NULL THEN {
                             book_id: pre_book.book_id,
                             title: pre_book.title,
                             category_id: pre_book.category_id
                         } ELSE null END) AS pre_list
                    OPTIONAL MATCH (b)-[:PREREQUISITE_FOR]->(next_book:Book)
                    RETURN l.loan_id AS loan_id, b.book_id AS book_id, b.title AS title,
                           cp.copy_id AS copy_id, l.issue_date AS issue_date,
                           l.due_date AS due_date,
                           CASE WHEN l.due_date < $today THEN 1 ELSE 0 END AS is_overdue,
                           fine_amount,
                           [x IN pre_list WHERE x IS NOT NULL] AS prerequisites,
                           [x IN collect(DISTINCT CASE WHEN next_book IS NOT NULL THEN {
                               book_id: next_book.book_id,
                               title: next_book.title,
                               category_id: next_book.category_id,
                               available_copies: size([(next_book)-[:HAS_COPY]->(nc:Copy {status:'Available'
    | }) | nc])
                           } ELSE null END) WHERE x IS NOT NULL] AS next_books
                    ORDER BY l.due_date ASC
                """, uid=user_id, today=today)
                data["current_loans"] = rows(res)
                data["active_loans"]  = data["current_loans"]

                # Accurate Committed Books Calculation (Active Loans + Awaiting Pickup)
                data["active_loans_count"] = len(data["current_loans"])
                await_res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest {status: 'Awaiting Pickup'})
```

```
                    RETURN count(r) AS c
                """, uid=user_id)
                data["awaiting_pickup_count"] = await_res.single()["c"]
                data["total_committed"] = data["active_loans_count"] + data["awaiting_pickup_count"]

                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
                    OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    RETURN coalesce(sum(f.fine_amount), 0) AS total_fines
                """, uid=user_id)
                data["total_fines"] = res.single()["total_fines"]

                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b:Book)
                    RETURN r.request_id AS request_id, b.book_id AS book_id,
                           r.copy_id AS copy_id, b.title AS book_title,
                           r.request_date AS request_date, r.status AS status
                    ORDER BY r.request_id DESC
                """, uid=user_id)
                data["loan_requests"] = rows(res)

                return render_template("dashboard.html", data=data)

            elif role == "Student":
                today = today_str()
                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]
    | -(b:Book)
                    WHERE l.return_date IS NULL
                    OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    WITH u, l, cp, b, coalesce(sum(f.fine_amount), 0) AS fine_amount
                    OPTIONAL MATCH (pre_book:Book)-[:PREREQUISITE_FOR]->(b)
                    WITH u, l, cp, b, fine_amount,
                         collect(DISTINCT CASE WHEN pre_book IS NOT NULL THEN {
                             book_id: pre_book.book_id,
                             title: pre_book.title,
                             category_id: pre_book.category_id
                         } ELSE null END) AS pre_list
                    OPTIONAL MATCH (b)-[:PREREQUISITE_FOR]->(next_book:Book)
                    RETURN l.loan_id AS loan_id, b.book_id AS book_id, b.title AS title,
                           cp.copy_id AS copy_id,
                           l.issue_date AS issue_date, l.due_date AS due_date,
                           CASE WHEN l.due_date < $today THEN 1 ELSE 0 END AS is_overdue,
                           fine_amount,
                           [x IN pre_list WHERE x IS NOT NULL] AS prerequisites,
                           [x IN collect(DISTINCT CASE WHEN next_book IS NOT NULL THEN {
                               book_id: next_book.book_id,
                               title: next_book.title,
                               category_id: next_book.category_id,
                               available_copies: size([(next_book)-[:HAS_COPY]->(nc:Copy {status:'Available'
    | }) | nc])
                           } ELSE null END) WHERE x IS NOT NULL] AS next_books
                    ORDER BY l.due_date ASC
                """, uid=user_id, today=today)
                data["active_loans"] = rows(res)

                # Accurate Committed Books Calculation (Active Loans + Awaiting Pickup)
                data["active_loans_count"] = len(data["active_loans"])
                await_res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest {status: 'Awaiting Pickup'})
                    RETURN count(r) AS c
                """, uid=user_id)
                data["awaiting_pickup_count"] = await_res.single()["c"]
                data["total_committed"] = data["active_loans_count"] + data["awaiting_pickup_count"]

                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
                    OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                    RETURN coalesce(sum(f.fine_amount), 0) AS total_fines
                """, uid=user_id)
                data["total_fines"] = res.single()["total_fines"]

                res = s.run("""
                    MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b:Book)
                    RETURN r.request_id AS request_id, b.book_id AS book_id,
                           r.copy_id AS copy_id, b.title AS book_title,
                           r.request_date AS request_date, r.status AS status
                    ORDER BY r.request_id DESC
                """, uid=user_id)
                data["loan_requests"] = rows(res)
```

```
0711:
0712:                 return render_template("dashboard.html", data=data)
0713:
0714:             else:
0715:                 flash("Unknown user role.", "danger")
0716:                 return redirect(url_for("logout"))
0717:
0718:     except Exception as e:
0719:         flash(f"Error fetching dashboard metrics: {str(e)}", "danger")
0720:         return render_template("dashboard.html", data={})
0721:
0722:
0723: # ------------------------------------------------------------------------
0724: # /catalog and /books
0725: # ------------------------------------------------------------------------
0726: @app.route("/catalog", methods=["GET"])
0727: @app.route("/books", methods=["GET"])
0728: @login_required
0729: def books():
0730:     category_id_param = request.args.get("category_id", type=int)
0731:     if category_id_param:
0732:         return redirect(url_for("category_books", category_id=category_id_param))
0733:
0734:     categories_list = []
0735:     try:
0736:         with get_session() as s:
0737:             res = s.run("""
0738:                 MATCH (c:Category)
0739:                 OPTIONAL MATCH (b:Book)-[:BELONGS_TO]->(c)
0740:                 RETURN c.category_id AS category_id, c.name AS name,
0741:                        c.section_code AS section_code,
0742:                        count(DISTINCT b) AS book_count
0743:                 ORDER BY c.name ASC
0744:             """)
0745:             categories_list = rows(res)
0746:     except Exception as e:
0747:         flash(f"Error retrieving category catalog: {str(e)}", "danger")
0748:
0749:     return render_template("catalog.html", categories=categories_list)
0750:
0751:
0752: # ------------------------------------------------------------------------
0753: # /category/<int:category_id>
0754: # ------------------------------------------------------------------------
0755: @app.route("/category/<int:category_id>", methods=["GET"])
0756: @login_required
0757: def category_books(category_id):
0758:     category    = None
0759:     book_list   = []
0760:     user_requests         = set()
0761:     user_pending_requests = set()
0762:     user_awaiting_pickup  = set()
0763:     user_active_loans     = set()
0764:     user_borrowing_blocked = False
0765:     block_reason = ""
0766:     q    = request.args.get("q", "").strip()
0767:     try:
0768:         page = max(1, int(request.args.get("page", 1)))
0769:     except (ValueError, TypeError):
0770:         page = 1
0771:
0772:     try:
0773:         with get_session() as s:
0774:             # Category metadata
0775:             res = s.run("""
0776:                 MATCH (c:Category {category_id: $cid})
0777:                 OPTIONAL MATCH (b:Book)-[:BELONGS_TO]->(c)
0778:                 RETURN c.category_id AS category_id, c.name AS name,
0779:                        c.section_code AS section_code,
0780:                        count(DISTINCT b) AS total_books
0781:             """, cid=category_id)
0782:             rec = res.single()
0783:             if not rec:
0784:                 flash("The requested category was not found.", "warning")
0785:                 return redirect(url_for("books"))
0786:             category = dict(rec)
0787:
0788:             # Books in category with optional search
0789:             if q:
0790:                 res = s.run("""
```

```
                    MATCH (b:Book)-[:BELONGS_TO]->(c:Category {category_id: $cid})
                    OPTIONAL MATCH (pub:Publisher)<-[:PUBLISHED_BY]-(b)
                    OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
                    WITH b, c, pub,
                         collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
                    WHERE toLower(b.title) CONTAINS toLower($q)
                       OR toLower(b.isbn) CONTAINS toLower($q)
                       OR replace(toLower(b.isbn), '-', '') CONTAINS replace(toLower($q), '-', '')
                       OR toLower(coalesce(pub.name, '')) CONTAINS toLower($q)
                       OR toString(b.publication_year) = $q
                       OR any(author IN author_list WHERE toLower(author) CONTAINS toLower($q))
                    RETURN b.book_id AS book_id, b.isbn AS isbn, b.title AS title,
                           b.publication_year AS publication_year,
                           b.category_id AS category_id, c.name AS category_name,
                           coalesce(pub.name, '') AS publisher_name,
                           reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS autho
    | rs,
                           size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status <> 'Retired' AND cp.status <> 'L
    | ost' | cp]) AS total_copies,
                           size([(b)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies
                    ORDER BY b.title ASC
                """, cid=category_id, q=q)
            else:
                limit  = 20
                offset = (page - 1) * limit
                res = s.run("""
                    MATCH (b:Book)-[:BELONGS_TO]->(c:Category {category_id: $cid})
                    OPTIONAL MATCH (pub:Publisher)<-[:PUBLISHED_BY]-(b)
                    OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
                    WITH b, c, pub,
                         collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
                    RETURN b.book_id AS book_id, b.isbn AS isbn, b.title AS title,
                           b.publication_year AS publication_year,
                           b.category_id AS category_id, c.name AS category_name,
                           coalesce(pub.name, '') AS publisher_name,
                           reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS autho
    | rs,
                           size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status <> 'Retired' AND cp.status <> 'L
    | ost' | cp]) AS total_copies,
                           size([(b)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies
                    ORDER BY b.title ASC
                    SKIP $offset LIMIT $limit
                """, cid=category_id, offset=offset, limit=limit)

            book_list = rows(res)

            # Frequently Borrowed Together: Co-borrowing Graph Analysis
            companion_res = s.run("""
                MATCH (b1:Book)-[:BELONGS_TO]->(:Category {category_id: $cid})
                OPTIONAL MATCH (b1)-[:HAS_COPY]->(:Copy)<-[:FOR_COPY]-(:Loan)<-[:BORROWED]-(u:User)-[:BORROW
    | ED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b2:Book)
                WHERE b1.book_id <> b2.book_id
                WITH b1, b2, count(DISTINCT u) AS common_patrons
                ORDER BY common_patrons DESC, b2.title ASC
                WITH b1, collect(CASE WHEN b2 IS NOT NULL THEN {id: b2.book_id, title: b2.title, count: comm
    | on_patrons} ELSE null END)[0] AS companion
                RETURN b1.book_id AS book_id, companion
            """, cid=category_id)
            companions_by_book = {r["book_id"]: r["companion"] for r in companion_res if r["companion"]}

            # Academic Progression: Prerequisites and Books to Study Next
            bids = [bk["book_id"] for bk in book_list]
            uid = session.get("user_id") or -1
            prereqs_by_book = {}
            nexts_by_book = {}
            if bids:
                prog_res = s.run("""
                    MATCH (b:Book) WHERE b.book_id IN $bids
                    OPTIONAL MATCH (pre:Book)-[:PREREQUISITE_FOR]->(b)
                    OPTIONAL MATCH (u:User {user_id: $uid})-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS
    | _COPY]-(pre)
                    WITH b, pre, (count(u) > 0) AS pre_completed
                    WITH b, collect(DISTINCT CASE WHEN pre IS NOT NULL THEN {
                        book_id: pre.book_id,
                        title: pre.title,
                        completed: pre_completed
                    } ELSE null END) AS prereqs
                    OPTIONAL MATCH (b)-[:PREREQUISITE_FOR]->(next_b:Book)
                    RETURN b.book_id AS book_id,
                           [x IN prereqs WHERE x IS NOT NULL] AS prerequisites,
```

```
0864:                             [x IN collect(DISTINCT CASE WHEN next_b IS NOT NULL THEN {
0865:                                 book_id: next_b.book_id,
0866:                                 title: next_b.title
0867:                             } ELSE null END) WHERE x IS NOT NULL] AS next_books
0868:                 """, bids=bids, uid=uid)
0869:                 for pr in rows(prog_res):
0870:                     prereqs_by_book[pr["book_id"]] = pr.get("prerequisites", [])
0871:                     nexts_by_book[pr["book_id"]] = pr.get("next_books", [])
0872:
0873:             for bk in book_list:
0874:                 bk["companion"] = companions_by_book.get(bk["book_id"])
0875:                 bk["prerequisites"] = prereqs_by_book.get(bk["book_id"], [])
0876:                 bk["next_books"] = nexts_by_book.get(bk["book_id"], [])
0877:
0878:             # User-specific flags
0879:             user_id   = session.get("user_id")
0880:             user_role = session.get("role")
0881:             if user_id and user_role in ["Student", "Staff"]:
0882:                 today = today_str()
0883:
0884:                 # Active requests
0885:                 res = s.run("""
0886:                     MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b:Book)
0887:                     WHERE r.status IN ['Pending', 'Awaiting Pickup']
0888:                     RETURN b.book_id AS book_id, r.status AS status
0889:                 """, uid=user_id)
0890:                 for r in res:
0891:                     bid = r["book_id"]
0892:                     user_requests.add(bid)
0893:                     if r["status"] == "Pending":
0894:                         user_pending_requests.add(bid)
0895:                     elif r["status"] == "Awaiting Pickup":
0896:                         user_awaiting_pickup.add(bid)
0897:
0898:                 # Active loans
0899:                 res = s.run("""
0900:                     MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan {return_date: null})-[:FOR_COPY]->(c
      | p:Copy)<-[:HAS_COPY]-(b:Book)
0901:                     RETURN DISTINCT b.book_id AS book_id
0902:                 """, uid=user_id)
0903:                 for r in res:
0904:                     user_active_loans.add(r["book_id"])
0905:
0906:                 # Unpaid fines check
0907:                 res = s.run("""
0908:                     MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payment_statu
      | s: 'Unpaid'})
0909:                     RETURN count(f) AS unpaid_count, coalesce(sum(f.fine_amount), 0) AS unpaid_total
0910:                 """, uid=user_id)
0911:                 f_check = res.single()
0912:
0913:                 # Overdue check
0914:                 res = s.run("""
0915:                     MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
0916:                     WHERE l.return_date IS NULL AND l.due_date < $today
0917:                     RETURN count(l) AS overdue_count
0918:                 """, uid=user_id, today=today)
0919:                 od_check = res.single()
0920:
0921:                 if f_check and f_check["unpaid_count"] > 0:
0922:                     user_borrowing_blocked = True
0923:                     block_reason = f"Unpaid fines (Rs. {f_check['unpaid_total']:.2f}). Please pay before bor
      | rowing."
0924:                 elif od_check and od_check["overdue_count"] > 0:
0925:                     user_borrowing_blocked = True
0926:                     block_reason = f"{od_check['overdue_count']} loan(s) overdue. Please return before borro
      | wing."
0927:
0928:     except Exception as e:
0929:         flash(f"Error retrieving category book collection: {str(e)}", "danger")
0930:         return redirect(url_for("books"))
0931:
0932:     return render_template(
0933:         "category_books.html",
0934:         category=category,
0935:         books=book_list,
0936:         user_requests=user_requests,
0937:         user_pending_requests=user_pending_requests,
0938:         user_awaiting_pickup=user_awaiting_pickup,
0939:         user_active_loans=user_active_loans,
```

```
0940:         user_borrowing_blocked=user_borrowing_blocked,
0941:         block_reason=block_reason,
0942:         page=page,
0943:         q=q
0944:     )
0945:
0946:
0947: # ---------------------------------------------------------------------------
0948: # /adjust_inventory/<int:book_id>
0949: # ---------------------------------------------------------------------------
0950: @app.route("/adjust_inventory/<int:book_id>", methods=["POST"])
0951: @login_required
0952: @role_required(["Admin", "Librarian"])
0953: def adjust_inventory(book_id):
0954:     action       = request.form.get("action")
0955:     category_id  = request.form.get("category_id")
0956:     quantity_str = request.form.get("quantity", "1").strip()
0957:
0958:     if not re.match(r"^\d+$", quantity_str):
0959:         flash("Invalid quantity: Only positive numerical digits are allowed.", "warning")
0960:         if category_id:
0961:             try:
0962:                 return redirect(url_for("category_books", category_id=int(category_id)))
0963:             except (ValueError, TypeError):
0964:                 pass
0965:         return redirect(url_for("books"))
0966:
0967:     qty = int(quantity_str)
0968:     if qty <= 0 or qty > 100:
0969:         flash("Quantity must be between 1 and 100.", "warning")
0970:         if category_id:
0971:             try:
0972:                 return redirect(url_for("category_books", category_id=int(category_id)))
0973:             except (ValueError, TypeError):
0974:                 pass
0975:         return redirect(url_for("books"))
0976:
0977:     try:
0978:         with get_session() as s:
0979:             res = s.run("MATCH (b:Book {book_id: $bid}) RETURN b.title AS title", bid=book_id)
0980:             rec = res.single()
0981:             if not rec:
0982:                 flash("Target book not found.", "warning")
0983:                 return redirect(url_for("books"))
0984:
0985:             book_title = rec["title"]
0986:
0987:             if action == "increase":
0988:                 cid_start = next_id(s, "Copy", "copy_id")
0989:                 for i in range(qty):
0990:                     new_cid = cid_start + i
0991:                     s.run("""
0992:                         MATCH (bk:Book {book_id: $bid})
0993:                         CREATE (cp:Copy {copy_id: $cid, book_id: $bid,
0994:                                          status: 'Available', condition: 'Good'})
0995:                         CREATE (bk)-[:HAS_COPY]->(cp)
0996:                     """, bid=book_id, cid=new_cid)
0997:
0998:                 res = s.run("""
0999:                     MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy)
1000:                     WHERE cp.status <> 'Retired'
1001:                     RETURN count(cp) AS total,
1002:                            size([(b)-[:HAS_COPY]->(c:Copy {status:'Available'}) | c]) AS avail
1003:                 """, bid=book_id)
1004:                 counts = res.single()
1005:                 flash(f"Added {qty} copies for '{book_title}' (Total: {counts['total']}, Available: {counts[
      | 'avail']}).", "success")
1006:
1007:             elif action == "decrease":
1008:                 res = s.run("""
1009:                     MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status: 'Available'})
1010:                     RETURN cp.copy_id AS copy_id LIMIT $qty
1011:                 """, bid=book_id, qty=qty)
1012:                 copy_ids = [r["copy_id"] for r in res]
1013:
1014:                 if not copy_ids:
1015:                     flash(f"No available copies in stock for '{book_title}'.", "warning")
1016:                 else:
1017:                     for cid in copy_ids:
1018:                         has_loans = s.run("""
```

```
                            MATCH (cp:Copy {copy_id: $cid})
                            RETURN EXISTS { MATCH (cp)<-[:FOR_COPY]-(:Loan) } AS has_loans
                        """, cid=cid).single()["has_loans"]

                        if has_loans:
                            s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Retired'", cid=cid)
                        else:
                            s.run("MATCH (cp:Copy {copy_id: $cid}) DETACH DELETE cp", cid=cid)
                    flash(f"Reduced inventory for '{book_title}' by {len(copy_ids)} copy/copies.", "success"
    | )

    except Exception as e:
        flash(f"Error updating inventory: {str(e)}", "danger")

    if category_id:
        try:
            return redirect(url_for("category_books", category_id=int(category_id)))
        except (ValueError, TypeError):
            pass
    return redirect(url_for("books"))


# ------------------------------------------------------------------------------
# Curriculum & Prerequisite Linking Helpers
# ------------------------------------------------------------------------------
def resolve_curriculum_track(s, track_choice, category_id):
    """Resolves specialization track name for academic prerequisite edges."""
    valid_tracks = [
        "Software Engineering",
        "Artificial Intelligence",
        "Data Engineering",
        "Systems & Infrastructure",
        "Theoretical CS"
    ]
    if track_choice in valid_tracks:
        return track_choice

    cat_name = ""
    if category_id:
        try:
            res_c = s.run("MATCH (c:Category {category_id: $cid}) RETURN c.name AS name", cid=int(category_i
    | d))
            rec_c = res_c.single()
            if rec_c and rec_c["name"]:
                cat_name = rec_c["name"]
        except Exception:
            pass

    c_low = cat_name.lower()
    if "data" in c_low or "database" in c_low:
        return "Data Engineering"
    elif "math" in c_low or "stat" in c_low:
        return "Theoretical CS"
    elif "physics" in c_low or "astronomy" in c_low or "hardware" in c_low or "network" in c_low or "operati
    | ng" in c_low:
        return "Systems & Infrastructure"
    elif "ai" in c_low or "intelligence" in c_low:
        return "Artificial Intelligence"
    elif "computer" in c_low or "software" in c_low or "code" in c_low:
        return "Software Engineering"
    return cat_name if cat_name else "Software Engineering"


def link_book_prerequisites(s, book_id, category_id, prereq_param, next_param, track_param, book_title=None)
    | :
    """
    Creates or updates [:PREREQUISITE_FOR] edges in Neo4j AuraDB.
    Supports smart auto-detection, prevents self-loops and direct 2-node cycles.
    """
    try:
        bid = int(book_id)
        cid = None
        if category_id is not None and str(category_id).strip():
            try:
                cid = int(str(category_id).strip())
            except (ValueError, TypeError):
                cid = None

        # Resolve track
        track = resolve_curriculum_track(s, track_param, cid)
```

```
1095:
1096:         # Resolve prereq ID
1097:         resolved_prereq_id = None
1098:         is_auto = False
1099:         if prereq_param == "auto" and cid:
1100:             # Find foundational book in the same Category with lowest incoming :PREREQUISITE_FOR degree
1101:             foundational_res = s.run("""
1102:                 MATCH (cand:Book)-[:BELONGS_TO]->(:Category {category_id: $cid})
1103:                 WHERE cand.book_id <> $bid
1104:                 OPTIONAL MATCH (cand)<-[r_in:PREREQUISITE_FOR]-()
1105:                 WITH cand, count(r_in) AS in_degree
1106:                 ORDER BY in_degree ASC, cand.book_id ASC
1107:                 LIMIT 1
1108:                 RETURN cand.book_id AS prereq_id, cand.title AS prereq_title
1109:             """, cid=cid, bid=bid)
1110:             found_rec = foundational_res.single()
1111:             if found_rec:
1112:                 resolved_prereq_id = found_rec["prereq_id"]
1113:                 is_auto = True
1114:         elif prereq_param and prereq_param != "auto":
1115:             try:
1116:                 resolved_prereq_id = int(prereq_param)
1117:             except (ValueError, TypeError):
1118:                 resolved_prereq_id = None
1119:
1120:         # Resolve next step ID
1121:         resolved_next_id = None
1122:         if next_param:
1123:             try:
1124:                 resolved_next_id = int(next_param)
1125:             except (ValueError, TypeError):
1126:                 resolved_next_id = None
1127:
1128:         # Guard 1: Prevent self-loops
1129:         if resolved_prereq_id and resolved_prereq_id == bid:
1130:             resolved_prereq_id = None
1131:         if resolved_next_id and resolved_next_id == bid:
1132:             resolved_next_id = None
1133:
1134:         # Guard 2: Prevent direct 2-node cycles (A -> B and B -> A)
1135:         if resolved_prereq_id and resolved_next_id and resolved_prereq_id == resolved_next_id:
1136:             resolved_next_id = None
1137:
1138:         # 1. Apply Prerequisite Link (pre -> b)
1139:         if resolved_prereq_id:
1140:             if is_auto:
1141:                 jump = "Category Progression"
1142:                 stage = "Stage 1 -> Stage 2"
1143:             else:
1144:                 jump = "Curriculum Progression"
1145:                 stage = "Stage 1 -> Stage 2"
1146:
1147:             s.run("""
1148:                 MATCH (pre:Book {book_id: $pre_id}), (b:Book {book_id: $bid})
1149:                 MERGE (pre)-[r:PREREQUISITE_FOR]->(b)
1150:                 SET r.difficulty_jump = $jump,
1151:                     r.stage = $stage,
1152:                     r.track = $track
1153:             """, pre_id=resolved_prereq_id, bid=bid, jump=jump, stage=stage, track=track)
1154:
1155:         # 2. Apply Next-Step Link (b -> nxt)
1156:         if resolved_next_id:
1157:             nxt_jump = "Advanced Specialization"
1158:             nxt_stage = "Stage 2 -> Stage 3"
1159:
1160:             s.run("""
1161:                 MATCH (b:Book {book_id: $bid}), (nxt:Book {book_id: $nxt_id})
1162:                 MERGE (b)-[r:PREREQUISITE_FOR]->(nxt)
1163:                 SET r.difficulty_jump = $nxt_jump,
1164:                     r.stage = $nxt_stage,
1165:                     r.track = $track
1166:             """, bid=bid, nxt_id=resolved_next_id, nxt_jump=nxt_jump, nxt_stage=nxt_stage, track=track)
1167:
1168:     except Exception as e:
1169:         app.logger.warning(f"link_book_prerequisites warning: {e}")
1170:
1171:
1172: # ------------------------------------------------------------------------------
1173: # /add_book
1174: # ------------------------------------------------------------------------------
```

```
@app.route("/add_book", methods=["GET", "POST"])
@login_required
@role_required(["Admin", "Librarian"])
def add_book():
    categories = []
    publishers = []
    authors    = []
    all_books  = []

    try:
        with get_session() as s:
            # Query all existing books with category and prerequisite hierarchy level
            book_res = s.run("""
                MATCH (b:Book)
                OPTIONAL MATCH (b)-[:BELONGS_TO]->(c:Category)
                OPTIONAL MATCH in_path = (root:Book)-[:PREREQUISITE_FOR*1..5]->(b)
                WHERE NOT ()-[:PREREQUISITE_FOR]->(root)
                WITH b, c, coalesce(max(length(in_path)) + 1, 1) AS level
                RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
                       c.category_id AS category_id,
                       coalesce(c.name, 'General') AS category_name,
                       level
                ORDER BY category_name ASC, level ASC, b.title ASC
            """)
            all_books = rows(book_res)

            res = s.run("MATCH (c:Category) RETURN c.category_id AS category_id, c.name AS name ORDER BY c.n
    | ame")
            categories = [dict(r) for r in res]

            res = s.run("MATCH (p:Publisher) RETURN p.publisher_id AS publisher_id, p.name AS name ORDER BY
    | p.name")
            publishers = [dict(r) for r in res]

            res = s.run("""
                MATCH (a:Author)
                RETURN a.author_id AS author_id,
                       (a.first_name + ' ' + a.last_name) AS full_name
                ORDER BY a.last_name
            """)
            authors = [dict(r) for r in res]

            if request.method == "POST":
                isbn            = request.form.get("isbn", "").strip()
                title           = request.form.get("title", "").strip()
                pub_year_str    = request.form.get("publication_year", "").strip()
                category_id     = request.form.get("category_id")
                selected_authors = request.form.getlist("author_ids")

                # Validate mandatory fields BEFORE creating any on-the-fly nodes in Neo4j
                if not isbn or not title:
                    flash("ISBN and Title are mandatory fields.", "warning")
                    return render_template("add_book.html", categories=categories, publishers=publishers, au
    | thors=authors, all_books=all_books)

                pub_year_val = None
                if pub_year_str:
                    try:
                        pub_year_val = int(pub_year_str)
                    except ValueError:
                        flash("Publication Year must be a valid numerical year.", "warning")
                        return render_template("add_book.html", categories=categories, publishers=publishers
    | , authors=authors, all_books=all_books)

                res_isbn = s.run("MATCH (b:Book {isbn: $isbn}) RETURN b.title AS title LIMIT 1", isbn=isbn)
                if res_isbn.single():
                    flash(f"A book with ISBN '{isbn}' already exists in the catalog.", "warning")
                    return render_template("add_book.html", categories=categories, publishers=publishers, au
    | thors=authors, all_books=all_books)

                try:
                    num_copies = min(max(1, int(request.form.get("num_copies", 1))), 50)
                except (ValueError, TypeError):
                    num_copies = 1

                publisher_id = request.form.get("publisher_id")

                # On-the-fly category (created only after validations pass)
                new_cat_name = request.form.get("new_category_name", "").strip()
                new_cat_code = request.form.get("new_category_code", "").strip()
```

```
1250:                 if new_cat_name:
1251:                     res = s.run("MATCH (c:Category) WHERE toLower(c.name)=toLower($n) RETURN c.category_id A
      | S cid", n=new_cat_name)
1252:                     rec = res.single()
1253:                     if rec:
1254:                         category_id = rec["cid"]
1255:                     else:
1256:                         new_cid = next_id(s, "Category", "category_id")
1257:                         cat_code_val = new_cat_code if new_cat_code else new_cat_name[:3].upper()
1258:                         s.run("CREATE (c:Category {category_id: $cid, name: $n, section_code: $code})",
1259:                               cid=new_cid, n=new_cat_name, code=cat_code_val)
1260:                         category_id = new_cid
1261:
1262:                 # On-the-fly publisher
1263:                 new_pub_name  = request.form.get("new_publisher_name", "").strip()
1264:                 new_pub_email = request.form.get("new_publisher_email", "").strip()
1265:                 if new_pub_name:
1266:                     res = s.run("MATCH (p:Publisher) WHERE toLower(p.name)=toLower($n) RETURN p.publisher_id
      |  AS pid", n=new_pub_name)
1267:                     rec = res.single()
1268:                     if rec:
1269:                         publisher_id = rec["pid"]
1270:                     else:
1271:                         new_pid = next_id(s, "Publisher", "publisher_id")
1272:                         s.run("CREATE (p:Publisher {publisher_id: $pid, name: $n, contact_email: $e})",
1273:                               pid=new_pid, n=new_pub_name, e=new_pub_email or "")
1274:                         publisher_id = new_pid
1275:
1276:                 # On-the-fly author (supports single-field authors without silently dropping them)
1277:                 new_auth_first = request.form.get("new_author_first", "").strip()
1278:                 new_auth_last  = request.form.get("new_author_last", "").strip()
1279:                 if new_auth_first or new_auth_last:
1280:                     fn = new_auth_first or ""
1281:                     ln = new_auth_last or ""
1282:                     res = s.run("""
1283:                         MATCH (a:Author)
1284:                         WHERE toLower(a.first_name)=toLower($fn) AND toLower(a.last_name)=toLower($ln)
1285:                         RETURN a.author_id AS aid
1286:                     """, fn=fn, ln=ln)
1287:                     rec = res.single()
1288:                     if rec:
1289:                         selected_authors.append(str(rec["aid"]))
1290:                     else:
1291:                         new_aid = next_id(s, "Author", "author_id")
1292:                         s.run("CREATE (a:Author {author_id: $aid, first_name: $fn, last_name: $ln})",
1293:                               aid=new_aid, fn=fn, ln=ln)
1294:                         selected_authors.append(str(new_aid))
1295:
1296:                 cat_id_int = None
1297:                 if category_id is not None and str(category_id).strip():
1298:                     try:
1299:                         cat_id_int = int(str(category_id).strip())
1300:                     except (ValueError, TypeError):
1301:                         cat_id_int = None
1302:
1303:                 pub_id_int = None
1304:                 if publisher_id is not None and str(publisher_id).strip():
1305:                     try:
1306:                         pub_id_int = int(str(publisher_id).strip())
1307:                     except (ValueError, TypeError):
1308:                         pub_id_int = None
1309:
1310:                 new_bid = next_id(s, "Book", "book_id")
1311:                 s.run("""
1312:                     CREATE (b:Book {
1313:                         book_id: $bid, isbn: $isbn, title: $title,
1314:                         publication_year: $year, category_id: $cat_id,
1315:                         publisher_id: $pub_id
1316:                     })
1317:                 """, bid=new_bid, isbn=isbn, title=title,
1318:                      year=pub_year_val,
1319:                      cat_id=cat_id_int,
1320:                      pub_id=pub_id_int)
1321:
1322:                 if cat_id_int:
1323:                     s.run("MATCH (b:Book {book_id: $bid}), (c:Category {category_id: $cid}) CREATE (b)-[:BEL
      | ONGS_TO]->(c)",
1324:                           bid=new_bid, cid=cat_id_int)
1325:                 if pub_id_int:
1326:                     s.run("MATCH (b:Book {book_id: $bid}), (p:Publisher {publisher_id: $pid}) CREATE (b)-[:P
```

```
      | UBLISHED_BY]->(p)",
1327:                           bid=new_bid, pid=pub_id_int)
1328:
1329:                 for aid_str in set(selected_authors):
1330:                     try:
1331:                         aid = int(aid_str)
1332:                         s.run("MATCH (a:Author {author_id: $aid}), (b:Book {book_id: $bid}) CREATE (a)-[:WRO
      | TE]->(b)",
1333:                               aid=aid, bid=new_bid)
1334:                     except (ValueError, TypeError):
1335:                         pass
1336:
1337:                 cid_start = next_id(s, "Copy", "copy_id")
1338:                 for i in range(num_copies):
1339:                     new_cid = cid_start + i
1340:                     s.run("""
1341:                         MATCH (bk:Book {book_id: $bid})
1342:                         CREATE (cp:Copy {copy_id: $cid, book_id: $bid,
1343:                                          status: 'Available', condition: 'New'})
1344:                         CREATE (bk)-[:HAS_COPY]->(cp)
1345:                     """, bid=new_bid, cid=new_cid)
1346:
1347:                 # Create academic prerequisite and next-step links
1348:                 prereq_book_id   = request.form.get("prereq_book_id", "").strip()
1349:                 next_book_id     = request.form.get("next_book_id", "").strip()
1350:                 curriculum_track = request.form.get("curriculum_track", "").strip()
1351:
1352:                 link_book_prerequisites(s, new_bid, category_id, prereq_book_id, next_book_id, curriculum_tr
      | ack, title)
1353:
1354:                 flash(f"Book '{title}' and {num_copies} copies created with academic roadmap links successfu
      | lly!", "success")
1355:                 return redirect(url_for("books"))
1356:
1357:     except Exception as e:
1358:         flash(f"Error in add book: {str(e)}", "danger")
1359:
1360:     return render_template("add_book.html", categories=categories, publishers=publishers, authors=authors, a
      | ll_books=all_books)
1361:
1362:
1363: # ------------------------------------------------------------------------
1364: # /edit_book/<int:book_id>
1365: # ------------------------------------------------------------------------
1366: @app.route("/edit_book/<int:book_id>", methods=["GET", "POST"])
1367: @login_required
1368: @role_required(["Admin", "Librarian"])
1369: def edit_book(book_id):
1370:     try:
1371:         with get_session() as s:
1372:             # Query book metadata along with existing incoming/outgoing prerequisite links, publisher, and a
      | uthors
1373:             res = s.run("""
1374:                 MATCH (b:Book {book_id: $bid})
1375:                 OPTIONAL MATCH (b)-[:BELONGS_TO]->(c:Category)
1376:                 OPTIONAL MATCH (b)-[:PUBLISHED_BY]->(p:Publisher)
1377:                 OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
1378:                 OPTIONAL MATCH (pre:Book)-[r_pre:PREREQUISITE_FOR]->(b)
1379:                 OPTIONAL MATCH (b)-[r_nxt:PREREQUISITE_FOR]->(nxt:Book)
1380:                 WITH b, c, p,
1381:                      collect(DISTINCT a.author_id) AS current_author_ids,
1382:                      collect(DISTINCT pre.book_id) AS prereq_ids,
1383:                      collect(DISTINCT nxt.book_id) AS next_ids,
1384:                      collect(DISTINCT coalesce(r_pre.track, r_nxt.track)) AS tracks
1385:                 RETURN b.book_id AS book_id, b.isbn AS isbn, b.title AS title,
1386:                        b.publication_year AS publication_year,
1387:                        coalesce(b.category_id, c.category_id) AS category_id,
1388:                        c.name AS category_name,
1389:                        coalesce(b.publisher_id, p.publisher_id) AS publisher_id,
1390:                        p.name AS publisher_name,
1391:                        current_author_ids,
1392:                        prereq_ids[0] AS current_prereq_id,
1393:                        next_ids[0] AS current_next_id,
1394:                        tracks[0] AS current_track
1395:             """, bid=book_id)
1396:             book = res.single()
1397:             if not book:
1398:                 flash("Target book not found.", "warning")
1399:                 return redirect(url_for("books"))
1400:             book = dict(book)
```

```
1401:
1402:             # Query all categories for dropdown
1403:             c_res = s.run("MATCH (c:Category) RETURN c.category_id AS category_id, c.name AS name ORDER BY c
      | .name")
1404:             categories = [dict(r) for r in c_res]
1405:
1406:             # Query all publishers for dropdown
1407:             p_res = s.run("MATCH (p:Publisher) RETURN p.publisher_id AS publisher_id, p.name AS name ORDER B
      | Y p.name")
1408:             publishers = [dict(r) for r in p_res]
1409:
1410:             # Query all authors for dropdown
1411:             a_res = s.run("""
1412:                 MATCH (a:Author)
1413:                 RETURN a.author_id AS author_id, (a.first_name + ' ' + a.last_name) AS full_name
1414:                 ORDER BY a.last_name, a.first_name
1415:             """)
1416:             authors = [dict(r) for r in a_res]
1417:
1418:             # Query all books with category and prerequisite hierarchy level
1419:             book_res = s.run("""
1420:                 MATCH (b:Book)
1421:                 OPTIONAL MATCH (b)-[:BELONGS_TO]->(c:Category)
1422:                 OPTIONAL MATCH in_path = (root:Book)-[:PREREQUISITE_FOR*1..5]->(b)
1423:                 WHERE NOT ()-[:PREREQUISITE_FOR]->(root)
1424:                 WITH b, c, coalesce(max(length(in_path)) + 1, 1) AS level
1425:                 RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
1426:                        c.category_id AS category_id,
1427:                        coalesce(c.name, 'General') AS category_name,
1428:                        level
1429:                 ORDER BY category_name ASC, level ASC, b.title ASC
1430:             """)
1431:             all_books = rows(book_res)
1432:
1433:             if request.method == "POST":
1434:                 title           = request.form.get("title", "").strip()
1435:                 isbn            = request.form.get("isbn", "").strip()
1436:                 pub_year        = request.form.get("publication_year", "").strip()
1437:                 category_id     = request.form.get("category_id")
1438:                 publisher_id    = request.form.get("publisher_id")
1439:                 selected_authors = request.form.getlist("author_ids")
1440:
1441:                 if not title or not isbn:
1442:                     flash("Title and ISBN are mandatory.", "warning")
1443:                     return render_template("edit_book.html", book=book, categories=categories, publishers=pu
      | blishers, authors=authors, all_books=all_books)
1444:
1445:                 try:
1446:                     pub_year_val = int(pub_year) if pub_year else None
1447:                 except ValueError:
1448:                     flash("Publication Year must be a valid numerical year.", "warning")
1449:                     return render_template("edit_book.html", book=book, categories=categories, publishers=pu
      | blishers, authors=authors, all_books=all_books)
1450:
1451:                 res_isbn = s.run("""
1452:                     MATCH (b:Book {isbn: $isbn})
1453:                     WHERE b.book_id <> $bid
1454:                     RETURN b.title AS title LIMIT 1
1455:                 """, isbn=isbn, bid=book_id)
1456:                 if res_isbn.single():
1457:                     flash(f"Another book with ISBN '{isbn}' already exists in the catalog.", "warning")
1458:                     return render_template("edit_book.html", book=book, categories=categories, publishers=pu
      | blishers, authors=authors, all_books=all_books)
1459:
1460:                 cat_id_val = None
1461:                 if category_id is not None and str(category_id).strip():
1462:                     try:
1463:                         cat_id_val = int(str(category_id).strip())
1464:                     except (ValueError, TypeError):
1465:                         cat_id_val = book.get("category_id")
1466:                 else:
1467:                     cat_id_val = book.get("category_id")
1468:
1469:                 pub_id_val = None
1470:                 if publisher_id is not None and str(publisher_id).strip():
1471:                     try:
1472:                         pub_id_val = int(str(publisher_id).strip())
1473:                     except (ValueError, TypeError):
1474:                         pub_id_val = book.get("publisher_id")
1475:                 else:
```

```
                    pub_id_val = book.get("publisher_id")

                # Update primary metadata, category, and publisher properties
                s.run("""
                    MATCH (b:Book {book_id: $bid})
                    SET b.title = $title, b.isbn = $isbn, b.publication_year = $year,
                        b.category_id = $cid, b.publisher_id = $pid
                """, bid=book_id, title=title, isbn=isbn, year=pub_year_val, cid=cat_id_val, pid=pub_id_val)

                # Synchronize [:BELONGS_TO] Category edge
                if cat_id_val:
                    s.run("""
                        MATCH (b:Book {book_id: $bid})
                        OPTIONAL MATCH (b)-[r:BELONGS_TO]->()
                        DELETE r
                        WITH b
                        MATCH (c:Category {category_id: $cid})
                        MERGE (b)-[:BELONGS_TO]->(c)
                    """, bid=book_id, cid=cat_id_val)

                # Synchronize [:PUBLISHED_BY] Publisher edge
                if pub_id_val:
                    s.run("""
                        MATCH (b:Book {book_id: $bid})
                        OPTIONAL MATCH (b)-[r:PUBLISHED_BY]->()
                        DELETE r
                        WITH b
                        MATCH (p:Publisher {publisher_id: $pid})
                        MERGE (b)-[:PUBLISHED_BY]->(p)
                    """, bid=book_id, pid=pub_id_val)

                # Synchronize [:WROTE] Author edges
                if selected_authors:
                    s.run("""
                        MATCH (a:Author)-[r:WROTE]->(b:Book {book_id: $bid})
                        DELETE r
                    """, bid=book_id)
                    for aid_str in set(selected_authors):
                        try:
                            aid = int(aid_str)
                            s.run("""
                                MATCH (a:Author {author_id: $aid}), (b:Book {book_id: $bid})
                                MERGE (a)-[:WROTE]->(b)
                            """, aid=aid, bid=book_id)
                        except (ValueError, TypeError):
                            pass

                # Clear old :PREREQUISITE_FOR edges before applying updated selections
                s.run("""
                    MATCH (b:Book {book_id: $bid})
                    OPTIONAL MATCH (old_pre:Book)-[r_in:PREREQUISITE_FOR]->(b)
                    OPTIONAL MATCH (b)-[r_out:PREREQUISITE_FOR]->(old_nxt:Book)
                    DELETE r_in, r_out
                """, bid=book_id)

                prereq_book_id   = request.form.get("prereq_book_id", "").strip()
                next_book_id     = request.form.get("next_book_id", "").strip()
                curriculum_track = request.form.get("curriculum_track", "").strip()

                link_book_prerequisites(s, book_id, cat_id_val, prereq_book_id, next_book_id, curriculum_tra
    | ck, title)

                flash(f"Book '{title}' and academic learning roadmap links updated successfully.", "success"
    | )
                if cat_id_val:
                    return redirect(url_for("category_books", category_id=int(cat_id_val)))
                return redirect(url_for("books"))

            return render_template("edit_book.html", book=book, categories=categories, publishers=publishers
    | , authors=authors, all_books=all_books)

    except Exception as e:
        flash(f"Error updating book: {str(e)}", "danger")
        return redirect(url_for("books"))


# ------------------------------------------------------------------------------
# /delete_book/<int:book_id>
# ------------------------------------------------------------------------------
@app.route("/delete_book/<int:book_id>", methods=["POST"])
```

```
@login_required
@role_required(["Admin"])
def delete_book(book_id):
    raw_cat = request.form.get("category_id")
    category_id_val = None
    if raw_cat is not None and str(raw_cat).strip():
        try:
            category_id_val = int(str(raw_cat).strip())
        except (ValueError, TypeError):
            category_id_val = None

    try:
        with get_session() as s:
            res = s.run("MATCH (b:Book {book_id: $bid}) RETURN b.title AS title, b.category_id AS category_id", bid=book_id)
            rec = res.single()
            if not rec:
                flash("Book record not found.", "warning")
                return redirect(url_for("books"))

            book_title = rec["title"]
            if category_id_val is None:
                category_id_val = rec.get("category_id")

            # Check active loans
            res = s.run("""
                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy)<-[:FOR_COPY]-(l:Loan)
                WHERE l.return_date IS NULL
                RETURN count(l) AS active_loans
            """, bid=book_id)
            active_loans = res.single()["active_loans"]

            if active_loans > 0:
                flash(f"Cannot delete '{book_title}': {active_loans} active loan(s) exist.", "danger")
                if category_id_val:
                    return redirect(url_for("category_books", category_id=category_id_val))
                return redirect(url_for("books"))

            # Check unpaid fines
            res = s.run("""
                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(:Copy)<-[:FOR_COPY]-(:Loan)-[:HAS_FINE]->(f:Fine {payment_status: 'Unpaid'})
                RETURN count(f) AS unpaid_fines
            """, bid=book_id)
            unpaid_fines = res.single()["unpaid_fines"]
            if unpaid_fines > 0:
                flash(f"Cannot delete '{book_title}': {unpaid_fines} unpaid fine(s) exist on loans for this book.", "danger")
                if category_id_val:
                    return redirect(url_for("category_books", category_id=category_id_val))
                return redirect(url_for("books"))

            # Check active awaiting pickup reservations
            res = s.run("""
                MATCH (r:LoanRequest {status: 'Awaiting Pickup'})-[:FOR_BOOK]->(b:Book {book_id: $bid})
                RETURN count(r) AS awaiting_count
            """, bid=book_id)
            awaiting_count = res.single()["awaiting_count"]
            if awaiting_count > 0:
                flash("Cannot delete book: Active pickup reservation(s) exist.", "danger")
                if category_id_val:
                    return redirect(url_for("category_books", category_id=category_id_val))
                return redirect(url_for("books"))

            # Delete loan requests
            s.run("MATCH (r:LoanRequest)-[:FOR_BOOK]->(b:Book {book_id: $bid}) DETACH DELETE r", bid=book_id)
            # Delete fines & loans via copies
            s.run("""
                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy)<-[:FOR_COPY]-(l:Loan)
                OPTIONAL MATCH (l)-[:HAS_FINE]->(f:Fine)
                DETACH DELETE f, l
            """, bid=book_id)
            # Delete copies and book
            s.run("MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy) DETACH DELETE cp", bid=book_id)
            s.run("MATCH (b:Book {book_id: $bid}) DETACH DELETE b", bid=book_id)

            flash(f"Book '{book_title}' and all associated records were permanently deleted.", "success")

    except Exception as e:
```

```
1629:         flash(f"Error archiving book: {str(e)}", "danger")
1630:
1631:     if category_id_val:
1632:         return redirect(url_for("category_books", category_id=category_id_val))
1633:     return redirect(url_for("books"))
1634:
1635:
1636: # ---------------------------------------------------------------------------
1637: # /issue_loan
1638: # ---------------------------------------------------------------------------
1639: @app.route("/issue_loan", methods=["GET", "POST"])
1640: @login_required
1641: @role_required(["Admin", "Librarian"])
1642: def issue_loan():
1643:     patrons          = []
1644:     available_copies = []
1645:     all_books        = []
1646:
1647:     try:
1648:         with get_session() as s:
1649:             sync_active_overdue_fines(s)
1650:             if request.method == "POST":
1651:                 copy_id_raw       = request.form.get("copy_id")
1652:                 book_id_raw       = request.form.get("book_id")
1653:                 borrower_username = (request.form.get("borrower_username") or request.form.get("patron_quick
      | _search") or "").strip()
1654:                 borrower_id_raw   = request.form.get("borrower_id")
1655:                 due_in_days_raw   = request.form.get("due_in_days", 14)
1656:                 issued_by         = session["user_id"]
1657:                 today             = today_str()
1658:
1659:                 try:
1660:                     due_in_days = int(due_in_days_raw)
1661:                 except (ValueError, TypeError):
1662:                     flash("Invalid loan duration: Must be a valid integer between 1 and 60 days.", "danger")
1663:                     return redirect(url_for("issue_loan"))
1664:
1665:                 if due_in_days < 1 or due_in_days > 60:
1666:                     flash("Invalid loan duration: Must be between 1 and 60 days.", "danger")
1667:                     return redirect(url_for("issue_loan"))
1668:
1669:                 copy_id = None
1670:                 if copy_id_raw:
1671:                     try:
1672:                         copy_id = int(copy_id_raw)
1673:                     except (ValueError, TypeError):
1674:                         flash("Invalid copy ID specified.", "danger")
1675:                         return redirect(url_for("issue_loan"))
1676:
1677:                 book_id = None
1678:                 if book_id_raw:
1679:                     try:
1680:                         book_id = int(book_id_raw)
1681:                     except (ValueError, TypeError):
1682:                         pass
1683:
1684:                 # Fallback: pick first available copy for a book
1685:                 if not copy_id and book_id:
1686:                     try:
1687:                         res = s.run("""
1688:                             MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy)
1689:                             WHERE cp.status = 'Available'
1690:                             RETURN cp.copy_id AS copy_id LIMIT 1
1691:                         """, bid=book_id)
1692:                         rec = res.single()
1693:                         if rec:
1694:                             copy_id = rec["copy_id"]
1695:                     except (ValueError, TypeError):
1696:                         pass
1697:
1698:                 # Resolve borrower
1699:                 borrower_status = None
1700:                 borrower_role   = None
1701:                 if borrower_username:
1702:                     res = s.run("""
1703:                         MATCH (u:User)
1704:                         WHERE toLower(u.username) = toLower($uname)
1705:                         RETURN u.user_id AS user_id, u.account_status AS account_status, u.role AS role
1706:                     """, uname=borrower_username)
1707:                     rec = res.single()
```

```
1708:                     if not rec:
1709:                         flash(f"Borrower '{borrower_username}' not found.", "danger")
1710:                         borrower_id = None
1711:                     else:
1712:                         borrower_id     = rec["user_id"]
1713:                         borrower_status = rec["account_status"]
1714:                         borrower_role   = rec["role"]
1715:                 elif borrower_id:
1716:                     try:
1717:                         borrower_id = int(borrower_id)
1718:                         res = s.run("""
1719:                             MATCH (u:User {user_id: $uid})
1720:                             RETURN u.account_status AS account_status, u.role AS role
1721:                         """, uid=borrower_id)
1722:                         rec = res.single()
1723:                         if rec:
1724:                             borrower_status = rec["account_status"]
1725:                             borrower_role   = rec["role"]
1726:                         else:
1727:                             borrower_id = None
1728:                             flash("Borrower not found.", "danger")
1729:                     except (ValueError, TypeError):
1730:                         borrower_id = None
1731:
1732:                 if not copy_id or not borrower_id:
1733:                     flash("Please provide both a valid copy and a patron username.", "warning")
1734:                 elif borrower_status != "Active":
1735:                     flash(f"Borrowing Prohibited: Borrower account is {borrower_status or 'Inactive'}.", "da
      | nger")
1736:                 elif borrower_role not in ["Student", "Staff"]:
1737:                     flash(f"Borrowing Prohibited: User role '{borrower_role}' is not eligible to borrow book
      | s (only Student or Staff accounts).", "danger")
1738:                 else:
1739:                     # Check copy status & reservation ownership
1740:                     res_cp = s.run("""
1741:                         MATCH (cp:Copy {copy_id: $cid})
1742:                         OPTIONAL MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)
1743:                         WHERE r.status = 'Awaiting Pickup' AND (r.copy_id = cp.copy_id OR (r)-[:RESERVED_COP
      | Y]->(cp))
1744:                         RETURN cp.status AS status, count(r) AS reserved_for_user
1745:                     """, cid=int(copy_id), uid=borrower_id)
1746:                     cp_rec = res_cp.single()
1747:
1748:                     # Unpaid fines check
1749:                     res = s.run("""
1750:                         MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payment_s
      | tatus:'Unpaid'})
1751:                         RETURN coalesce(sum(f.fine_amount),0) AS unpaid_total
1752:                     """, uid=borrower_id)
1753:                     unpaid_total = res.single()["unpaid_total"]
1754:
1755:                     # Overdue check
1756:                     res = s.run("""
1757:                         MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
1758:                         WHERE l.return_date IS NULL AND l.due_date < $today
1759:                         RETURN count(l) AS overdue_count
1760:                     """, uid=borrower_id, today=today)
1761:                     overdue_count = res.single()["overdue_count"]
1762:
1763:                     # Quota check (active loans + awaiting pickup, excluding this book)
1764:                     res = s.run("""
1765:                         MATCH (cp:Copy {copy_id: $cid})<-[:HAS_COPY]-(b:Book)
1766:                         WITH b.book_id AS this_book_id
1767:                         MATCH (u:User {user_id: $uid})
1768:                         OPTIONAL MATCH (u)-[:BORROWED]->(al:Loan) WHERE al.return_date IS NULL
1769:                         OPTIONAL MATCH (u)-[:REQUESTED]->(ar:LoanRequest)-[:FOR_BOOK]->(ab:Book)
1770:                         WHERE ar.status = 'Awaiting Pickup' AND ab.book_id <> this_book_id
1771:                         RETURN count(DISTINCT al) + count(DISTINCT ar) AS other_commitments
1772:                     """, cid=int(copy_id), uid=borrower_id)
1773:                     other_commitments = res.single()["other_commitments"]
1774:
1775:                     # Same-book duplicate check
1776:                     res = s.run("""
1777:                         MATCH (cp:Copy {copy_id: $cid})<-[:HAS_COPY]-(b:Book)
1778:                         MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan {return_date:null})-[:FOR_COPY]-
      | >(c2:Copy)<-[:HAS_COPY]-(b)
1779:                         RETURN count(l) AS same_book_count
1780:                     """, cid=int(copy_id), uid=borrower_id)
1781:                     same_book_count = res.single()["same_book_count"]
1782:
```

```
                    if not cp_rec:
                        flash(f"Copy #{copy_id} not found in inventory.", "danger")
                    elif cp_rec["status"] != "Available" and not (cp_rec["status"] == "Reserved" and cp_rec[
    | "reserved_for_user"] > 0):
                        flash(f"Checkout Prohibited: Copy #{copy_id} is currently {cp_rec['status']}.", "war
    | ning")
                    elif unpaid_total > 0:
                        flash(f"Borrowing Prohibited: Patron has unpaid fines of Rs. {unpaid_total:.2f}.", "
    | warning")
                    elif overdue_count > 0:
                        flash(f"Borrowing Prohibited: Patron has {overdue_count} overdue loan(s).", "warning
    | ")
                    elif (other_commitments + 1) > 3:
                        flash(f"Borrowing Prohibited: Patron would exceed 3-book threshold ({other_commitmen
    | ts} existing).", "warning")
                    elif same_book_count > 0:
                        flash("Duplicate Loan Prohibited: Patron already has this book on loan.", "warning")
                    else:
                        # Issue the loan
                        issue_dt = datetime.now()
                        due_dt   = issue_dt + timedelta(days=due_in_days)
                        new_lid  = next_id(s, "Loan", "loan_id")

                        s.run("""
                            CREATE (l:Loan {
                                loan_id: $lid, copy_id: $cid, borrower_id: $bid,
                                issued_by: $issby,
                                issue_date: $issue_date, due_date: $due_date,
                                return_date: null
                            })
                        """, lid=new_lid, cid=int(copy_id), bid=borrower_id,
                             issby=issued_by,
                             issue_date=issue_dt.strftime("%Y-%m-%d"),
                             due_date=due_dt.strftime("%Y-%m-%d"))

                        s.run("""
                            MATCH (u:User {user_id: $uid}), (l:Loan {loan_id: $lid})
                            CREATE (u)-[:BORROWED]->(l)
                        """, uid=borrower_id, lid=new_lid)

                        s.run("""
                            MATCH (l:Loan {loan_id: $lid}), (cp:Copy {copy_id: $cid})
                            CREATE (l)-[:FOR_COPY]->(cp)
                        """, lid=new_lid, cid=int(copy_id))

                        s.run("""
                            MATCH (lib:User {user_id: $uid}), (l:Loan {loan_id: $lid})
                            CREATE (lib)-[:ISSUED]->(l)
                        """, uid=issued_by, lid=new_lid)

                        s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Borrowed'", cid=int(copy_id)
    | )

                        # Fulfil Awaiting Pickup or Pending request if exists
                        s.run("""
                            MATCH (cp:Copy {copy_id: $cid})<-[:HAS_COPY]-(b:Book)
                            MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b)
                            WHERE r.status IN ['Awaiting Pickup', 'Pending']
                            WITH r, cp, r.copy_id AS old_cid
                            OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
                            DELETE rel
                            SET r.status = 'Fulfilled', r.copy_id = cp.copy_id
                            WITH old_cid, cp
                            WHERE old_cid IS NOT NULL AND old_cid <> cp.copy_id
                            MATCH (rcp:Copy {copy_id: old_cid, status: 'Reserved'})
                            SET rcp.status = 'Available'
                        """, cid=int(copy_id), uid=borrower_id)

                        flash(f"Loan successfully issued for Copy #{copy_id}.", "success")
                        return redirect(url_for("dashboard"))

            # GET: fetch dropdown data
            res = s.run("""
                MATCH (b:Book)-[:BELONGS_TO]->(c:Category)
                OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
                WITH b, c, collect(DISTINCT a.first_name + ' ' + a.last_name) AS alist
                RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
                       c.name AS category_name,
                       reduce(s='', a IN alist | CASE WHEN s='' THEN a ELSE s+', '+a END) AS authors,
                       size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status = 'Available' | cp]) AS available_co
```

```
      | pies_count,
1857:                       size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status <> 'Retired' AND cp.status <> 'Lost'
      |  | cp]) AS total_copies_count
1858:                   ORDER BY b.title ASC
1859:               """)
1860:               all_books = [dict(r) for r in res]
1861:
1862:               today_cur = today_str()
1863:               res = s.run("""
1864:                   MATCH (u:User)
1865:                   WHERE u.role IN ['Student','Staff'] AND u.account_status = 'Active'
1866:                   WITH u,
1867:                        size([(u)-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payment_status:'Unpaid'}) | f]) A
      | S unpaid_fines_count,
1868:                        size([(u)-[:BORROWED]->(ol:Loan) WHERE ol.return_date IS NULL AND ol.due_date < $today
      | | ol]) AS overdue_count
1869:                   RETURN u.user_id AS user_id, u.username AS username,
1870:                          u.role AS role, u.email AS email,
1871:                          (unpaid_fines_count > 0 OR overdue_count > 0) AS has_blocking_issue,
1872:                          unpaid_fines_count, overdue_count
1873:                   ORDER BY u.role ASC, u.username ASC
1874:               """, today=today_cur)
1875:               patrons = [dict(r) for r in res]
1876:
1877:               res = s.run("""
1878:                   MATCH (b:Book)-[:HAS_COPY]->(cp:Copy)
1879:                   WHERE cp.status = 'Available'
1880:                   RETURN cp.copy_id AS copy_id, cp.book_id AS book_id,
1881:                          b.title AS book_title, b.isbn AS isbn,
1882:                          cp.status AS status, cp.condition AS condition
1883:                   ORDER BY b.title ASC, cp.copy_id ASC
1884:               """)
1885:               available_copies = [dict(r) for r in res]
1886:
1887:       except Exception as e:
1888:           flash(f"Failed to issue loan: {str(e)}", "danger")
1889:
1890:       return render_template("issue_loan.html", patrons=patrons,
1891:                              available_copies=available_copies, all_books=all_books)
1892:
1893:
1894: # ------------------------------------------------------------------------------
1895: # /my_fines
1896: # ------------------------------------------------------------------------------
1897: @app.route("/my_fines", methods=["GET"])
1898: @login_required
1899: def my_fines():
1900:     user_id    = session.get("user_id")
1901:     fines_list = []
1902:
1903:     try:
1904:         with get_session() as s:
1905:             sync_active_overdue_fines(s)
1906:             res = s.run("""
1907:                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine)
1908:                 OPTIONAL MATCH (l)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book)
1909:                 RETURN f.fine_id AS fine_id, f.fine_amount AS fine_amount,
1910:                        f.payment_status AS payment_status,
1911:                        l.loan_id AS loan_id, l.issue_date AS issue_date,
1912:                        l.due_date AS due_date, l.return_date AS return_date,
1913:                        coalesce(b.title, 'Archived/Deleted Title') AS book_title,
1914:                        coalesce(b.title, 'Archived/Deleted Title') AS title,
1915:                        cp.copy_id AS copy_id
1916:                 ORDER BY f.fine_id DESC
1917:             """, uid=user_id)
1918:             fines_list = rows(res)
1919:
1920:     except Exception as e:
1921:         flash(f"Error fetching fine statements: {str(e)}", "danger")
1922:
1923:     return render_template("my_fines.html", fines=fines_list)
1924:
1925:
1926: # ------------------------------------------------------------------------------
1927: # /manage_users (Admin only)
1928: # ------------------------------------------------------------------------------
1929: @app.route("/manage_users", methods=["GET", "POST"])
1930: @login_required
1931: @role_required(["Admin"])
1932: def manage_users():
```

```
    users_list = []
    try:
        with get_session() as s:
            if request.method == "POST":
                action = request.form.get("action")

                if action == "add_user":
                    new_uname = request.form.get("username", "").strip()
                    new_email = request.form.get("email", "").lower().strip()
                    new_pass  = request.form.get("password", "")
                    new_role  = request.form.get("role", "Student")

                    if not new_uname or not new_email or not new_pass:
                        flash("All fields are required.", "warning")
                    elif new_role not in ["Admin", "Librarian", "Staff", "Student"]:
                        flash(f"Invalid role: '{new_role}'. Permitted roles are Admin, Librarian, Staff, or
    | Student.", "danger")
                    else:
                        is_email_valid, email_err = validate_email_format(new_email)
                        if not is_email_valid:
                            flash(f"Invalid Email: {email_err}", "warning")
                        elif not (new_email.endswith("@vit.edu") or new_email.endswith("@vit.ac.in")):
                            flash("Registration requires a valid institutional email domain (@vit.edu or @vi
    | t.ac.in).", "danger")
                        else:
                            is_pass_valid, pass_err = validate_password_strength(new_pass)
                            if not is_pass_valid:
                                flash(f"Invalid Password: {pass_err}", "warning")
                            else:
                                res_chk = s.run("""
                                    MATCH (u:User)
                                    WHERE toLower(u.username) = toLower($uname) OR toLower(u.email) = toLowe
    | r($email)
                                    RETURN u.username AS username, u.email AS email LIMIT 1
                                """, uname=new_uname, email=new_email)
                                if res_chk.single():
                                    flash(f"User with username '{new_uname}' or email '{new_email}' already
    | exists.", "warning")
                                else:
                                    pw_hash = generate_password_hash(new_pass)
                                    new_uid = next_id(s, "User", "user_id")
                                    s.run("""
                                        CREATE (u:User {
                                            user_id: $uid, username: $uname,
                                            password_hash: $pw, email: $email,
                                            role: $role, account_status: 'Active'
                                        })
                                    """, uid=new_uid, uname=new_uname, pw=pw_hash,
                                         email=new_email, role=new_role)
                                    flash(f"User '{new_uname}' ({new_role}) created successfully!", "success
    | ")

                elif action == "toggle_status":
                    target_uid_raw = request.form.get("user_id")
                    current_status = request.form.get("current_status")

                    try:
                        target_uid = int(target_uid_raw)
                    except (ValueError, TypeError):
                        flash("Invalid user ID specified.", "danger")
                        return redirect(url_for("manage_users"))

                    if target_uid == session.get("user_id"):
                        flash("Action Prohibited: Cannot suspend your own session.", "danger")
                        return redirect(url_for("manage_users"))

                    # Protect Admin from suspension
                    res = s.run("MATCH (u:User {user_id: $uid}) RETURN u.role AS role, u.username AS usernam
    | e", uid=target_uid)
                    u_rec = res.single()
                    if u_rec and u_rec["role"] == "Admin":
                        flash(f"Action Prohibited: Admin account '{u_rec['username']}' cannot be suspended b
    | y system policy.", "danger")
                        return redirect(url_for("manage_users"))

                    new_status = "Suspended" if current_status == "Active" else "Active"
                    s.run("""
                        MATCH (u:User {user_id: $uid}) SET u.account_status = $status
                    """, uid=target_uid, status=new_status)
                    flash(f"User #{target_uid} status updated to '{new_status}'.", "info")
```

```
2006:
2007:                 elif action == "reset_password":
2008:                     target_uid_raw = request.form.get("user_id")
2009:                     try:
2010:                         target_uid = int(target_uid_raw)
2011:                     except (ValueError, TypeError):
2012:                         flash("Invalid user ID specified.", "danger")
2013:                         return redirect(url_for("manage_users"))
2014:
2015:                     try:
2016:                         res = s.run("MATCH (u:User {user_id: $uid}) RETURN u.role AS role, u.username AS use
     | rname", uid=target_uid)
2017:                         u_rec = res.single()
2018:                         if not u_rec:
2019:                             flash("User not found.", "warning")
2020:                         elif u_rec["role"] not in ["Student", "Staff", "Librarian"]:
2021:                             flash("Action Prohibited: Password reset via this panel is not permitted for Adm
     | in accounts.", "danger")
2022:                         else:
2023:                             uname = u_rec["username"]
2024:                             temp_pass = f"{uname.capitalize()}#Lib2026"
2025:                             new_hash = generate_password_hash(temp_pass)
2026:                             s.run("MATCH (u:User {user_id: $uid}) SET u.password_hash = $ph", uid=int(target
     | _uid), ph=new_hash)
2027:                             flash(f"Password for {u_rec['role']} '{uname}' has been reset to temporary passw
     | ord: {temp_pass}", "success")
2028:                     except Exception as e:
2029:                         flash(f"Error resetting password: {str(e)}", "danger")
2030:
2031:             res = s.run("""
2032:                 MATCH (u:User)
2033:                 RETURN u.user_id AS user_id, u.username AS username,
2034:                        u.email AS email, u.role AS role, u.account_status AS account_status
2035:                 ORDER BY u.user_id ASC
2036:             """)
2037:             users_list = [dict(r) for r in res]
2038:
2039:     except Exception as e:
2040:         flash(f"Error managing users: {str(e)}", "danger")
2041:
2042:     return render_template("manage_users.html", users=users_list)
2043:
2044:
2045: # ------------------------------------------------------------------------------
2046: # /delete_user/<int:user_id>
2047: # ------------------------------------------------------------------------------
2048: @app.route("/delete_user/<int:user_id>", methods=["POST"])
2049: @login_required
2050: @role_required(["Admin"])
2051: def delete_user(user_id):
2052:     if user_id == session.get("user_id"):
2053:         flash("Action Prohibited: Cannot delete your own Admin account.", "danger")
2054:         return redirect(url_for("manage_users"))
2055:
2056:     try:
2057:         with get_session() as s:
2058:             res = s.run("""
2059:                 MATCH (u:User {user_id: $uid})
2060:                 RETURN u.role AS role, u.username AS username
2061:             """, uid=user_id)
2062:             rec = res.single()
2063:             if not rec:
2064:                 flash("User not found.", "warning")
2065:                 return redirect(url_for("manage_users"))
2066:
2067:             if rec["role"] == "Admin":
2068:                 flash(f"Action Prohibited: Admin account '{rec['username']}' cannot be deleted by policy.",
     | "danger")
2069:                 return redirect(url_for("manage_users"))
2070:
2071:             # Check active loans
2072:             res = s.run("""
2073:                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
2074:                 WHERE l.return_date IS NULL
2075:                 RETURN count(l) AS active_loans
2076:             """, uid=user_id)
2077:             if res.single()["active_loans"] > 0:
2078:                 flash("Cannot delete user: Active loans exist.", "danger")
2079:                 return redirect(url_for("manage_users"))
2080:
```

```
            # Check unpaid fines
            res = s.run("""
                MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payment_status:'U
    | npaid'})
                RETURN count(f) AS unpaid_fines
            """, uid=user_id)
            if res.single()["unpaid_fines"] > 0:
                flash("Cannot delete user: Unpaid fines exist.", "danger")
                return redirect(url_for("manage_users"))

            # Release reserved copies before deleting loan requests
            s.run("""
                MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)
                OPTIONAL MATCH (r)-[:RESERVED_COPY]->(cp:Copy)
                OPTIONAL MATCH (cp2:Copy {copy_id: r.copy_id})
                FOREACH (c IN CASE WHEN cp IS NOT NULL THEN [cp] ELSE [] END | SET c.status = 'Available')
                FOREACH (c IN CASE WHEN cp2 IS NOT NULL THEN [cp2] ELSE [] END | SET c.status = 'Available')
                DETACH DELETE r
            """, uid=user_id)

            # Detach returned loans (anonymize to user_id=0)
            s.run("""
                MATCH (u:User {user_id: $uid})-[rel:BORROWED]->(l:Loan)
                WHERE l.return_date IS NOT NULL
                DELETE rel
                SET l.borrower_id = 0
            """, uid=user_id)

            # Delete user node
            s.run("MATCH (u:User {user_id: $uid}) DETACH DELETE u", uid=user_id)
            flash(f"User '{rec['username']}' (ID #{user_id}) deleted successfully.", "success")

    except Exception as e:
        flash(f"Error during account deletion: {str(e)}", "danger")

    return redirect(url_for("manage_users"))


# ------------------------------------------------------------------------------
# /change_password
# ------------------------------------------------------------------------------
@app.route("/change_password", methods=["GET", "POST"])
@login_required
def change_password():
    user_id = session.get("user_id")

    if request.method == "POST":
        current_pass = request.form.get("current_password", "")
        new_pass     = request.form.get("new_password", "")
        confirm_pass = request.form.get("confirm_password", "")

        if not current_pass or not new_pass or not confirm_pass:
            flash("All password fields are required.", "warning")
            return render_template("change_password.html")

        if new_pass != confirm_pass:
            flash("New password and confirmation do not match.", "warning")
            return render_template("change_password.html")

        is_valid, err_msg = validate_password_strength(new_pass)
        if not is_valid:
            flash(f"Invalid New Password: {err_msg}", "warning")
            return render_template("change_password.html")

        try:
            with get_session() as s:
                res = s.run("""
                    MATCH (u:User {user_id: $uid})
                    RETURN u.password_hash AS password_hash
                """, uid=user_id)
                rec = res.single()

                if not rec or not check_password_hash(rec["password_hash"], current_pass):
                    flash("Current password verification failed.", "danger")
                    return render_template("change_password.html")

                if check_password_hash(rec["password_hash"], new_pass):
                    flash("New password cannot be identical to your current password.", "warning")
                    return render_template("change_password.html")

```

```
2160:                 new_hash = generate_password_hash(new_pass)
2161:                 s.run("MATCH (u:User {user_id: $uid}) SET u.password_hash = $ph", uid=user_id, ph=new_hash)
2162:
2163:             flash("Password changed successfully! Please log in again.", "success")
2164:             session.clear()
2165:             return redirect(url_for("login"))
2166:
2167:         except Exception as e:
2168:             flash(f"Error updating password: {str(e)}", "danger")
2169:
2170:     return render_template("change_password.html")
2171:
2172:
2173: # ------------------------------------------------------------------------
2174: # /request_loan/<int:book_id>
2175: # ------------------------------------------------------------------------
2176: @app.route("/request_loan/<int:book_id>", methods=["POST"])
2177: @login_required
2178: def request_loan(book_id):
2179:     user_id = session.get("user_id")
2180:     role    = session.get("role")
2181:     today   = today_str()
2182:
2183:     if role not in ["Student", "Staff"]:
2184:         flash("Only Students and Staff may request book loans.", "warning")
2185:         return redirect(url_for("books"))
2186:
2187:     try:
2188:         with get_session() as s:
2189:             sync_active_overdue_fines(s)
2190:             # 1. Unpaid fines check
2191:             res = s.run("""
2192:                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payment_status:'U
      | npaid'})
2193:                 RETURN coalesce(sum(f.fine_amount), 0) AS unpaid_total
2194:             """, uid=user_id)
2195:             unpaid_total = res.single()["unpaid_total"]
2196:             if unpaid_total > 0:
2197:                 flash(f"You have unpaid fines of Rs. {unpaid_total:.2f}. Settle them before requesting.", "d
      | anger")
2198:                 return redirect(url_for("my_fines"))
2199:
2200:             # 2. Overdue check
2201:             res = s.run("""
2202:                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
2203:                 WHERE l.return_date IS NULL AND l.due_date < $today
2204:                 RETURN count(l) AS overdue_count
2205:             """, uid=user_id, today=today)
2206:             overdue_count = res.single()["overdue_count"]
2207:             if overdue_count > 0:
2208:                 flash(f"You have {overdue_count} overdue book(s). Return them before requesting.", "danger")
2209:                 return redirect(url_for("dashboard"))
2210:
2211:             # 3. Duplicate active loan check
2212:             res = s.run("""
2213:                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan {return_date:null})-[:FOR_COPY]->(cp:Cop
      | y)<-[:HAS_COPY]-(b:Book {book_id: $bid})
2214:                 RETURN count(l) AS same_book_count
2215:             """, uid=user_id, bid=book_id)
2216:             if res.single()["same_book_count"] > 0:
2217:                 flash("Duplicate Loan: You already have this book on active loan.", "warning")
2218:                 return redirect(url_for("books"))
2219:
2220:             # 4. Existing pending/awaiting check
2221:             res = s.run("""
2222:                 MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]->(b:Book {book_id:
      | $bid})
2223:                 WHERE r.status IN ['Pending', 'Awaiting Pickup']
2224:                 RETURN count(r) AS pending_count
2225:             """, uid=user_id, bid=book_id)
2226:             if res.single()["pending_count"] > 0:
2227:                 flash("You already have an active request for this title.", "info")
2228:                 return redirect(url_for("books"))
2229:
2230:             # 5. Quota check (active + awaiting + pending >= 3)
2231:             res = s.run("""
2232:                 MATCH (u:User {user_id: $uid})
2233:                 OPTIONAL MATCH (u)-[:BORROWED]->(al:Loan) WHERE al.return_date IS NULL
2234:                 OPTIONAL MATCH (u)-[:REQUESTED]->(ap:LoanRequest) WHERE ap.status = 'Awaiting Pickup'
2235:                 OPTIONAL MATCH (u)-[:REQUESTED]->(pnd:LoanRequest) WHERE pnd.status = 'Pending'
```

```
2236:                 RETURN count(DISTINCT al) AS act_cnt,
2237:                        count(DISTINCT ap)  AS ap_cnt,
2238:                        count(DISTINCT pnd) AS pend_cnt
2239:             """, uid=user_id)
2240:             q = res.single()
2241:             act_cnt  = q["act_cnt"]  or 0
2242:             ap_cnt   = q["ap_cnt"]   or 0
2243:             pend_cnt = q["pend_cnt"] or 0
2244:             combined = act_cnt + ap_cnt + pend_cnt
2245:
2246:             if combined >= 3:
2247:                 flash(f"Quota Reached: {combined}/3 books ({act_cnt} active, {ap_cnt} awaiting, {pend_cnt} p
      | ending).", "warning")
2248:                 cat_id = request.form.get("category_id")
2249:                 if cat_id:
2250:                     try:
2251:                         return redirect(url_for("category_books", category_id=int(cat_id)))
2252:                     except (ValueError, TypeError):
2253:                         pass
2254:                 return redirect(url_for("books"))
2255:
2256:             # 6. Get book title
2257:             res = s.run("MATCH (b:Book {book_id: $bid}) RETURN b.title AS title", bid=book_id)
2258:             rec = res.single()
2259:             book_title = rec["title"] if rec else f"Book #{book_id}"
2260:
2261:             # 7. Create LoanRequest node and relationships
2262:             new_rid  = next_id(s, "LoanRequest", "request_id")
2263:             req_date = datetime.now().strftime("%Y-%m-%d")
2264:
2265:             s.run("""
2266:                 CREATE (r:LoanRequest {
2267:                     request_id: $rid, book_id: $bid, borrower_id: $uid,
2268:                     copy_id: null, request_date: $rdate, status: 'Pending'
2269:                 })
2270:             """, rid=new_rid, bid=book_id, uid=user_id, rdate=req_date)
2271:
2272:             s.run("""
2273:                 MATCH (u:User {user_id: $uid}), (r:LoanRequest {request_id: $rid})
2274:                 CREATE (u)-[:REQUESTED]->(r)
2275:             """, uid=user_id, rid=new_rid)
2276:
2277:             s.run("""
2278:                 MATCH (r:LoanRequest {request_id: $rid}), (b:Book {book_id: $bid})
2279:                 CREATE (r)-[:FOR_BOOK]->(b)
2280:             """, rid=new_rid, bid=book_id)
2281:
2282:             flash(f"Loan request for '{book_title}' submitted! Awaiting Librarian approval.", "success")
2283:
2284:     except Exception as e:
2285:         flash(f"Failed to submit loan request: {str(e)}", "danger")
2286:
2287:     if request.form.get("from_dashboard"):
2288:         return redirect(url_for("dashboard"))
2289:     cat_id = request.form.get("category_id")
2290:     if cat_id:
2291:         try:
2292:             return redirect(url_for("category_books", category_id=int(cat_id)))
2293:         except (ValueError, TypeError):
2294:             pass
2295:     return redirect(url_for("books"))
2296:
2297:
2298: # ------------------------------------------------------------------------
2299: # /loan_requests (Admin/Librarian approval workflow)
2300: # ------------------------------------------------------------------------
2301: @app.route("/loan_requests", methods=["GET", "POST"])
2302: @login_required
2303: @role_required(["Admin", "Librarian"])
2304: def loan_requests():
2305:     requests_list       = []
2306:     awaiting_pickup_list = []
2307:
2308:     try:
2309:         with get_session() as s:
2310:             sync_fulfilled_loan_requests(s)
2311:             sync_active_overdue_fines(s)
2312:             today    = today_str()
2313:             issuer_id = session.get("user_id")
2314:
```

```
2315:             if request.method == "POST":
2316:                 req_id_raw = request.form.get("request_id")
2317:                 action     = request.form.get("action")
2318:
2319:                 try:
2320:                     req_id = int(req_id_raw)
2321:                 except (ValueError, TypeError):
2322:                     flash("Invalid or missing request ID.", "warning")
2323:                     return redirect(url_for("loan_requests"))
2324:
2325:                 if action == "approve":
2326:                     res = s.run("""
2327:                         MATCH (r:LoanRequest {request_id: $rid})
2328:                         RETURN r.book_id AS book_id, r.borrower_id AS borrower_id, r.status AS status
2329:                     """, rid=req_id)
2330:                     req_data = res.single()
2331:
2332:                     if not req_data or req_data["status"] != "Pending":
2333:                         flash("Invalid or already processed request.", "warning")
2334:                     else:
2335:                         borrower_id = req_data["borrower_id"]
2336:                         b_id        = req_data["book_id"]
2337:
2338:                         res = s.run("MATCH (u:User {user_id: $uid}) RETURN u.username AS username, u.account
      | _status AS account_status", uid=borrower_id)
2339:                         u_info = res.single()
2340:                         b_name = (u_info or {}).get("username", f"User #{borrower_id}")
2341:                         b_status = (u_info or {}).get("account_status", "Active")
2342:
2343:                         # Fines check
2344:                         res = s.run("""
2345:                             MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {payme
      | nt_status:'Unpaid'})
2346:                             RETURN coalesce(sum(f.fine_amount),0) AS unpaid_total
2347:                         """, uid=borrower_id)
2348:                         unpaid_total = res.single()["unpaid_total"]
2349:
2350:                         # Overdue check
2351:                         res = s.run("""
2352:                             MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
2353:                             WHERE l.return_date IS NULL AND l.due_date < $today
2354:                             RETURN count(l) AS overdue_count
2355:                         """, uid=borrower_id, today=today)
2356:                         overdue_count = res.single()["overdue_count"]
2357:
2358:                         # Quota (active + awaiting)
2359:                         res = s.run("""
2360:                             MATCH (u:User {user_id: $uid})
2361:                             OPTIONAL MATCH (u)-[:BORROWED]->(al:Loan) WHERE al.return_date IS NULL
2362:                             OPTIONAL MATCH (u)-[:REQUESTED]->(ap:LoanRequest {status:'Awaiting Pickup'})
2363:                             RETURN count(DISTINCT al) AS active_loans_cnt,
2364:                                    count(DISTINCT ap)  AS awaiting_pickup_cnt
2365:                         """, uid=borrower_id)
2366:                         q = res.single()
2367:                         active_cnt  = q["active_loans_cnt"]  or 0
2368:                         await_cnt   = q["awaiting_pickup_cnt"] or 0
2369:                         committed   = active_cnt + await_cnt
2370:
2371:                         # Same-book check
2372:                         res = s.run("""
2373:                             MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan {return_date:null})-[:FOR_CO
      | PY]->(cp:Copy)<-[:HAS_COPY]-(b:Book {book_id: $bid})
2374:                             RETURN count(l) AS same_book_count
2375:                         """, uid=borrower_id, bid=b_id)
2376:                         same_book_count = res.single()["same_book_count"]
2377:
2378:                         if b_status == "Suspended":
2379:                             flash(f"Approval Prohibited: Patron '{b_name}' account is currently suspended.",
      |  "danger")
2380:                         elif unpaid_total > 0:
2381:                             flash(f"Approval Prohibited: '{b_name}' has unpaid fines of Rs. {unpaid_total:.2
      | f}.", "danger")
2382:                         elif overdue_count > 0:
2383:                             flash(f"Approval Prohibited: '{b_name}' has {overdue_count} overdue loan(s).", "
      | danger")
2384:                         elif same_book_count > 0:
2385:                             # Auto-fulfill lingering request since user already holds active loan for this b
      | ook
2386:                             s.run("""
2387:                                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)
```

```
                                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp)
                                WHERE l.return_date IS NULL
                                MATCH (u)-[:REQUESTED]->(r:LoanRequest {request_id: $rid})-[:FOR_BOOK]->(b)
                                WHERE r.status IN ['Pending', 'Awaiting Pickup']
                                WITH r, cp, r.copy_id AS old_cid
                                OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
                                DELETE rel
                                SET r.status = 'Fulfilled', r.copy_id = cp.copy_id
                                WITH old_cid, cp
                                WHERE old_cid IS NOT NULL AND old_cid <> cp.copy_id
                                MATCH (rcp:Copy {copy_id: old_cid, status: 'Reserved'})
                                SET rcp.status = 'Available'
                            """, uid=borrower_id, bid=b_id, rid=req_id)
                            flash(f"Approval Prohibited: '{b_name}' already has this book on active loan. Re
    | quest #{req_id} marked Fulfilled.", "warning")
                        elif committed >= 3:
                            flash(f"Approval Prohibited: '{b_name}' has reached the 3-book threshold ({commi
    | tted}).", "danger")
                        else:
                            # Find available copy
                            res = s.run("""
                                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status:'Available'})
                                RETURN cp.copy_id AS copy_id LIMIT 1
                            """, bid=b_id)
                            avail = res.single()

                            if not avail:
                                flash("Cannot approve: No physical copies available in inventory (0 stock)."
    | , "danger")
                            else:
                                copy_id = avail["copy_id"]
                                s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Reserved'", cid=copy
    | _id)
                                s.run("""
                                    MATCH (r:LoanRequest {request_id: $rid}), (cp:Copy {copy_id: $cid})
                                    SET r.status = 'Awaiting Pickup', r.copy_id = $cid
                                    MERGE (r)-[:RESERVED_COPY]->(cp)
                                """, rid=req_id, cid=copy_id)

                                flash(f"Request #{req_id} approved! Copy #{copy_id} reserved for '{b_name}'.
    | ", "success")

                elif action == "confirm_pickup":
                    res = s.run("""
                        MATCH (r:LoanRequest {request_id: $rid, status: 'Awaiting Pickup'})-[:FOR_BOOK]->(b:
    | Book)
                        MATCH (u:User {user_id: r.borrower_id})
                        RETURN r.borrower_id AS borrower_id, r.copy_id AS copy_id,
                               r.book_id AS book_id, b.title AS book_title,
                               u.username AS borrower_name, u.account_status AS borrower_status
                    """, rid=req_id)
                    pickup_req = res.single()

                    if not pickup_req:
                        flash("Request not found or not in 'Awaiting Pickup' status.", "warning")
                    else:
                        c_id  = pickup_req["copy_id"]
                        b_id2 = pickup_req["borrower_id"]
                        b_name = pickup_req["borrower_name"]
                        b_title = pickup_req["book_title"]

                        if not c_id:
                            res = s.run("""
                                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status:'Reserved'})
                                WHERE NOT (cp)<-[:RESERVED_COPY]-(:LoanRequest)
                                RETURN min(cp.copy_id) AS copy_id
                            """, bid=pickup_req["book_id"])
                            c_id = (res.single() or {}).get("copy_id")

                        if not c_id:
                            flash(f"No reserved copy found for '{b_title}'.", "danger")
                        else:
                            today_cur = today_str()
                            # Re-verify borrower eligibility before physical handover
                            res_fine = s.run("""
                                MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:HAS_FINE]->(f:Fine {p
    | ayment_status:'Unpaid'})
                                RETURN coalesce(sum(f.fine_amount), 0) AS unpaid_total
                            """, uid=b_id2)
                            unpaid_total = res_fine.single()["unpaid_total"]
```

```
2461:
2462:                             res_ovd = s.run("""
2463:                                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
2464:                                 WHERE l.return_date IS NULL AND l.due_date < $today
2465:                                 RETURN count(l) AS overdue_count
2466:                             """, uid=b_id2, today=today_cur)
2467:                             overdue_count = res_ovd.single()["overdue_count"]
2468:
2469:                             res_act = s.run("""
2470:                                 MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)
2471:                                 WHERE l.return_date IS NULL
2472:                                 RETURN count(l) AS active_loans_count
2473:                             """, uid=b_id2)
2474:                             active_loans_count = res_act.single()["active_loans_count"]
2475:
2476:                             if pickup_req.get("borrower_status") == "Suspended":
2477:                                 flash(f"Handover Prohibited: Patron '{b_name}' account is currently suspende
     | d.", "danger")
2478:                             elif unpaid_total > 0:
2479:                                 flash(f"Handover Prohibited: '{b_name}' has unpaid disciplinary fines of Rs.
     |  {unpaid_total:.2f}.", "danger")
2480:                             elif overdue_count > 0:
2481:                                 flash(f"Handover Prohibited: '{b_name}' has {overdue_count} overdue loan(s).
     | ", "danger")
2482:                             elif active_loans_count >= 3:
2483:                                 flash(f"Handover Prohibited: '{b_name}' has reached the 3 active loans limit
     |  ({active_loans_count} active).", "danger")
2484:                             else:
2485:                                 due_days = 14
2486:                                 issue_dt = datetime.now()
2487:                                 due_dt   = issue_dt + timedelta(days=due_days)
2488:                                 new_lid  = next_id(s, "Loan", "loan_id")
2489:
2490:                                 s.run("""
2491:                                     CREATE (l:Loan {
2492:                                         loan_id: $lid, copy_id: $cid, borrower_id: $bid,
2493:                                         issued_by: $issby,
2494:                                         issue_date: $issue_date, due_date: $due_date,
2495:                                         return_date: null
2496:                                     })
2497:                                 """, lid=new_lid, cid=c_id, bid=b_id2, issby=issuer_id,
2498:                                      issue_date=issue_dt.strftime("%Y-%m-%d"),
2499:                                      due_date=due_dt.strftime("%Y-%m-%d"))
2500:
2501:                                 s.run("""
2502:                                     MATCH (u:User {user_id: $uid}), (l:Loan {loan_id: $lid})
2503:                                     CREATE (u)-[:BORROWED]->(l)
2504:                                 """, uid=b_id2, lid=new_lid)
2505:
2506:                                 s.run("""
2507:                                     MATCH (l:Loan {loan_id: $lid}), (cp:Copy {copy_id: $cid})
2508:                                     CREATE (l)-[:FOR_COPY]->(cp)
2509:                                 """, lid=new_lid, cid=c_id)
2510:
2511:                                 s.run("""
2512:                                     MATCH (lib:User {user_id: $uid}), (l:Loan {loan_id: $lid})
2513:                                     CREATE (lib)-[:ISSUED]->(l)
2514:                                 """, uid=issuer_id, lid=new_lid)
2515:
2516:                                 s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Borrowed'", cid=c_id
     | )
2517:                                 s.run("""
2518:                                     MATCH (r:LoanRequest {request_id: $rid})
2519:                                     OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
2520:                                     DELETE rel
2521:                                     SET r.status = 'Fulfilled', r.copy_id = $cid
2522:                                 """, rid=req_id, cid=c_id)
2523:
2524:                                 # Clean up any other duplicate/lingering requests for the same book and borr
     | ower
2525:                                 s.run("""
2526:                                     MATCH (u:User {user_id: $uid})-[:REQUESTED]->(r:LoanRequest)-[:FOR_BOOK]
     | ->(b:Book {book_id: $bid})
2527:                                     WHERE r.status IN ['Pending', 'Awaiting Pickup']
2528:                                     WITH r, r.copy_id AS old_cid
2529:                                     OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
2530:                                     DELETE rel
2531:                                     SET r.status = 'Fulfilled', r.copy_id = $cid
2532:                                     WITH old_cid
2533:                                     WHERE old_cid IS NOT NULL AND old_cid <> $cid
```

```
                                    MATCH (rcp:Copy {copy_id: old_cid, status: 'Reserved'})
                                    SET rcp.status = 'Available'
                                """, uid=b_id2, bid=pickup_req["book_id"], cid=c_id)

                                flash(f"Collection Confirmed! '{b_title}' issued to {b_name}.", "success")

                elif action == "cancel_pickup":
                    res = s.run("""
                        MATCH (r:LoanRequest {request_id: $rid, status: 'Awaiting Pickup'})-[:FOR_BOOK]->(b:
    | Book)
                        MATCH (u:User {user_id: r.borrower_id})
                        RETURN r.copy_id AS copy_id, r.book_id AS book_id,
                               u.username AS borrower_name
                    """, rid=req_id)
                    c_req = res.single()
                    if c_req:
                        s.run("""
                            MATCH (r:LoanRequest {request_id: $rid})
                            OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
                            DELETE rel
                            SET r.status = 'Rejected'
                        """, rid=req_id)
                        if c_req["copy_id"]:
                            s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Available'", cid=c_req["
    | copy_id"])
                        else:
                            res2 = s.run("""
                                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status:'Reserved'})
                                WHERE NOT (cp)<-[:RESERVED_COPY]-(:LoanRequest)
                                RETURN min(cp.copy_id) AS copy_id
                            """, bid=c_req["book_id"])
                            reserved_cid = (res2.single() or {}).get("copy_id")
                            if reserved_cid:
                                s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Available'", cid=res
    | erved_cid)

                        flash(f"Pickup #{req_id} for '{c_req['borrower_name']}' cancelled.", "info")

                elif action == "reject":
                    res = s.run("""
                        MATCH (r:LoanRequest {request_id: $rid})
                        RETURN r.book_id AS book_id
                    """, rid=req_id)
                    rej_data = res.single()
                    s.run("""
                        MATCH (r:LoanRequest {request_id: $rid})
                        OPTIONAL MATCH (r)-[rel:RESERVED_COPY]->()
                        DELETE rel
                        SET r.status = 'Rejected'
                    """, rid=req_id)

                    if rej_data:
                        r_bid = rej_data["book_id"]
                        # Release one reserved copy if more reserved than awaiting
                        res = s.run("""
                            MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status:'Reserved'})
                            RETURN count(cp) AS res_cnt
                        """, bid=r_bid)
                        res_cnt = res.single()["res_cnt"]

                        res = s.run("""
                            MATCH (r2:LoanRequest {status:'Awaiting Pickup'})-[:FOR_BOOK]->(b:Book {book_id:
    |  $bid})
                            RETURN count(r2) AS await_cnt
                        """, bid=r_bid)
                        await_cnt = res.single()["await_cnt"]

                        if res_cnt > await_cnt:
                            res3 = s.run("""
                                MATCH (b:Book {book_id: $bid})-[:HAS_COPY]->(cp:Copy {status:'Reserved'})
                                WHERE NOT (cp)<-[:RESERVED_COPY]-(:LoanRequest)
                                RETURN cp.copy_id AS copy_id LIMIT 1
                            """, bid=r_bid)
                            rec3 = res3.single()
                            if rec3:
                                s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Available'", cid=rec
    | 3["copy_id"])

                    flash(f"Loan Request #{req_id} rejected.", "info")

```

```
            # Fetch pending requests
            res = s.run("""
                MATCH (r:LoanRequest {status:'Pending'})-[:FOR_BOOK]->(b:Book)
                MATCH (u:User {user_id: r.borrower_id})
                RETURN r.request_id AS request_id, b.book_id AS book_id,
                       b.title AS book_title, b.isbn AS isbn,
                       r.borrower_id AS borrower_id,
                       u.username AS borrower_name, u.email AS borrower_email,
                       u.role AS borrower_role, r.request_date AS request_date,
                       size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status = 'Available' | cp]) AS available_co
    | pies_count,
                       size([(u)-[:BORROWED]->(al:Loan) WHERE al.return_date IS NULL | al]) AS borrower_acti
    | ve_loans,
                       size([(u)-[:REQUESTED]->(ap:LoanRequest {status:'Awaiting Pickup'}) | ap]) AS borrowe
    | r_awaiting_pickup,
                       size([(u)-[:BORROWED]->(al2:Loan) WHERE al2.return_date IS NULL | al2])
                       + size([(u)-[:REQUESTED]->(ap2:LoanRequest {status:'Awaiting Pickup'}) | ap2]) AS bor
    | rower_committed_total,
                       size([(u)-[:BORROWED]->(ol:Loan) WHERE ol.return_date IS NULL AND ol.due_date < $toda
    | y | ol]) AS borrower_overdue_loans,
                       coalesce(reduce(total=0.0, f IN [(u)-[:BORROWED]->(ol2:Loan)-[:HAS_FINE]->(fin:Fine {
    | payment_status:'Unpaid'}) | fin.fine_amount] | total + f), 0.0) AS borrower_unpaid_fines
                ORDER BY r.request_date ASC
            """, today=today)
            requests_list = rows(res)

            # Fetch awaiting pickup
            res = s.run("""
                MATCH (r:LoanRequest {status:'Awaiting Pickup'})-[:FOR_BOOK]->(b:Book)-[:BELONGS_TO]->(c:Cat
    | egory)
                MATCH (u:User {user_id: r.borrower_id})
                OPTIONAL MATCH (r)-[:RESERVED_COPY]->(rcp:Copy)
                OPTIONAL MATCH (cp_fallback:Copy {copy_id: r.copy_id})
                WITH r, b, c, u, coalesce(rcp, cp_fallback) AS cp
                RETURN r.request_id AS request_id, b.book_id AS book_id,
                       coalesce(cp.copy_id, r.copy_id) AS copy_id, b.title AS book_title,
                       b.isbn AS isbn, c.name AS category_name,
                       cp.condition AS copy_condition,
                       r.borrower_id AS borrower_id,
                       u.username AS borrower_name, u.email AS borrower_email,
                       u.role AS borrower_role, r.request_date AS request_date
                ORDER BY r.request_date ASC
            """)
            awaiting_pickup_list = rows(res)

    except Exception as e:
        flash(f"Error handling loan requests: {str(e)}", "danger")
        requests_list        = []
        awaiting_pickup_list = []

    return render_template("loan_requests.html", requests=requests_list,
                           awaiting_pickup=awaiting_pickup_list)


# ---------------------------------------------------------------------------
# /return_book/<int:loan_id>
# ---------------------------------------------------------------------------
@app.route("/return_book/<int:loan_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def return_book(loan_id):
    try:
        with get_session() as s:
            res = s.run("""
                MATCH (l:Loan {loan_id: $lid})-[:FOR_COPY]->(cp:Copy)
                RETURN l.copy_id AS copy_id, l.due_date AS due_date,
                       l.return_date AS return_date
            """, lid=loan_id)
            rec = res.single()

            if not rec:
                flash(f"Loan #{loan_id} not found.", "warning")
                return redirect(url_for("dashboard"))

            if rec["return_date"] is not None:
                flash(f"Loan #{loan_id} is already returned.", "info")
                return redirect(url_for("dashboard"))

            return_dt = datetime.now()
            return_str = return_dt.strftime("%Y-%m-%d")
```

```
            due_str    = rec["due_date"]

            s.run("""
                MATCH (l:Loan {loan_id: $lid}) SET l.return_date = $rdt
            """, lid=loan_id, rdt=return_str)

            s.run("""
                MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Available'
            """, cid=rec["copy_id"])

            # Calculate overdue fine (Rs. 10/day)
            try:
                due_d    = datetime.strptime(due_str, "%Y-%m-%d")
                ret_d    = return_dt
                days_late = max(0, (ret_d.date() - due_d.date()).days)
                if days_late > 0:
                    fine_amount = days_late * 10.0
                    res_fine = s.run("""
                        MATCH (l:Loan {loan_id: $lid})
                        OPTIONAL MATCH (l)-[:HAS_FINE]->(existing_f:Fine)
                        RETURN existing_f.fine_id AS fine_id, existing_f.payment_status AS payment_status
                    """, lid=loan_id).single()

                    if res_fine and res_fine["fine_id"] is not None:
                        if res_fine["payment_status"] == "Unpaid":
                            s.run("""
                                MATCH (f:Fine {fine_id: $fid})
                                SET f.fine_amount = $amt
                            """, fid=res_fine["fine_id"], amt=fine_amount)
                            flash(f"Loan #{loan_id} returned. Overdue fine updated to Rs. {fine_amount:.2f}
    | ({days_late} day(s) late).", "warning")
                        else:
                            flash(f"Loan #{loan_id} returned. Pre-existing fine is already {res_fine['paymen
    | t_status']}.", "info")
                    else:
                        new_fid = next_id(s, "Fine", "fine_id")
                        s.run("""
                            MATCH (l:Loan {loan_id: $lid})
                            CREATE (f:Fine {
                                fine_id: $fid, loan_id: $lid,
                                fine_amount: $amt, payment_status: 'Unpaid',
                                payment_date: null, collected_by: null
                            })
                            CREATE (l)-[:HAS_FINE]->(f)
                        """, fid=new_fid, lid=loan_id, amt=fine_amount)
                        flash(f"Loan #{loan_id} returned. Overdue fine of Rs. {fine_amount:.2f} assessed ({d
    | ays_late} day(s) late).", "warning")
                else:
                    flash(f"Loan #{loan_id} returned successfully. No overdue fine.", "success")
            except Exception:
                flash(f"Loan #{loan_id} returned. Could not calculate fine.", "info")

    except Exception as e:
        flash(f"Error processing book return: {str(e)}", "danger")

    return redirect(url_for("dashboard"))


# ---------------------------------------------------------------------------
# /pay_fine/<int:fine_id>
# ---------------------------------------------------------------------------
@app.route("/pay_fine/<int:fine_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def pay_fine(fine_id):
    payment_action = request.form.get("payment_action", "Paid").strip()
    if payment_action not in ["Paid", "Waived"]:
        payment_action = "Paid"

    try:
        with get_session() as s:
            res = s.run("""
                MATCH (l:Loan)-[:HAS_FINE]->(f:Fine {fine_id: $fid})
                OPTIONAL MATCH (u:User)-[:BORROWED]->(l)
                RETURN f.fine_id AS fine_id, f.fine_amount AS fine_amount,
                       f.payment_status AS payment_status, coalesce(u.username, 'Deleted Patron') AS usernam
    | e
            """, fid=fine_id)
            fine = res.single()

```

```
2758:             if not fine:
2759:                 flash(f"Fine #{fine_id} not found.", "warning")
2760:                 return redirect(url_for("dashboard"))
2761:
2762:             if fine["payment_status"] in ["Paid", "Waived"]:
2763:                 flash(f"Fine #{fine_id} was already {fine['payment_status']}.", "info")
2764:                 return redirect(url_for("dashboard"))
2765:
2766:             pay_date = datetime.now().strftime("%Y-%m-%d")
2767:             s.run("""
2768:                 MATCH (f:Fine {fine_id: $fid})
2769:                 SET f.payment_status = $action, f.payment_date = $pdate,
2770:                     f.collected_by = $admin_id
2771:             """, fid=fine_id, action=payment_action, pdate=pay_date,
2772:                  admin_id=session.get("user_id"))
2773:
2774:             action_desc = "waived" if payment_action == "Waived" else "marked as Paid"
2775:             flash(f"Fine #{fine_id} (Rs. {fine['fine_amount']:.2f}) for '{fine['username']}' {action_desc}!"
      | , "success")
2776:
2777:     except Exception as e:
2778:         flash(f"Error updating fine: {str(e)}", "danger")
2779:
2780:     return redirect(url_for("dashboard"))
2781:
2782:
2783: # ------------------------------------------------------------------------
2784: # /delete_loan/<int:loan_id>
2785: # ------------------------------------------------------------------------
2786: @app.route("/delete_loan/<int:loan_id>", methods=["POST"])
2787: @login_required
2788: @role_required(["Admin", "Librarian"])
2789: def delete_loan(loan_id):
2790:     try:
2791:         with get_session() as s:
2792:             res = s.run("""
2793:                 MATCH (l:Loan {loan_id: $lid})-[:FOR_COPY]->(cp:Copy)
2794:                 OPTIONAL MATCH (u:User)-[:BORROWED]->(l)
2795:                 OPTIONAL MATCH (cp)<-[:HAS_COPY]-(b:Book)
2796:                 RETURN l.return_date AS return_date, cp.copy_id AS copy_id,
2797:                        coalesce(u.username, 'Deleted Patron') AS borrower_name, u.user_id AS borrower_id,
2798:                        coalesce(b.title, 'Unknown') AS book_title
2799:             """, lid=loan_id)
2800:             loan = res.single()
2801:
2802:             if not loan:
2803:                 flash(f"Loan #{loan_id} not found.", "warning")
2804:                 return redirect(url_for("dashboard"))
2805:
2806:             was_active    = loan["return_date"] is None
2807:             copy_id       = loan["copy_id"]
2808:             borrower_name = loan["borrower_name"]
2809:             book_title    = loan["book_title"]
2810:
2811:             # Delete fines then loan
2812:             s.run("""
2813:                 MATCH (l:Loan {loan_id: $lid})
2814:                 OPTIONAL MATCH (l)-[:HAS_FINE]->(f1:Fine)
2815:                 OPTIONAL MATCH (f2:Fine {loan_id: $lid})
2816:                 DETACH DELETE f1, f2
2817:             """, lid=loan_id)
2818:
2819:             if was_active:
2820:                 s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Available'", cid=copy_id)
2821:
2822:             s.run("MATCH (l:Loan {loan_id: $lid}) DETACH DELETE l", lid=loan_id)
2823:
2824:             if was_active:
2825:                 flash(f"Loan #{loan_id} for '{borrower_name}' ('{book_title}') deleted. Copy #{copy_id} rest
      | ored.", "success")
2826:             else:
2827:                 flash(f"Historical loan #{loan_id} for '{borrower_name}' ('{book_title}') deleted.", "succes
      | s")
2828:
2829:     except Exception as e:
2830:         flash(f"Error deleting loan: {str(e)}", "danger")
2831:
2832:     return redirect(url_for("dashboard"))
2833:
2834:
```

```
# ------------------------------------------------------------------------
# /report_lost/<int:loan_id>
# ------------------------------------------------------------------------
@app.route("/report_lost/<int:loan_id>", methods=["POST"])
@login_required
@role_required(["Admin", "Librarian"])
def report_lost(loan_id):
    try:
        with get_session() as s:
            res = s.run("""
                MATCH (l:Loan {loan_id: $lid})-[:FOR_COPY]->(cp:Copy)
                OPTIONAL MATCH (u:User)-[:BORROWED]->(l)
                OPTIONAL MATCH (cp)<-[:HAS_COPY]-(b:Book)
                RETURN l.return_date AS return_date, cp.copy_id AS copy_id,
                       coalesce(u.username, 'Deleted Patron') AS borrower_name,
                       coalesce(b.title, 'Unknown') AS book_title
            """, lid=loan_id)
            loan = res.single()

            if not loan:
                flash(f"Loan #{loan_id} not found.", "warning")
                return redirect(url_for("dashboard"))

            if loan["return_date"] is not None:
                flash(f"Loan #{loan_id} is already returned/closed. Cannot report lost.", "info")
                return redirect(url_for("dashboard"))

            copy_id       = loan["copy_id"]
            borrower_name = loan["borrower_name"]
            book_title    = loan["book_title"]

            s.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Lost'", cid=copy_id)
            s.run("""
                MATCH (l:Loan {loan_id: $lid})
                SET l.return_date = $rdt
            """, lid=loan_id, rdt=datetime.now().strftime("%Y-%m-%d"))

            # Check if loan has pre-existing Unpaid overdue fine node
            res_fine = s.run("""
                MATCH (l:Loan {loan_id: $lid})
                OPTIONAL MATCH (l)-[:HAS_FINE]->(existing_f:Fine)
                RETURN existing_f.fine_id AS fine_id, existing_f.payment_status AS payment_status
            """, lid=loan_id).single()

            if res_fine and res_fine["fine_id"] is not None and res_fine["payment_status"] == "Unpaid":
                s.run("""
                    MATCH (f:Fine {fine_id: $fid})
                    SET f.fine_amount = 1500.0
                """, fid=res_fine["fine_id"])
            else:
                new_fid = next_id(s, "Fine", "fine_id")
                s.run("""
                    MATCH (l:Loan {loan_id: $lid})
                    CREATE (f:Fine {
                        fine_id: $fid, loan_id: $lid,
                        fine_amount: 1500.0, payment_status: 'Unpaid',
                        payment_date: null, collected_by: null
                    })
                    CREATE (l)-[:HAS_FINE]->(f)
                """, fid=new_fid, lid=loan_id)

            flash(f"'{book_title}' (Copy #{copy_id}) marked Lost for '{borrower_name}'. Replacement fine of
    | Rs. 1500.00 assessed.", "warning")

    except Exception as e:
        flash(f"Error recording lost book: {str(e)}", "danger")

    return redirect(url_for("dashboard"))


# ========================================================================
# Task 4 — Graph Collaborative Filtering: Book Recommendations
# ========================================================================
@app.route("/recommendations")
@login_required
def recommendations():
    """
    Graph-based book recommendation using multi-hop Cypher traversal.
    For Student/Staff: Personal collaborative filtering based on personal borrowing history.
    For Admin/Librarian: Global Patron Collaborative Filtering Network showcasing peer borrowing clusters.
```

```
    """
    role = session.get("role")
    user_id = session.get("user_id")
    is_admin_view = role in ["Admin", "Librarian"]
    recs = []

    try:
        with get_session() as s:
            if is_admin_view:
                # Global Peer Borrowing Clusters across all students and patrons
                res = s.run("""
                    MATCH (u1:User)-[:BORROWED]->(l1:Loan)-[:FOR_COPY]->(cp1:Copy)<-[:HAS_COPY]-(b1:Book)-[:
| BELONGS_TO]->(cat:Category)
                    MATCH (u2:User)-[:BORROWED]->(l2:Loan)-[:FOR_COPY]->(cp2:Copy)<-[:HAS_COPY]-(b2:Book)-[:
| BELONGS_TO]->(cat)
                    WHERE u1.user_id <> u2.user_id AND b1.book_id <> b2.book_id AND u1.role IN ['Student', '
| Staff'] AND u2.role IN ['Student', 'Staff']
                    WITH b2, count(DISTINCT u2) AS mutual_borrowers, collect(DISTINCT cat.name)[0] AS catego
| ry_name
                    OPTIONAL MATCH (a:Author)-[:WROTE]->(b2)
                    WITH b2, mutual_borrowers, category_name, collect(DISTINCT a.first_name + ' ' + a.last_n
| ame) AS author_list
                    RETURN b2.book_id AS book_id, b2.title AS title,
                           b2.isbn AS isbn, category_name,
                           reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS autho
| rs,
                           mutual_borrowers,
                           size([(b2)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies
                    ORDER BY mutual_borrowers DESC, b2.title ASC
                    LIMIT 6
                """)
                recs = [dict(r) for r in res]

                # Fallback: top circulated books
                if not recs:
                    res = s.run("""
                        MATCH (u:User)-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book)
                        WITH b, count(DISTINCT u) AS borrow_count
                        OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
                        OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
                        WITH b, borrow_count, cat,
                             collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
                        RETURN b.book_id AS book_id, b.title AS title,
                               b.isbn AS isbn, cat.name AS category_name,
                               reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS a
| uthors,
                               borrow_count AS mutual_borrowers,
                               size([(b)-[:HAS_COPY]->(cp2:Copy {status:'Available'}) | cp2]) AS available_c
| opies
                        ORDER BY borrow_count DESC, b.title ASC
                        LIMIT 6
                    """)
                    recs = [dict(r) for r in res]
            else:
                # Personal collaborative filtering for Student/Staff (strictly matches users within the same
|  role)
                res = s.run("""
                    MATCH (me:User {user_id: $uid})-[:BORROWED]->(l1:Loan)-[:FOR_COPY]->(cp1:Copy)<-[:HAS_CO
| PY]-(b1:Book)-[:BELONGS_TO]->(cat:Category)
                    MATCH (other:User)-[:BORROWED]->(l2:Loan)-[:FOR_COPY]->(cp2:Copy)<-[:HAS_COPY]-(b2:Book)
| -[:BELONGS_TO]->(cat)
                    WHERE other.user_id <> $uid AND other.role = me.role
                      AND NOT (me)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b2)
                      AND NOT (me)-[:REQUESTED]->(:LoanRequest)-[:FOR_BOOK]->(b2)
                    WITH b2, count(DISTINCT other) AS mutual_borrowers,
                         collect(DISTINCT cat.name)[0] AS category_name
                    OPTIONAL MATCH (a:Author)-[:WROTE]->(b2)
                    WITH b2, mutual_borrowers, category_name,
                         collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
                    RETURN b2.book_id AS book_id, b2.title AS title,
                           b2.isbn AS isbn, category_name,
                           reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS autho
| rs,
                           mutual_borrowers,
                           size([(b2)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies
                    ORDER BY mutual_borrowers DESC, b2.title ASC
                    LIMIT 6
                """, uid=user_id)
                recs = [dict(r) for r in res]

```

```
2982:                 # Graph-content traversal fallback for Staff when peer count is 0:
2983:                 # Traverse shared Categories, Authors, and Prerequisite chains without mixing in student pee
      | rs
2984:                 if not recs and role == "Staff":
2985:                     res = s.run("""
2986:                         MATCH (u:User {user_id: $uid})-[:BORROWED|REQUESTED]->()-[:FOR_COPY|FOR_BOOK]->()-[:
      | HAS_COPY*0..1]-(b:Book)
2987:                         MATCH (b)-[:BELONGS_TO|WROTE|PREREQUISITE_FOR*1..2]-(rec:Book)
2988:                         WHERE rec.book_id <> b.book_id
2989:                           AND NOT (u)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(rec)
2990:                           AND NOT (u)-[:REQUESTED]->(:LoanRequest)-[:FOR_BOOK]->(rec)
2991:                         WITH rec, count(DISTINCT b) AS shared_connections
2992:                         OPTIONAL MATCH (cat:Category)<-[:BELONGS_TO]-(rec)
2993:                         OPTIONAL MATCH (a:Author)-[:WROTE]->(rec)
2994:                         WITH rec, shared_connections, collect(DISTINCT cat.name)[0] AS category_name,
2995:                              collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list,
2996:                              size([(rec)<-[:HAS_COPY]-(:Copy)<-[:FOR_COPY]-(:Loan)<-[:BORROWED]-(st:User {ro
      | le: 'Staff'}) WHERE st.user_id <> $uid | st]) AS staff_readers
2997:                         RETURN rec.book_id AS book_id, rec.title AS title,
2998:                                rec.isbn AS isbn, category_name,
2999:                                reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS a
      | uthors,
3000:                                staff_readers AS mutual_borrowers,
3001:                                CASE WHEN staff_readers > 0 THEN NULL ELSE 'Curriculum Match' END AS badge_la
      | bel,
3002:                                size([(rec)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_c
      | opies
3003:                         ORDER BY shared_connections DESC, rec.title ASC
3004:                         LIMIT 6
3005:                     """, uid=user_id)
3006:                     recs = [dict(r) for r in res]
3007:
3008:                 # Fallback: most circulated books the user hasn't read (isolated for Staff)
3009:                 if not recs:
3010:                     if role == "Staff":
3011:                         res = s.run("""
3012:                             MATCH (b:Book)<-[:HAS_COPY]-(cp:Copy)<-[:FOR_COPY]-(l:Loan)
3013:                             WHERE NOT (b)<-[:FOR_BOOK]-(:LoanRequest)<-[:REQUESTED]-(:User {user_id: $uid})
3014:                               AND NOT (b)<-[:HAS_COPY]-(:Copy)<-[:FOR_COPY]-(:Loan)<-[:BORROWED]-(:User {use
      | r_id: $uid})
3015:                             WITH b, size([(b)<-[:HAS_COPY]-(:Copy)<-[:FOR_COPY]-(:Loan)<-[:BORROWED]-(st:Use
      | r {role: 'Staff'}) WHERE st.user_id <> $uid | st]) AS staff_readers
3016:                             OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
3017:                             OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
3018:                             WITH b, staff_readers, cat,
3019:                                  collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
3020:                             RETURN b.book_id AS book_id, b.title AS title,
3021:                                    b.isbn AS isbn, cat.name AS category_name,
3022:                                    reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END)
      | AS authors,
3023:                                    staff_readers AS mutual_borrowers,
3024:                                    CASE WHEN staff_readers > 0 THEN NULL ELSE 'Curriculum Match' END AS badg
      | e_label,
3025:                                    size([(b)-[:HAS_COPY]->(cp2:Copy {status:'Available'}) | cp2]) AS availab
      | le_copies
3026:                             ORDER BY staff_readers DESC, b.title ASC
3027:                             LIMIT 6
3028:                         """, uid=user_id)
3029:                     else:
3030:                         res = s.run("""
3031:                             MATCH (u:User)-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(cp:Copy)<-[:HAS_COPY]-(b:Book
      | )
3032:                             WHERE NOT (b)<-[:FOR_BOOK]-(:LoanRequest)<-[:REQUESTED]-(:User {user_id: $uid})
3033:                               AND NOT (b)<-[:HAS_COPY]-(:Copy)<-[:FOR_COPY]-(:Loan)<-[:BORROWED]-(:User {use
      | r_id: $uid})
3034:                             WITH b, count(DISTINCT u) AS borrow_count
3035:                             OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
3036:                             OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
3037:                             WITH b, borrow_count, cat,
3038:                                  collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
3039:                             RETURN b.book_id AS book_id, b.title AS title,
3040:                                    b.isbn AS isbn, cat.name AS category_name,
3041:                                    reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END)
      | AS authors,
3042:                                    borrow_count AS mutual_borrowers,
3043:                                    size([(b)-[:HAS_COPY]->(cp2:Copy {status:'Available'}) | cp2]) AS availab
      | le_copies
3044:                             ORDER BY borrow_count DESC, b.title ASC
3045:                             LIMIT 6
3046:                         """, uid=user_id)
```

```
                    recs = [dict(r) for r in res]

            # ---------------------------------------------------------------
            # Role-Scoped Jaccard Similarity "Study Partner" Matchmaker
            # ---------------------------------------------------------------
            study_partners = []
            if role == "Student" and user_id:
                # Student view: Personalized peer study partners based STRICTLY on actual borrowed books
                j_res = s.run("""
                    MATCH (u1:User {user_id: $uid})-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b:Book)
                    WITH u1, collect(DISTINCT b) AS u1_books
                    MATCH (u2:User)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b2:Book)
                    WHERE u2.user_id <> u1.user_id AND u2.role = 'Student'
                    WITH u1_books, u2, collect(DISTINCT b2) AS u2_books
                    WITH u2, [x IN u1_books WHERE x IN u2_books] AS shared, u1_books + [x IN u2_books WHERE NOT x IN u1_books] AS union_books
                    WHERE size(shared) > 0
                    RETURN u2.username AS peer_name, size(shared) AS shared_count, round(toFloat(size(shared)) / size(union_books) * 100, 1) AS jaccard_pct, [b IN shared | b.title] AS shared_titles
                    ORDER BY jaccard_pct DESC LIMIT 5
                """, uid=user_id)
                study_partners = rows(j_res)
            elif role in ["Admin", "Librarian"]:
                # Admin / Librarian view: Global Student Cohort Study Pairs based STRICTLY on actual borrowed books
                gj_res = s.run("""
                    MATCH (u1:User)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b:Book)
                    WHERE u1.role = 'Student'
                    WITH u1, collect(DISTINCT b) AS u1_books
                    MATCH (u2:User)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b2:Book)
                    WHERE u1.user_id < u2.user_id AND u2.role = 'Student'
                    WITH u1, u1_books, u2, collect(DISTINCT b2) AS u2_books
                    WITH u1, u2, [x IN u1_books WHERE x IN u2_books] AS shared, u1_books + [x IN u2_books WHERE NOT x IN u1_books] AS union_books
                    WHERE size(shared) > 0
                    RETURN u1.username + ' & ' + u2.username AS peer_name, size(shared) AS shared_count, round(toFloat(size(shared)) / size(union_books) * 100, 1) AS jaccard_pct, [b IN shared | b.title] AS shared_titles
                    ORDER BY jaccard_pct DESC, shared_count DESC LIMIT 5
                """)
                study_partners = rows(gj_res)
            # For Staff (Faculty): study_partners remains empty [] as faculty members only need book recommendations

    except Exception as e:
        flash(f"Error loading recommendations: {str(e)}", "danger")

    return render_template(
        "recommendations.html",
        recommendations=recs,
        study_partners=study_partners,
        is_admin_view=is_admin_view,
        admin_banner=f"{role} View: Global Patron Collaborative Filtering Network" if is_admin_view else None
    )


# ============================================================================
# Task 5 — Interactive Graph Visualizer & Graph API (Vis.js Network)
# ============================================================================
def serialize_graph_val(v):
    """Safely converts Neo4j data types into JSON-serializable primitives."""
    if v is None:
        return None
    if isinstance(v, (datetime, date)):
        return v.isoformat()
    if hasattr(v, "iso_format"):
        return v.iso_format()
    if hasattr(v, "to_native"):
        native = v.to_native()
        if hasattr(native, "isoformat"):
            return native.isoformat()
        return str(native)
    if isinstance(v, (int, float, bool, str)):
        return v
    return str(v)


def format_graph_node(n, custom_group=None, custom_label=None):
```

```
3118:     """Formats a Neo4j node into a Vis.js Network node structure."""
3119:     labels = list(n.labels) if hasattr(n, "labels") else []
3120:     primary_label = custom_group or (labels[0] if labels else "Node")
3121:     raw_props = dict(n) if hasattr(n, "items") else {}
3122:     props = {k: serialize_graph_val(v) for k, v in raw_props.items() if k != "password_hash"}
3123: 
3124:     if custom_label:
3125:         display_label = custom_label
3126:     elif primary_label == "Book":
3127:         title = props.get("title", f"Book #{props.get('book_id')}")
3128:         display_label = title if len(title) <= 24 else title[:22] + "..."
3129:     elif primary_label == "User":
3130:         display_label = f"{props.get('username')} ({props.get('role', 'User')})"
3131:     elif primary_label == "Author":
3132:         display_label = f"{props.get('first_name', '')} {props.get('last_name', '')}".strip()
3133:     elif primary_label == "Category":
3134:         display_label = f"{props.get('name', 'Category')}"
3135:     elif primary_label == "Publisher":
3136:         display_label = f"{props.get('name', 'Publisher')}"
3137:     elif primary_label == "Copy":
3138:         display_label = f"Copy #{props.get('copy_id')} [{props.get('status', 'Available')}]"
3139:     elif primary_label == "Loan":
3140:         ret_date = props.get("return_date")
3141:         loan_status = "Returned" if (ret_date is not None and str(ret_date).strip() not in ["", "None", "nul
      | l"]) else props.get("status", "Active")
3142:         display_label = f"Loan #{props.get('loan_id')} [{loan_status}]"
3143:     elif primary_label == "Fine":
3144:         display_label = f"Fine #{props.get('fine_id')}: Rs.{props.get('fine_amount', 0)}"
3145:     elif primary_label == "LoanRequest":
3146:         display_label = f"Req #{props.get('request_id')} [{props.get('status', 'Pending')}]"
3147:     else:
3148:         display_label = f"{primary_label}"
3149: 
3150:     tooltip_rows = [f"<b>{html.escape(str(k))}:</b> {html.escape(str(v))}" for k, v in list(props.items())[:
      | 6]]
3151:     tooltip = f"<div style='font-family:sans-serif;'><b>{html.escape(str(primary_label))}</b><br/>" + "<br/>
      | ".join(tooltip_rows) + "</div>"
3152: 
3153:     return {
3154:         "id": n.element_id,
3155:         "label": display_label,
3156:         "group": primary_label,
3157:         "title": tooltip,
3158:         "properties": props
3159:     }
3160: 
3161: 
3162: def format_graph_edge(r, from_id, to_id, custom_label=None, is_dashed=False):
3163:     """Formats a Neo4j relationship into a Vis.js Network edge structure."""
3164:     edge_id = r.element_id if hasattr(r, "element_id") else f"e_{from_id}_{to_id}_{custom_label or 'REL'}"
3165:     rel_type = custom_label or (r.type if hasattr(r, "type") else "RELATED_TO")
3166:     raw_props = dict(r) if hasattr(r, "items") else {}
3167:     props = {k: serialize_graph_val(v) for k, v in raw_props.items()}
3168: 
3169:     edge_dict = {
3170:         "id": edge_id,
3171:         "from": from_id,
3172:         "to": to_id,
3173:         "label": rel_type,
3174:         "title": f":{rel_type}" + (f" ({props})" if props else ""),
3175:         "arrows": "to"
3176:     }
3177:     if is_dashed:
3178:         edge_dict["dashes"] = True
3179:         edge_dict["width"] = 2
3180:     return edge_dict
3181: 
3182: 
3183: @app.route("/graph_visualizer")
3184: @login_required
3185: def graph_visualizer():
3186:     """
3187:     Renders the interactive Graph Visualizer canvas powered by Vis-Network.
3188:     Allows dynamic exploration of knowledge graph entities and borrowing subgraphs.
3189:     """
3190:     return render_template(
3191:         "graph_visualizer.html",
3192:         user_role=session.get("role", "Student"),
3193:         username=session.get("username", "User"),
3194:         user_id=session.get("user_id")
```

```
    )


@app.route("/api/graph_data")
@login_required
def api_graph_data():
    """
    REST API returning graph nodes and edges for Vis-Network visualization.
    Supports filtering by view ('personal', 'catalog', 'full', 'recommendations').
    """
    user_id = session.get("user_id")
    user_role = session.get("role", "Student")

    default_view = "full" if user_role in ["Admin", "Librarian"] else "personal"
    view = request.args.get("view", default_view).strip().lower()

    # Enforce role-based view boundary: regular students/staff can view personal or catalog graphs
    if user_role in ["Student", "Staff"] and view == "full":
        view = "catalog"

    nodes = {}
    edges = []

    try:
        with get_session() as s:
            sync_fulfilled_loan_requests(s)

            if view in ["personal", "circulation", "borrowing"]:
                if user_role in ["Admin", "Librarian"] or view == "circulation":
                    # Global Circulation & Fines Subgraph across the entire library
                    # Strictly return only circulation and financial nodes:
                    # User (only users who have loans/requests/fines), Loan, Fine, LoanRequest (status IN ['Pending', 'Awaiting Pickup']),
                    # Copy (only copies linked to loans), and Book (only books linked to those copies/requests).
                    # Completely exclude Author, Category, and Publisher nodes.
                    circ_res = s.run("""
                        MATCH (u:User)-[r1:BORROWED]->(l:Loan)-[r2:FOR_COPY]->(cp:Copy)<-[r3:HAS_COPY]-(b:Book)
                        OPTIONAL MATCH (l)-[r4:HAS_FINE]->(f:Fine)
                        RETURN u, l, cp, b, f, r1, r2, r3, r4
                        LIMIT 300
                    """)
                    for rec in circ_res:
                        for node_key in ["u", "l", "cp", "b", "f"]:
                            item = rec[node_key]
                            if item is not None and item.element_id not in nodes:
                                nodes[item.element_id] = format_graph_node(item)
                        for r_key in ["r1", "r2", "r3", "r4"]:
                            r = rec[r_key]
                            if r is not None and r.element_id not in [e["id"] for e in edges]:
                                edges.append(format_graph_edge(r, r.start_node.element_id, r.end_node.element_id))

                    req_res = s.run("""
                        MATCH (u:User)-[r1:REQUESTED]->(lr:LoanRequest)-[r2:FOR_BOOK]->(b:Book)
                        WHERE lr.status IN ['Pending', 'Awaiting Pickup']
                        RETURN u, lr, b, r1, r2
                        LIMIT 150
                    """)
                    for rec in req_res:
                        for node_key in ["u", "lr", "b"]:
                            item = rec[node_key]
                            if item is not None and item.element_id not in nodes:
                                nodes[item.element_id] = format_graph_node(item)
                        for r_key in ["r1", "r2"]:
                            r = rec[r_key]
                            if r is not None and r.element_id not in [e["id"] for e in edges]:
                                edges.append(format_graph_edge(r, r.start_node.element_id, r.end_node.element_id))
                else:
                    # Student & Staff personal borrowing graph: You, Active Loans, Fines, Active Requests, Copies, and Books.
                    # Filter out historical returned loans (WHERE l.return_date IS NULL) so personal view reflects active loans without conflict.
                    res = s.run("""
                        MATCH (u:User {user_id: $uid})
                        OPTIONAL MATCH (u)-[r1:BORROWED]->(l:Loan)
                        WHERE l.return_date IS NULL
                        OPTIONAL MATCH (l)-[r2:FOR_COPY]->(cp:Copy)
```

```
                        OPTIONAL MATCH (b1:Book)-[r3:HAS_COPY]->(cp)
                        OPTIONAL MATCH (u)-[r_all:BORROWED]->(all_l:Loan)-[r4:HAS_FINE]->(f:Fine)
                        WHERE f.payment_status = 'Unpaid' OR l.loan_id = all_l.loan_id
                        OPTIONAL MATCH (all_l)-[r_cp:FOR_COPY]->(f_cp:Copy)<-[r_bk:HAS_COPY]-(f_b:Book)
                        OPTIONAL MATCH (u)-[r5:REQUESTED]->(lr:LoanRequest)
                        WHERE lr.status IN ['Pending', 'Awaiting Pickup']
                        OPTIONAL MATCH (lr)-[r6:FOR_BOOK]->(b2:Book)
                        RETURN u, l, cp, b1, all_l, f_cp, f_b, f, lr, b2,
                               r1, r2, r3, r_all, r_cp, r_bk, r4, r5, r6
                    """, uid=user_id)
                    for rec in res:
                        u = rec["u"]
                        if u is not None and u.element_id not in nodes:
                            nodes[u.element_id] = format_graph_node(u, custom_label=f"■ You ({u.get('usernam
    | e')})", custom_group="User")
                        for node_key in ["l", "cp", "b1", "all_l", "f_cp", "f_b", "f", "lr", "b2"]:
                            item = rec[node_key]
                            if item is not None and item.element_id not in nodes:
                                nodes[item.element_id] = format_graph_node(item)
                        for r_key in ["r1", "r2", "r3", "r_all", "r_cp", "r_bk", "r4", "r5", "r6"]:
                            r = rec[r_key]
                            if r is not None and r.element_id not in [e["id"] for e in edges]:
                                edges.append(format_graph_edge(r, r.start_node.element_id, r.end_node.elemen
    | t_id))

            elif view == "catalog":
                # Catalog Knowledge Graph (Books, Categories, Authors, Publishers, Copies)
                res = s.run("""
                    MATCH (b:Book)
                    OPTIONAL MATCH (b)-[r1:BELONGS_TO]->(cat:Category)
                    OPTIONAL MATCH (a:Author)-[r2:WROTE]->(b)
                    OPTIONAL MATCH (b)-[r3:PUBLISHED_BY]->(p:Publisher)
                    OPTIONAL MATCH (b)-[r4:HAS_COPY]->(cp:Copy)
                    RETURN b, cat, a, p, cp, r1, r2, r3, r4
                    LIMIT 200
                """)
                for rec in res:
                    for node_key in ["b", "cat", "a", "p", "cp"]:
                        item = rec[node_key]
                        if item is not None and item.element_id not in nodes:
                            nodes[item.element_id] = format_graph_node(item)
                    for r_key in ["r1", "r2", "r3", "r4"]:
                        r = rec[r_key]
                        if r is not None and r.element_id not in [e["id"] for e in edges]:
                            edges.append(format_graph_edge(r, r.start_node.element_id, r.end_node.element_id
    | ))

            elif view == "recommendations":
                target_uid = user_id
                is_admin_librarian = user_role in ["Admin", "Librarian"]
                target_label = f"You ({session.get('username', 'User')})"

                if is_admin_librarian:
                    # Preview top active student borrower's collaborative path for Admin/Librarian
                    top_user = s.run("""
                        MATCH (u:User {role: 'Student'})-[:BORROWED]->(l:Loan)
                        RETURN u.user_id AS uid, u.username AS username, count(l) AS c
                        ORDER BY c DESC LIMIT 1
                    """).single()
                    if top_user:
                        target_uid = top_user["uid"]
                        target_label = f"Active Patron ({top_user['username']})"
                    else:
                        target_label = "Active Patron"

                # Multi-hop Collaborative Filtering Path (strictly matches same role)
                res = s.run("""
                    MATCH (me:User {user_id: $uid})-[:BORROWED]->(l1:Loan)-[:FOR_COPY]->(cp1:Copy)<-[:HAS_CO
    | PY]-(b1:Book)-[:BELONGS_TO]->(cat:Category)
                    MATCH (other:User)-[:BORROWED]->(l2:Loan)-[:FOR_COPY]->(cp2:Copy)<-[:HAS_COPY]-(b2:Book)
    | -[:BELONGS_TO]->(cat)
                    WHERE other.user_id <> $uid AND other.role = me.role
                      AND NOT (me)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(b2)
                    RETURN me, b1, other, b2, cat
                    LIMIT 15
                """, uid=target_uid)
                recs_found = False

                def add_synth_edge(from_id, to_id, label):
                    eid = f"rec_{from_id}_{to_id}_{label}"
```

```
                    if eid not in [e["id"] for e in edges]:
                        edges.append({
                            "id": eid, "from": from_id, "to": to_id, "label": label,
                            "title": label, "arrows": "to",
                            "font": {"size": 10, "align": "middle", "background": "#0f172a", "color": "#94A3
    | B8"}
                        })

                for rec in res:
                    recs_found = True
                    me = rec["me"]
                    b1 = rec["b1"]
                    other = rec["other"]
                    b2 = rec["b2"]
                    cat = rec["cat"]

                    # Level 0 (Column 1 - Left): You / Active Patron
                    if me.element_id not in nodes:
                        n = format_graph_node(me, custom_label=f"■ {target_label}", custom_group="CurrentUse
    | r")
                        n["level"] = 0
                        nodes[me.element_id] = n

                    # Level 1 (Column 2 - Center-Left): Your Read Books / Shared Categories
                    if b1.element_id not in nodes:
                        n = format_graph_node(b1, custom_label=f"■ Read:\n{b1.get('title')}", custom_group="
    | SharedBook")
                        n["level"] = 1
                        nodes[b1.element_id] = n
                    if cat and cat.element_id not in nodes:
                        n = format_graph_node(cat, custom_label=f"■ Category:\n{cat.get('name')}", custom_gr
    | oup="Category")
                        n["level"] = 1
                        nodes[cat.element_id] = n

                    # Level 2 (Column 3 - Center-Right): Peer Readers
                    if other.element_id not in nodes:
                        n = format_graph_node(other, custom_label=f"■ Peer:\n{other.get('username')}", custo
    | m_group="PeerUser")
                        n["level"] = 2
                        nodes[other.element_id] = n

                    # Level 3 (Column 4 - Right): Recommended Books
                    if b2.element_id not in nodes:
                        n = format_graph_node(b2, custom_label=f"■ Recommended:\n{b2.get('title')}", custom_
    | group="RecommendedBook")
                        n["level"] = 3
                        nodes[b2.element_id] = n

                    # Sequential multi-hop flow: Level 0 -> Level 1 -> Level 2 -> Level 3
                    add_synth_edge(me.element_id, b1.element_id, "borrowed")
                    add_synth_edge(b1.element_id, other.element_id, "shared")
                    if cat:
                        add_synth_edge(me.element_id, cat.element_id, "shared")
                        add_synth_edge(cat.element_id, other.element_id, "shared")
                    add_synth_edge(other.element_id, b2.element_id, "recommends")

                if not recs_found:
                    if user_role == "Staff":
                        # Graph-content traversal fallback for Staff without peer matches:
                        # (You) -> [:borrowed] -> (Borrowed Book) -> [:shared] -> (Shared Category/Author/Pr
    | erequisite) -> [:recommends] -> (Recommended Book)
                        staff_fallback_res = s.run("""
                            MATCH (me:User {user_id: $uid})-[:BORROWED|REQUESTED]->()-[:FOR_COPY|FOR_BOOK]->
    | ()-[:HAS_COPY*0..1]-(b1:Book)
                            MATCH path = (b1)-[r:BELONGS_TO|WROTE|PREREQUISITE_FOR*1..2]-(rec:Book)
                            WHERE rec.book_id <> b1.book_id
                              AND NOT (me)-[:BORROWED]->(:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(rec)
                              AND NOT (me)-[:REQUESTED]->(:LoanRequest)-[:FOR_BOOK]->(rec)
                            RETURN me, b1, nodes(path) AS p_nodes, relationships(path) AS p_rels, rec
                            LIMIT 15
                        """, uid=target_uid)
                        for row in staff_fallback_res:
                            me = row["me"]
                            b1 = row["b1"]
                            rec = row["rec"]
                            p_nodes = row["p_nodes"]
                            p_rels = row["p_rels"]

                            # Level 0 (Column 1 - Left): You
```

```
                            if me.element_id not in nodes:
                                n = format_graph_node(me, custom_label=f"■ {target_label}", custom_group="Cu
    | rrentUser")
                                n["level"] = 0
                                nodes[me.element_id] = n

                            # Level 1 (Column 2 - Center-Left): Borrowed Book
                            if b1.element_id not in nodes:
                                n = format_graph_node(b1, custom_label=f"■ Borrowed:\n{b1.get('title')}", cu
    | stom_group="SharedBook")
                                n["level"] = 1
                                nodes[b1.element_id] = n

                            add_synth_edge(me.element_id, b1.element_id, "borrowed")

                            # Level 2 (Column 3 - Center-Right): Shared Category / Author / Prerequisite
                            inter_id = None
                            if len(p_nodes) == 3:
                                inter = p_nodes[1]
                                inter_id = inter.element_id
                                if inter_id not in nodes:
                                    labels = list(inter.labels)
                                    if "Category" in labels:
                                        n = format_graph_node(inter, custom_label=f"■ Category:\n{inter.get(
    | 'name')}", custom_group="Category")
                                    elif "Author" in labels:
                                        n = format_graph_node(inter, custom_label=f"✍■ Author:\n{inter.get('
    | first_name', '')} {inter.get('last_name', '')}", custom_group="Author")
                                    else:
                                        n = format_graph_node(inter, custom_label=f"■ Prerequisite:\n{inter.
    | get('title')}", custom_group="Prerequisite")
                                    n["level"] = 2
                                    nodes[inter_id] = n
                                add_synth_edge(b1.element_id, inter_id, "shared")
                            else:
                                rel_type = p_rels[0].type if p_rels else "PREREQUISITE_FOR"
                                synth_node_id = f"synth_prereq_{b1.element_id}_{rec.element_id}"
                                if synth_node_id not in nodes:
                                    nodes[synth_node_id] = {
                                        "id": synth_node_id,
                                        "label": "■ Prerequisite\nPathway",
                                        "group": "Prerequisite",
                                        "level": 2,
                                        "title": "Prerequisite Academic Pathway"
                                    }
                                inter_id = synth_node_id
                                add_synth_edge(b1.element_id, inter_id, "requires" if rel_type == "PREREQUIS
    | ITE_FOR" else "shared")

                            # Level 3 (Column 4 - Right): Recommended Book
                            if rec.element_id not in nodes:
                                n = format_graph_node(rec, custom_label=f"■ Recommended:\n{rec.get('title')}
    | ", custom_group="RecommendedBook")
                                n["level"] = 3
                                nodes[rec.element_id] = n

                            if inter_id:
                                add_synth_edge(inter_id, rec.element_id, "recommends")
                    else:
                        # Fallback graph for users with no loan history yet
                        fallback_res = s.run("""
                            MATCH (me:User {user_id: $uid})
                            MATCH (b:Book)<-[:HAS_COPY]-(cp:Copy)<-[:FOR_COPY]-(l:Loan)<-[:BORROWED]-(other:
    | User {role: 'Student'})
                            OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
                            RETURN me, b, other, cat, count(l) as borrow_count
                            ORDER BY borrow_count DESC
                            LIMIT 6
                        """, uid=target_uid)
                        for rec in fallback_res:
                            me = rec["me"]
                            b = rec["b"]
                            other = rec["other"]
                            cat = rec["cat"]
                            if me.element_id not in nodes:
                                n = format_graph_node(me, custom_label=f"■ {target_label}", custom_group="Cu
    | rrentUser")
                                n["level"] = 0
                                nodes[me.element_id] = n
                            if cat and cat.element_id not in nodes:
```

```
                                n = format_graph_node(cat, custom_label=f"■ Category:\n{cat.get('name')}", c
    | ustom_group="Category")
                                n["level"] = 1
                                nodes[cat.element_id] = n
                            if other and other.element_id not in nodes:
                                n = format_graph_node(other, custom_label=f"■ Peer:\n{other.get('username')}
    | ", custom_group="PeerUser")
                                n["level"] = 2
                                nodes[other.element_id] = n
                            if b.element_id not in nodes:
                                n = format_graph_node(b, custom_label=f"■ Recommended:\n{b.get('title')}", c
    | ustom_group="RecommendedBook")
                                n["level"] = 3
                                nodes[b.element_id] = n

                            if cat:
                                add_synth_edge(me.element_id, cat.element_id, "shared")
                                if other:
                                    add_synth_edge(cat.element_id, other.element_id, "shared")
                            if other:
                                add_synth_edge(other.element_id, b.element_id, "recommends")

            else:
                # Full system graph (Admin / Librarian)
                res = s.run("""
                    MATCH (n)
                    OPTIONAL MATCH (n)-[r]->(m)
                    RETURN n, r, m
                """)
                for rec in res:
                    n = rec["n"]
                    m = rec["m"]
                    r = rec["r"]
                    if n is not None and n.element_id not in nodes:
                        nodes[n.element_id] = format_graph_node(n)
                    if m is not None and m.element_id not in nodes:
                        nodes[m.element_id] = format_graph_node(m)
                    if r is not None and r.element_id not in [e["id"] for e in edges]:
                        edges.append(format_graph_edge(r, n.element_id, m.element_id))

    except Exception as e:
        return jsonify({"error": str(e), "nodes": [], "edges": [], "stats": {}}), 500

    # Build group statistics
    counts = {}
    for node in nodes.values():
        grp = node.get("group", "Other")
        counts[grp] = counts.get(grp, 0) + 1

    return jsonify({
        "nodes": list(nodes.values()),
        "edges": edges,
        "view": view,
        "stats": {
            "total_nodes": len(nodes),
            "total_edges": len(edges),
            "counts": counts
        }
    })


# ============================================================================
# Task 6 — The "Book-to-Book Bridge" Shortest Path Route (/book_bridge)
# ============================================================================
def format_bridge_node(n, idx, total_nodes, user_role):
    """
    Formats a graph node for the Book-to-Book Bridge with role-based privacy masking.
    If the path traverses :User or :Loan nodes and the user is a Student or Staff,
    masks identities as 'Anonymous Reader' and 'Circulation Link' to protect patron privacy.
    Admins and Librarians view full patron usernames and loan details.
    """
    labels = list(n.labels) if hasattr(n, "labels") else []
    primary_label = labels[0] if labels else "Node"
    is_privacy_masked = user_role in ["Student", "Staff"]

    custom_grp = None
    custom_lbl = None
    if idx == 0:
        custom_grp = "StartBook"
```

```
3563:         custom_lbl = f"■ Origin: {n.get('title', '')[:22]}"
3564:     elif idx == total_nodes - 1:
3565:         custom_grp = "TargetBook"
3566:         custom_lbl = f"■ Destination: {n.get('title', '')[:22]}"
3567:     elif "Book" in labels:
3568:         custom_grp = "IntermediateBook"
3569:         book_title = n.get("title", "")
3570:         short_title = book_title if len(book_title) <= 22 else book_title[:20] + "..."
3571:         custom_lbl = f"■ {short_title}"
3572:
3573:     if is_privacy_masked and "User" in labels:
3574:         node_dict = {
3575:             "id": n.element_id,
3576:             "label": "Anonymous Reader",
3577:             "group": "User",
3578:             "title": "<div style='font-family:sans-serif;'><b>Anonymous Reader</b><br/>Privacy Masked: Patro
      | n Identity Hidden</div>",
3579:             "properties": {"role": "Reader", "status": "Anonymous Reader"}
3580:         }
3581:         step_item = {"step": idx + 1, "type": "User", "name": "Anonymous Reader"}
3582:         return node_dict, step_item
3583:     elif is_privacy_masked and "Loan" in labels:
3584:         node_dict = {
3585:             "id": n.element_id,
3586:             "label": "Circulation Link",
3587:             "group": "Loan",
3588:             "title": "<div style='font-family:sans-serif;'><b>Circulation Link</b><br/>Confidential Circulat
      | ion Record</div>",
3589:             "properties": {"status": "Circulation Link"}
3590:         }
3591:         step_item = {"step": idx + 1, "type": "Loan", "name": "Circulation Link"}
3592:         return node_dict, step_item
3593:     elif is_privacy_masked and "LoanRequest" in labels:
3594:         node_dict = {
3595:             "id": n.element_id,
3596:             "label": "Circulation Request",
3597:             "group": "LoanRequest",
3598:             "title": "<div style='font-family:sans-serif;'><b>Circulation Request</b><br/>Confidential Reque
      | st Record</div>",
3599:             "properties": {"status": "Circulation Request"}
3600:         }
3601:         step_item = {"step": idx + 1, "type": "LoanRequest", "name": "Circulation Request"}
3602:         return node_dict, step_item
3603:     else:
3604:         node_dict = format_graph_node(n, custom_group=custom_grp, custom_label=custom_lbl)
3605:         item_name = n.get("title") or n.get("name") or n.get("username") or f"{primary_label} #{n.get('book_
      | id', n.get('copy_id', n.get('loan_id', '')))}"
3606:         step_type = "Intermediate Book" if custom_grp == "IntermediateBook" else primary_label
3607:         step_item = {"step": idx + 1, "type": step_type, "name": item_name}
3608:         return node_dict, step_item
3609:
3610:
3611: @app.route("/book_bridge", methods=["GET", "POST"])
3612: @login_required
3613: def book_bridge():
3614:     """
3615:     Renders and executes the 'Book-to-Book Bridge' shortest path traversal.
3616:     Calculates degrees of separation connecting two library volumes using Neo4j shortestPath().
3617:     """
3618:     book_a_id = request.values.get("book_a", type=int)
3619:     book_b_id = request.values.get("book_b", type=int)
3620:
3621:     all_books = []
3622:     path_nodes = []
3623:     path_edges = []
3624:     path_steps = []
3625:     degrees_of_separation = 0
3626:     path_found = False
3627:     error_msg = None
3628:
3629:     try:
3630:         with get_session() as s:
3631:             # Fetch all books for dropdown selection
3632:             books_res = s.run("""
3633:                 MATCH (b:Book)
3634:                 OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
3635:                 OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
3636:                 RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
3637:                        cat.name AS category_name,
3638:                        collect(DISTINCT a.first_name + ' ' + a.last_name) AS authors
```

```
3639:                 ORDER BY b.title ASC
3640:             """)
3641:             raw_books = rows(books_res)
3642:             all_books = []
3643:             for rb in raw_books:
3644:                 auth_list = [a for a in rb.get("authors", []) if a and a.strip()]
3645:                 rb["author_name"] = ", ".join(auth_list) if auth_list else "Author"
3646:                 all_books.append(rb)
3647: 
3648:             # Default to first and fifth book if none specified
3649:             if not book_a_id and not book_b_id and len(all_books) >= 2:
3650:                 book_a_id = all_books[0]["book_id"]
3651:                 book_b_id = all_books[4]["book_id"] if len(all_books) > 4 else all_books[1]["book_id"]
3652: 
3653:             if book_a_id and book_b_id:
3654:                 if book_a_id == book_b_id:
3655:                     error_msg = "Please select two different books to calculate the shortest path bridge."
3656:                 else:
3657:                     # 1. Primary shortest path across metadata and prerequisite learning hierarchy
3658:                     res = s.run("""
3659:                         MATCH p = shortestPath((b1:Book {book_id: $id1})-[:BELONGS_TO|WROTE|PUBLISHED_BY|PRE
      | REQUISITE_FOR*1..5]-(b2:Book {book_id: $id2}))
3660:                         RETURN p
3661:                     """, id1=book_a_id, id2=book_b_id)
3662:                     rec = res.single()
3663: 
3664:                     # 2. Resilient multi-hop fallback across readers/copies if cross-domain
3665:                     if not (rec and rec["p"]):
3666:                         res = s.run("""
3667:                             MATCH p = shortestPath((b1:Book {book_id: $id1})-[:BELONGS_TO|WROTE|PREREQUISITE
      | _FOR|PUBLISHED_BY|HAS_COPY|FOR_COPY|BORROWED|FOR_BOOK*1..10]-(b2:Book {book_id: $id2}))
3668:                             RETURN p
3669:                         """, id1=book_a_id, id2=book_b_id)
3670:                         rec = res.single()
3671: 
3672:                     if rec and rec["p"]:
3673:                         path = rec["p"]
3674:                         path_found = True
3675:                         degrees_of_separation = len(path.relationships)
3676: 
3677:                         # Format nodes with role-based privacy masking
3678:                         user_role = session.get("role") or session.get("Role")
3679:                         total_nodes = len(path.nodes)
3680:                         for idx, n in enumerate(path.nodes):
3681:                             f_node, step_item = format_bridge_node(n, idx, total_nodes, user_role)
3682:                             path_nodes.append(f_node)
3683:                             path_steps.append(step_item)
3684: 
3685:                         # Format relationships preserving true Neo4j schema edge direction
3686:                         for idx, r in enumerate(path.relationships):
3687:                             from_id = r.start_node.element_id
3688:                             to_id = r.end_node.element_id
3689:                             edge_dict = format_graph_edge(r, from_id, to_id)
3690:                             edge_dict["label"] = f"Step {idx+1}: {r.type}"
3691:                             path_edges.append(edge_dict)
3692:                     else:
3693:                         error_msg = f"No connecting graph bridge discovered between Book #{book_a_id} and Bo
      | ok #{book_b_id} within maximum traversal depth."
3694: 
3695:     except Exception as e:
3696:         error_msg = f"Database error evaluating graph shortest path: {str(e)}"
3697: 
3698:     return render_template(
3699:         "book_bridge.html",
3700:         all_books=all_books,
3701:         book_a_id=book_a_id,
3702:         book_b_id=book_b_id,
3703:         path_nodes=path_nodes,
3704:         path_edges=path_edges,
3705:         path_steps=path_steps,
3706:         degrees=degrees_of_separation,
3707:         path_found=path_found,
3708:         error_msg=error_msg
3709:     )
3710: 
3711: 
3712: @app.route("/api/book_bridge")
3713: @login_required
3714: def api_book_bridge():
3715:     """JSON API endpoint returning shortest path data for dynamic UI rendering."""
```

```
3716:     book_a_id = request.args.get("book_a", type=int)
3717:     book_b_id = request.args.get("book_b", type=int)
3718:
3719:     if not book_a_id or not book_b_id:
3720:         return jsonify({"error": "Parameters book_a and book_b are required.", "nodes": [], "edges": []}), 4
      | 00
3721:
3722:     if book_a_id == book_b_id:
3723:         return jsonify({"error": "Book A and Book B must be different.", "nodes": [], "edges": []}), 400
3724:
3725:     try:
3726:         with get_session() as s:
3727:             res = s.run("""
3728:                 MATCH p = shortestPath((b1:Book {book_id: $id1})-[:BELONGS_TO|WROTE|PUBLISHED_BY|PREREQUISIT
      | E_FOR*1..5]-(b2:Book {book_id: $id2}))
3729:                 RETURN p
3730:             """, id1=book_a_id, id2=book_b_id)
3731:             rec = res.single()
3732:
3733:             if not (rec and rec["p"]):
3734:                 res = s.run("""
3735:                     MATCH p = shortestPath((b1:Book {book_id: $id1})-[:BELONGS_TO|WROTE|PREREQUISITE_FOR|PUB
      | LISHED_BY|HAS_COPY|FOR_COPY|BORROWED|FOR_BOOK*1..10]-(b2:Book {book_id: $id2}))
3736:                     RETURN p
3737:                 """, id1=book_a_id, id2=book_b_id)
3738:                 rec = res.single()
3739:
3740:             if not (rec and rec["p"]):
3741:                 return jsonify({"found": False, "degrees": 0, "nodes": [], "edges": [], "steps": [], "messag
      | e": "No path discovered."})
3742:
3743:             path = rec["p"]
3744:             nodes = []
3745:             edges = []
3746:             steps = []
3747:
3748:             user_role = session.get("role") or session.get("Role")
3749:             total_nodes = len(path.nodes)
3750:             for idx, n in enumerate(path.nodes):
3751:                 f_node, step_item = format_bridge_node(n, idx, total_nodes, user_role)
3752:                 nodes.append(f_node)
3753:                 steps.append(step_item)
3754:
3755:             for idx, r in enumerate(path.relationships):
3756:                 from_id = r.start_node.element_id
3757:                 to_id = r.end_node.element_id
3758:                 e = format_graph_edge(r, from_id, to_id)
3759:                 e["label"] = f"Step {idx+1}: {r.type}"
3760:                 edges.append(e)
3761:
3762:             return jsonify({
3763:                 "found": True,
3764:                 "degrees": len(path.relationships),
3765:                 "nodes": nodes,
3766:                 "edges": edges,
3767:                 "steps": steps
3768:             })
3769:
3770:     except Exception as e:
3771:         return jsonify({"error": str(e), "nodes": [], "edges": []}), 500
3772:
3773:
3774: # ============================================================================
3775: # Task 7 — Author Centrality & Influencer Analytics Dashboard (/library_analytics)
3776: # ============================================================================
3777: @app.route("/library_analytics")
3778: @login_required
3779: def library_analytics():
3780:     """
3781:     Renders the Author Centrality & Graph Influencer Analytics Dashboard.
3782:     Available to Staff, Librarian, and Admin roles to analyze connection density,
3783:     borrowing centrality, and knowledge graph structure.
3784:     """
3785:     role = session.get("role")
3786:     if role not in ["Admin", "Librarian", "Staff"]:
3787:         flash("Library Analytics & Centrality Dashboard is accessible to Staff, Librarians, and Administrato
      | rs.", "warning")
3788:         return redirect(url_for("dashboard"))
3789:
3790:     try:
```

```
        with get_session() as s:
            # 1. Total System Loans for Share Calculation
            total_system_loans = s.run("MATCH (l:Loan) RETURN count(l) AS c").single()["c"] or 0

            # 2. Author Centrality & Borrowing Density (Degree Centrality)
            # Separate OPTIONAL MATCH clauses so copies without loans are still counted accurately
            author_res = s.run("""
                MATCH (a:Author)-[:WROTE]->(b:Book)
                OPTIONAL MATCH (b)-[:HAS_COPY]->(cp:Copy)
                OPTIONAL MATCH (cp)<-[:FOR_COPY]-(l:Loan)
                RETURN a.author_id AS author_id,
                       a.first_name + ' ' + a.last_name AS author_name,
                       count(DISTINCT l) AS total_borrowings,
                       count(DISTINCT b) AS authored_books,
                       count(DISTINCT cp) AS active_copies
                ORDER BY total_borrowings DESC, authored_books DESC, active_copies DESC
                LIMIT 5
            """)
            top_authors = rows(author_res)

            # Calculate Borrowing Density Share % against total system loans and progress bar width relative
    |  to top author
            max_author_loans = top_authors[0]["total_borrowings"] if top_authors and top_authors[0]["total_b
    | orrowings"] > 0 else 1
            for auth in top_authors:
                borrowings = auth.get("total_borrowings", 0)
                auth["share_pct"] = round(borrowings / total_system_loans * 100, 1) if total_system_loans el
    | se 0.0
                auth["bar_width"] = round((borrowings / max_author_loans) * 100, 1) if max_author_loans else
    |  0.0

            # 3. Book Degree Centrality & Circulation Influence
            # Fetch co-authors and join with ", " in Python so all authors are displayed
            book_res = s.run("""
                MATCH (b:Book)
                OPTIONAL MATCH (b)-[:HAS_COPY]->(cp:Copy)
                OPTIONAL MATCH (cp)<-[:FOR_COPY]-(l:Loan)
                OPTIONAL MATCH (b)<-[:FOR_BOOK]-(lr:LoanRequest)
                OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
                OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
                RETURN b.book_id AS book_id, b.title AS title,
                       cat.name AS category_name,
                       collect(DISTINCT a.first_name + ' ' + a.last_name) AS authors_list,
                       count(DISTINCT cp) AS copies_count,
                       count(DISTINCT l) AS loans_count,
                       count(DISTINCT lr) AS requests_count,
                       (count(DISTINCT cp) * 2 + count(DISTINCT l) * 3 + count(DISTINCT lr)) AS degree_score
                ORDER BY degree_score DESC, loans_count DESC
                LIMIT 5
            """)
            top_books = rows(book_res)
            for bk in top_books:
                authors = bk.get("authors_list", [])
                bk["author_name"] = ", ".join(authors) if authors else "Unknown"

            # 4. Category Topic Closeness & Circulation Density
            cat_res = s.run("""
                MATCH (cat:Category)<-[:BELONGS_TO]-(b:Book)
                OPTIONAL MATCH (b)-[:HAS_COPY]->(cp:Copy)<-[:FOR_COPY]-(l:Loan)
                RETURN cat.name AS category_name,
                       cat.section_code AS section_code,
                       count(DISTINCT b) AS books_count,
                       count(l) AS total_loans
                ORDER BY total_loans DESC, books_count DESC
            """)
            categories = rows(cat_res)

            # 5. Overall Knowledge Graph Topology Metrics
            nodes_count = s.run("MATCH (n) RETURN count(n) AS c").single()["c"]
            rels_count = s.run("MATCH ()-[r]->() RETURN count(r) AS c").single()["c"]

            # Query both distinct active borrowers (unreturned loans) and active loans
            active_res = s.run("""
                MATCH (u:User)-[:BORROWED]->(l:Loan)
                WHERE l.return_date IS NULL
                RETURN count(DISTINCT u) AS active_borrowers, count(DISTINCT l) AS active_loans
            """).single()
            active_borrowers = active_res["active_borrowers"] if active_res else 0
            active_loans = active_res["active_loans"] if active_res else 0

```

```
3867:             avg_degree = round((float(rels_count) * 2.0 / nodes_count), 2) if nodes_count else 0.0
3868:
3869:             graph_metrics = {
3870:                 "nodes_count": nodes_count,
3871:                 "rels_count": rels_count,
3872:                 "active_borrowers": active_borrowers,
3873:                 "active_loans": active_loans,
3874:                 "avg_degree": avg_degree,
3875:                 "density_rating": "High (Multi-clustered)" if avg_degree >= 2.0 else "Sparse"
3876:             }
3877:
3878:             return render_template(
3879:                 "library_analytics.html",
3880:                 top_authors=top_authors,
3881:                 top_books=top_books,
3882:                 categories=categories,
3883:                 metrics=graph_metrics
3884:             )
3885:
3886:     except Exception as e:
3887:         flash(f"Error loading graph centrality analytics: {str(e)}", "danger")
3888:         return redirect(url_for("dashboard"))
3889:
3890:
3891: # ============================================================================
3892: # Advanced Graph Feature — Academic Learning Roadmap ([:PREREQUISITE_FOR])
3893: # ============================================================================
3894: def ensure_prerequisite_hierarchy(s=None):
3895:     """
3896:     Idempotently seeds logical (:Book)-[:PREREQUISITE_FOR]->(:Book) relationships
3897:     in Neo4j AuraDB if not already present. Reuses existing active session s if provided.
3898:     """
3899:     def _run_seeding(session_tx):
3900:         prereq_specs = [
3901:             (2, 3, 'Code Quality to Algorithms', 'Stage 1 -> Stage 2', 'Software Engineering'),
3902:             (3, 4, 'Algorithms to Intelligent Systems', 'Stage 2 -> Stage 3', 'Artificial Intelligence'),
3903:             (3, 9, 'Algorithms to Theoretical Mastery', 'Stage 2 -> Stage 4', 'Theoretical CS'),
3904:             (2, 8, 'Clean Code to Domain Architecture', 'Stage 1 -> Stage 3', 'Software Engineering'),
3905:             (6, 7, 'OS Internals to Network Protocols', 'Stage 1 -> Stage 2', 'Systems & Infrastructure'),
3906:             (7, 1, 'Networking to Distributed Storage', 'Stage 2 -> Stage 3', 'Data Engineering'),
3907:             (5, 1, 'Relational Theory to Distributed Scale', 'Stage 1 -> Stage 3', 'Data Engineering')
3908:         ]
3909:         for src_id, dst_id, jump, stage, track in prereq_specs:
3910:             session_tx.run("""
3911:                 MATCH (src:Book {book_id: $src_id}), (dst:Book {book_id: $dst_id})
3912:                 MERGE (src)-[r:PREREQUISITE_FOR]->(dst)
3913:                 ON CREATE SET r.difficulty_jump = $jump,
3914:                               r.stage = $stage,
3915:                               r.track = $track
3916:             """, src_id=src_id, dst_id=dst_id, jump=jump, stage=stage, track=track)
3917:
3918:     try:
3919:         if s is not None:
3920:             _run_seeding(s)
3921:         else:
3922:             with get_session() as sess:
3923:                 _run_seeding(sess)
3924:     except Exception as e:
3925:         app.logger.warning(f"Error checking/seeding prerequisite hierarchy: {e}")
3926:
3927:
3928: @app.route("/learning_paths", methods=["GET"])
3929: @login_required
3930: def learning_paths():
3931:     """
3932:     Renders the Academic Learning Roadmap based on variable-length
3933:     (:Book)-[:PREREQUISITE_FOR*1..4]->(:Book) hierarchical trees with real-time stock availability.
3934:     """
3935:     try:
3936:         with get_session() as s:
3937:             ensure_prerequisite_hierarchy(s)
3938:             # 1. Variable-length paths query: full end-to-end Root-to-Mastery sequences
3939:             path_res = s.run("""
3940:                 MATCH path = (root:Book)-[:PREREQUISITE_FOR*1..4]->(target:Book)
3941:                 WHERE NOT ()-[:PREREQUISITE_FOR]->(root) AND NOT (target)-[:PREREQUISITE_FOR]->()
3942:                 RETURN [n IN nodes(path) | {id: n.book_id, title: n.title}] AS nodes_in_path,
3943:                        length(path) AS depth,
3944:                        [r IN relationships(path) | {jump: r.difficulty_jump, track: r.track, stage: r.stage}
     | ] AS edges_in_path
3945:             """)
```

```
3946:             raw_paths = rows(path_res)
3947:
3948:             # Assign clear specialization badges for distinct branching tracks
3949:             for p in raw_paths:
3950:                 nodes_in_p = p.get("nodes_in_path") or []
3951:                 root_title = (nodes_in_p[0].get("title") or "") if nodes_in_p and isinstance(nodes_in_p[0],
     | dict) else ""
3952:                 target_title = (nodes_in_p[-1].get("title") or "") if nodes_in_p and isinstance(nodes_in_p[-
     | 1], dict) else ""
3953:                 edge_tracks = [(e.get("track") or "") for e in (p.get("edges_in_path") or []) if isinstance(
     | e, dict)]
3954:
3955:                 if "Artificial Intelligence" in target_title or any("Artificial Intelligence" in t for t in
     | edge_tracks):
3956:                     p["track_badge"] = "AI Specialization Track"
3957:                     p["track_color"] = "#8B5CF6"
3958:                 elif "Art of Computer Programming" in target_title or any("Theoretical" in t for t in edge_t
     | racks):
3959:                     p["track_badge"] = "Algorithms Mastery Track"
3960:                     p["track_color"] = "#F59E0B"
3961:                 elif "Domain-Driven Design" in target_title:
3962:                     p["track_badge"] = "Software Architecture Track"
3963:                     p["track_color"] = "#3B82F6"
3964:                 elif "Modern Operating Systems" in root_title:
3965:                     p["track_badge"] = "Systems & Infrastructure Track"
3966:                     p["track_color"] = "#06B6D4"
3967:                 elif "Database System Concepts" in root_title or any("Data Engineering" in t for t in edge_t
     | racks):
3968:                     p["track_badge"] = "Data Engineering Track"
3969:                     p["track_color"] = "#10B981"
3970:                 else:
3971:                     last_track = edge_tracks[-1] if edge_tracks else "Curriculum"
3972:                     p["track_badge"] = f"{last_track} Track"
3973:                     p["track_color"] = "#818CF8"
3974:
3975:             # Fetch patron learning progress (completed reads and active loans)
3976:             uid = session.get("user_id") or -1
3977:             user_completed_bids = set()
3978:             user_active_bids = set()
3979:             if uid and uid > 0:
3980:                 prog_res = s.run("""
3981:                     MATCH (u:User {user_id: $uid})-[:BORROWED]->(l:Loan)-[:FOR_COPY]->(:Copy)<-[:HAS_COPY]-(
     | b:Book)
3982:                     RETURN b.book_id AS book_id, l.return_date AS return_date
3983:                 """, uid=uid)
3984:                 for row in rows(prog_res):
3985:                     if row.get("return_date"):
3986:                         user_completed_bids.add(row["book_id"])
3987:                     else:
3988:                         user_active_bids.add(row["book_id"])
3989:
3990:             # 2. Roadmap books with real-time stock availability, category metadata, and level attribution
3991:             nodes_res = s.run("""
3992:                 MATCH (b:Book)
3993:                 WHERE (b)-[:PREREQUISITE_FOR]-()
3994:                 OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
3995:                 OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
3996:                 OPTIONAL MATCH (b)-[:PREREQUISITE_FOR]->(nxt:Book)
3997:                 OPTIONAL MATCH (prv:Book)-[:PREREQUISITE_FOR]->(b)
3998:                 WITH b, cat,
3999:                      collect(DISTINCT a.first_name + ' ' + a.last_name) AS authors,
4000:                      collect(DISTINCT prv.book_id) AS prereqs,
4001:                      collect(DISTINCT nxt.book_id) AS unlocks
4002:                 RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
4003:                        coalesce(cat.category_id, b.category_id, 1) AS category_id,
4004:                        coalesce(cat.name, 'General') AS category_name,
4005:                        reduce(s='', x IN authors | CASE WHEN s='' THEN x ELSE s+', '+x END) AS authors,
4006:                        size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status <> 'Retired' AND cp.status <> 'Lost'
     |  | cp]) AS total_copies,
4007:                        size([(b)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies,
4008:                        prereqs,
4009:                        unlocks
4010:                 ORDER BY book_id
4011:             """)
4012:             book_nodes = rows(nodes_res)
4013:
4014:             # 3. Directed edges
4015:             edge_res = s.run("""
4016:                 MATCH (src:Book)-[r:PREREQUISITE_FOR]->(dst:Book)
4017:                 RETURN src.book_id AS source, dst.book_id AS target,
```

```
                        r.difficulty_jump AS difficulty_jump,
                        r.stage AS stage,
                        coalesce(r.track, 'General') AS track
            """)
            edge_rows = rows(edge_res)

            # Build level mapping (root nodes in-degree 0 -> level 1, etc.)
            in_degrees = {b["book_id"]: len(b["prereqs"]) for b in book_nodes}
            levels = {}
            for b in book_nodes:
                if in_degrees.get(b["book_id"], 0) == 0:
                    levels[b["book_id"]] = 1

            # Honour explicit target stages (e.g. Donald Knuth reaching Stage 4)
            for e in edge_rows:
                stage_str = e.get("stage") or ""
                if "->" in stage_str and "Stage" in stage_str:
                    try:
                        dest_stage = int(stage_str.split("->")[-1].replace("Stage", "").strip())
                        levels[e["target"]] = max(levels.get(e["target"], 1), dest_stage)
                    except Exception:
                        pass

            # Relax levels down the edges
            changed = True
            iterations = 0
            while changed and iterations < 10:
                changed = False
                iterations += 1
                for e in edge_rows:
                    src_lvl = levels.get(e["source"])
                    if src_lvl is not None:
                        target_lvl = src_lvl + 1
                        if levels.get(e["target"]) is None or levels[e["target"]] < target_lvl:
                            levels[e["target"]] = target_lvl
                            changed = True

            # Format Vis.js network data
            vis_nodes = []
            for b in book_nodes:
                bid = b["book_id"]
                lvl = levels.get(bid, 1)
                avail = b["available_copies"]
                total = b["total_copies"]
                is_avail = avail > 0

                user_status = "unattempted"
                if bid in user_completed_bids:
                    user_status = "completed"
                elif bid in user_active_bids:
                    user_status = "reading"

                stage_label = f"Level {lvl}"
                if lvl == 1:
                    stage_title = "Level 1: Foundational"
                    color_bg = "#1E293B"
                    color_border = "#3B82F6"
                elif lvl == 2:
                    stage_title = "Level 2: Core Engineering"
                    color_bg = "#1E293B"
                    color_border = "#8B5CF6"
                elif lvl == 3:
                    stage_title = "Level 3: Systems Specialization"
                    color_bg = "#1E293B"
                    color_border = "#EC4899"
                else:
                    stage_title = "Level 4: Advanced Mastery"
                    color_bg = "#1E293B"
                    color_border = "#F59E0B"

                if user_status == "completed":
                    node_border = "#10B981"
                    status_text = f"✓ Mastered ({avail}/{total} In Library)"
                    node_label = f"[{stage_label} • ✓ Mastered]\n{b['title']}\n{status_text}"
                elif user_status == "reading":
                    node_border = "#38BDF8"
                    status_text = f"■ Active Loan ({avail}/{total} In Library)"
                    node_label = f"[{stage_label} • ■ Reading]\n{b['title']}\n{status_text}"
                else:
                    node_border = color_border if is_avail else "#EF4444"
```

```
4098:                     status_text = f"● {avail}/{total} Avail" if is_avail else f"● Out of Stock (0/{total})"
4099:                     node_label = f"[{stage_label}]\n{b['title']}\n{status_text}"
4100:
4101:                 vis_nodes.append({
4102:                     "id": bid,
4103:                     "label": node_label,
4104:                     "level": lvl,
4105:                     "title": f"<b>{b['title']}</b><br>Authors: {b['authors']}<br>Category: {b['category_name
     | ']}<br>Stage: {stage_title}<br>Stock: {avail}/{total} available",
4106:                     "shape": "box",
4107:                     "margin": 14,
4108:                     "color": {
4109:                         "background": color_bg,
4110:                         "border": node_border,
4111:                         "highlight": { "background": "#334155", "border": "#60A5FA" }
4112:                     },
4113:                     "borderWidth": 3 if user_status != "unattempted" else 2,
4114:                     "font": { "color": "#F8FAFC", "size": 15, "multi": True, "face": "Inter, sans-serif" },
4115:                     "properties": {
4116:                         "book_id": bid,
4117:                         "title": b["title"],
4118:                         "isbn": b["isbn"],
4119:                         "authors": b["authors"],
4120:                         "category_id": b.get("category_id") or 1,
4121:                         "category_name": b["category_name"],
4122:                         "level": lvl,
4123:                         "stage_title": stage_title,
4124:                         "available_copies": avail,
4125:                         "total_copies": total,
4126:                         "is_available": is_avail,
4127:                         "user_status": user_status,
4128:                         "prereqs": b["prereqs"],
4129:                         "unlocks": b["unlocks"]
4130:                     }
4131:                 })
4132:
4133:             vis_edges = []
4134:             for idx, e in enumerate(edge_rows):
4135:                 track = e.get("track") or "General"
4136:                 diff_jump = e.get("difficulty_jump") or "Prerequisite"
4137:                 edge_color = "#3B82F6"
4138:                 if "Intelligence" in track or "Software" in track:
4139:                     edge_color = "#8B5CF6"
4140:                 elif "Data" in track or "Systems" in track:
4141:                     edge_color = "#06B6D4"
4142:                 elif "Theoretical" in track:
4143:                     edge_color = "#F59E0B"
4144:
4145:                 vis_edges.append({
4146:                     "id": f"e_{e['source']}_{e['target']}",
4147:                     "from": e["source"],
4148:                     "to": e["target"],
4149:                     "label": diff_jump,
4150:                     "arrows": "to",
4151:                     "color": { "color": edge_color, "highlight": "#60A5FA" },
4152:                     "font": { "color": "#94A3B8", "size": 10, "align": "horizontal", "background": "rgba(15,
     |  23, 42, 0.85)" },
4153:                     "width": 2,
4154:                     "smooth": { "type": "cubicBezier", "forceDirection": "vertical", "roundness": 0.4 },
4155:                     "properties": {
4156:                         "difficulty_jump": diff_jump,
4157:                         "stage": e.get("stage") or "General",
4158:                         "track": track
4159:                     }
4160:                 })
4161:
4162:             # Calculate curriculum statistics
4163:             total_volumes = len(book_nodes)
4164:             available_volumes = sum(1 for b in book_nodes if b["available_copies"] > 0)
4165:             max_depth = max(levels.values()) if levels else 1
4166:             total_edges = len(vis_edges)
4167:
4168:             stats = {
4169:                 "total_volumes": total_volumes,
4170:                 "available_volumes": available_volumes,
4171:                 "max_depth": max_depth,
4172:                 "total_edges": total_edges,
4173:                 "total_paths": len(raw_paths)
4174:             }
4175:
```

```
4176:             return render_template(
4177:                 "learning_paths.html",
4178:                 vis_nodes=vis_nodes,
4179:                 vis_edges=vis_edges,
4180:                 book_nodes=book_nodes,
4181:                 edge_rows=edge_rows,
4182:                 stats=stats,
4183:                 paths=raw_paths
4184:             )
4185:
4186:     except Exception as e:
4187:         flash(f"Error loading Academic Learning Roadmaps: {str(e)}", "danger")
4188:         return redirect(url_for("dashboard"))
4189:
4190:
4191: @app.route("/api/learning_paths", methods=["GET"])
4192: @login_required
4193: def api_learning_paths():
4194:     """Returns JSON graph data for the academic prerequisite hierarchy."""
4195:     try:
4196:         with get_session() as s:
4197:             ensure_prerequisite_hierarchy(s)
4198:             edge_res = s.run("""
4199:                 MATCH (src:Book)-[r:PREREQUISITE_FOR]->(dst:Book)
4200:                 RETURN src.book_id AS source, dst.book_id AS target,
4201:                        r.difficulty_jump AS difficulty_jump, r.stage AS stage, r.track AS track
4202:             """)
4203:             edges = rows(edge_res)
4204:
4205:             node_res = s.run("""
4206:                 MATCH (b:Book) WHERE (b)-[:PREREQUISITE_FOR]-()
4207:                 OPTIONAL MATCH (b)-[:BELONGS_TO]->(cat:Category)
4208:                 OPTIONAL MATCH (a:Author)-[:WROTE]->(b)
4209:                 WITH b, cat, collect(DISTINCT a.first_name + ' ' + a.last_name) AS author_list
4210:                 RETURN b.book_id AS book_id, b.title AS title, b.isbn AS isbn,
4211:                        coalesce(cat.category_id, b.category_id, 1) AS category_id,
4212:                        coalesce(cat.name, 'General') AS category_name,
4213:                        reduce(s='', a IN author_list | CASE WHEN s='' THEN a ELSE s+', '+a END) AS authors,
4214:                        size([(b)-[:HAS_COPY]->(cp:Copy {status:'Available'}) | cp]) AS available_copies,
4215:                        size([(b)-[:HAS_COPY]->(cp:Copy) WHERE cp.status <> 'Retired' AND cp.status <> 'Lost'
     |  | cp]) AS total_copies
4216:                 ORDER BY b.book_id
4217:             """)
4218:             nodes = rows(node_res)
4219:             return jsonify({"status": "success", "nodes": nodes, "edges": edges})
4220:     except Exception as e:
4221:         return jsonify({"status": "error", "message": str(e)}), 500
4222:
4223:
4224: # ============================================================================
4225: # Entry Point
4226: # ============================================================================
4227: if __name__ == "__main__":
4228:     app.run(debug=True, host="127.0.0.1", port=5000)
4229:
```

Lines: 503 | **Lang:** Python 3.12 / Cypher | **Category:** Database Seeding & Schema Engine

MODULE SOURCE: seed_all_data.py

Core Role & Purpose: Neo4j AuraDB database seeding script establishing uniqueness constraints, schema indexes, sample users (Admin, Librarian, Staff, Student), book catalog, multi-copy relationships, active loans, and disciplinary fines.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:

- Verify creation of UNIQUE constraints on User(username), User(email), Book(book_id), and Copy(copy_id).
- Verify password hashing for seeded accounts using generate_password_hash.
- Verify relationship consistency: (:User)-[:BORROWED]->(:Book), (:Book)-[:HAS_COPY]->(:Copy), (:User)-[:HAS_FINE]->(:Fine).
- Verify that available_copies on Book matches the count of Copy nodes where is_borrowed = false.

```
0001: """
0002: Neo4j AuraDB Seeding Script - VITCC Library Management System
0003: Migrated from Oracle 21c to Neo4j Graph Database.
0004: Seeds all 9 entity types with 10+ records each using explicit graph relationships.
0005: """
0006:
0007: from neo4j import GraphDatabase
0008: from werkzeug.security import generate_password_hash
0009: from datetime import datetime, timedelta
0010:
0011: NEO4J_URI      = "neo4j+s://e7f1b8d0.databases.neo4j.io"
0012: NEO4J_USERNAME = "e7f1b8d0"
0013: NEO4J_PASSWORD = "ROzf6G2s6B2tP6C61ODmEGMbmG-pzeCq5KZgI6CmxHI"
0014: NEO4J_DATABASE = "e7f1b8d0"
0015:
0016:
0017: def run_seed():
0018:     driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USERNAME, NEO4J_PASSWORD))
0019:     print("Connected to Neo4j AuraDB. Beginning comprehensive seeding...")
0020:
0021:     with driver.session(database=NEO4J_DATABASE) as session:
0022:         try:
0023:             # ================================================================
0024:             # WIPE existing data for a clean seed
0025:             # ================================================================
0026:             print("\n[0/9] Clearing existing graph data...")
0027:             session.run("MATCH (n) DETACH DELETE n")
0028:             print(" -> Graph cleared.")
0029:
0030:             # ================================================================
0031:             # 0.1 CREATE UNIQUENESS CONSTRAINTS
```

```
0032:             # ================================================================
0033:             print("\n[0.1] Ensuring schema uniqueness constraints...")
0034:             constraints = [
0035:                 ("user_id_unique", "User", "user_id"),
0036:                 ("user_username_unique", "User", "username"),
0037:                 ("user_email_unique", "User", "email"),
0038:                 ("book_id_unique", "Book", "book_id"),
0039:                 ("book_isbn_unique", "Book", "isbn"),
0040:                 ("category_id_unique", "Category", "category_id"),
0041:                 ("publisher_id_unique", "Publisher", "publisher_id"),
0042:                 ("author_id_unique", "Author", "author_id"),
0043:                 ("copy_id_unique", "Copy", "copy_id"),
0044:                 ("loan_id_unique", "Loan", "loan_id"),
0045:                 ("fine_id_unique", "Fine", "fine_id"),
0046:                 ("request_id_unique", "LoanRequest", "request_id"),
0047:             ]
0048:             for c_name, label, prop in constraints:
0049:                 session.run(f"CREATE CONSTRAINT {c_name} IF NOT EXISTS FOR (n:{label}) REQUIRE n.{prop} IS U
      | NIQUE")
0050:             print(" -> All 12 uniqueness constraints verified.")
0051:
0052:             # ================================================================
0053:             # 1. SEED USERS (11 users across all 4 RBAC roles)
0054:             # ================================================================
0055:             print("\n[1/9] Seeding Users...")
0056:             users = [
0057:                 ("admin",        "Admin@123",   "admin@vit.edu",         "Admin"),
0058:                 ("librarian",    "Lib@12345",   "librarian@vit.edu",     "Librarian"),
0059:                 ("staff",        "Staff@123",   "staff@vit.edu",         "Staff"),
0060:                 ("prof_turing",  "Staff@123",   "turing@vit.edu",        "Staff"),
0061:                 ("prof_shannon", "Staff@123",   "shannon@vit.edu",       "Staff"),
0062:                 ("student",      "Student@1",   "student@vit.edu",       "Student"),
0063:                 ("alex_stu",     "Student@1",   "alex@vit.edu",          "Student"),
0064:                 ("priya_stu",    "Student@1",   "priya@vit.edu",         "Student"),
0065:                 ("rahul_stu",    "Student@1",   "rahul@vit.edu",         "Student"),
0066:                 ("emily_stu",    "Student@1",   "emily@vit.edu",         "Student"),
0067:                 ("marcus_stu",   "Student@1",   "marcus@vit.edu",        "Student"),
0068:             ]
0069:
0070:             for idx, u in enumerate(users, start=1):
0071:                 pw_hash = generate_password_hash(u[1])
0072:                 session.run("""
0073:                     CREATE (u:User {
0074:                         user_id: $uid,
0075:                         username: $username,
0076:                         password_hash: $pw,
0077:                         email: $email,
0078:                         role: $role,
0079:                         account_status: 'Active'
```

```
                    })
                """, uid=idx, username=u[0], pw=pw_hash, email=u[2], role=u[3])

            result = session.run("MATCH (u:User) RETURN u.username AS username, u.user_id AS user_id")
            user_id_map = {r["username"]: r["user_id"] for r in result}
            print(f" -> Seeded {len(users)} users.")

            # ================================================================
            # 2. SEED CATEGORIES (10 records)
            # ================================================================
            print("[2/9] Seeding Categories...")
            categories = [
                ("Computer Science & AI",        "CS"),
                ("Physics & Astronomy",           "PHY"),
                ("Mathematics & Statistics",      "MAT"),
                ("Electrical Engineering",        "EE"),
                ("Mechanical Engineering",        "ME"),
                ("World History & Civilizations", "HIS"),
                ("Philosophy & Ethics",           "PHI"),
                ("Classical Literature",          "LIT"),
                ("Economics & Public Policy",     "ECO"),
                ("Data Architecture & Databases", "DB"),
            ]

            for idx, cat in enumerate(categories, start=1):
                session.run("""
                    CREATE (c:Category {
                        category_id: $cid,
                        name: $name,
                        section_code: $code
                    })
                """, cid=idx, name=cat[0], code=cat[1])

            result = session.run("MATCH (c:Category) RETURN c.name AS name, c.category_id AS cid")
            category_id_map = {r["name"]: r["cid"] for r in result}
            print(f" -> Seeded {len(categories)} categories.")

            # ================================================================
            # 3. SEED PUBLISHERS (10 records)
            # ================================================================
            print("[3/9] Seeding Publishers...")
            publishers = [
                ("O'Reilly Media",              "contact@oreilly.com"),
                ("Pearson Education",           "support@pearson.com"),
                ("MIT Press",                   "info@mitpress.mit.edu"),
                ("McGraw-Hill Education",       "help@mheducation.com"),
                ("Addison-Wesley Professional", "inquiries@addisonwesley.com"),
                ("Cambridge University Press",  "academic@cambridge.org"),
                ("Oxford University Press",     "custserv@oup.com"),
                ("Springer Nature",             "contact@springernature.com"),
                ("John Wiley & Sons",           "support@wiley.com"),
                ("Morgan Kaufmann",             "info@mkp.com"),
            ]

            for idx, pub in enumerate(publishers, start=1):
                session.run("""
                    CREATE (p:Publisher {
                        publisher_id: $pid,
                        name: $name,
                        contact_email: $email
                    })
                """, pid=idx, name=pub[0], email=pub[1])

            result = session.run("MATCH (p:Publisher) RETURN p.name AS name, p.publisher_id AS pid")
            publisher_id_map = {r["name"]: r["pid"] for r in result}
            print(f" -> Seeded {len(publishers)} publishers.")

            # ================================================================
            # 4. SEED AUTHORS (10 records)
            # ================================================================
            print("[4/9] Seeding Authors...")
            authors = [
                ("Donald",  "Knuth"),
                ("Martin",  "Fowler"),
                ("Robert",  "Martin"),
                ("Eric",    "Evans"),
                ("Abraham", "Silberschatz"),
                ("Andrew",  "Tanenbaum"),
                ("Thomas",  "Cormen"),
                ("Stuart",  "Russell"),
```

```
0160:                 ("Peter",   "Norvig"),
0161:                 ("Richard", "Feynman"),
0162:             ]
0163:
0164:             for idx, auth in enumerate(authors, start=1):
0165:                 session.run("""
0166:                     CREATE (a:Author {
0167:                         author_id: $aid,
0168:                         first_name: $fn,
0169:                         last_name: $ln
0170:                     })
0171:                 """, aid=idx, fn=auth[0], ln=auth[1])
0172:
0173:             result = session.run("""
0174:                 MATCH (a:Author)
0175:                 RETURN (a.first_name + ' ' + a.last_name) AS full_name, a.author_id AS aid
0176:             """)
0177:             author_id_map = {r["full_name"]: r["aid"] for r in result}
0178:             print(f" -> Seeded {len(authors)} authors.")
0179:
0180:             # ================================================================
0181:             # 5. SEED BOOKS + graph relationships (10 records)
0182:             # ================================================================
0183:             print("[5/9] Seeding Books + graph relationships...")
0184:             books = [
0185:                 ("978-1449373320", "Designing Data-Intensive Applications",     2017,
0186:                  "Data Architecture & Databases", "O'Reilly Media",             ["Martin Fowler"]),
0187:                 ("978-0132350884", "Clean Code: Agile Software Craftsmanship",   2008,
0188:                  "Computer Science & AI",         "Pearson Education",          ["Robert Martin"]),
0189:                 ("978-0262033848", "Introduction to Algorithms (CLRS)",          2009,
0190:                  "Computer Science & AI",         "MIT Press",                  ["Thomas Cormen"]),
0191:                 ("978-0136042594", "Artificial Intelligence: A Modern Approach", 2020,
0192:                  "Computer Science & AI",         "Pearson Education",          ["Stuart Russell", "Peter N
      | orvig"]),
0193:                 ("978-0078022159", "Database System Concepts (7th Ed)",          2019,
0194:                  "Data Architecture & Databases", "McGraw-Hill Education",      ["Abraham Silberschatz"]),
0195:                 ("978-0133591620", "Modern Operating Systems (4th Ed)",          2014,
0196:                  "Computer Science & AI",         "Addison-Wesley Professional", ["Andrew Tanenbaum"]),
0197:                 ("978-0132126953", "Computer Networks (5th Ed)",                 2010,
0198:                  "Computer Science & AI",         "Pearson Education",          ["Andrew Tanenbaum"]),
0199:                 ("978-0321125217", "Domain-Driven Design: Tackling Complexity",  2003,
0200:                  "Computer Science & AI",         "Addison-Wesley Professional", ["Eric Evans"]),
0201:                 ("978-0201896831", "The Art of Computer Programming, Vol 1",     1997,
0202:                  "Mathematics & Statistics",      "Addison-Wesley Professional", ["Donald Knuth"]),
0203:                 ("978-0465023820", "The Feynman Lectures on Physics",            2011,
0204:                  "Physics & Astronomy",           "Addison-Wesley Professional", ["Richard Feynman"]),
0205:             ]
0206:
0207:             book_id_map = {}  # isbn -> book_id
0208:             for idx, b in enumerate(books, start=1):
0209:                 isbn, title, year, cat_name, pub_name, auth_names = b
0210:                 cat_id = category_id_map.get(cat_name)
0211:                 pub_id = publisher_id_map.get(pub_name)
0212:
0213:                 session.run("""
0214:                     CREATE (bk:Book {
0215:                         book_id: $bid,
0216:                         isbn: $isbn,
0217:                         title: $title,
0218:                         publication_year: $year,
0219:                         category_id: $cat_id,
0220:                         publisher_id: $pub_id
0221:                     })
0222:                 """, bid=idx, isbn=isbn, title=title, year=year,
0223:                      cat_id=cat_id, pub_id=pub_id)
0224:
0225:                 session.run("""
0226:                     MATCH (bk:Book {book_id: $bid}), (c:Category {category_id: $cid})
0227:                     CREATE (bk)-[:BELONGS_TO]->(c)
0228:                 """, bid=idx, cid=cat_id)
0229:
0230:                 session.run("""
0231:                     MATCH (bk:Book {book_id: $bid}), (p:Publisher {publisher_id: $pid})
0232:                     CREATE (bk)-[:PUBLISHED_BY]->(p)
0233:                 """, bid=idx, pid=pub_id)
0234:
0235:                 for a_name in auth_names:
0236:                     a_id = author_id_map.get(a_name)
0237:                     if a_id:
0238:                         session.run("""
```

```
                            MATCH (a:Author {author_id: $aid}), (bk:Book {book_id: $bid})
                            CREATE (a)-[:WROTE]->(bk)
                        """, aid=a_id, bid=idx)

            book_id_map[isbn] = idx

    print(f" -> Seeded {len(books)} books.")

    # ================================================================
    # 6. SEED BOOK COPIES (2 per book = 20 copies)
    # ================================================================
    print("[6/9] Seeding Book Copies...")
    cid_counter = 1
    copy_rows = []

    for isbn, b_id in book_id_map.items():
        for condition in ["New", "Good"]:
            session.run("""
                CREATE (cp:Copy {
                    copy_id: $cid,
                    book_id: $bid,
                    status: 'Available',
                    condition: $cond
                })
            """, cid=cid_counter, bid=b_id, cond=condition)

            session.run("""
                MATCH (bk:Book {book_id: $bid}), (cp:Copy {copy_id: $cid})
                CREATE (bk)-[:HAS_COPY]->(cp)
            """, bid=b_id, cid=cid_counter)

            copy_rows.append((cid_counter, b_id))
            cid_counter += 1

    print(f" -> Seeded {len(copy_rows)} book copies.")

    # ================================================================
    # 7. SEED LOANS (12 scenarios)
    # ================================================================
    print("[7/9] Seeding Loans...")
    librarian_id = user_id_map.get("librarian", user_id_map.get("admin"))
    student_id   = user_id_map.get("student")
    alex_id      = user_id_map.get("alex_stu")
    priya_id     = user_id_map.get("priya_stu")
    rahul_id     = user_id_map.get("rahul_stu")
    staff_id     = user_id_map.get("staff")
    turing_id    = user_id_map.get("prof_turing")

    now = datetime.now()

    loan_configs = [
        (0,  student_id, 3,  -11, None),
        (1,  alex_id,   5,  -9,  None),
        (2,  staff_id,  7,  -7,  None),
        (3,  student_id, 25, 12,  1),   # Returned Oct 17, 2026 (11 days late = Rs. 110.00 Paid)
        (4,  rahul_id,  20,  6,  None),
        (5,  priya_id,  30, 16,  None),
        (6,  student_id, 31, 17,  14),  # Due Oct 1, returned Oct 4 (3 days late = Rs. 30.00 Paid)
        (7,  alex_id,   20,  10,  6),   # Returned 4 days late = Rs. 40.00 Unpaid
        (8,  turing_id, 15,  5,   2),   # Returned 3 days late = Rs. 30.00 Paid
        (9,  student_id, 40, 26,  20),
        (10, rahul_id,  35, 21,  14),
        (11, priya_id,  50, 36,  25),
        (12, staff_id,  22, 12,   7),   # Returned 5 days late = Rs. 50.00 Paid
    ]

    loans_list = []

    for loan_idx, cfg in enumerate(loan_configs, start=1):
        c_idx, b_id, issue_days, due_days, ret_days = cfg
        c_id = copy_rows[c_idx][0]

        issue_dt = now - timedelta(days=issue_days)
        due_dt   = now - timedelta(days=due_days)
        ret_dt   = (now - timedelta(days=ret_days)) if ret_days is not None else None

        session.run("""
            CREATE (l:Loan {
                loan_id: $lid,
                copy_id: $cid,
```

```
0319:                         borrower_id: $bid,
0320:                         issued_by: $issby,
0321:                         issue_date: $issue_date,
0322:                         due_date: $due_date,
0323:                         return_date: $return_date
0324:                     })
0325:                 """, lid=loan_idx, cid=c_id, bid=b_id, issby=librarian_id,
0326:                      issue_date=issue_dt.strftime("%Y-%m-%d"),
0327:                      due_date=due_dt.strftime("%Y-%m-%d"),
0328:                      return_date=ret_dt.strftime("%Y-%m-%d") if ret_dt else None)
0329:
0330:                 session.run("""
0331:                     MATCH (u:User {user_id: $uid}), (l:Loan {loan_id: $lid})
0332:                     CREATE (u)-[:BORROWED]->(l)
0333:                 """, uid=b_id, lid=loan_idx)
0334:
0335:                 session.run("""
0336:                     MATCH (l:Loan {loan_id: $lid}), (cp:Copy {copy_id: $cid})
0337:                     CREATE (l)-[:FOR_COPY]->(cp)
0338:                 """, lid=loan_idx, cid=c_id)
0339:
0340:                 session.run("""
0341:                     MATCH (lib:User {user_id: $uid}), (l:Loan {loan_id: $lid})
0342:                     CREATE (lib)-[:ISSUED]->(l)
0343:                 """, uid=librarian_id, lid=loan_idx)
0344:
0345:                 if ret_dt is None:
0346:                     session.run("MATCH (cp:Copy {copy_id: $cid}) SET cp.status = 'Borrowed'", cid=c_id)
0347:
0348:                 loans_list.append((loan_idx, b_id, issue_dt, due_dt, ret_dt))
0349:
0350:             print(f" -> Seeded {len(loans_list)} loans.")
0351:
0352:             # ================================================================
0353:             # 8. SEED FINES (10 records)
0354:             # ================================================================
0355:             print("[8/9] Seeding Fines...")
0356:             fine_specs = [
0357:                 (3,  110.00, "Paid"),    # Loan #4 (Clean Code, returned 11 days late = Rs. 110 Paid)
0358:                 (4,   60.00, "Unpaid"),
0359:                 (5,  160.00, "Unpaid"),
0360:                 (9,   60.00, "Paid"),
0361:                 (10,  70.00, "Paid"),
0362:                 (11, 110.00, "Waived"),
0363:                 (6,   30.00, "Paid"),    # Loan #7 (AI, returned 3 days late = Rs. 30 Paid)
0364:                 (7,   40.00, "Unpaid"),
0365:                 (8,   30.00, "Paid"),
0366:                 (12,  50.00, "Paid"),    # Loan #13 (staff, returned 5 days late = Rs. 50 Paid)
0367:             ]
0368:
0369:             for fine_idx, spec in enumerate(fine_specs, start=1):
0370:                 loan_0idx, amount, status = spec
0371:                 l_id = loans_list[loan_0idx][0]
0372:                 l_ret_dt = loans_list[loan_0idx][4]
0373:                 pay_dt = (l_ret_dt.strftime("%Y-%m-%d") if l_ret_dt else "2026-10-17") if status == "Paid" e
      | lse None
0374:                 col_by = librarian_id if status == "Paid" else None
0375:
0376:                 session.run("""
0377:                     CREATE (f:Fine {
0378:                         fine_id: $fid,
0379:                         loan_id: $lid,
0380:                         fine_amount: $amt,
0381:                         payment_status: $status,
0382:                         payment_date: $pay_date,
0383:                         collected_by: $col_by
0384:                     })
0385:                 """, fid=fine_idx, lid=l_id, amt=amount, status=status,
0386:                      pay_date=pay_dt, col_by=col_by)
0387:
0388:                 session.run("""
0389:                     MATCH (l:Loan {loan_id: $lid}), (f:Fine {fine_id: $fid})
0390:                     CREATE (l)-[:HAS_FINE]->(f)
0391:                 """, lid=l_id, fid=fine_idx)
0392:
0393:             print(f" -> Seeded {len(fine_specs)} fines.")
0394:
0395:             # ================================================================
0396:             # 9. SEED LOAN REQUESTS (10 records)
0397:             # ================================================================
```

```
0398:             print("[9/9] Seeding Loan Requests...")
0399:             book_ids_list = list(book_id_map.values())
0400:
0401:             request_specs = [
0402:                 ("emily_stu",    0, "Pending",         2),
0403:                 ("marcus_stu",   1, "Pending",         1),
0404:                 ("alex_stu",     2, "Pending",         5),  # CLRS copies 5 & 6 already borrowed; re-synced
      | to Pending
0405:                 ("marcus_stu",   3, "Awaiting Pickup", 4),  # Eligible student in good standing
0406:                 ("rahul_stu",    4, "Fulfilled",       10),
0407:                 ("student",      5, "Fulfilled",       15),
0408:                 ("emily_stu",    6, "Rejected",         7),
0409:                 ("marcus_stu",   7, "Pending",          3),
0410:                 ("prof_turing",  8, "Fulfilled",        20),
0411:                 ("prof_shannon", 9, "Pending",           1),
0412:             ]
0413:
0414:             for req_idx, spec in enumerate(request_specs, start=1):
0415:                 uname, b_idx, status, days_ago = spec
0416:                 borrower_id = user_id_map.get(uname)
0417:                 b_id = book_ids_list[b_idx]
0418:                 req_date = now - timedelta(days=days_ago)
0419:
0420:                 # For Awaiting Pickup (e.g. Marcus REQ-4 for AI Book), assign available copy 8
0421:                 assigned_cid = 8 if req_idx == 4 and status == "Awaiting Pickup" else None
0422:
0423:                 session.run("""
0424:                     CREATE (r:LoanRequest {
0425:                         request_id: $rid,
0426:                         book_id: $bid,
0427:                         borrower_id: $borrower_id,
0428:                         copy_id: $cid,
0429:                         request_date: $rdate,
0430:                         status: $status
0431:                     })
0432:                 """, rid=req_idx, bid=b_id, borrower_id=borrower_id,
0433:                      cid=assigned_cid,
0434:                      rdate=req_date.strftime("%Y-%m-%d"), status=status)
0435:
0436:                 session.run("""
0437:                     MATCH (u:User {user_id: $uid}), (r:LoanRequest {request_id: $rid})
0438:                     CREATE (u)-[:REQUESTED]->(r)
0439:                 """, uid=borrower_id, rid=req_idx)
0440:
0441:                 session.run("""
0442:                     MATCH (r:LoanRequest {request_id: $rid}), (bk:Book {book_id: $bid})
0443:                     CREATE (r)-[:FOR_BOOK]->(bk)
0444:                 """, rid=req_idx, bid=b_id)
0445:
0446:                 if assigned_cid:
0447:                     session.run("""
0448:                         MATCH (cp:Copy {copy_id: $cid})
0449:                         SET cp.status = 'Reserved'
0450:                     """, cid=assigned_cid)
0451:                     session.run("""
0452:                         MATCH (r:LoanRequest {request_id: $rid}), (cp:Copy {copy_id: $cid})
0453:                         CREATE (r)-[:RESERVED_COPY]->(cp)
0454:                     """, rid=req_idx, cid=assigned_cid)
0455:
0456:             print(f" -> Seeded {len(request_specs)} loan requests.")
0457:
0458:             # ============================================================
0459:             # 10. SEED ACADEMIC PREREQUISITES ([:PREREQUISITE_FOR])
0460:             # ============================================================
0461:             print("\n[10/10] Seeding Academic Prerequisite Hierarchy ([:PREREQUISITE_FOR])...")
0462:             prereq_edges = [
0463:                 (2, 3, "Code Quality to Algorithms"),
0464:                 (3, 4, "Algorithms to Intelligent Systems"),
0465:                 (3, 9, "Algorithms to Theoretical Mastery"),
0466:                 (2, 8, "Clean Code to Domain Architecture"),
0467:                 (6, 7, "OS Internals to Network Protocols"),
0468:                 (7, 1, "Networking to Distributed Storage"),
0469:                 (5, 1, "Relational Theory to Distributed Scale"),
0470:             ]
0471:             for src_id, dst_id, jump in prereq_edges:
0472:                 session.run("""
0473:                     MATCH (src:Book {book_id: $src_id}), (dst:Book {book_id: $dst_id})
0474:                     MERGE (src)-[r:PREREQUISITE_FOR]->(dst)
0475:                     SET r.difficulty_jump = $jump
0476:                 """, src_id=src_id, dst_id=dst_id, jump=jump)
```

```
0477:             print(f" -> Seeded {len(prereq_edges)} prerequisite relationships.")
0478:
0479:             print("\n[VERIFICATION] Node and Relationship counts:")
0480:             for label in ["User", "Category", "Publisher", "Author", "Book", "Copy", "Loan", "Fine", "LoanRe
      | quest"]:
0481:                 result = session.run(f"MATCH (n:{label}) RETURN count(n) AS cnt")
0482:                 cnt = result.single()["cnt"]
0483:                 print(f"  Node {label}: {cnt}")
0484:
0485:             rel_res = session.run("MATCH ()-[r:PREREQUISITE_FOR]->() RETURN count(r) AS cnt")
0486:             print(f"  Relationship PREREQUISITE_FOR: {rel_res.single()['cnt']}")
0487:
0488:             print("\n[SUCCESS] All 9 entity types and prerequisite roadmap seeded into Neo4j AuraDB!")
0489:             print("\nLogin credentials:")
0490:             print("  admin       / Admin@123")
0491:             print("  librarian   / Lib@12345")
0492:             print("  staff       / Staff@123")
0493:             print("  student     / Student@1")
0494:
0495:         except Exception as e:
0496:             print(f"\n[ERROR] Seeding aborted: {e}")
0497:             raise
0498:         finally:
0499:             driver.close()
0500:
0501:
0502: if __name__ == "__main__":
0503:     run_seed()
```

MODULE SOURCE: requirements.txt

Lines: 18 | **Lang:** Pip Requirements | **Category:** Dependencies & Environment

Core Role & Purpose: Python package dependency manifest declaring Flask, neo4j, Flask-WTF, ReportLab, and supporting utilities.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify neo4j official driver dependency.
- Verify Flask and Flask-WTF for CSRF protection.
- Verify ReportLab for programmatic PDF audit generation.

```
0001: blinker==1.9.0
0002: cachelib==0.17.0
0003: cffi==2.1.1
0004: click==8.5.0
0005: cryptography==50.0.2
0006: Flask==3.1.3
0007: Flask-Session==0.8.0
0008: Flask-WTF==1.3.0
0009: WTForms==3.2.2
0010: itsdangerous==2.2.0
0011: Jinja2==3.1.6
0012: MarkupSafe==3.0.4
0013: msgspec==0.22.0
0014: neo4j>=5.0.0
0015: pytz>=2024.1
0016: pycparser==3.0
0017: typing_extensions==4.16.0
0018: Werkzeug==3.1.9
```

MODULE SOURCE: templates/base.html

Lines: 662 | **Lang:** HTML5 / Jinja2 / Vanilla CSS | **Category:** Master UI Shell & Navigation

Core Role & Purpose: Global master layout template providing the dark glassmorphic design system, responsive sidebar, topbar, role-based navigation scoping, notification toasts, and security headers.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify that sidebar navigation items are strictly role-gated (Manage Users visible only to Admin; Issue Book visible only to Admin/Librarian).
- Verify CSRF token meta tags or form hook availability.
- Verify breadcrumb and active page highlight logic.

```
0001: <!DOCTYPE html>
0002: <html lang="en">
0003: <head>
0004:     <meta charset="UTF-8">
0005:     <meta name="viewport" content="width=device-width, initial-scale=1.0">
0006:     <title>{% block title %}VITCC LIBRARY | Enterprise Library System{% endblock %}</title>
0007:     <!-- Modern Sans Font: Inter & Plus Jakarta Sans -->
0008:     <link rel="preconnect" href="https://fonts.googleapis.com">
0009:     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
0010:     <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&fami
      | ly=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
0011:
0012:     <style>
0013:         :root {
0014:             --bg-base: #0B0F19;
0015:             --bg-surface: #111827;
0016:             --bg-elevated: #1F2937;
0017:             --bg-card: rgba(17, 24, 39, 0.7);
0018:             --border-subtle: rgba(255, 255, 255, 0.08);
0019:             --border-hover: rgba(255, 255, 255, 0.16);
0020:             --accent-primary: #3B82F6;
0021:             --accent-primary-hover: #2563EB;
0022:             --accent-glow: rgba(59, 130, 246, 0.25);
0023:             --text-primary: #F9FAFB;
0024:             --text-secondary: #9CA3AF;
```

```
0025:             --text-muted: #6B7280;
0026:             --success-bg: rgba(16, 185, 129, 0.12);
0027:             --success-text: #34D399;
0028:             --success-border: rgba(16, 185, 129, 0.25);
0029:             --warning-bg: rgba(245, 158, 11, 0.12);
0030:             --warning-text: #FBBF24;
0031:             --warning-border: rgba(245, 158, 11, 0.25);
0032:             --danger-bg: rgba(239, 68, 68, 0.12);
0033:             --danger-text: #F87171;
0034:             --danger-border: rgba(239, 68, 68, 0.25);
0035:             --radius-sm: 6px;
0036:             --radius-md: 10px;
0037:             --radius-lg: 16px;
0038:             --radius-full: 9999px;
0039:             --sidebar-width: 260px;
0040:             --topbar-height: 70px;
0041:             --font-sans: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, sans-serif;
0042:             --font-mono: 'JetBrains Mono', monospace;
0043:         }
0044:
0045:         * {
0046:             box-sizing: border-box;
0047:             margin: 0;
0048:             padding: 0;
0049:         }
0050:
0051:         body {
0052:             font-family: var(--font-sans);
0053:             background-color: var(--bg-base);
0054:             color: var(--text-primary);
0055:             line-height: 1.5;
0056:             -webkit-font-smoothing: antialiased;
0057:             min-height: 100vh;
0058:             display: flex;
0059:         }
0060:
0061:         /* App Wrapper */
0062:         .app-layout {
0063:             display: flex;
0064:             width: 100%;
0065:             min-height: 100vh;
0066:         }
0067:
0068:         /* Sidebar Styling */
0069:         .sidebar {
0070:             width: var(--sidebar-width);
0071:             background: var(--bg-surface);
0072:             border-right: 1px solid var(--border-subtle);
0073:             display: flex;
0074:             flex-direction: column;
0075:             position: fixed;
0076:             top: 0;
0077:             bottom: 0;
0078:             left: 0;
0079:             z-index: 50;
```

```
            padding: 24px 16px;
        }

        .brand-container {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 0 12px 24px 12px;
            border-bottom: 1px solid var(--border-subtle);
            margin-bottom: 24px;
        }

        .brand-logo {
            width: 38px;
            height: 38px;
            background: linear-gradient(135deg, #3B82F6 0%, #1E40AF 100%);
            border-radius: var(--radius-md);
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 4px 12px var(--accent-glow);
        }

        .brand-logo svg {
            width: 20px;
            height: 20px;
            fill: #FFFFFF;
        }

        .brand-title {
            font-size: 1.15rem;
            font-weight: 700;
            letter-spacing: -0.02em;
            color: var(--text-primary);
        }

        /* Nav Links */
        .nav-section-title {
            font-size: 0.72rem;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 0.08em;
            color: var(--text-muted);
            padding: 8px 12px;
            margin-top: 12px;
        }

        .nav-list {
            list-style: none;
            display: flex;
            flex-direction: column;
            gap: 4px;
        }

        .nav-item a {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 10px 12px;
            color: var(--text-secondary);
            text-decoration: none;
            font-size: 0.9rem;
            font-weight: 500;
            border-radius: var(--radius-md);
            transition: all 0.15s ease-in-out;
        }

        .nav-item a:hover {
            background: rgba(255, 255, 255, 0.05);
            color: var(--text-primary);
        }

        .nav-item a.active {
            background: var(--accent-primary);
            color: #FFFFFF;
            font-weight: 600;
            box-shadow: 0 4px 14px var(--accent-glow);
        }

        .nav-item svg {
```

```
            width: 18px;
            height: 18px;
            stroke-width: 2;
            stroke: currentColor;
            fill: none;
        }

        /* Sidebar Footer / User Profile */
        .sidebar-footer {
            margin-top: auto;
            padding-top: 16px;
            border-top: 1px solid var(--border-subtle);
        }

        .user-card {
            display: flex;
            align-items: center;
            gap: 12px;
            padding: 10px;
            border-radius: var(--radius-md);
            background: rgba(255, 255, 255, 0.03);
            border: 1px solid var(--border-subtle);
        }

        .user-avatar {
            width: 36px;
            height: 36px;
            border-radius: var(--radius-full);
            background: linear-gradient(135deg, #4F46E5, #06B6D4);
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: 700;
            font-size: 0.85rem;
            color: #FFFFFF;
        }

        .user-info {
            flex: 1;
            overflow: hidden;
        }

        .user-name {
            font-size: 0.85rem;
            font-weight: 600;
            color: var(--text-primary);
            white-space: nowrap;
            overflow: hidden;
            text-overflow: ellipsis;
        }

        .user-role-badge {
            font-size: 0.7rem;
            color: var(--accent-primary);
            font-weight: 600;
        }

        .btn-logout {
            color: var(--text-muted);
            background: none;
            border: none;
            cursor: pointer;
            padding: 6px;
            border-radius: var(--radius-sm);
            transition: color 0.15s;
            display: flex;
            align-items: center;
            text-decoration: none;
        }

        .btn-logout:hover {
            color: var(--danger-text);
        }

        /* Main Workspace Content */
        .main-wrapper {
            margin-left: var(--sidebar-width);
            flex: 1;
            display: flex;
            flex-direction: column;
```

```
0240:             min-height: 100vh;
0241:         }
0242:
0243:         /* Top Navbar */
0244:         .topbar {
0245:             height: var(--topbar-height);
0246:             border-bottom: 1px solid var(--border-subtle);
0247:             background: rgba(11, 15, 25, 0.7);
0248:             backdrop-filter: blur(12px);
0249:             display: flex;
0250:             align-items: center;
0251:             justify-content: space-between;
0252:             padding: 0 32px;
0253:             position: sticky;
0254:             top: 0;
0255:             z-index: 40;
0256:         }
0257:
0258:         .topbar-breadcrumb {
0259:             display: flex;
0260:             align-items: center;
0261:             gap: 8px;
0262:             font-size: 0.85rem;
0263:             color: var(--text-muted);
0264:         }
0265:
0266:         .topbar-breadcrumb span.current {
0267:             color: var(--text-primary);
0268:             font-weight: 600;
0269:         }
0270:
0271:         .page-content {
0272:             padding: 32px;
0273:             flex: 1;
0274:         }
0275:
0276:         /* Alerts & Flash Messages */
0277:         .flash-messages {
0278:             margin-bottom: 24px;
0279:             display: flex;
0280:             flex-direction: column;
0281:             gap: 10px;
0282:         }
0283:
0284:         .flash-alert {
0285:             padding: 12px 16px;
0286:             padding-right: 1.5rem;
0287:             border-radius: var(--radius-md);
0288:             font-size: 0.9rem;
0289:             display: flex;
0290:             align-items: center;
0291:             justify-content: space-between;
0292:             border: 1px solid;
0293:             backdrop-filter: blur(8px);
0294:         }
0295:
0296:         .flash-alert.success {
0297:             background: var(--success-bg);
0298:             color: var(--success-text);
0299:             border-color: var(--success-border);
0300:         }
0301:
0302:         .flash-alert.danger, .flash-alert.error {
0303:             background: var(--danger-bg);
0304:             color: var(--danger-text);
0305:             border-color: var(--danger-border);
0306:         }
0307:
0308:         .flash-alert.warning {
0309:             background: var(--warning-bg);
0310:             color: var(--warning-text);
0311:             border-color: var(--warning-border);
0312:         }
0313:
0314:         .flash-alert.info {
0315:             background: rgba(59, 130, 246, 0.12);
0316:             color: var(--accent-primary);
0317:             border-color: rgba(59, 130, 246, 0.25);
0318:         }
0319:
```

```
0320:         /* Standard Buttons */
0321:         .btn {
0322:             display: inline-flex;
0323:             align-items: center;
0324:             justify-content: center;
0325:             gap: 8px;
0326:             padding: 10px 18px;
0327:             border-radius: var(--radius-md);
0328:             font-size: 0.88rem;
0329:             font-weight: 600;
0330:             text-decoration: none;
0331:             cursor: pointer;
0332:             transition: all 0.15s ease-in-out;
0333:             border: 1px solid transparent;
0334:         }
0335:
0336:         .btn-primary {
0337:             background: var(--accent-primary);
0338:             color: #FFFFFF;
0339:             box-shadow: 0 4px 12px var(--accent-glow);
0340:         }
0341:
0342:         .btn-primary:hover {
0343:             background: var(--accent-primary-hover);
0344:             transform: translateY(-1px);
0345:         }
0346:
0347:         .btn-secondary {
0348:             background: var(--bg-elevated);
0349:             color: var(--text-primary);
0350:             border: 1px solid var(--border-subtle);
0351:         }
0352:
0353:         .btn-secondary:hover {
0354:             background: rgba(255, 255, 255, 0.1);
0355:             border-color: var(--border-hover);
0356:         }
0357:
0358:         .badge {
0359:             display: inline-flex;
0360:             align-items: center;
0361:             padding: 4px 10px;
0362:             border-radius: var(--radius-full);
0363:             font-size: 0.75rem;
0364:             font-weight: 600;
0365:         }
0366:
0367:         .badge-success { background: var(--success-bg); color: var(--success-text); border: 1px solid var(--
      | success-border); }
0368:         .badge-warning { background: var(--warning-bg); color: var(--warning-text); border: 1px solid var(--
      | warning-border); }
0369:         .badge-danger  { background: var(--danger-bg);  color: var(--danger-text);  border: 1px solid var(--
      | danger-border); }
0370:         .badge-info    { background: rgba(59, 130, 246, 0.12); color: var(--accent-primary); border: 1px sol
      | id rgba(59, 130, 246, 0.25); }
0371:
0372:         /* Topbar Graph & AI Action Buttons */
0373:         .topbar-actions {
0374:             display: flex;
0375:             align-items: center;
0376:             gap: 12px;
0377:         }
0378:
0379:         .topbar-nav-btn {
0380:             display: inline-flex;
0381:             align-items: center;
0382:             gap: 8px;
0383:             padding: 7px 15px;
0384:             border-radius: var(--radius-full);
0385:             font-size: 0.82rem;
0386:             font-weight: 600;
0387:             text-decoration: none;
0388:             transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
0389:             border: 1px solid transparent;
0390:             cursor: pointer;
0391:         }
0392:
0393:         .btn-ai-recs {
0394:             background: linear-gradient(135deg, rgba(139, 92, 246, 0.15), rgba(168, 85, 247, 0.1));
0395:             color: #C084FC;
```

```
0396:             border-color: rgba(139, 92, 246, 0.35);
0397:         }
0398:
0399:         .btn-ai-recs:hover {
0400:             background: linear-gradient(135deg, rgba(139, 92, 246, 0.3), rgba(168, 85, 247, 0.2));
0401:             color: #F3E8FF;
0402:             border-color: rgba(139, 92, 246, 0.7);
0403:             box-shadow: 0 0 16px rgba(168, 85, 247, 0.35);
0404:             transform: translateY(-1px);
0405:         }
0406:
0407:         .btn-graph-explorer {
0408:             background: linear-gradient(135deg, rgba(6, 182, 212, 0.15), rgba(59, 130, 246, 0.1));
0409:             color: #22D3EE;
0410:             border-color: rgba(6, 182, 212, 0.35);
0411:         }
0412:
0413:         .btn-graph-explorer:hover {
0414:             background: linear-gradient(135deg, rgba(6, 182, 212, 0.3), rgba(59, 130, 246, 0.2));
0415:             color: #E0F2FE;
0416:             border-color: rgba(6, 182, 212, 0.7);
0417:             box-shadow: 0 0 16px rgba(6, 182, 212, 0.35);
0418:             transform: translateY(-1px);
0419:         }
0420:
0421:         /* Responsive */
0422:         @media (max-width: 900px) {
0423:             .sidebar {
0424:                 transform: translateX(-100%);
0425:                 transition: transform 0.3s ease;
0426:             }
0427:             .sidebar.open {
0428:                 transform: translateX(0);
0429:             }
0430:             .main-wrapper {
0431:                 margin-left: 0;
0432:             }
0433:         }
0434:     </style>
0435:     {% block extra_css %}{% endblock %}
0436: </head>
0437: <body>
0438:     <div class="app-layout">
0439:         <!-- Sidebar Navigation -->
0440:         {% if session.get('user_id') %}
0441:         <aside class="sidebar">
0442:             <div class="brand-container">
0443:                 <div class="brand-logo">
0444:                     <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4
      | 5.5v14z"></path></svg>
0445:                 </div>
0446:                 <div>
0447:                     <div class="brand-title">VITCC LIBRARY</div>
0448:                 </div>
0449:             </div>
0450:
0451:             <!-- Role-Based Navigation Links -->
0452:             <div class="nav-section-title">Workspace</div>
0453:             <ul class="nav-list">
0454:                 <!-- 1. Admin & Librarian Navigation -->
0455:                 {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Lib
      | rarian'] %}
0456:                     <li class="nav-item">
0457:                         <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}
      | active{% endif %}">
0458:                             <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="
      | 14" y="3" width="7" height="7"></rect><rect x="14" y="14" width="7" height="7"></rect><rect x="3" y="1
      | 4" width="7" height="7"></rect></svg>
0459:                             Dashboard
0460:                         </a>
0461:                     </li>
0462:                     <li class="nav-item">
0463:                         <a href="{{ url_for('books') }}" class="{% if request.endpoint in ['books', 'categor
      | y_books', 'add_book', 'edit_book'] %}active{% endif %}">
0464:                             <svg viewBox="0 0 24 24"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5
      | 2.5 0 0 1-2.5-2.5Z"></path><path d="M6 6h10"></path><path d="M6 10h10"></path></svg>
0465:                             Catalog
0466:                         </a>
0467:                     </li>
0468:                     <li class="nav-item">
```

```
0469:                         <a href="{{ url_for('issue_loan') }}" class="{% if request.endpoint == 'issue_loan'
      | %}active{% endif %}">
0470:                             <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="10"></circle><polyline point
      | s="12 6 12 12 14 14"></polyline></svg>
0471:                             Issue Book
0472:                         </a>
0473:                     </li>
0474:                     <li class="nav-item">
0475:                         <a href="{{ url_for('loan_requests') }}" class="{% if request.endpoint == 'loan_requ
      | ests' %}active{% endif %}">
0476:                             <svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0
      |  0 1-2-2V6a2 2 0 0 1 2-2h2"></path><rect x="8" y="2" width="8" height="4" rx="1" ry="1"></rect><path d
      | ="M9 14l2 2 4-4"></path></svg>
0477:                             Loan Requests
0478:                         </a>
0479:                     </li>
0480:                     {% if session.get('role') == 'Admin' or session.get('Role') == 'Admin' %}
0481:                     <li class="nav-item">
0482:                         <a href="{{ url_for('manage_users') }}" class="{% if request.endpoint == 'manage_use
      | rs' %}active{% endif %}" title="Administrative Access">
0483:                             <svg viewBox="0 0 24 24"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></p
      | ath><circle cx="9" cy="7" r="4"></circle><path d="M22 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13
      | a4 4 0 0 1 0 7.75"></path></svg>
0484:                             Manage Users
0485:                         </a>
0486:                     </li>
0487:                     {% endif %}
0488:                     <li class="nav-item">
0489:                         <a href="{{ url_for('learning_paths') }}" class="{% if request.endpoint == 'learning
      | _paths' %}active{% endif %}" title="Academic Learning Roadmap & Prerequisite Hierarchy">
0490:                             <svg viewBox="0 0 24 24"><polygon points="12 2 2 7 12 12 22 7 12 2"></polygon><p
      | olyline points="2 17 12 22 22 17"></polyline><polyline points="2 12 12 17 22 12"></polyline></svg>
0491:                             Learning Roadmaps
0492:                         </a>
0493:                     </li>
0494:                     <li class="nav-item">
0495:                         <a href="{{ url_for('recommendations') }}" class="{% if request.endpoint == 'recomme
      | ndations' %}active{% endif %}" title="Collaborative Filtering Book Recommendations">
0496:                             <svg viewBox="0 0 24 24"><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18
      |  21.02 12 17.77 5.82 21.02 7 14.14 2 9.27 8.91 8.26 12 2"></polygon></svg>
0497:                             AI Recommendations
0498:                         </a>
0499:                     </li>
0500:
0501:                 <!-- 2. Staff Navigation -->
0502:                 {% elif session.get('role') == 'Staff' or session.get('Role') == 'Staff' %}
0503:                     <li class="nav-item">
0504:                         <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}
      | active{% endif %}">
0505:                             <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="
      | 14" y="3" width="7" height="7"></rect><rect x="14" y="14" width="7" height="7"></rect><rect x="3" y="1
      | 4" width="7" height="7"></rect></svg>
0506:                             Dashboard
0507:                         </a>
0508:                     </li>
0509:                     <li class="nav-item">
0510:                         <a href="{{ url_for('books') }}" class="{% if request.endpoint in ['books', 'categor
      | y_books', 'add_book', 'edit_book'] %}active{% endif %}">
0511:                             <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1
      | ="21" x2="16.65" y2="16.65"></line></svg>
0512:                             Search Books
0513:                         </a>
0514:                     </li>
0515:                     <li class="nav-item">
0516:                         <a href="{{ url_for('learning_paths') }}" class="{% if request.endpoint == 'learning
      | _paths' %}active{% endif %}" title="Academic Learning Roadmap & Prerequisite Hierarchy">
0517:                             <svg viewBox="0 0 24 24"><polygon points="12 2 2 7 12 12 22 7 12 2"></polygon><p
      | olyline points="2 17 12 22 22 17"></polyline><polyline points="2 12 12 17 22 12"></polyline></svg>
0518:                             Learning Roadmaps
0519:                         </a>
0520:                     </li>
0521:                     <li class="nav-item">
0522:                         <a href="{{ url_for('recommendations') }}" class="{% if request.endpoint == 'recomme
      | ndations' %}active{% endif %}" title="Collaborative Filtering Book Recommendations">
0523:                             <svg viewBox="0 0 24 24"><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18
      |  21.02 12 17.77 5.82 21.02 7 14.14 2 9.27 8.91 8.26 12 2"></polygon></svg>
0524:                             Recommended For You
0525:                         </a>
0526:                     </li>
0527:                     <li class="nav-item">
```

```
0528:                         <a href="{{ url_for('my_fines') }}" class="{% if request.endpoint == 'my_fines' %}ac
      | tive{% endif %}">
0529:                             <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M1
      | 7 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 0 1 0 7H6"></path></svg>
0530:                             My Fines
0531:                         </a>
0532:                     </li>
0533:
0534:                 <!-- 3. Student Navigation -->
0535:                 {% elif session.get('role') == 'Student' or session.get('Role') == 'Student' %}
0536:                     <li class="nav-item">
0537:                         <a href="{{ url_for('dashboard') }}" class="{% if request.endpoint == 'dashboard' %}
      | active{% endif %}">
0538:                             <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"></rect><rect x="
      | 14" y="3" width="7" height="7"></rect><rect x="14" y="14" width="7" height="7"></rect><rect x="3" y="1
      | 4" width="7" height="7"></rect></svg>
0539:                             My Dashboard
0540:                         </a>
0541:                     </li>
0542:                     <li class="nav-item">
0543:                         <a href="{{ url_for('books') }}" class="{% if request.endpoint in ['books', 'categor
      | y_books', 'add_book', 'edit_book'] %}active{% endif %}">
0544:                             <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1
      | ="21" x2="16.65" y2="16.65"></line></svg>
0545:                             Search Books
0546:                         </a>
0547:                     </li>
0548:                     <li class="nav-item">
0549:                         <a href="{{ url_for('learning_paths') }}" class="{% if request.endpoint == 'learning
      | _paths' %}active{% endif %}" title="Academic Learning Roadmap & Prerequisite Hierarchy">
0550:                             <svg viewBox="0 0 24 24"><polygon points="12 2 2 7 12 12 22 7 12 2"></polygon><p
      | olyline points="2 17 12 22 22 17"></polyline><polyline points="2 12 12 17 22 12"></polyline></svg>
0551:                             Learning Roadmaps
0552:                         </a>
0553:                     </li>
0554:                     <li class="nav-item">
0555:                         <a href="{{ url_for('recommendations') }}" class="{% if request.endpoint == 'recomme
      | ndations' %}active{% endif %}" title="Collaborative Filtering Book Recommendations">
0556:                             <svg viewBox="0 0 24 24"><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18
      |  21.02 12 17.77 5.82 21.02 7 14.14 2 9.27 8.91 8.26 12 2"></polygon></svg>
0557:                             Recommended For You
0558:                         </a>
0559:                     </li>
0560:                     <li class="nav-item">
0561:                         <a href="{{ url_for('my_fines') }}" class="{% if request.endpoint == 'my_fines' %}ac
      | tive{% endif %}">
0562:                             <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M1
      | 7 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 0 1 0 7H6"></path></svg>
0563:                             My Fines
0564:                         </a>
0565:                     </li>
0566:                 {% endif %}
0567:             </ul>
0568:
0569:             <!-- NEO4J ENGINE LAB Section (Subtle, Clearly Separated at the Bottom) -->
0570:             <div class="nav-section-title" style="margin-top: 22px; display: flex; align-items: center; just
      | ify-content: space-between;">
0571:                 <span>NEO4J ENGINE LAB</span>
0572:                 <span style="font-size: 0.62rem; background: rgba(59, 130, 246, 0.2); color: #93C5FD; paddin
      | g: 1px 6px; border-radius: 9999px; letter-spacing: 0.05em;">PROOF</span>
0573:             </div>
0574:             <ul class="nav-list">
0575:                 <li class="nav-item">
0576:                     <a href="{{ url_for('graph_visualizer') }}" class="{% if request.endpoint == 'graph_visu
      | alizer' %}active{% endif %}" title="Interactive Knowledge Graph Explorer">
0577:                         <svg viewBox="0 0 24 24"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12
      | " r="3"></circle><circle cx="18" cy="19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.4
      | 9"></line><line x1="15.41" y1="6.51" x2="8.59" y2="10.49"></line></svg>
0578:                         Live Graph Explorer
0579:                     </a>
0580:                 </li>
0581:                 <li class="nav-item">
0582:                     <a href="{{ url_for('book_bridge') }}" class="{% if request.endpoint == 'book_bridge' %}
      | active{% endif %}" title="Shortest Path Degrees of Separation Between Books">
0583:                         <svg viewBox="0 0 24 24"><path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1
      |  2-2h6"></path><polyline points="15 3 21 3 21 9"></polyline><line x1="10" y1="14" x2="21" y2="3"></lin
      | e></svg>
0584:                         Book Bridge
0585:                     </a>
0586:                 </li>
```

```
0587:                 {% if session.get('role') in ['Admin', 'Librarian', 'Staff'] %}
0588:                 <li class="nav-item">
0589:                     <a href="{{ url_for('library_analytics') }}" class="{% if request.endpoint == 'library_a
      | nalytics' %}active{% endif %}" title="Author Centrality & Graph Influencer Analytics">
0590:                         <svg viewBox="0 0 24 24"><line x1="18" y1="20" x2="18" y2="10"></line><line x1="12"
      | y1="20" x2="12" y2="4"></line><line x1="6" y1="20" x2="6" y2="14"></line></svg>
0591:                         Centrality Analytics
0592:                     </a>
0593:                 </li>
0594:                 {% endif %}
0595:
0596:                 <li class="nav-item">
0597:                     <a href="{{ url_for('change_password') }}" class="{% if request.endpoint == 'change_pass
      | word' %}active{% endif %}" title="Update account credentials">
0598:                         <svg viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></r
      | ect><path d="M7 11V7a5 5 0 0 1 10 0v4"></path></svg>
0599:                         Change Password
0600:                     </a>
0601:                 </li>
0602:             </ul>
0603:
0604:             <!-- Authenticated User Profile Summary -->
0605:             <div class="sidebar-footer">
0606:                 <div class="user-card">
0607:                     <div class="user-avatar">
0608:                         {{ session.get('username', 'U')[0]|upper }}
0609:                     </div>
0610:                     <div class="user-info">
0611:                         <div class="user-name">{{ session.get('username') }}</div>
0612:                         <div class="user-role-badge">{{ session.get('role') }}</div>
0613:                     </div>
0614:                     <a href="{{ url_for('change_password') }}" class="btn-logout" title="Change Password" st
      | yle="color: var(--text-muted); margin-right: 2px;">
0615:                         <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none" st
      | roke-width="2"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></rect><path d="M7 11V7a5 5 0 0
      |  1 10 0v4"></path></svg>
0616:                     </a>
0617:                     <a href="{{ url_for('logout') }}" class="btn-logout" title="Sign Out">
0618:                         <svg viewBox="0 0 24 24" width="18" height="18" stroke="currentColor" fill="none" st
      | roke-width="2"><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4"></path><polyline points="16 17 21 12
      | 16 7"></polyline><line x1="21" y1="12" x2="9" y2="12"></line></svg>
0619:                     </a>
0620:                 </div>
0621:             </div>
0622:         </aside>
0623:         {% endif %}
0624:
0625:         <!-- Main Workspace Area -->
0626:         <div class="main-wrapper" {% if not session.get('user_id') %}style="margin-left: 0;"{% endif %}>
0627:             {% if session.get('user_id') %}
0628:             <header class="topbar">
0629:                 <div class="topbar-breadcrumb">
0630:                     <span>VITCC LIBRARY</span>
0631:                     <span>/</span>
0632:                     <span class="current">{% block breadcrumb %}Workspace{% endblock %}</span>
0633:                 </div>
0634:                 <div class="topbar-actions">
0635:                     <span class="badge badge-info">{{ session.get('role') }} Portal</span>
0636:                 </div>
0637:             </header>
0638:             {% endif %}
0639:
0640:             <main class="page-content">
0641:                 <!-- Flash Notification Banner Stack -->
0642:                 {% with messages = get_flashed_messages(with_categories=true) %}
0643:                     {% if messages %}
0644:                         <div class="flash-messages">
0645:                             {% for category, message in messages %}
0646:                                 <div class="flash-alert {{ category }}">
0647:                                     <span>{{ message }}</span>
0648:                                     <span style="cursor: pointer;" onclick="this.parentElement.remove();">&t
      | imes;</span>
0649:                                 </div>
0650:                             {% endfor %}
0651:                         </div>
0652:                     {% endif %}
0653:                 {% endwith %}
0654:
0655:                 {% block content %}{% endblock %}
0656:             </main>
```

```
        </div>
    </div>

    {% block extra_js %}{% endblock %}
</body>
</html>
```

MODULE SOURCE: templates/manage_users.html

Lines: 351 | **Lang:** HTML5 / Jinja2 | **Category:** User Administration & RBAC

Core Role & Purpose: Admin user management interface providing user listing, role status toggles, user suspension, user deletion, and Librarian password reset modal.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify that Admin rows display NO delete, suspend, or password reset buttons (hardened against peer admin tampering).
- Verify that Librarian rows DO render the 'Reset Password' button triggering the reset modal.
- Verify password reset modal form POSTs to /reset_user_password with CSRF token.

```
{% extends "base.html" %}

{% block title %}User Management | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}User Management{% endblock %}

{% block extra_css %}
<style>
    .users-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 28px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .users-title {
        font-size: 1.6rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
    }

    .users-sub {
        font-size: 0.88rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    .form-panel {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px;
        margin-bottom: 32px;
    }

    .form-grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
        gap: 16px;
        align-items: flex-end;
    }

    .status-btn {
        background: none;
        border: 1px solid;
        border-radius: var(--radius-sm);
        padding: 4px 10px;
        font-size: 0.75rem;
        font-weight: 600;
        cursor: pointer;
        transition: all 0.15s ease;
    }

    .status-btn.suspend {
        color: var(--danger-text);
        border-color: var(--danger-border);
        background: var(--danger-bg);
    }

    .status-btn.suspend:hover {
        background: rgba(239, 68, 68, 0.25);
    }

    .status-btn.activate {
        color: var(--success-text);
        border-color: var(--success-border);
        background: var(--success-bg);
    }

    .status-btn.activate:hover {
        background: rgba(16, 185, 129, 0.25);
    }

    .btn-delete-account {
        display: inline-flex;
        align-items: center;
        gap: 6px;
        background: rgba(239, 68, 68, 0.14);
```

```
0081:         color: #F87171;
0082:         border: 1px solid rgba(239, 68, 68, 0.35);
0083:         border-radius: var(--radius-sm);
0084:         padding: 5px 12px;
0085:         font-size: 0.78rem;
0086:         font-weight: 700;
0087:         cursor: pointer;
0088:         transition: all 0.15s ease;
0089:     }
0090:
0091:     .btn-delete-account:hover {
0092:         background: #DC2626;
0093:         color: #FFFFFF;
0094:         border-color: #DC2626;
0095:         box-shadow: 0 2px 8px rgba(220, 38, 38, 0.4);
0096:         transform: translateY(-1px);
0097:     }
0098:
0099:     .form-input,
0100:     input[type="text"].form-input,
0101:     input[type="email"].form-input,
0102:     input[type="password"].form-input,
0103:     select.form-input {
0104:         background: #0f172a !important;
0105:         color: #f8fafc !important;
0106:         border: 1px solid #334155 !important;
0107:         border-radius: var(--radius-md) !important;
0108:         padding: 0.65rem 0.85rem !important;
0109:         width: 100% !important;
0110:         box-sizing: border-box !important;
0111:         font-size: 0.88rem;
0112:         outline: none;
0113:         transition: all 0.15s ease-in-out;
0114:     }
0115:
0116:     .form-input:focus {
0117:         border-color: #3b82f6 !important;
0118:         box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.25) !important;
0119:     }
0120:
0121:     select.form-input option {
0122:         background: #0f172a;
0123:         color: #f8fafc;
0124:     }
0125:
0126:     .table-container {
0127:         width: 100%;
0128:         overflow-x: auto;
0129:     }
0130:
0131:     .data-table {
0132:         width: 100%;
0133:         min-width: 800px;
0134:         border-collapse: collapse;
0135:         text-align: left;
0136:         font-size: 0.88rem;
0137:     }
0138:
0139:     .data-table th {
0140:         background: rgba(255, 255, 255, 0.02);
0141:         color: var(--text-muted);
0142:         font-size: 0.72rem;
0143:         text-transform: uppercase;
0144:         letter-spacing: 0.06em;
0145:         font-weight: 600;
0146:         padding: 0.85rem 1rem;
0147:         border-bottom: 1px solid var(--border-subtle);
0148:         white-space: nowrap;
0149:     }
0150:
0151:     .data-table td {
0152:         padding: 0.85rem 1rem;
0153:         border-bottom: 1px solid var(--border-subtle);
0154:         color: var(--text-secondary);
0155:         vertical-align: middle;
0156:     }
0157:
0158:     .data-table tr:hover td {
0159:         background: rgba(255, 255, 255, 0.015);
0160:         color: var(--text-primary);
```

```
0161:     }
0162:
0163:     .btn-reset-pw {
0164:         display: inline-flex;
0165:         align-items: center;
0166:         gap: 5px;
0167:         background: rgba(59, 130, 246, 0.12);
0168:         color: #60A5FA;
0169:         border: 1px solid rgba(59, 130, 246, 0.3);
0170:         border-radius: var(--radius-sm);
0171:         padding: 4px 10px;
0172:         font-size: 0.75rem;
0173:         font-weight: 600;
0174:         cursor: pointer;
0175:         transition: all 0.15s ease;
0176:     }
0177:
0178:     .btn-reset-pw:hover {
0179:         background: #2563EB;
0180:         color: #FFFFFF;
0181:         border-color: #2563EB;
0182:     }
0183: </style>
0184: {% endblock %}
0185:
0186: {% block content %}
0187: <div class="users-header">
0188:     <div>
0189:         <h1 class="users-title">User Account Administration</h1>
0190:         <p class="users-sub">Manage institutional patrons, role-based authorization, and account statuses</p
      | >
0191:     </div>
0192: </div>
0193:
0194: <!-- Provision New User Form -->
0195: <div class="form-panel">
0196:     <h2 style="font-size: 1.05rem; font-weight: 700; margin-bottom: 16px; color: var(--text-primary); displa
      | y: flex; align-items: center; gap: 8px;">
0197:         <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" stroke-wi
      | dth="2"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></circl
      | e><line x1="19" y1="8" x2="19" y2="14"></line><line x1="22" y1="11" x2="16" y2="11"></line></svg>
0198:         Provision New System User
0199:     </h2>
0200:
0201:     <!-- Policy info banner -->
0202:     <div style="background: rgba(30, 41, 59, 0.6); border: 1px solid rgba(59, 130, 246, 0.25); border-radius
      | : var(--radius-md); padding: 10px 14px; margin-bottom: 14px; font-size: 0.77rem; color: var(--text-sec
      | ondary); display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 8px
      | ;">
0203:         <div>
0204:             <strong style="color: #93C5FD;">Credential Requirements:</strong>
0205:             Email must contain <strong>'@'</strong> and <strong>'.'</strong>.
0206:             Password must be <strong>8 to 64 characters</strong> with at least <strong>1 uppercase</strong>,
      |  <strong>1 lowercase</strong>, <strong>1 digit</strong>, and <strong>1 special character</strong>.
0207:         </div>
0208:     </div>
0209:
0210:     <form method="POST" action="{{ url_for('manage_users') }}">
0211:         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0212:         <input type="hidden" name="action" value="add_user">
0213:         <div class="form-grid">
0214:             <div class="form-group" style="margin-bottom: 0;">
0215:                 <label class="form-label" for="username">Username *</label>
0216:                 <input type="text" id="username" name="username" class="form-input" placeholder="e.g. jdoe"
      | required>
0217:             </div>
0218:
0219:             <div class="form-group" style="margin-bottom: 0;">
0220:                 <label class="form-label" for="email">Institutional Email *</label>
0221:                 <input type="email" id="email" name="email" class="form-input" placeholder="jdoe@vit.edu" re
      | quired>
0222:             </div>
0223:
0224:             <div class="form-group" style="margin-bottom: 0;">
0225:                 <label class="form-label" for="password">Initial Password *</label>
0226:                 <input type="password" id="password" name="password" class="form-input" placeholder="•••••••
      | •••••" minlength="8" maxlength="64" required>
0227:             </div>
0228:
0229:             <div class="form-group" style="margin-bottom: 0;">
```

```
0230:                 <label class="form-label" for="role">Assign Role *</label>
0231:                 <select id="role" name="role" class="form-input" style="cursor: pointer;">
0232:                     <option value="Student">Student</option>
0233:                     <option value="Staff">Staff</option>
0234:                     <option value="Librarian">Librarian</option>
0235:                     <option value="Admin">Admin</option>
0236:                 </select>
0237:             </div>
0238:
0239:             <div>
0240:                 <button type="submit" class="btn btn-primary" style="width: 100%; padding: 11px;">
0241:                     + Create User
0242:                 </button>
0243:             </div>
0244:         </div>
0245:     </form>
0246: </div>
0247:
0248: <!-- Registered Users Data Table -->
0249: <div class="data-panel">
0250:     <div class="panel-header">
0251:         <span class="panel-title">Active System Users</span>
0252:         <span class="badge badge-info">{{ users|length if users else 0 }} Registered</span>
0253:     </div>
0254:
0255:     <div class="table-container">
0256:         <table class="data-table">
0257:             <thead>
0258:                 <tr>
0259:                     <th>User ID</th>
0260:                     <th>Username</th>
0261:                     <th>Email Address</th>
0262:                     <th>Role</th>
0263:                     <th>Status</th>
0264:                     <th style="text-align: right;">Action</th>
0265:                 </tr>
0266:             </thead>
0267:             <tbody>
0268:                 {% if users %}
0269:                     {% for u in users %}
0270:                     <tr>
0271:                         <td style="font-family: var(--font-mono); font-size: 0.8rem; white-space: nowrap;">#
      | USR-{{ u.user_id }}</td>
0272:                         <td style="color: var(--text-primary); font-weight: 600; white-space: nowrap;">{{ u.
      | username }}</td>
0273:                         <td>{{ u.email }}</td>
0274:                         <td>
0275:                             {% if u.role == 'Admin' %}
0276:                                 <span class="badge badge-info">Admin</span>
0277:                             {% elif u.role == 'Librarian' %}
0278:                                 <span class="badge badge-warning">Librarian</span>
0279:                             {% elif u.role == 'Staff' %}
0280:                                 <span class="badge" style="background: rgba(168, 85, 247, 0.12); color: #C08
      | 4FC; border: 1px solid rgba(168, 85, 247, 0.25);">Staff</span>
0281:                             {% else %}
0282:                                 <span class="badge" style="background: rgba(255, 255, 255, 0.05); color: var
      | (--text-secondary); border: 1px solid var(--border-subtle);">Student</span>
0283:                             {% endif %}
0284:                         </td>
0285:                         <td>
0286:                             {% if u.account_status == 'Active' %}
0287:                                 <span class="badge badge-success">Active</span>
0288:                             {% else %}
0289:                                 <span class="badge badge-danger">Suspended</span>
0290:                             {% endif %}
0291:                         </td>
0292:                         <td style="text-align: right;">
0293:                             <div style="display: flex; gap: 8px; justify-content: flex-end; align-items: cen
      | ter; flex-wrap: wrap;">
0294:                                 <!-- Reset Password action button for Student/Staff/Librarian rows -->
0295:                                 {% if u.role in ['Student', 'Staff', 'Librarian'] %}
0296:                                 <form method="POST" action="{{ url_for('manage_users') }}" style="display: i
      | nline;">
0297:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0298:                                     <input type="hidden" name="action" value="reset_password">
0299:                                     <input type="hidden" name="user_id" value="{{ u.user_id }}">
0300:                                     <button type="submit" class="btn-reset-pw" title="Reset password for {{
      | u.username }}" onclick="return confirm('Reset password for {{ u.username }} (#{{ u.user_id }}) to temp
      | orary default?');">
0301:                                         <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="
```

```
      | currentColor" stroke-width="2"><path d="M21 2l-2 2m-6 6l7-7M3 10a7 7 0 1 0 14 0 7 7 0 1 0-14 0z"></pat
      | h></svg>
0302:                                       Reset Password
0303:                                   </button>
0304:                               </form>
0305:                               {% endif %}
0306:
0307:                               <!-- Admin Protection from Suspension and Deletion -->
0308:                               {% if u.role == 'Admin' %}
0309:                                   <span class="badge badge-info" style="background: rgba(59, 130, 246, 0.1
      | 5); color: #93c5fd; border: 1px solid rgba(59, 130, 246, 0.35); font-size: 0.75rem; padding: 5px 10px;
      | " title="Admin account is protected by system policy from suspension and deletion.">
0310:                                       Protected (Admin)
0311:                                   </span>
0312:                               {% else %}
0313:                               <form method="POST" action="{{ url_for('manage_users') }}" style="display: i
      | nline;">
0314:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0315:                                   <input type="hidden" name="action" value="toggle_status">
0316:                                   <input type="hidden" name="user_id" value="{{ u.user_id }}">
0317:                                   <input type="hidden" name="current_status" value="{{ u.account_status }}
      | ">
0318:                                   {% if u.account_status == 'Active' %}
0319:                                       <button type="submit" class="status-btn suspend" title="Suspend acco
      | unt access">
0320:                                           Suspend
0321:                                       </button>
0322:                                   {% else %}
0323:                                       <button type="submit" class="status-btn activate" title="Reactivate
      | account">
0324:                                           Activate
0325:                                       </button>
0326:                                   {% endif %}
0327:                               </form>
0328:
0329:                               <!-- Highly Visible Styled Delete Account Button -->
0330:                               <form method="POST" action="{{ url_for('delete_user', user_id=u.user_id) }}"
      |  style="display: inline;" onsubmit="return confirm('WARNING: Are you sure you want to permanently dele
      | te user {{ u.username }} (#{{ u.user_id }})? This action cannot be undone.');">
0331:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0332:                                   <button type="submit" class="btn-delete-account" title="Permanently dele
      | te user account">
0333:                                       <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6v14a2 2 0 0 1-
      | 2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></path></svg>
0334:                                       Delete Account
0335:                                   </button>
0336:                               </form>
0337:                               {% endif %}
0338:                           </div>
0339:                       </td>
0340:                   </tr>
0341:                   {% endfor %}
0342:               {% else %}
0343:                   <tr>
0344:                       <td colspan="6" class="empty-state">No users registered in database.</td>
0345:                   </tr>
0346:               {% endif %}
0347:           </tbody>
0348:       </table>
0349:     </div>
0350: </div>
0351: {% endblock %}
```

MODULE SOURCE: templates/dashboard.html

Lines: 961 | **Lang:** HTML5 / Jinja2 | **Category:** Circulation & Core Dashboard

Core Role & Purpose: Central role-scoped dashboard rendering disciplinary liabilities, borrowing statistics, quick actions, and recent activity tables.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:

- Verify that the liabilities card displays 'Disciplinary Liabilities' and 'Disciplinary Fines' without the word 'outstanding'.
- Verify role-conditional rendering of action buttons (e.g. Issue Book for Librarians, Request Book for Students).

```
0001: {% extends "base.html" %}
0002: {% block title %}Dashboard | VITCC LIBRARY{% endblock %}
0003: {% block breadcrumb %}Dashboard{% endblock %}
0004: {% block extra_css %}
0005: <style>
0006:     /* Welcome Header */
0007:     .dashboard-header {
0008:         display: flex;
0009:         align-items: center;
0010:         justify-content: space-between;
0011:         margin-bottom: 32px;
0012:         flex-wrap: wrap;
0013:         gap: 16px;
0014:     }
0015:     .welcome-title {
0016:         font-size: 1.75rem;
0017:         font-weight: 800;
0018:         letter-spacing: -0.03em;
0019:         color: var(--text-primary);
0020:     }
0021:     .welcome-sub {
0022:         font-size: 0.92rem;
0023:         color: var(--text-secondary);
0024:         margin-top: 4px;
0025:     }
0026:     .header-actions {
0027:         display: flex;
0028:         gap: 12px;
0029:     }
```

```
0030:     /* Bento Grid Stat Cards */
0031:     .stat-grid {
0032:         display: grid;
0033:         grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
0034:         gap: 20px;
0035:         margin-bottom: 36px;
0036:     }
0037:     .stat-card {
0038:         background: var(--bg-surface);
0039:         border: 1px solid var(--border-subtle);
0040:         border-radius: var(--radius-lg);
0041:         padding: 24px;
0042:         display: flex;
0043:         flex-direction: column;
0044:         justify-content: space-between;
0045:         position: relative;
0046:         overflow: hidden;
0047:         transition: transform 0.2s ease, border-color 0.2s ease;
0048:     }
0049:     .stat-card:hover {
0050:         transform: translateY(-2px);
0051:         border-color: var(--border-hover);
0052:     }
0053:     .stat-top {
0054:         display: flex;
0055:         align-items: center;
0056:         justify-content: space-between;
0057:         margin-bottom: 16px;
0058:     }
0059:     .stat-label {
0060:         font-size: 0.8rem;
0061:         font-weight: 600;
0062:         text-transform: uppercase;
0063:         letter-spacing: 0.05em;
0064:         color: var(--text-muted);
0065:     }
0066:     .stat-icon {
0067:         width: 36px;
0068:         height: 36px;
0069:         border-radius: var(--radius-md);
0070:         display: flex;
0071:         align-items: center;
0072:         justify-content: center;
0073:     }
0074:     .stat-icon svg {
0075:         width: 18px;
0076:         height: 18px;
0077:         stroke-width: 2;
0078:         stroke: currentColor;
0079:         fill: none;
0080:     }
```

```
0081:     .stat-value {
0082:         font-size: 2.1rem;
0083:         font-weight: 800;
0084:         letter-spacing: -0.04em;
0085:         color: var(--text-primary);
0086:         font-feature-settings: "tnum";
0087:     }
0088:     .stat-footnote {
0089:         font-size: 0.78rem;
0090:         color: var(--text-muted);
0091:         margin-top: 6px;
0092:         display: flex;
0093:         align-items: center;
0094:         gap: 6px;
0095:     }
0096:     /* Icon Color Themes */
0097:     .icon-blue   { background: rgba(59, 130, 246, 0.12); color: #60A5FA; }
0098:     .icon-emerald{ background: rgba(16, 185, 129, 0.12); color: #34D399; }
0099:     .icon-amber  { background: rgba(245, 158, 11, 0.12); color: #FBBF24; }
0100:     .icon-rose   { background: rgba(239, 68, 68, 0.12);  color: #F87171; }
0101:     .icon-purple { background: rgba(168, 85, 247, 0.12); color: #C084FC; }
0102:     /* Action Deck */
0103:     .section-title {
0104:         font-size: 1.15rem;
0105:         font-weight: 700;
0106:         letter-spacing: -0.02em;
0107:         margin-bottom: 16px;
0108:         margin-top: 32px;
0109:         display: flex;
0110:         align-items: center;
0111:         gap: 10px;
0112:     }
0113:     .quick-actions-bar {
0114:         display: flex;
0115:         gap: 14px;
0116:         margin-bottom: 32px;
0117:         flex-wrap: wrap;
0118:     }
0119:     .action-btn {
0120:         background: var(--bg-surface);
0121:         border: 1px solid var(--border-subtle);
0122:         padding: 14px 20px;
0123:         border-radius: var(--radius-md);
0124:         display: flex;
0125:         align-items: center;
0126:         gap: 12px;
0127:         color: var(--text-primary);
0128:         text-decoration: none;
0129:         font-weight: 600;
0130:         font-size: 0.88rem;
0131:         transition: all 0.2s ease;
0132:     }
0133:     .action-btn:hover {
0134:         border-color: var(--accent-primary);
0135:         background: var(--bg-elevated);
0136:         transform: translateY(-1px);
0137:     }
0138:     .action-btn svg {
0139:         width: 18px;
0140:         height: 18px;
0141:         stroke: var(--accent-primary);
0142:         stroke-width: 2;
0143:         fill: none;
0144:     }
0145:     /* Modern Data Card Table */
0146:     .data-panel {
0147:         background: var(--bg-surface);
0148:         border: 1px solid var(--border-subtle);
0149:         border-radius: var(--radius-lg);
0150:         overflow: hidden;
0151:         margin-bottom: 28px;
0152:     }
0153:     .panel-header {
0154:         padding: 18px 24px;
0155:         border-bottom: 1px solid var(--border-subtle);
0156:         display: flex;
0157:         align-items: center;
0158:         justify-content: space-between;
0159:     }
0160:     .panel-title {
```

```
0161:         font-size: 0.95rem;
0162:         font-weight: 700;
0163:         color: var(--text-primary);
0164:     }
0165:     .table-container,
0166:     .table-responsive {
0167:         width: 100%;
0168:         overflow-x: auto;
0169:         -webkit-overflow-scrolling: touch;
0170:         display: block;
0171:     }
0172:     .data-table {
0173:         width: 100%;
0174:         min-width: 780px;
0175:         border-collapse: collapse;
0176:         text-align: left;
0177:         font-size: 0.88rem;
0178:     }
0179:     .data-table th {
0180:         background: rgba(255, 255, 255, 0.02);
0181:         color: var(--text-muted);
0182:         font-size: 0.72rem;
0183:         text-transform: uppercase;
0184:         letter-spacing: 0.06em;
0185:         font-weight: 600;
0186:         padding: 14px 24px;
0187:         border-bottom: 1px solid var(--border-subtle);
0188:     }
0189:     .data-table td {
0190:         padding: 16px 24px;
0191:         border-bottom: 1px solid var(--border-subtle);
0192:         color: var(--text-secondary);
0193:     }
0194:     .data-table tr:last-child td {
0195:         border-bottom: none;
0196:     }
0197:     .data-table tr:hover td {
0198:         background: rgba(255, 255, 255, 0.015);
0199:         color: var(--text-primary);
0200:     }
0201:     .book-title-cell {
0202:         color: var(--text-primary);
0203:         font-weight: 600;
0204:     }
0205:     .empty-state {
0206:         text-align: center;
0207:         padding: 48px 20px;
0208:         color: var(--text-muted);
0209:     }
0210:     .empty-state svg {
0211:         width: 44px;
0212:         height: 44px;
0213:         stroke: var(--text-muted);
0214:         stroke-width: 1.5;
0215:         fill: none;
0216:         margin-bottom: 12px;
0217:     }
0218: </style>
0219: {% endblock %}
0220: {% block content %}
0221: <div class="dashboard-header">
0222:     <div>
0223:         <h1 class="welcome-title">Welcome back, {{ session.get('username', 'User') }}</h1>
0224:         <p class="welcome-sub">
0225:             Logged in as <strong style="color: var(--accent-primary);">{{ session.get('role') }}</strong>
0226:         </p>
0227:     </div>
0228:     <div class="header-actions">
0229:         <a href="{{ url_for('books') }}" class="btn btn-secondary">
0230:             <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line></svg>
0231:             Browse Catalog
0232:         </a>
0233:         {% if session.get('role') in ['Admin', 'Librarian'] %}
0234:         <a href="{{ url_for('loan_requests') }}" class="btn btn-secondary">
0235:             <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2"></path><rect x="
      | 8" y="2" width="8" height="4" rx="1" ry="1"></rect><path d="M9 14l2 2 4-4"></path></svg>
0236:             Loan Requests
0237:         </a>
```

```
0238:         <a href="{{ url_for('issue_loan') }}" class="btn btn-primary">
0239:             <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyline></svg>
0240:             Issue Book
0241:         </a>
0242:         {% endif %}
0243:     </div>
0244: </div>
0245: <!-- ========================================================================== -->
0246: <!-- 1. ADMIN & LIBRARIAN VIEW -->
0247: <!-- ========================================================================== -->
0248: {% if session.get('role') in ['Admin', 'Librarian'] %}
0249: <div class="stat-grid">
0250:     <div class="stat-card">
0251:         <div class="stat-top">
0252:             <span class="stat-label">Total Titles</span>
0253:             <div class="stat-icon icon-blue">
0254:                 <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v
      | 14z"></path></svg>
0255:             </div>
0256:         </div>
0257:         <div class="stat-value">{{ data.total_books or 0 }}</div>
0258:         <div class="stat-footnote">Unique registered titles in catalog</div>
0259:     </div>
0260:     <div class="stat-card">
0261:         <div class="stat-top">
0262:             <span class="stat-label">Active Loans</span>
0263:             <div class="stat-icon icon-emerald">
0264:                 <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 1
      | 2 14 14"></polyline></svg>
0265:             </div>
0266:         </div>
0267:         <div class="stat-value">{{ data.active_loans or data.borrowed_copies or 0 }}</div>
0268:         <div class="stat-footnote">Physical copies currently checked out</div>
0269:     </div>
0270:     <div class="stat-card">
0271:         <div class="stat-top">
0272:             <span class="stat-label">Overdue Loans</span>
0273:             <div class="stat-icon icon-rose">
0274:                 <svg viewBox="0 0 24 24"><path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-
      | 3L13.71 3.86a2 2 0 0 0-3.42 0z"></path><line x1="12" y1="9" x2="12" y2="13"></line><line x1="12" y1="1
      | 7" x2="12.01" y2="17"></line></svg>
0275:             </div>
0276:         </div>
0277:         <div class="stat-value">{{ data.overdue_loans or 0 }}</div>
0278:         <div class="stat-footnote">Loans exceeding return threshold</div>
0279:     </div>
0280:     <div class="stat-card">
0281:         <div class="stat-top">
0282:             <span class="stat-label">Disciplinary Fines</span>
0283:             <div class="stat-icon icon-amber">
0284:                 <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5
      | 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6"></path></svg>
0285:             </div>
0286:         </div>
0287:         <div class="stat-value">Rs. {{ "%.2f"|format(data.unpaid_fines or 0) }}</div>
0288:         <div class="stat-footnote">Disciplinary Liabilities</div>
0289:     </div>
0290: </div>
0291: <h2 class="section-title">Quick Administrative Actions</h2>
0292: <div class="quick-actions-bar">
0293:     <a href="{{ url_for('add_book') }}" class="action-btn">
0294:         <svg viewBox="0 0 24 24"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2
      | ="12"></line></svg>
0295:         Register New Book
0296:     </a>
0297:     <a href="{{ url_for('loan_requests') }}" class="action-btn">
0298:         <svg viewBox="0 0 24 24"><path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1
      |  2-2h2"></path><rect x="8" y="2" width="8" height="4" rx="1" ry="1"></rect><path d="M9 14l2 2 4-4"></p
      | ath></svg>
0299:         Loan Requests Queue
0300:     </a>
0301:     <a href="{{ url_for('issue_loan') }}" class="action-btn">
0302:         <svg viewBox="0 0 24 24"><polyline points="9 11 12 14 22 4"></polyline><path d="M21 12v7a2 2 0 0 1-2
      |  2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"></path></svg>
0303:         Issue Physical Copy
0304:     </a>
0305:     <a href="{{ url_for('books') }}" class="action-btn">
0306:         <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></p
      | ath></svg>
```

```
0307:         Audit Catalog & Stock
0308:     </a>
0309:     {% if session.get('role') == 'Admin' %}
0310:     <a href="{{ url_for('manage_users') }}" class="action-btn">
0311:         <svg viewBox="0 0 24 24"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="9" c
      | y="7" r="4"></circle><path d="M22 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13a4 4 0 0 1 0 7.75"><
      | /path></svg>
0312:         Manage Users
0313:     </a>
0314:     {% endif %}
0315: </div>
0316: <!-- ========================================================================== -->
0317: <!-- ADMIN & LIBRARIAN OPERATIONAL MONITORS & ACTIONS -->
0318: <!-- ========================================================================== -->
0319: {% if session.get('role') in ['Admin', 'Librarian'] %}
0320:     <!-- Monitor 1: Who and all are currently loaning books (With Return Book Action) -->
0321:     <h2 class="section-title">
0322:         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" stroke-wi
      | dth="2"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyline></svg>
0323:         Currently Active Book Loans
0324:     </h2>
0325:     <div class="data-panel">
0326:         <div class="panel-header">
0327:             <span class="panel-title">Active Borrowers Ledger</span>
0328:             <span class="badge badge-info">{{ data.active_loans_list|length if data.active_loans_list else 0
      |  }} Active Loans</span>
0329:         </div>
0330:         <div class="table-container table-responsive">
0331:             <table class="data-table">
0332:                 <thead>
0333:                     <tr>
0334:                         <th>Loan Ref</th>
0335:                         <th>Borrower</th>
0336:                         <th>Book Title</th>
0337:                         <th>Copy Ref</th>
0338:                         <th>Issue Date</th>
0339:                         <th>Due Date</th>
0340:                         <th>Fine Status</th>
0341:                         <th style="text-align: right;">Circulation Action</th>
0342:                     </tr>
0343:                 </thead>
0344:                 <tbody>
0345:                     {% if data.active_loans_list %}
0346:                         {% for loan in data.active_loans_list %}
0347:                         <tr>
0348:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ loan.loan_i
      | d }}</td>
0349:                             <td>
0350:                                 <div style="font-weight: 600; color: var(--text-primary);">{{ loan.borrower_
      | name }}</div>
0351:                                 <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; ali
      | gn-items: center; gap: 6px; margin-top: 2px;">
0352:                                     <span>{{ loan.borrower_email }}</span>
0353:                                     {% if loan.borrower_role == 'Staff' %}
0354:                                         <span class="badge" style="background: rgba(168, 85, 247, 0.12); col
      | or: #C084FC; font-size: 0.65rem; padding: 1px 5px;">Staff</span>
0355:                                     {% else %}
0356:                                         <span class="badge" style="background: rgba(59, 130, 246, 0.12); col
      | or: #60A5FA; font-size: 0.65rem; padding: 1px 5px;">Student</span>
0357:                                     {% endif %}
0358:                                 </div>
0359:                             </td>
0360:                             <td class="book-title-cell">{{ loan.book_title }}</td>
0361:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">Copy #{{ loan.copy
      | _id }}</td>
0362:                             <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</t
      | d>
0363:                             <td style="font-weight: 600;">{{ loan.due_date.strftime('%b %d, %Y') if loan.due
      | _date else 'N/A' }}</td>
0364:                             <td>
0365:                                 {% if (loan.fine_amount or 0) > 0 %}
0366:                                     <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount)
      | }} Unpaid</span>
0367:                                 {% else %}
0368:                                     <span class="badge badge-success" style="background: rgba(16, 185, 129,
      | 0.12); color: #34D399; border: 1px solid rgba(16, 185, 129, 0.3);">Zero Fine</span>
0369:                                 {% endif %}
0370:                             </td>
0371:                             <td style="text-align: right; white-space: nowrap;">
0372:                                 <form method="POST" action="{{ url_for('return_book', loan_id=loan.loan_id)
```

```
      | }}" style="display: inline;" onsubmit="return confirm('Process return for {{ loan.book_title }} (Copy
      | #{{ loan.copy_id }})?');">
0373:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0374:                                   <button type="submit" class="btn" style="background: rgba(59, 130, 246,
      | 0.15); color: #60A5FA; border: 1px solid rgba(59, 130, 246, 0.35); padding: 5px 12px; font-size: 0.78r
      | em; font-weight: 700; border-radius: var(--radius-sm);">
0375:                                       <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><polyline points="9 14 4 9 9 4"></polyline><path d="M20 20v-7a4 4 0 0
      |  0-4-4H4"></path></svg>
0376:                                       Mark Returned
0377:                                   </button>
0378:                               </form>
0379:                               <form method="POST" action="{{ url_for('delete_loan', loan_id=loan.loan_id)
      | }}" style="display: inline; margin-left: 6px;" onsubmit="return confirm('Delete this active loan for {
      | { loan.borrower_name }} ({{ loan.book_title }})? This will release Copy #{{ loan.copy_id }} back to Av
      | ailable stock.');">
0380:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0381:                                   <button type="submit" class="btn" style="background: rgba(239, 68, 68, 0
      | .15); color: #F87171; border: 1px solid rgba(239, 68, 68, 0.35); padding: 5px 12px; font-size: 0.78rem
      | ; font-weight: 700; border-radius: var(--radius-sm);">
0382:                                       <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6v14a2 2 0 0
      | 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></path></svg>
0383:                                       Delete Loan
0384:                                   </button>
0385:                               </form>
0386:                               <form method="POST" action="{{ url_for('report_lost', loan_id=loan.loan_id)
      | }}" style="display: inline; margin-left: 6px;" onsubmit="return confirm('Report book as LOST for {{ lo
      | an.borrower_name }} ({{ loan.book_title }})? This will mark copy as Lost, close the loan, and charge a
      |  replacement penalty of Rs. 1500.00.');">
0387:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0388:                                   <button type="submit" class="btn" style="background: rgba(245, 158, 11,
      | 0.15); color: #FBBF24; border: 1px solid rgba(245, 158, 11, 0.35); padding: 5px 12px; font-size: 0.78r
      | em; font-weight: 700; border-radius: var(--radius-sm);">
0389:                                       <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="8" x2="12"
      | y2="12"></line><line x1="12" y1="16" x2="12.01" y2="16"></line></svg>
0390:                                       Mark as Lost
0391:                                   </button>
0392:                               </form>
0393:                           </td>
0394:                       </tr>
0395:                       {% endfor %}
0396:                   {% else %}
0397:                       <tr>
0398:                           <td colspan="8" class="empty-state">No books currently loaned out. All items are
      |  in inventory.</td>
0399:                       </tr>
0400:                   {% endif %}
0401:               </tbody>
0402:           </table>
0403:       </div>
0404:     </div>
0405:     <!-- Monitor 2: Who are overdue (With Fast Return & Penalty Check & Delete Overdue) -->
0406:     <h2 class="section-title" style="color: var(--danger-text);">
0407:         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--danger-text)" stroke-width
      | ="2"><path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z
      | "></path><line x1="12" y1="9" x2="12" y2="13"></line><line x1="12" y1="17" x2="12.01" y2="17"></line><
      | /svg>
0408:         Overdue Borrowers Watchlist
0409:     </h2>
0410:     <div class="data-panel" style="border-color: rgba(239, 68, 68, 0.25);">
0411:         <div class="panel-header">
0412:             <span class="panel-title" style="color: var(--danger-text);">Overdue Accounts Requiring Action</
      | span>
0413:             <span class="badge badge-danger">{{ data.overdue_loans_list|length if data.overdue_loans_list el
      | se 0 }} Overdue</span>
0414:         </div>
0415:         <div class="table-container table-responsive">
0416:             <table class="data-table">
0417:                 <thead>
0418:                     <tr>
0419:                         <th>Loan Ref</th>
0420:                         <th>Delinquent Borrower</th>
0421:                         <th>Book Title</th>
0422:                         <th>Copy Ref</th>
0423:                         <th>Due Date</th>
0424:                         <th>Days Overdue</th>
0425:                         <th style="text-align: right;">Circulation Action</th>
0426:                     </tr>
```

```
0427:                 </thead>
0428:                 <tbody>
0429:                     {% if data.overdue_loans_list %}
0430:                         {% for ov in data.overdue_loans_list %}
0431:                         <tr>
0432:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ ov.loan_id
     | }}</td>
0433:                             <td>
0434:                                 <div style="font-weight: 600; color: var(--text-primary);">{{ ov.borrower_na
     | me }}</div>
0435:                                 <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; ali
     | gn-items: center; gap: 6px; margin-top: 2px;">
0436:                                     <span>{{ ov.borrower_email }}</span>
0437:                                     {% if ov.borrower_role == 'Staff' %}
0438:                                         <span class="badge" style="background: rgba(168, 85, 247, 0.12); col
     | or: #C084FC; font-size: 0.65rem; padding: 1px 5px;">Staff</span>
0439:                                     {% else %}
0440:                                         <span class="badge" style="background: rgba(59, 130, 246, 0.12); col
     | or: #60A5FA; font-size: 0.65rem; padding: 1px 5px;">Student</span>
0441:                                     {% endif %}
0442:                                 </div>
0443:                             </td>
0444:                             <td class="book-title-cell">{{ ov.book_title }}</td>
0445:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">Copy #{{ ov.copy_i
     | d }}</td>
0446:                             <td style="color: var(--danger-text); font-weight: 600;">
0447:                                 {{ ov.due_date.strftime('%b %d, %Y') if ov.due_date else 'N/A' }}
0448:                             </td>
0449:                             <td>
0450:                                 <span class="badge badge-danger">
0451:                                     {{ ov.days_overdue or 1 }} Day(s) Overdue
0452:                                 </span>
0453:                             </td>
0454:                             <td style="text-align: right; white-space: nowrap;">
0455:                                 <form method="POST" action="{{ url_for('return_book', loan_id=ov.loan_id) }}
     | " style="display: inline;" onsubmit="return confirm('Process return for overdue book {{ ov.book_title
     | }}? Overdue fine will be calculated.');">
0456:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0457:                                     <button type="submit" class="btn" style="background: rgba(239, 68, 68, 0
     | .15); color: #F87171; border: 1px solid rgba(239, 68, 68, 0.35); padding: 5px 12px; font-size: 0.78rem
     | ; font-weight: 700; border-radius: var(--radius-sm);">
0458:                                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
     | currentColor" stroke-width="2.5"><polyline points="9 14 4 9 9 4"></polyline><path d="M20 20v-7a4 4 0 0
     |  0-4-4H4"></path></svg>
0459:                                         Receive Overdue
0460:                                     </button>
0461:                                 </form>
0462:                                 <form method="POST" action="{{ url_for('delete_loan', loan_id=ov.loan_id) }}
     | " style="display: inline; margin-left: 6px;" onsubmit="return confirm('Delete this overdue loan for {{
     |  ov.borrower_name }} ({{ ov.book_title }})? This will delete the active loan for this patron, remove a
     | ssociated fines, and restore Copy #{{ ov.copy_id }} to Available stock.');">
0463:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0464:                                     <button type="submit" class="btn" style="background: rgba(220, 38, 38, 0
     | .25); color: #FCA5A5; border: 1px solid rgba(220, 38, 38, 0.5); padding: 5px 12px; font-size: 0.78rem;
     |  font-weight: 700; border-radius: var(--radius-sm);">
0465:                                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
     | currentColor" stroke-width="2.5"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6v14a2 2 0 0
     | 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></path></svg>
0466:                                         Delete Overdue
0467:                                     </button>
0468:                                 </form>
0469:                                 <form method="POST" action="{{ url_for('report_lost', loan_id=ov.loan_id) }}
     | " style="display: inline; margin-left: 6px;" onsubmit="return confirm('Report overdue book as LOST for
     |  {{ ov.borrower_name }} ({{ ov.book_title }})? This will mark copy as Lost, close the loan, and charge
     |  a replacement penalty of Rs. 1500.00.');">
0470:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0471:                                     <button type="submit" class="btn" style="background: rgba(245, 158, 11,
     | 0.15); color: #FBBF24; border: 1px solid rgba(245, 158, 11, 0.35); padding: 5px 12px; font-size: 0.78r
     | em; font-weight: 700; border-radius: var(--radius-sm);">
0472:                                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
     | currentColor" stroke-width="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="8" x2="12"
     | y2="12"></line><line x1="12" y1="16" x2="12.01" y2="16"></line></svg>
0473:                                         Mark as Lost
0474:                                     </button>
0475:                                 </form>
0476:                             </td>
0477:                         </tr>
0478:                         {% endfor %}
0479:                     {% else %}
0480:                         <tr>
```

```
0481:                             <td colspan="7" class="empty-state">No loans currently overdue. All patrons in g
      | ood standing!</td>
0482:                         </tr>
0483:                     {% endif %}
0484:                 </tbody>
0485:             </table>
0486:         </div>
0487:     </div>
0488:     <!-- Monitor 3: Who didn't pay fine (With Collect & Mark Paid Action) -->
0489:     <h2 class="section-title" style="color: var(--warning-text);">
0490:         <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="var(--warning-text)" stroke-widt
      | h="2"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5 3.5 0 0 0 0 0 7h5a3.5 3.5 0 0 1
      |  0 7H6"></path></svg>
0491:         Disciplinary Fines & Liabilities Ledger
0492:     </h2>
0493:     <div class="data-panel" style="border-color: rgba(245, 158, 11, 0.25);">
0494:         <div class="panel-header">
0495:             <span class="panel-title" style="color: var(--warning-text);">Disciplinary Liabilities</span>
0496:             <span class="badge badge-warning">{{ data.unpaid_fines_list|length if data.unpaid_fines_list els
      | e 0 }} Unpaid Penalties</span>
0497:         </div>
0498:         <div class="table-container table-responsive">
0499:             <table class="data-table">
0500:                 <thead>
0501:                     <tr>
0502:                         <th>Fine Ref</th>
0503:                         <th>Borrower</th>
0504:                         <th>Book Title</th>
0505:                         <th>Originating Loan</th>
0506:                         <th>Due Date</th>
0507:                         <th>Disciplinary Fine</th>
0508:                         <th>Status</th>
0509:                         <th style="text-align: right;">Cashier Action</th>
0510:                     </tr>
0511:                 </thead>
0512:                 <tbody>
0513:                     {% if data.unpaid_fines_list %}
0514:                         {% for fine in data.unpaid_fines_list %}
0515:                         <tr>
0516:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">#FN-{{ fine.fine_i
      | d }}</td>
0517:                             <td>
0518:                                 <div style="font-weight: 600; color: var(--text-primary);">{{ fine.borrower_
      | name }}</div>
0519:                                 <div style="font-size: 0.76rem; color: var(--text-muted);">{{ fine.borrower_
      | email }}</div>
0520:                             </td>
0521:                             <td class="book-title-cell">{{ fine.book_title }}</td>
0522:                             <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ fine.loan_i
      | d }}</td>
0523:                             <td>{{ fine.due_date.strftime('%b %d, %Y') if fine.due_date else 'N/A' }}</td>
0524:                             <td style="font-weight: 700; color: var(--warning-text);">
0525:                                 Rs. {{ "%.2f"|format(fine.fine_amount) }}
0526:                             </td>
0527:                             <td>
0528:                                 <span class="badge badge-warning">Unpaid</span>
0529:                             </td>
0530:                             <td style="text-align: right;">
0531:                                 <div style="display: inline-flex; gap: 8px; justify-content: flex-end; align
      | -items: center;">
0532:                                     <form method="POST" action="{{ url_for('pay_fine', fine_id=fine.fine_id)
      |  }}" style="display: inline;" onsubmit="return confirm('Confirm payment collection of Rs. {{ fine.fine
      | _amount }} from {{ fine.borrower_name }}?');">
0533:                                         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0534:                                         <input type="hidden" name="payment_action" value="Paid">
0535:                                         <button type="submit" class="btn" style="background: rgba(16, 185, 1
      | 29, 0.15); color: #34D399; border: 1px solid rgba(16, 185, 129, 0.35); padding: 5px 12px; font-size: 0
      | .78rem; font-weight: 700; border-radius: var(--radius-sm);">
0536:                                             <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="20 6 9 17 4 12"></polyline></svg>
0537:                                             Collect & Mark Paid
0538:                                         </button>
0539:                                     </form>
0540:                                     <form method="POST" action="{{ url_for('pay_fine', fine_id=fine.fine_id)
      |  }}" style="display: inline;" onsubmit="return confirm('Confirm waiving penalty of Rs. {{ fine.fine_am
      | ount }} for {{ fine.borrower_name }}?');">
0541:                                         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0542:                                         <input type="hidden" name="payment_action" value="Waived">
0543:                                         <button type="submit" class="btn" style="background: rgba(99, 102, 2
      | 41, 0.15); color: #818CF8; border: 1px solid rgba(99, 102, 241, 0.35); padding: 5px 12px; font-size: 0
```

```
      | .78rem; font-weight: 700; border-radius: var(--radius-sm);">
0544:                                           <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="4.93" y1="4.93"
      |  x2="19.07" y2="19.07"></line></svg>
0545:                                           Waive Penalty
0546:                                       </button>
0547:                                   </form>
0548:                               </div>
0549:                           </td>
0550:                       </tr>
0551:                       {% endfor %}
0552:                   {% else %}
0553:                       <tr>
0554:                           <td colspan="8" class="empty-state">No unpaid fines recorded. All fines are sett
      | led.</td>
0555:                       </tr>
0556:                   {% endif %}
0557:               </tbody>
0558:           </table>
0559:       </div>
0560:     </div>
0561: {% endif %}
0562: <!-- ========================================================================== -->
0563: <!-- 2. STUDENT VIEW -->
0564: <!-- ========================================================================== -->
0565: {% elif session.get('role') == 'Student' %}
0566: <div class="stat-grid">
0567:     <div class="stat-card">
0568:         <div class="stat-top">
0569:             <span class="stat-label">Committed Books</span>
0570:             <div class="stat-icon icon-blue">
0571:                 <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v
      | 14z"></path></svg>
0572:             </div>
0573:         </div>
0574:         <div class="stat-value">{{ data.total_committed or 0 }} / 3</div>
0575:         <div class="stat-footnote">{{ data.active_loans_count or 0 }} Checked Out &bull; {{ data.awaiting_pi
      | ckup_count or 0 }} Awaiting Pickup</div>
0576:     </div>
0577:     {% if (data.awaiting_pickup_count or 0) > 0 %}
0578:     <div class="stat-card" style="border: 1px solid rgba(59, 130, 246, 0.4); background: rgba(59, 130, 246,
      | 0.06);">
0579:         <div class="stat-top">
0580:             <span class="stat-label" style="color: #60A5FA; font-weight: 700;">Ready for Pickup</span>
0581:             <div class="stat-icon icon-emerald">
0582:                 <svg viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"></polyline></svg>
0583:             </div>
0584:         </div>
0585:         <div class="stat-value" style="color: #60A5FA;">{{ data.awaiting_pickup_count }} {{ 'Book' if data.a
      | waiting_pickup_count == 1 else 'Books' }}</div>
0586:         <div class="stat-footnote" style="color: #93C5FD; font-weight: 600;">Reserved at circulation desk</d
      | iv>
0587:     </div>
0588:     {% endif %}
0589:     <div class="stat-card">
0590:         <div class="stat-top">
0591:             <span class="stat-label">Fine</span>
0592:             <div class="stat-icon icon-amber">
0593:                 <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5
      | 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6"></path></svg>
0594:             </div>
0595:         </div>
0596:         {% if (data.total_fines or 0) > 0 %}
0597:             <div class="stat-value" style="color: var(--danger-text);">Rs. {{ "%.2f"|format(data.total_fines
      | ) }}</div>
0598:             <div class="stat-footnote">
0599:                 <span style="color: var(--danger-text); font-weight: 600;">Clear balance to borrow</span>
0600:             </div>
0601:         {% else %}
0602:             <div class="stat-value" style="font-size: 1.25rem; color: #34D399; font-weight: 800;">Zero Fine<
      | /div>
0603:             <div class="stat-footnote">
0604:                 <span style="color: var(--success-text); font-weight: 600;">Rs. {{ "%.2f"|format(data.total_
      | fines or 0) }}</span>
0605:             </div>
0606:         {% endif %}
0607:     </div>
0608: </div>
0609: <h2 class="section-title">My Active Loans</h2>
0610: <div class="data-panel">
```

```
0611:     <div class="panel-header">
0612:         <span class="panel-title">Currently Borrowed Books</span>
0613:         <span class="badge badge-info">{{ data.active_loans|length if data.active_loans else 0 }} Items</spa
      | n>
0614:     </div>
0615:     <div class="table-container table-responsive">
0616:         <table class="data-table">
0617:             <thead>
0618:                 <tr>
0619:                     <th>Loan Ref</th>
0620:                     <th>Book Title</th>
0621:                     <th>Copy ID</th>
0622:                     <th>Checkout Date</th>
0623:                     <th>Due Date</th>
0624:                     <th>Fine Status</th>
0625:                     <th>Status</th>
0626:                 </tr>
0627:             </thead>
0628:             <tbody>
0629:                 {% if data.active_loans %}
0630:                     {% for loan in data.active_loans %}
0631:                     <tr>
0632:                         <td style="font-family: var(--font-mono); font-size: 0.8rem;">#LN-{{ loan.loan_id }}
      | </td>
0633:                         <td class="book-title-cell">
0634:                             <div style="font-weight: 600; color: var(--text-primary);">{{ loan.title }}</div
      | >
0635:                             {% if loan.prerequisites or loan.next_books %}
0636:                             <div style="display: flex; flex-wrap: wrap; gap: 0.4rem; margin-top: 0.35rem; al
      | ign-items: center;">
0637:                                 {% if loan.prerequisites %}
0638:                                     {% for pre in loan.prerequisites %}
0639:                                     <a href="{{ url_for('category_books', category_id=pre.category_id) if pr
      | e.category_id else url_for('books') }}" style="text-decoration: none;" title="Explore prerequisite tit
      | le in catalog">
0640:                                         <span class="badge" style="background: rgba(148, 163, 184, 0.12); co
      | lor: #CBD5E1; border: 1px solid rgba(148, 163, 184, 0.25); font-size: 0.72rem; padding: 2px 7px; borde
      | r-radius: 9999px; display: inline-flex; align-items: center; gap: 4px; cursor: pointer; transition: al
      | l 0.2s ease;">
0641:                                             <svg width="10" height="10" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="15 18 9 12 15 6"></polyline></svg>
0642:                                             Prerequisite: {{ pre.title }}
0643:                                         </span>
0644:                                     </a>
0645:                                     {% endfor %}
0646:                                 {% endif %}
0647:
0648:                                 {% if loan.next_books %}
0649:                                     {% for nxt in loan.next_books %}
0650:                                     <a href="{{ url_for('category_books', category_id=nxt.category_id) if nx
      | t.category_id else url_for('books') }}" style="text-decoration: none;" title="Explore downstream recom
      | mended title in catalog">
0651:                                         <span class="badge" style="background: rgba(99, 102, 241, 0.15); col
      | or: #A5B4FC; border: 1px solid rgba(99, 102, 241, 0.35); font-size: 0.72rem; padding: 2px 8px; border-
      | radius: 9999px; display: inline-flex; align-items: center; gap: 5px; cursor: pointer; transition: all
      | 0.2s ease;">
0652:                                             <svg width="10" height="10" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="9 18 15 12 9 6"></polyline></svg>
0653:                                             Next to Study: <strong style="color: #E0E7FF;">{{ nxt.title }}</
      | strong>
0654:                                             {% if nxt.available_copies > 0 %}
0655:                                                 <span style="color: #34D399; font-weight: 700; margin-left:
      | 2px;">● {{ nxt.available_copies }} Avail</span>
0656:                                             {% else %}
0657:                                                 <span style="color: #F87171; font-weight: 700; margin-left:
      | 2px;">● Out of Stock</span>
0658:                                             {% endif %}
0659:                                         </span>
0660:                                     </a>
0661:                                     {% endfor %}
0662:                                 {% endif %}
0663:                             </div>
0664:                             {% endif %}
0665:                         </td>
0666:                         <td>Copy #{{ loan.copy_id }}</td>
0667:                         <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</td>
0668:                         <td>{{ loan.due_date.strftime('%b %d, %Y') if loan.due_date else 'N/A' }}</td>
0669:                         <td>
0670:                             {% if (loan.fine_amount or 0) > 0 %}
0671:                                 <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount) }} U
```

```
      | npaid</span>
0672:                             {% else %}
0673:                                 <span class="badge badge-success" style="background: rgba(16, 185, 129, 0.12
      | ); color: #34D399; border: 1px solid rgba(16, 185, 129, 0.3);">Zero Fine</span>
0674:                             {% endif %}
0675:                         </td>
0676:                         <td>
0677:                             {% if loan.is_overdue == 1 %}
0678:                                 <span class="badge badge-danger">Overdue</span>
0679:                             {% else %}
0680:                                 <span class="badge badge-success">Active Loan</span>
0681:                             {% endif %}
0682:                         </td>
0683:                     </tr>
0684:                     {% endfor %}
0685:                 {% else %}
0686:                     <tr>
0687:                         <td colspan="7">
0688:                             <div class="empty-state">
0689:                                 <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5
      | 2.5 0 0 0 4 5.5v14z"></path></svg>
0690:                                 <div>You have no active loans checked out.</div>
0691:                                 <a href="{{ url_for('books') }}" class="btn btn-secondary" style="margin-top
      | : 14px;">Browse Books Catalog</a>
0692:                             </div>
0693:                         </td>
0694:                     </tr>
0695:                 {% endif %}
0696:             </tbody>
0697:         </table>
0698:     </div>
0699: </div>
0700: <!-- Student Loan Requests Queue Status -->
0701: <h2 class="section-title">My Loan Requests</h2>
0702: <div class="data-panel">
0703:     <div class="panel-header">
0704:         <span class="panel-title">Submitted Permission Requests</span>
0705:         <span class="badge badge-info">{{ data.loan_requests|length if data.loan_requests else 0 }} Requests
      | </span>
0706:     </div>
0707:     <div class="table-container table-responsive">
0708:         <table class="data-table">
0709:             <thead>
0710:                 <tr>
0711:                     <th>Request Ref</th>
0712:                     <th>Book Title</th>
0713:                     <th>Requested On</th>
0714:                     <th>Status</th>
0715:                     <th>Action</th>
0716:                 </tr>
0717:             </thead>
0718:             <tbody>
0719:                 {% if data.loan_requests %}
0720:                     {% for req in data.loan_requests %}
0721:                     <tr>
0722:                         <td style="font-family: var(--font-mono); font-size: 0.8rem;">#REQ-{{ req.request_id
      |  }}</td>
0723:                         <td class="book-title-cell">{{ req.book_title }}</td>
0724:                         <td>{{ req.request_date.strftime('%b %d, %Y') if req.request_date else 'N/A' }}</td>
0725:                         <td>
0726:                             {% if req.status == 'Pending' %}
0727:                                 <span class="badge badge-warning">Awaiting Approval</span>
0728:                             {% elif req.status == 'Awaiting Pickup' %}
0729:                                 <span class="badge badge-info" style="background: rgba(59, 130, 246, 0.15);
      | color: #60A5FA; border: 1px solid rgba(59, 130, 246, 0.35);">Approved &ndash; Ready for Pickup</span>
0730:                             {% elif req.status == 'Approved' or req.status == 'Fulfilled' %}
0731:                                 <span class="badge badge-success">Fulfilled</span>
0732:                             {% elif req.status == 'Rejected' %}
0733:                                 <span class="badge badge-danger">Rejected</span>
0734:                             {% else %}
0735:                                 <span class="badge badge-secondary">{{ req.status }}</span>
0736:                             {% endif %}
0737:                         </td>
0738:                         <td>
0739:                             {% if req.status == 'Rejected' and req.book_id %}
0740:                                 <form method="POST" action="{{ url_for('request_loan', book_id=req.book_id)
      | }}" style="display: inline; margin: 0;">
0741:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0742:                                     <input type="hidden" name="from_dashboard" value="1"/>
0743:                                     <button type="submit" class="btn btn-sm" style="padding: 4px 10px; font-
```

```
      | size: 0.76rem; font-weight: 700; background: rgba(59, 130, 246, 0.15); color: #60A5FA; border: 1px sol
      | id rgba(59, 130, 246, 0.35); border-radius: var(--radius-sm); cursor: pointer;" onclick="return confir
      | m('Submit new loan request for \'{{ req.book_title|escape }}\'?');">
0744:                                     ■ Request Again
0745:                                 </button>
0746:                             </form>
0747:                         {% elif req.status == 'Awaiting Pickup' %}
0748:                             <span style="font-size: 0.76rem; color: #60A5FA; font-weight: 600;">{% if re
      | q.copy_id %}Copy #{{ req.copy_id }} &bull; {% endif %}Collect at Desk</span>
0749:                         {% elif req.status == 'Pending' %}
0750:                             <span style="font-size: 0.76rem; color: var(--text-muted);">In Queue</span>
0751:                         {% else %}
0752:                             <span style="font-size: 0.76rem; color: var(--text-muted);">&mdash;</span>
0753:                         {% endif %}
0754:                     </td>
0755:                 </tr>
0756:                 {% endfor %}
0757:             {% else %}
0758:                 <tr>
0759:                     <td colspan="5" class="empty-state">No loan requests submitted. You can request book
      | s from the Catalog.</td>
0760:                 </tr>
0761:             {% endif %}
0762:         </tbody>
0763:     </table>
0764: </div>
0765: </div>
0766: <!-- ========================================================================== -->
0767: <!-- 3. STAFF VIEW -->
0768: <!-- ========================================================================== -->
0769: {% elif session.get('role') == 'Staff' %}
0770: <div class="stat-grid">
0771:     <div class="stat-card">
0772:         <div class="stat-top">
0773:             <span class="stat-label">Committed Books</span>
0774:             <div class="stat-icon icon-blue">
0775:                 <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v
      | 14z"></path></svg>
0776:             </div>
0777:         </div>
0778:         <div class="stat-value">{{ data.total_committed or 0 }} / 3</div>
0779:         <div class="stat-footnote">{{ data.active_loans_count or 0 }} Checked Out &bull; {{ data.awaiting_pi
      | ckup_count or 0 }} Awaiting Pickup</div>
0780:     </div>
0781:     {% if (data.awaiting_pickup_count or 0) > 0 %}
0782:     <div class="stat-card" style="border: 1px solid rgba(59, 130, 246, 0.4); background: rgba(59, 130, 246,
      | 0.06);">
0783:         <div class="stat-top">
0784:             <span class="stat-label" style="color: #60A5FA; font-weight: 700;">Ready for Pickup</span>
0785:             <div class="stat-icon icon-emerald">
0786:                 <svg viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"></polyline></svg>
0787:             </div>
0788:         </div>
0789:         <div class="stat-value" style="color: #60A5FA;">{{ data.awaiting_pickup_count }} {{ 'Book' if data.a
      | waiting_pickup_count == 1 else 'Books' }}</div>
0790:         <div class="stat-footnote" style="color: #93C5FD; font-weight: 600;">Reserved at circulation desk</d
      | iv>
0791:     </div>
0792:     {% endif %}
0793:     <div class="stat-card">
0794:         <div class="stat-top">
0795:             <span class="stat-label">Fine</span>
0796:             <div class="stat-icon icon-amber">
0797:                 <svg viewBox="0 0 24 24"><line x1="12" y1="1" x2="12" y2="23"></line><path d="M17 5H9.5a3.5
      | 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6"></path></svg>
0798:             </div>
0799:         </div>
0800:         {% if (data.total_fines or 0) > 0 %}
0801:             <div class="stat-value" style="color: var(--danger-text);">Rs. {{ "%.2f"|format(data.total_fines
      | ) }}</div>
0802:             <div class="stat-footnote">
0803:                 <span style="color: var(--danger-text); font-weight: 600;">Clear balance to borrow</span>
0804:             </div>
0805:         {% else %}
0806:             <div class="stat-value" style="font-size: 1.25rem; color: #34D399; font-weight: 800;">Zero Fine<
      | /div>
0807:             <div class="stat-footnote">
0808:                 <span style="color: var(--success-text); font-weight: 600;">Rs. {{ "%.2f"|format(data.total_
      | fines or 0) }}</span>
0809:             </div>
```

```
0810:         {% endif %}
0811:     </div>
0812: </div>
0813: <h2 class="section-title">My Checked Out Books</h2>
0814: <div class="data-panel">
0815:     <div class="table-container table-responsive">
0816:         <table class="data-table">
0817:             <thead>
0818:                 <tr>
0819:                     <th>Loan #</th>
0820:                     <th>Book Title</th>
0821:                     <th>Copy ID</th>
0822:                     <th>Issued On</th>
0823:                     <th>Due Date</th>
0824:                     <th>Fine Status</th>
0825:                     <th>Status</th>
0826:                 </tr>
0827:             </thead>
0828:             <tbody>
0829:                 {% if data.current_loans %}
0830:                     {% for loan in data.current_loans %}
0831:                     <tr>
0832:                         <td style="font-family: var(--font-mono);">#{{ loan.loan_id }}</td>
0833:                         <td class="book-title-cell">
0834:                             <div style="font-weight: 600; color: var(--text-primary);">{{ loan.title }}</div
      | >
0835:                             {% if loan.prerequisites or loan.next_books %}
0836:                             <div style="display: flex; flex-wrap: wrap; gap: 0.4rem; margin-top: 0.35rem; al
      | ign-items: center;">
0837:                                 {% if loan.prerequisites %}
0838:                                     {% for pre in loan.prerequisites %}
0839:                                     <a href="{{ url_for('category_books', category_id=pre.category_id) if pr
      | e.category_id else url_for('books') }}" style="text-decoration: none;" title="Explore prerequisite tit
      | le in catalog">
0840:                                         <span class="badge" style="background: rgba(148, 163, 184, 0.12); co
      | lor: #CBD5E1; border: 1px solid rgba(148, 163, 184, 0.25); font-size: 0.72rem; padding: 2px 7px; borde
      | r-radius: 9999px; display: inline-flex; align-items: center; gap: 4px; cursor: pointer; transition: al
      | l 0.2s ease;">
0841:                                             <svg width="10" height="10" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="15 18 9 12 15 6"></polyline></svg>
0842:                                             Prerequisite: {{ pre.title }}
0843:                                         </span>
0844:                                     </a>
0845:                                     {% endfor %}
0846:                                 {% endif %}
0847:
0848:                                 {% if loan.next_books %}
0849:                                     {% for nxt in loan.next_books %}
0850:                                     <a href="{{ url_for('category_books', category_id=nxt.category_id) if nx
      | t.category_id else url_for('books') }}" style="text-decoration: none;" title="Explore downstream recom
      | mended title in catalog">
0851:                                         <span class="badge" style="background: rgba(99, 102, 241, 0.15); col
      | or: #A5B4FC; border: 1px solid rgba(99, 102, 241, 0.35); font-size: 0.72rem; padding: 2px 8px; border-
      | radius: 9999px; display: inline-flex; align-items: center; gap: 5px; cursor: pointer; transition: all
      | 0.2s ease;">
0852:                                             <svg width="10" height="10" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="9 18 15 12 9 6"></polyline></svg>
0853:                                             Next to Study: <strong style="color: #E0E7FF;">{{ nxt.title }}</
      | strong>
0854:                                             {% if nxt.available_copies > 0 %}
0855:                                                 <span style="color: #34D399; font-weight: 700; margin-left:
      | 2px;">● {{ nxt.available_copies }} Avail</span>
0856:                                             {% else %}
0857:                                                 <span style="color: #F87171; font-weight: 700; margin-left:
      | 2px;">● Out of Stock</span>
0858:                                             {% endif %}
0859:                                         </span>
0860:                                     </a>
0861:                                     {% endfor %}
0862:                                 {% endif %}
0863:                             </div>
0864:                             {% endif %}
0865:                         </td>
0866:                         <td>Copy #{{ loan.copy_id }}</td>
0867:                         <td>{{ loan.issue_date.strftime('%b %d, %Y') if loan.issue_date else 'N/A' }}</td>
0868:                         <td>{{ loan.due_date.strftime('%b %d, %Y') if loan.due_date else 'N/A' }}</td>
0869:                         <td>
0870:                             {% if (loan.fine_amount or 0) > 0 %}
0871:                                 <span class="badge badge-danger">Rs. {{ "%.2f"|format(loan.fine_amount) }} U
      | npaid</span>
```

```
                            {% else %}
                                <span class="badge badge-success" style="background: rgba(16, 185, 129, 0.12
    | ); color: #34D399; border: 1px solid rgba(16, 185, 129, 0.3);">Zero Fine</span>
                            {% endif %}
                        </td>
                        <td>
                            {% if loan.is_overdue == 1 %}
                                <span class="badge badge-danger">Overdue</span>
                            {% else %}
                                <span class="badge badge-success">Active Loan</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="7" class="empty-state">No items currently checked out.</td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
<!-- Staff Loan Requests Queue Status -->
<h2 class="section-title">My Loan Requests</h2>
<div class="data-panel">
    <div class="panel-header">
        <span class="panel-title">Submitted Permission Requests</span>
        <span class="badge badge-info">{{ data.loan_requests|length if data.loan_requests else 0 }} Requests
    | </span>
    </div>
    <div class="table-container table-responsive">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Request Ref</th>
                    <th>Book Title</th>
                    <th>Requested On</th>
                    <th>Status</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody>
                {% if data.loan_requests %}
                    {% for req in data.loan_requests %}
                    <tr>
                        <td style="font-family: var(--font-mono); font-size: 0.8rem;">#REQ-{{ req.request_id
    |  }}</td>
                        <td class="book-title-cell">{{ req.book_title }}</td>
                        <td>{{ req.request_date.strftime('%b %d, %Y') if req.request_date else 'N/A' }}</td>
                        <td>
                            {% if req.status == 'Pending' %}
                                <span class="badge badge-warning">Awaiting Approval</span>
                            {% elif req.status == 'Awaiting Pickup' %}
                                <span class="badge badge-info" style="background: rgba(59, 130, 246, 0.15);
    | color: #60A5FA; border: 1px solid rgba(59, 130, 246, 0.35);">Approved &ndash; Ready for Pickup</span>
                            {% elif req.status == 'Approved' or req.status == 'Fulfilled' %}
                                <span class="badge badge-success">Fulfilled</span>
                            {% elif req.status == 'Rejected' %}
                                <span class="badge badge-danger">Rejected</span>
                            {% else %}
                                <span class="badge badge-secondary">{{ req.status }}</span>
                            {% endif %}
                        </td>
                        <td>
                            {% if req.status == 'Rejected' and req.book_id %}
                                <form method="POST" action="{{ url_for('request_loan', book_id=req.book_id)
    | }}" style="display: inline; margin: 0;">
                                    <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
                                    <input type="hidden" name="from_dashboard" value="1"/>
                                    <button type="submit" class="btn btn-sm" style="padding: 4px 10px; font-
    | size: 0.76rem; font-weight: 700; background: rgba(59, 130, 246, 0.15); color: #60A5FA; border: 1px sol
    | id rgba(59, 130, 246, 0.35); border-radius: var(--radius-sm); cursor: pointer;" onclick="return confir
    | m('Submit new loan request for \'{{ req.book_title|escape }}\'?');">
                                        ■ Request Again
                                    </button>
                                </form>
                            {% elif req.status == 'Awaiting Pickup' %}
                                <span style="font-size: 0.76rem; color: #60A5FA; font-weight: 600;">{% if re
    | q.copy_id %}Copy #{{ req.copy_id }} &bull; {% endif %}Collect at Desk</span>
```

```
                            {% elif req.status == 'Pending' %}
                                <span style="font-size: 0.76rem; color: var(--text-muted);">In Queue</span>
                            {% else %}
                                <span style="font-size: 0.76rem; color: var(--text-muted);">&mdash;</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="5" class="empty-state">No loan requests submitted. You can request book
    | s from the Catalog.</td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
{% endif %}
{% endblock %}
```

MODULE SOURCE: templates/issue_loan.html

Lines: 669 | **Lang:** HTML5 / Jinja2 / JavaScript | **Category:** Circulation Console

Core Role & Purpose: Manual book checkout interface featuring live substring autocomplete dropdowns for user selection and book selection.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:

- Verify that member search input uses live substring filtering (matching any part of the query against name or username).
- Verify that book title search filters titles containing the substring.
- Verify client-side validation that at least one physical copy is available before allowing form submission.

```
{% extends "base.html" %}

{% block title %}Issue Book Loan | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Issue Loan{% endblock %}

{% block extra_css %}
<style>
    .form-input,
    select.form-input,
    input[type="text"].form-input,
    input[type="number"].form-input {
        background: #0f172a !important;
        color: #f8fafc !important;
        border: 1px solid #334155 !important;
        border-radius: 0.5rem !important;
        padding: 0.65rem 0.85rem !important;
        width: 100% !important;
        box-sizing: border-box !important;
        font-size: 0.88rem;
        outline: none;
        transition: border-color 0.15s ease, box-shadow 0.15s ease;
    }
    .form-input:focus {
        border-color: #3b82f6 !important;
        box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.25) !important;
    }
    select.form-input option,
    select.form-input optgroup {
        background: #0f172a;
        color: #f8fafc;
    }
    .filter-wrapper {
        position: relative;
        width: 100%;
        box-sizing: border-box;
    }
    .filter-wrapper input.form-input {
        padding-left: 38px !important;
        width: 100% !important;
    }
    .filter-icon {
        position: absolute;
        left: 12px;
        top: 50%;
        transform: translateY(-50%);
        color: var(--text-muted);
        pointer-events: none;
        width: 16px;
        height: 16px;
        z-index: 2;
    }
    .autocomplete-dropdown {
        position: absolute;
        top: calc(100% + 4px);
        left: 0;
        right: 0;
        background: #0f172a;
        border: 1px solid #334155;
        border-radius: 0.5rem;
        max-height: 250px;
        overflow-y: auto;
        z-index: 1050;
        box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.7), 0 8px 10px -6px rgba(0, 0, 0, 0.6);
    }
    .autocomplete-dropdown::-webkit-scrollbar {
        width: 6px;
    }
    .autocomplete-dropdown::-webkit-scrollbar-thumb {
        background: #334155;
        border-radius: 3px;
```

```
0071:     }
0072:     .autocomplete-item {
0073:         padding: 0.65rem 0.85rem;
0074:         cursor: pointer;
0075:         border-bottom: 1px solid rgba(51, 65, 85, 0.4);
0076:         display: flex;
0077:         align-items: center;
0078:         justify-content: space-between;
0079:         gap: 10px;
0080:         transition: background 0.15s ease;
```

```
0081:     }
0082:     .autocomplete-item:last-child {
0083:         border-bottom: none;
0084:     }
0085:     .autocomplete-item:hover,
0086:     .autocomplete-item.active {
0087:         background: #1e293b;
0088:     }
0089:     .autocomplete-highlight {
0090:         color: #60a5fa;
0091:         font-weight: 700;
0092:         text-decoration: underline;
0093:         text-decoration-color: rgba(96, 165, 250, 0.6);
0094:     }
0095:     .autocomplete-empty {
0096:         padding: 0.85rem 1rem;
0097:         color: #94a3b8;
0098:         font-size: 0.82rem;
0099:         text-align: center;
0100:     }
0101: </style>
0102: {% endblock %}
0103:
0104: {% block content %}
0105: <div style="max-width: 720px; margin: 0 auto;">
0106:     <div style="margin-bottom: 28px;">
0107:         <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">Issue Book Loan</h1>
0108:         <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">Assign an available ph
      | ysical copy to a registered patron</p>
0109:     </div>
0110:
0111:     <div class="login-card" style="max-width: 100%; padding: 32px; background: var(--bg-surface); border: 1p
      | x solid var(--border-subtle); border-radius: var(--radius-lg);">
0112:         <form method="POST" action="{{ url_for('issue_loan') }}" id="issueLoanForm">
0113:             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0114:
0115:             <!-- 1. BOOK SELECTION DROPDOWN MENU -->
0116:             <div class="form-group" style="margin-bottom: 22px;">
0117:                 <label class="form-label" for="book_id" style="display: block; margin-bottom: 8px; font-weig
      | ht: 600;">Select Book from Catalog *</label>
0118:                 <div style="display: flex; flex-direction: column; gap: 8px; width: 100%;">
0119:                     <!-- Quick Book Filter Search Input (100% width, no duplicate emoji) -->
0120:                     <div class="filter-wrapper" id="book_filter_wrapper">
0121:                         <input
0122:                             type="text"
0123:                             id="book_quick_search"
0124:                             class="form-input"
0125:                             placeholder="Type to search books by title, author, or ISBN..."
0126:                             autocomplete="off"
0127:                         >
0128:                         <svg class="filter-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strok
      | e-width="2">
0129:                             <circle cx="11" cy="11" r="8"></circle>
0130:                             <line x1="21" y1="21" x2="16.65" y2="16.65"></line>
0131:                         </svg>
0132:                         <div id="book_dropdown_list" class="autocomplete-dropdown" style="display: none;"></
      | div>
0133:                     </div>
0134:
0135:                     <!-- Books Dropdown Menu -->
0136:                     <select id="book_id" name="book_id" class="form-input" style="cursor: pointer;">
0137:                         <option value="">-- Choose Book from Dropdown Menu --</option>
0138:                         {% if all_books %}
0139:                             <optgroup label="Available in Inventory" id="availBooksGroup">
0140:                                 {% for b in all_books if (b.available_copies_count or 0) > 0 %}
0141:                                     <option
0142:                                         value="{{ b.book_id }}"
0143:                                         data-avail="{{ b.available_copies_count }}"
0144:                                         data-search="{{ b.title|lower }} {{ (b.authors or '')|lower }} {{ b.
      | category_name|lower }} {{ b.isbn|lower }}"
0145:                                     >
0146:                                         {{ b.title }} &mdash; [{{ b.category_name }}] ({{ b.available_copies
      | _count }} avail / {{ b.total_copies_count }} total)
0147:                                     </option>
0148:                                 {% endfor %}
0149:                             </optgroup>
0150:                             <optgroup label="Currently Out of Stock" id="outOfStockGroup">
0151:                                 {% for b in all_books if (b.available_copies_count or 0) == 0 %}
0152:                                     <option
0153:                                         value="{{ b.book_id }}"
```

```
0154:                                       data-avail="0"
0155:                                       data-search="{{ b.title|lower }} {{ (b.authors or '')|lower }} {{ b.
      | category_name|lower }} {{ b.isbn|lower }}"
0156:                                       style="opacity: 0.6;"
0157:                                   >
0158:                                       {{ b.title }} &mdash; [{{ b.category_name }}] (0 Available - Out of
      | Stock)
0159:                                   </option>
0160:                               {% endfor %}
0161:                           </optgroup>
0162:                       {% endif %}
0163:                   </select>
0164:               </div>
0165:               <small style="color: var(--text-muted); font-size: 0.75rem; margin-top: 4px; display: block;
      | ">Select the book you wish to issue to automatically populate available physical copies.</small>
0166:           </div>
0167:
0168:           <!-- 2. PHYSICAL COPY ID / BARCODE SELECTION -->
0169:           <div class="form-group" style="margin-bottom: 22px;">
0170:               <label class="form-label" for="copy_select" style="display: block; margin-bottom: 8px; font-
      | weight: 600;">Physical Copy ID / Barcode *</label>
0171:               <div style="display: flex; flex-direction: column; gap: 8px; width: 100%;">
0172:                   <!-- Dropdown for specific physical copy of the chosen book -->
0173:                   <select id="copy_select" name="copy_id" class="form-input" style="cursor: pointer;" requ
      | ired>
0174:                       <option value="">-- Choose Specific Copy --</option>
0175:                       {% if available_copies %}
0176:                           {% for c in available_copies %}
0177:                               <option
0178:                                   value="{{ c.copy_id }}"
0179:                                   data-book-id="{{ c.book_id }}"
0180:                                   data-title="{{ c.book_title }}"
0181:                               >
0182:                                   Copy #{{ c.copy_id }} &mdash; {{ c.book_title }} [Condition: {{ c.condit
      | ion }} | Status: {{ c.status }}]
0183:                               </option>
0184:                           {% endfor %}
0185:                       {% endif %}
0186:                   </select>
0187:
0188:                   <!-- Hidden raw Copy ID input kept synchronized for script compatibility without raw whi
      | te input box -->
0189:                   <input type="hidden" id="copy_id" value="">
0190:
0191:                   <div>
0192:                       <span id="copy_status_badge" style="font-size: 0.78rem; font-weight: 700; padding: 6
      | px 12px; border-radius: var(--radius-sm); display: none; white-space: nowrap;"></span>
0193:                   </div>
0194:               </div>
0195:               <small style="color: var(--text-muted); font-size: 0.75rem; margin-top: 4px; display: block;
      | ">Selecting a book above automatically picks the first available physical copy from inventory.</small>
0196:           </div>
0197:
0198:           <!-- 3. PATRON USERNAME DROPDOWN MENU WITH ROLES -->
0199:           <div class="form-group" style="margin-bottom: 22px;">
0200:               <label class="form-label" for="borrower_username" style="display: block; margin-bottom: 8px;
      |  font-weight: 600;">Patron Username (Select Member &amp;amp; Role) *</label>
0201:               <div style="display: flex; flex-direction: column; gap: 8px; width: 100%;">
0202:                   <!-- Quick Member Filter Search Input (100% width, no duplicate emoji) -->
0203:                   <div class="filter-wrapper" id="patron_filter_wrapper">
0204:                       <input
0205:                           type="text"
0206:                           id="patron_quick_search"
0207:                           name="patron_quick_search"
0208:                           class="form-input"
0209:                           placeholder="Type patron name substring to search..."
0210:                           autocomplete="off"
0211:                       >
0212:                       <svg class="filter-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" strok
      | e-width="2">
0213:                           <circle cx="11" cy="11" r="8"></circle>
0214:                           <line x1="21" y1="21" x2="16.65" y2="16.65"></line>
0215:                       </svg>
0216:                       <div id="patron_dropdown_list" class="autocomplete-dropdown" style="display: none;">
      | </div>
0217:                   </div>
0218:
0219:                   <!-- Direct Patron Username Dropdown Menu -->
0220:                   <select id="borrower_username" name="borrower_username" class="form-input" required size
      | ="1" style="cursor: pointer;">
```

```
0221:                         <option value="">-- Choose Member from Dropdown Menu --</option>
0222:                         {% if patrons %}
0223:                             <optgroup label="Staff Members" id="staffGroup">
0224:                                 {% for p in patrons if p.role == 'Staff' %}
0225:                                     <option value="{{ p.username }}" data-role="staff" data-search="{{ p.use
      | rname|lower }} staff {{ p.email|lower }}{% if p.has_blocking_issue %} overdue fine warning{% endif %}"
      | >
0226:                                         {{ p.username }} &mdash; [Staff] (ID: #{{ p.user_id }}){% if p.has_b
      | locking_issue %} [■■ Has Overdue / Unpaid Fine]{% endif %}
0227:                                     </option>
0228:                                 {% endfor %}
0229:                             </optgroup>
0230:                             <optgroup label="Students" id="studentGroup">
0231:                                 {% for p in patrons if p.role == 'Student' %}
0232:                                     <option value="{{ p.username }}" data-role="student" data-search="{{ p.u
      | sername|lower }} student {{ p.email|lower }}{% if p.has_blocking_issue %} overdue fine warning{% endif
      |  %}">
0233:                                         {{ p.username }} &mdash; [Student] (ID: #{{ p.user_id }}){% if p.has
      | _blocking_issue %} [■■ Has Overdue / Unpaid Fine]{% endif %}
0234:                                     </option>
0235:                                 {% endfor %}
0236:                             </optgroup>
0237:                         {% endif %}
0238:                     </select>
0239:                 </div>
0240:                 <small style="color: var(--text-muted); font-size: 0.75rem; margin-top: 4px; display: block;
      | ">Select the borrowing member from the dropdown menu (grouped by Staff and Student).</small>
0241:             </div>
0242:
0243:             <!-- 4. LOAN DURATION (Block Label) -->
0244:             <div class="form-group" style="margin-bottom: 28px;">
0245:                 <label class="form-label" for="due_in_days" style="display: block; margin-bottom: 8px; font-
      | weight: 600;">Loan Duration (Days)</label>
0246:                 <input type="number" id="due_in_days" name="due_in_days" class="form-input" value="14" min="
      | 1" max="60" required>
0247:                 <small style="color: var(--text-muted); font-size: 0.75rem; margin-top: 6px; display: block;
      | ">Standard policy is 14 days before Rs. 10/day overdue fine applies.</small>
0248:             </div>
0249:
0250:             <div style="display: flex; gap: 12px; justify-content: flex-end;">
0251:                 <a href="{{ url_for('dashboard') }}" class="btn btn-secondary">Cancel</a>
0252:                 <button type="submit" class="btn btn-primary" id="btnIssueLoan">Confirm &amp; Issue Book</bu
      | tton>
0253:             </div>
0254:         </form>
0255:     </div>
0256: </div>
0257: {% endblock %}
0258:
0259: {% block extra_js %}
0260: <script id="patrons-data" type="application/json">{{ (patrons or [])|tojson }}</script>
0261: <script id="books-data" type="application/json">{{ (all_books or [])|tojson }}</script>
0262: <script>
0263:     document.addEventListener('DOMContentLoaded', function() {
0264:         // Elements
0265:         const bookSearch = document.getElementById('book_quick_search');
0266:         const bookDropdown = document.getElementById('book_dropdown_list');
0267:         const bookWrapper = document.getElementById('book_filter_wrapper');
0268:         const bookSelect = document.getElementById('book_id');
0269:         const copySelect = document.getElementById('copy_select');
0270:         const copyInput = document.getElementById('copy_id');
0271:         const copyBadge = document.getElementById('copy_status_badge');
0272:
0273:         const patronSearch = document.getElementById('patron_quick_search');
0274:         const patronDropdown = document.getElementById('patron_dropdown_list');
0275:         const patronWrapper = document.getElementById('patron_filter_wrapper');
0276:         const patronSelect = document.getElementById('borrower_username');
0277:         const issueLoanForm = document.getElementById('issueLoanForm');
0278:
0279:         const patronsData = JSON.parse(document.getElementById('patrons-data') ? document.getElementById('pa
      | trons-data').textContent : '[]');
0280:         const booksData = JSON.parse(document.getElementById('books-data') ? document.getElementById('books-
      | data').textContent : '[]');
0281:
0282:         // All copy options
0283:         const allCopyOptions = copySelect ? Array.from(copySelect.querySelectorAll('option:not([value=""])')
      | ) : [];
0284:
0285:         function escapeHtml(str) {
0286:             if (!str) return '';
```

```
0287:             return String(str)
0288:                 .replace(/&/g, '&amp;')
0289:                 .replace(/</g, '&lt;')
0290:                 .replace(/>/g, '&gt;')
0291:                 .replace(/"/g, '&quot;')
0292:                 .replace(/'/g, '&#039;');
0293:         }
0294:
0295:         function highlightSubstring(text, query) {
0296:             if (!query || !text) return escapeHtml(text || '');
0297:             const lowerText = text.toLowerCase();
0298:             const lowerQuery = query.toLowerCase();
0299:             const idx = lowerText.indexOf(lowerQuery);
0300:             if (idx === -1) return escapeHtml(text);
0301:             return escapeHtml(text.substring(0, idx)) +
0302:                    '<span class="autocomplete-highlight">' +
0303:                    escapeHtml(text.substring(idx, idx + query.length)) +
0304:                    '</span>' +
0305:                    escapeHtml(text.substring(idx + query.length));
0306:         }
0307:
0308:         function updateActiveItem(items, activeIdx) {
0309:             items.forEach((it, idx) => {
0310:                 if (idx === activeIdx) {
0311:                     it.classList.add('active');
0312:                     it.scrollIntoView({ block: 'nearest' });
0313:                 } else {
0314:                     it.classList.remove('active');
0315:                 }
0316:             });
0317:         }
0318:
0319:         // ========================================
0320:         // 1. PATRON SUBSTRING DROPDOWN AUTOCOMPLETE
0321:         // ========================================
0322:         let activePatronIdx = -1;
0323:
0324:         function renderPatronDropdown(filtered, query) {
0325:             if (!patronDropdown) return;
0326:             patronDropdown.innerHTML = '';
0327:             activePatronIdx = -1;
0328:
0329:             if (filtered.length === 0) {
0330:                 patronDropdown.innerHTML = `<div class="autocomplete-empty">No members found containing "<st
      | rong>${escapeHtml(query)}</strong>"</div>`;
0331:                 patronDropdown.style.display = 'block';
0332:                 return;
0333:             }
0334:
0335:             filtered.forEach((p, idx) => {
0336:                 const item = document.createElement('div');
0337:                 item.className = 'autocomplete-item';
0338:                 item.setAttribute('data-username', p.username);
0339:                 item.setAttribute('data-index', idx);
0340:
0341:                 const roleBadge = p.role === 'Staff'
0342:                     ? `<span class="badge" style="background: rgba(168, 85, 247, 0.15); color: #C084FC; bord
      | er: 1px solid rgba(168, 85, 247, 0.3); font-size: 0.7rem; padding: 2px 7px;">Staff</span>`
0343:                     : `<span class="badge" style="background: rgba(59, 130, 246, 0.15); color: #93c5fd; bord
      | er: 1px solid rgba(59, 130, 246, 0.3); font-size: 0.7rem; padding: 2px 7px;">Student</span>`;
0344:
0345:                 const issueBadge = p.has_blocking_issue
0346:                     ? `<span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px 6px;" title=
      | "Overdue loans or unpaid fines exist">■■ Issue</span>`
0347:                     : '';
0348:
0349:                 item.innerHTML = `
0350:                     <div style="display: flex; flex-direction: column; gap: 2px; overflow: hidden; text-alig
      | n: left;">
0351:                         <div style="font-weight: 600; color: #f8fafc; display: flex; align-items: center; ga
      | p: 6px;">
0352:                             <span style="font-size: 0.9rem;">${highlightSubstring(p.username, query)}</span>
0353:                             <span style="font-size: 0.72rem; color: #94a3b8; font-family: var(--font-mono);"
      | >#USR-${p.user_id}</span>
0354:                         </div>
0355:                         <div style="font-size: 0.76rem; color: #94a3b8; text-overflow: ellipsis; overflow: h
      | idden; white-space: nowrap;">
0356:                             ${highlightSubstring(p.email || '', query)}
0357:                         </div>
0358:                     </div>
```

```
                    <div style="display: flex; align-items: center; gap: 6px; flex-shrink: 0;">
                        ${roleBadge}
                        ${issueBadge}
                    </div>
                `;

                item.addEventListener('click', function(e) {
                    e.stopPropagation();
                    selectPatron(p.username);
                });

                patronDropdown.appendChild(item);
            });

            patronDropdown.style.display = 'block';
        }

        function selectPatron(username) {
            if (!username) return;
            if (patronSearch) patronSearch.value = username;
            if (patronSelect) {
                patronSelect.value = username;
                patronSelect.dispatchEvent(new Event('change'));
            }
            if (patronDropdown) patronDropdown.style.display = 'none';
        }

        function filterPatrons() {
            const query = (patronSearch ? patronSearch.value : '').trim().toLowerCase();
            if (!query) {
                renderPatronDropdown(patronsData, '');
                return;
            }
            const filtered = patronsData.filter(p => {
                const unameMatch = p.username && p.username.toLowerCase().includes(query);
                const emailMatch = p.email && p.email.toLowerCase().includes(query);
                const roleMatch = p.role && p.role.toLowerCase().includes(query);
                return unameMatch || emailMatch || roleMatch;
            });
            renderPatronDropdown(filtered, query);
        }

        if (patronSearch && patronDropdown) {
            patronSearch.addEventListener('input', filterPatrons);
            patronSearch.addEventListener('focus', filterPatrons);

            patronSearch.addEventListener('keydown', function(e) {
                const items = patronDropdown.querySelectorAll('.autocomplete-item');
                if (!items.length || patronDropdown.style.display === 'none') {
                    if (e.key === 'ArrowDown') {
                        filterPatrons();
                        e.preventDefault();
                    }
                    return;
                }

                if (e.key === 'ArrowDown') {
                    e.preventDefault();
                    activePatronIdx = (activePatronIdx + 1) % items.length;
                    updateActiveItem(items, activePatronIdx);
                } else if (e.key === 'ArrowUp') {
                    e.preventDefault();
                    activePatronIdx = (activePatronIdx - 1 + items.length) % items.length;
                    updateActiveItem(items, activePatronIdx);
                } else if (e.key === 'Enter') {
                    e.preventDefault();
                    if (activePatronIdx >= 0 && activePatronIdx < items.length) {
                        items[activePatronIdx].click();
                    } else if (items.length > 0) {
                        items[0].click();
                    }
                } else if (e.key === 'Escape') {
                    patronDropdown.style.display = 'none';
                }
            });
        }

        // ========================================
        // 2. BOOK AUTOCOMPLETE DROPDOWN
        // ========================================
```

```
0439:         let activeBookIdx = -1;
0440:
0441:         function renderBookDropdown(filtered, query) {
0442:             if (!bookDropdown) return;
0443:             bookDropdown.innerHTML = '';
0444:             activeBookIdx = -1;
0445:
0446:             if (filtered.length === 0) {
0447:                 bookDropdown.innerHTML = `<div class="autocomplete-empty">No books found containing "<strong
      | >${escapeHtml(query)}</strong>"</div>`;
0448:                 bookDropdown.style.display = 'block';
0449:                 return;
0450:             }
0451:
0452:             filtered.forEach((b, idx) => {
0453:                 const item = document.createElement('div');
0454:                 item.className = 'autocomplete-item';
0455:                 item.setAttribute('data-book-id', b.book_id);
0456:                 item.setAttribute('data-index', idx);
0457:
0458:                 const avail = b.available_copies_count || 0;
0459:                 const availBadge = avail > 0
0460:                     ? `<span class="badge" style="background: rgba(16, 185, 129, 0.15); color: #34D399; bord
      | er: 1px solid rgba(16, 185, 129, 0.3); font-size: 0.7rem; padding: 2px 7px;">${avail} Avail</span>`
0461:                     : `<span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px 6px;">Out of
      |  Stock</span>`;
0462:
0463:                 item.innerHTML = `
0464:                     <div style="display: flex; flex-direction: column; gap: 2px; overflow: hidden; text-alig
      | n: left;">
0465:                         <div style="font-weight: 600; color: #f8fafc; font-size: 0.88rem;">
0466:                             ${highlightSubstring(b.title, query)}
0467:                         </div>
0468:                         <div style="font-size: 0.75rem; color: #94a3b8; text-overflow: ellipsis; overflow: h
      | idden; white-space: nowrap;">
0469:                             [${escapeHtml(b.category_name)}] &middot; ${highlightSubstring(b.authors || 'Unk
      | nown Author', query)} &middot; ISBN: ${highlightSubstring(b.isbn || '', query)}
0470:                         </div>
0471:                     </div>
0472:                     <div style="display: flex; align-items: center; gap: 6px; flex-shrink: 0;">
0473:                         ${availBadge}
0474:                     </div>
0475:                 `;
0476:
0477:                 item.addEventListener('click', function(e) {
0478:                     e.stopPropagation();
0479:                     selectBook(b.book_id, b.title);
0480:                 });
0481:
0482:                 bookDropdown.appendChild(item);
0483:             });
0484:
0485:             bookDropdown.style.display = 'block';
0486:         }
0487:
0488:         function selectBook(bookId, title) {
0489:             if (!bookId) return;
0490:             if (bookSearch) bookSearch.value = title || '';
0491:             if (bookSelect) {
0492:                 bookSelect.value = String(bookId);
0493:                 bookSelect.dispatchEvent(new Event('change'));
0494:             }
0495:             if (bookDropdown) bookDropdown.style.display = 'none';
0496:         }
0497:
0498:         function filterBooks() {
0499:             const query = (bookSearch ? bookSearch.value : '').trim().toLowerCase();
0500:             if (!query) {
0501:                 renderBookDropdown(booksData, '');
0502:                 return;
0503:             }
0504:             const filtered = booksData.filter(b => {
0505:                 const titleMatch = b.title && b.title.toLowerCase().includes(query);
0506:                 const isbnMatch = b.isbn && b.isbn.toLowerCase().includes(query);
0507:                 const authorMatch = b.authors && b.authors.toLowerCase().includes(query);
0508:                 const catMatch = b.category_name && b.category_name.toLowerCase().includes(query);
0509:                 return titleMatch || isbnMatch || authorMatch || catMatch;
0510:             });
0511:             renderBookDropdown(filtered, query);
0512:         }
```

```
0513:
0514:         if (bookSearch && bookDropdown) {
0515:             bookSearch.addEventListener('input', filterBooks);
0516:             bookSearch.addEventListener('focus', filterBooks);
0517:
0518:             bookSearch.addEventListener('keydown', function(e) {
0519:                 const items = bookDropdown.querySelectorAll('.autocomplete-item');
0520:                 if (!items.length || bookDropdown.style.display === 'none') {
0521:                     if (e.key === 'ArrowDown') {
0522:                         filterBooks();
0523:                         e.preventDefault();
0524:                     }
0525:                     return;
0526:                 }
0527:
0528:                 if (e.key === 'ArrowDown') {
0529:                     e.preventDefault();
0530:                     activeBookIdx = (activeBookIdx + 1) % items.length;
0531:                     updateActiveItem(items, activeBookIdx);
0532:                 } else if (e.key === 'ArrowUp') {
0533:                     e.preventDefault();
0534:                     activeBookIdx = (activeBookIdx - 1 + items.length) % items.length;
0535:                     updateActiveItem(items, activeBookIdx);
0536:                 } else if (e.key === 'Enter') {
0537:                     e.preventDefault();
0538:                     if (activeBookIdx >= 0 && activeBookIdx < items.length) {
0539:                         items[activeBookIdx].click();
0540:                     } else if (items.length > 0) {
0541:                         items[0].click();
0542:                     }
0543:                 } else if (e.key === 'Escape') {
0544:                     bookDropdown.style.display = 'none';
0545:                 }
0546:             });
0547:         }
0548:
0549:         // Close dropdowns when clicking outside
0550:         document.addEventListener('click', function(e) {
0551:             if (patronWrapper && !patronWrapper.contains(e.target)) {
0552:                 if (patronDropdown) patronDropdown.style.display = 'none';
0553:             }
0554:             if (bookWrapper && !bookWrapper.contains(e.target)) {
0555:                 if (bookDropdown) bookDropdown.style.display = 'none';
0556:             }
0557:         });
0558:
0559:         // Sync native select changes to text inputs
0560:         if (patronSelect && patronSearch) {
0561:             patronSelect.addEventListener('change', function() {
0562:                 if (this.value) {
0563:                     patronSearch.value = this.value;
0564:                 }
0565:             });
0566:         }
0567:
0568:         if (bookSelect && bookSearch) {
0569:             bookSelect.addEventListener('change', function() {
0570:                 const opt = this.options[this.selectedIndex];
0571:                 if (opt && opt.value) {
0572:                     const found = booksData.find(b => String(b.book_id) === String(opt.value));
0573:                     if (found) {
0574:                         bookSearch.value = found.title;
0575:                     }
0576:                 }
0577:             });
0578:         }
0579:
0580:         // Auto copy selection when book changes
0581:         if (bookSelect && copySelect && copyInput) {
0582:             bookSelect.addEventListener('change', function() {
0583:                 const selectedBookId = this.value;
0584:                 if (!selectedBookId) {
0585:                     allCopyOptions.forEach(opt => opt.style.display = '');
0586:                     if (copyBadge) copyBadge.style.display = 'none';
0587:                     return;
0588:                 }
0589:
0590:                 let matchingCopies = [];
0591:                 allCopyOptions.forEach(opt => {
0592:                     if (opt.getAttribute('data-book-id') === selectedBookId) {
```

```
                        opt.style.display = '';
                        matchingCopies.push(opt);
                    } else {
                        opt.style.display = 'none';
                    }
                });

                if (matchingCopies.length > 0) {
                    copySelect.value = matchingCopies[0].value;
                    copyInput.value = matchingCopies[0].value;

                    if (copyBadge) {
                        copyBadge.style.display = 'inline-block';
                        copyBadge.style.background = 'rgba(16, 185, 129, 0.15)';
                        copyBadge.style.color = '#34D399';
                        copyBadge.style.border = '1px solid rgba(16, 185, 129, 0.35)';
                        copyBadge.textContent = `✓ ${matchingCopies.length} Copy/Copies Available`;
                    }
                } else {
                    copySelect.value = '';
                    copyInput.value = '';
                    if (copyBadge) {
                        copyBadge.style.display = 'inline-block';
                        copyBadge.style.background = 'rgba(239, 68, 68, 0.15)';
                        copyBadge.style.color = '#F87171';
                        copyBadge.style.border = '1px solid rgba(239, 68, 68, 0.35)';
                        copyBadge.textContent = '■■ Out of Stock';
                    }
                }
            });
        }

        if (copySelect && copyInput) {
            copySelect.addEventListener('change', function() {
                if (this.value) {
                    copyInput.value = this.value;
                    const opt = copySelect.querySelector(`option[value="${this.value}"]`);
                    if (opt && bookSelect) {
                        const bId = opt.getAttribute('data-book-id');
                        if (bId && bookSelect.value !== bId) {
                            bookSelect.value = bId;
                            bookSelect.dispatchEvent(new Event('change'));
                        }
                    }
                    if (copyBadge) copyBadge.style.display = 'none';
                }
            });
        }

        // Form submit safety
        if (issueLoanForm) {
            issueLoanForm.addEventListener('submit', function() {
                if (bookSelect && !bookSelect.value && bookSearch && bookSearch.value.trim()) {
                    const bq = bookSearch.value.trim().toLowerCase();
                    const bmatch = booksData.find(b =>
                        (b.title && b.title.toLowerCase() === bq) ||
                        (b.isbn && b.isbn.toLowerCase() === bq) ||
                        (b.title && b.title.toLowerCase().includes(bq)) ||
                        (b.isbn && b.isbn.toLowerCase().includes(bq))
                    );
                    if (bmatch) {
                        bookSelect.value = bmatch.book_id;
                        bookSelect.dispatchEvent(new Event('change'));
                    }
                }
                if (patronSelect && !patronSelect.value && patronSearch && patronSearch.value.trim()) {
                    const q = patronSearch.value.trim().toLowerCase();
                    const match = patronsData.find(p => p.username.toLowerCase() === q || p.username.toLower
    | Case().includes(q));
                    if (match) {
                        patronSelect.value = match.username;
                    }
                }
            });
        }
    });
</script>
{% endblock %}
```

MODULE SOURCE: templates/loan_requests.html

Lines: 419 | **Lang:** HTML5 / Jinja2 | **Category:** Circulation Queue

Core Role & Purpose: Administrative loan request review and approval queue.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify table header typo fix: column displays 'REQUEST TIMESTAMP' with a space.
- Verify Approve and Reject buttons include CSRF tokens and target correct route endpoints.

```
{% extends "base.html" %}

{% block title %}Loan Requests & Collection Desk | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Loan Requests Queue{% endblock %}

{% block extra_css %}
<style>
    .queue-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 28px;
        flex-wrap: wrap;
        gap: 16px;
```

```
0015:     }
0016:
0017:     .queue-title {
0018:         font-size: 1.6rem;
0019:         font-weight: 800;
0020:         letter-spacing: -0.03em;
0021:         color: var(--text-primary);
0022:     }
0023:
0024:     .queue-sub {
0025:         font-size: 0.88rem;
0026:         color: var(--text-secondary);
0027:         margin-top: 4px;
0028:     }
0029:
0030:     .btn-approve {
0031:         display: inline-flex;
0032:         align-items: center;
0033:         gap: 6px;
0034:         background: rgba(16, 185, 129, 0.15);
0035:         color: #34D399;
0036:         border: 1px solid rgba(16, 185, 129, 0.35);
0037:         border-radius: var(--radius-sm);
0038:         padding: 6px 14px;
0039:         font-size: 0.8rem;
0040:         font-weight: 700;
0041:         cursor: pointer;
0042:         transition: all 0.15s ease;
0043:     }
0044:
0045:     .btn-approve:hover {
0046:         background: #059669;
0047:         color: #FFFFFF;
0048:         border-color: #059669;
0049:         box-shadow: 0 2px 8px rgba(5, 150, 105, 0.35);
0050:         transform: translateY(-1px);
0051:     }
0052:
0053:     .btn-confirm-pickup {
0054:         display: inline-flex;
0055:         align-items: center;
0056:         gap: 6px;
0057:         background: #059669;
0058:         color: #FFFFFF;
0059:         border: 1px solid #059669;
0060:         border-radius: var(--radius-sm);
0061:         padding: 7px 16px;
0062:         font-size: 0.82rem;
0063:         font-weight: 700;
0064:         cursor: pointer;
0065:         transition: all 0.15s ease;
0066:         box-shadow: 0 2px 8px rgba(5, 150, 105, 0.3);
0067:     }
0068:
0069:     .btn-confirm-pickup:hover {
0070:         background: #10B981;
0071:         border-color: #10B981;
0072:         box-shadow: 0 4px 12px rgba(16, 185, 129, 0.4);
0073:         transform: translateY(-1px);
0074:     }
0075:
0076:     .btn-reject {
0077:         display: inline-flex;
0078:         align-items: center;
0079:         gap: 6px;
0080:         background: rgba(239, 68, 68, 0.12);
```

```
0081:         color: #F87171;
0082:         border: 1px solid rgba(239, 68, 68, 0.3);
0083:         border-radius: var(--radius-sm);
0084:         padding: 6px 14px;
0085:         font-size: 0.8rem;
0086:         font-weight: 700;
0087:         cursor: pointer;
0088:         transition: all 0.15s ease;
0089:     }
0090:
0091:     .btn-reject:hover {
0092:         background: #DC2626;
0093:         color: #FFFFFF;
0094:         border-color: #DC2626;
0095:         box-shadow: 0 2px 8px rgba(220, 38, 38, 0.35);
0096:         transform: translateY(-1px);
0097:     }
0098:
0099:     .table-container {
0100:         width: 100%;
0101:         overflow-x: auto;
0102:     }
0103:
0104:     .data-table {
0105:         width: 100%;
0106:         min-width: 780px;
0107:         border-collapse: collapse;
0108:         text-align: left;
0109:         font-size: 0.88rem;
0110:     }
0111:
0112:     .data-table th {
0113:         background: rgba(255, 255, 255, 0.02);
0114:         color: var(--text-muted);
0115:         font-size: 0.72rem;
0116:         text-transform: uppercase;
0117:         letter-spacing: 0.06em;
0118:         font-weight: 600;
0119:         padding: 0.85rem 1rem;
0120:         border-bottom: 1px solid var(--border-subtle);
0121:         white-space: nowrap;
0122:     }
0123:
0124:     .data-table td {
0125:         padding: 0.85rem 1rem;
0126:         border-bottom: 1px solid var(--border-subtle);
0127:         color: var(--text-secondary);
0128:         vertical-align: middle;
0129:     }
0130:
0131:     .data-table tr:last-child td {
0132:         border-bottom: none;
0133:     }
0134:
0135:     .data-table tr:hover td {
0136:         background: rgba(255, 255, 255, 0.015);
0137:         color: var(--text-primary);
0138:     }
0139: </style>
0140: {% endblock %}
0141:
0142: {% block content %}
0143: <div class="queue-header">
0144:     <div>
0145:         <h1 class="queue-title">Borrowing Permission Requests &amp;amp; Collection Desk</h1>
0146:         <p class="queue-sub">Authorize student/staff loan requests and record physical book collections when
      |  patrons pick up their books</p>
0147:     </div>
0148:     <div style="display: flex; gap: 10px;">
0149:         <span class="badge" style="background: rgba(59, 130, 246, 0.15); color: #60A5FA; border: 1px solid r
      | gba(59, 130, 246, 0.35); font-size: 0.85rem; padding: 6px 14px;">
0150:             {{ awaiting_pickup|length if awaiting_pickup else 0 }} Awaiting Pickup
0151:         </span>
0152:         <span class="badge badge-info" style="font-size: 0.85rem; padding: 6px 14px;">
0153:             {{ requests|length if requests else 0 }} Pending Review
0154:         </span>
0155:     </div>
0156: </div>
0157:
0158: <!-- ========================================================================== -->
```

```
0159: <!-- 1. COLLECTION DESK: AWAITING PHYSICAL PICKUP TABLE                        -->
0160: <!-- ========================================================================== -->
0161: <div class="data-panel" style="margin-bottom: 32px; border: 1px solid rgba(59, 130, 246, 0.25);">
0162:     <div class="panel-header" style="background: rgba(59, 130, 246, 0.05);">
0163:         <div>
0164:             <span class="panel-title" style="color: #60A5FA; display: flex; align-items: center; gap: 8px;">
0165:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2.5"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></path><polyline p
      | oints="10 2 10 10 13 7 16 10 16 2"></polyline></svg>
0166:                 Circulation Desk: Ready for Physical Collection
0167:             </span>
0168:             <div style="font-size: 0.78rem; color: var(--text-muted); margin-top: 2px;">
0169:                 Patrons who have had their requests approved and physical copies reserved. Confirm collectio
      | n when the patron receives the book.
0170:             </div>
0171:         </div>
0172:         <div>
0173:             <span class="badge" style="background: rgba(59, 130, 246, 0.2); color: #93C5FD; font-size: 0.78r
      | em; padding: 4px 10px;">
0174:                 {{ awaiting_pickup|length if awaiting_pickup else 0 }} Ready for Handover
0175:             </span>
0176:         </div>
0177:     </div>
0178:
0179:     <div class="table-container">
0180:         <table class="data-table">
0181:             <thead>
0182:                 <tr>
0183:                     <th>Request ID</th>
0184:                     <th>Patron Name &amp;amp; Role</th>
0185:                     <th>Book Details</th>
0186:                     <th>Reserved Copy ID</th>
0187:                     <th>Request Timestamp</th>
0188:                     <th style="text-align: right;">Handover Action</th>
0189:                 </tr>
0190:             </thead>
0191:             <tbody>
0192:                 {% if awaiting_pickup %}
0193:                     {% for p in awaiting_pickup %}
0194:                     <tr>
0195:                         <td style="font-family: var(--font-mono); font-size: 0.82rem; font-weight: 600;">
0196:                             #REQ-{{ p.request_id }}
0197:                         </td>
0198:                         <td>
0199:                             <div style="font-weight: 600; color: var(--text-primary);">{{ p.borrower_name }}
      | </div>
0200:                             <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; align-i
      | tems: center; gap: 6px; margin-top: 2px;">
0201:                                 <span>{{ p.borrower_email }}</span>
0202:                                 {% if p.borrower_role == 'Staff' %}
0203:                                     <span class="badge" style="background: rgba(168, 85, 247, 0.15); color:
      | #C084FC; font-size: 0.68rem; padding: 1px 6px;">Staff</span>
0204:                                 {% else %}
0205:                                     <span class="badge" style="background: rgba(59, 130, 246, 0.15); color:
      | #60A5FA; font-size: 0.68rem; padding: 1px 6px;">Student</span>
0206:                                 {% endif %}
0207:                             </div>
0208:                         </td>
0209:                         <td>
0210:                             <div class="book-title-cell">{{ p.book_title }}</div>
0211:                             <div style="font-size: 0.76rem; color: var(--text-muted); margin-top: 2px;">
0212:                                 [{{ p.category_name }}] &bull; ISBN: {{ p.isbn }}
0213:                             </div>
0214:                         </td>
0215:                         <td>
0216:                             {% if p.copy_id %}
0217:                                 <span class="badge" style="background: rgba(16, 185, 129, 0.15); color: #34D
      | 399; border: 1px solid rgba(16, 185, 129, 0.35); font-weight: 700; font-size: 0.82rem; padding: 4px 10
      | px;">
0218:                                     Copy #{{ p.copy_id }} Reserved
0219:                                 </span>
0220:                             {% else %}
0221:                                 <span class="badge badge-warning">Copy Reserved</span>
0222:                             {% endif %}
0223:                         </td>
0224:                         <td style="font-size: 0.82rem; white-space: nowrap;">
0225:                             {{ p.request_date.strftime('%b %d, %Y') if p.request_date else 'N/A' }}
0226:                         </td>
0227:                         <td style="text-align: right;">
0228:                             <div style="display: flex; gap: 8px; justify-content: flex-end; align-items: cen
```

```
      | ter;">
0229:                                 <!-- Confirm Pickup / Hand Over Button -->
0230:                                 <form method="POST" action="{{ url_for('loan_requests') }}" style="display:
      | inline;">
0231:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0232:                                     <input type="hidden" name="request_id" value="{{ p.request_id }}">
0233:                                     <input type="hidden" name="action" value="confirm_pickup">
0234:                                     <button
0235:                                         type="submit"
0236:                                         class="btn-confirm-pickup"
0237:                                         title="Confirm physical book handover to {{ p.borrower_name }} and a
      | ctivate loan"
0238:                                         onclick="return confirm('Confirm physical handover of \'{{ p.book_ti
      | tle|escape }}\' (Copy #{{ p.copy_id }}) to {{ p.borrower_name }}? Active loan will be issued for 14 da
      | ys.');"
0239:                                     >
0240:                                         <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><polyline points="20 6 9 17 4 12"></polyline></svg>
0241:                                         ✓ Hand Over &amp;amp; Confirm Collection
0242:                                     </button>
0243:                                 </form>
0244:
0245:                                 <!-- Cancel Reservation Button -->
0246:                                 <form method="POST" action="{{ url_for('loan_requests') }}" style="display:
      | inline;" onsubmit="return confirm('Cancel pickup hold for {{ p.borrower_name }}? Physical copy will be
      |  returned to inventory.');">
0247:                                     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0248:                                     <input type="hidden" name="request_id" value="{{ p.request_id }}">
0249:                                     <input type="hidden" name="action" value="cancel_pickup">
0250:                                     <button type="submit" class="btn-reject" title="Cancel hold and return c
      | opy to inventory">
0251:                                         Cancel Hold
0252:                                     </button>
0253:                                 </form>
0254:                             </div>
0255:                         </td>
0256:                     </tr>
0257:                     {% endfor %}
0258:                 {% else %}
0259:                     <tr>
0260:                         <td colspan="6" class="empty-state" style="padding: 24px;">
0261:                             <div style="font-size: 0.95rem; font-weight: 600; color: var(--text-secondary);"
      | >No Patrons Awaiting Pickup</div>
0262:                             <p style="font-size: 0.8rem; margin-top: 4px; color: var(--text-muted);">Approve
      | d requests waiting for physical desk collection will appear here.</p>
0263:                         </td>
0264:                     </tr>
0265:                 {% endif %}
0266:             </tbody>
0267:         </table>
0268:     </div>
0269: </div>
0270:
0271: <!-- ============================================================================ -->
0272: <!-- 2. PENDING AUTHORIZATION QUEUE (WAITING APPROVAL)                            -->
0273: <!-- ============================================================================ -->
0274: <div class="data-panel">
0275:     <div class="panel-header">
0276:         <div>
0277:             <span class="panel-title">Pending Authorization Queue</span>
0278:             <div style="font-size: 0.78rem; color: var(--text-muted); margin-top: 2px;">
0279:                 Review submitted patron borrowing permission requests and reserve inventory copies
0280:             </div>
0281:         </div>
0282:         <div>
0283:             <span class="badge badge-info" style="font-size: 0.78rem; padding: 4px 10px;">
0284:                 {{ requests|length if requests else 0 }} Pending Review
0285:             </span>
0286:         </div>
0287:     </div>
0288:
0289:     <div class="table-container">
0290:         <table class="data-table">
0291:             <thead>
0292:                 <tr>
0293:                     <th>Request ID</th>
0294:                     <th>Borrower Name &amp;amp; Role</th>
0295:                     <th>Book Details</th>
0296:                     <th>Available Copies</th>
0297:                     <th>Request&amp;nbsp;Timestamp</th>
```

```
0298:                     <th style="text-align: right;">Authorization</th>
0299:                 </tr>
0300:             </thead>
0301:             <tbody>
0302:                 {% if requests %}
0303:                     {% for req in requests %}
0304:                     <tr>
0305:                         <td style="font-family: var(--font-mono); font-size: 0.82rem; font-weight: 600;">
0306:                             #REQ-{{ req.request_id }}
0307:                         </td>
0308:                         <td>
0309:                             <div style="font-weight: 600; color: var(--text-primary);">{{ req.borrower_name
     | }}</div>
0310:                             <div style="font-size: 0.76rem; color: var(--text-muted); display: flex; align-i
     | tems: center; gap: 6px; margin-top: 2px;">
0311:                                 <span>{{ req.borrower_email }}</span>
0312:                                 {% if req.borrower_role == 'Staff' %}
0313:                                     <span class="badge" style="background: rgba(168, 85, 247, 0.15); color:
     | #C084FC; font-size: 0.68rem; padding: 1px 6px;">Staff</span>
0314:                                 {% else %}
0315:                                     <span class="badge" style="background: rgba(59, 130, 246, 0.15); color:
     | #60A5FA; font-size: 0.68rem; padding: 1px 6px;">Student</span>
0316:                                 {% endif %}
0317:                             </div>
0318:                             <div style="margin-top: 4px; display: flex; flex-wrap: wrap; gap: 4px;">
0319:                                 {% if (req.borrower_committed_total or 0) >= 3 %}
0320:                                     <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px
     |  7px;" title="{{ req.borrower_active_loans or 0 }} active loans + {{ req.borrower_awaiting_pickup or 0
     |  }} approved holds awaiting pickup">
0321:                                         Threshold Full ({{ req.borrower_committed_total }}/3: {{ req.borrowe
     | r_active_loans or 0 }} Loans, {{ req.borrower_awaiting_pickup or 0 }} Pickup)
0322:                                     </span>
0323:                                 {% else %}
0324:                                     <span class="badge" style="background: rgba(255, 255, 255, 0.06); color:
     |  var(--text-secondary); font-size: 0.68rem; padding: 2px 7px; border: 1px solid var(--border-subtle);"
     | >
0325:                                         Committed: {{ req.borrower_committed_total or 0 }} / 3 ({{ req.borro
     | wer_active_loans or 0 }} Loans, {{ req.borrower_awaiting_pickup or 0 }} Pickup)
0326:                                     </span>
0327:                                 {% endif %}
0328:                                 {% if (req.borrower_overdue_loans or 0) > 0 %}
0329:                                     <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px
     |  7px;">
0330:                                         {{ req.borrower_overdue_loans }} Overdue Loan(s)
0331:                                     </span>
0332:                                 {% endif %}
0333:                                 {% if (req.borrower_unpaid_fines or 0) > 0 %}
0334:                                     <span class="badge badge-danger" style="font-size: 0.68rem; padding: 2px
     |  7px;">
0335:                                         Unpaid Fine: Rs. {{ "%.2f"|format(req.borrower_unpaid_fines) }}
0336:                                     </span>
0337:                                 {% endif %}
0338:                             </div>
0339:                         </td>
0340:                         <td>
0341:                             <div class="book-title-cell">{{ req.book_title }}</div>
0342:                             <div style="font-size: 0.76rem; font-family: var(--font-mono); color: var(--text
     | -muted); margin-top: 2px;">
0343:                                 ISBN: {{ req.isbn }}
0344:                             </div>
0345:                         </td>
0346:                         <td>
0347:                             {% if req.available_copies_count > 0 %}
0348:                                 <span class="badge badge-success">{{ req.available_copies_count }} In Stock<
     | /span>
0349:                             {% else %}
0350:                                 <span class="badge badge-danger">0 Copies Available</span>
0351:                             {% endif %}
0352:                         </td>
0353:                         <td style="font-size: 0.82rem; white-space: nowrap;">
0354:                             {{ req.request_date.strftime('%b %d, %Y') if req.request_date else 'N/A' }}
0355:                         </td>
0356:                         <td style="text-align: right;">
0357:                             <div style="display: flex; gap: 8px; justify-content: flex-end; align-items: cen
     | ter;">
0358:                                 {% if req.available_copies_count <= 0 or (req.borrower_unpaid_fines or 0) >
     | 0 or (req.borrower_overdue_loans or 0) > 0 or (req.borrower_committed_total or 0) >= 3 %}
0359:                                     <button
0360:                                         type="button"
0361:                                         class="btn"
```

```
0362:                                       disabled
0363:                                       style="background: rgba(239, 68, 68, 0.12); color: #F87171; border:
      | 1px solid rgba(239, 68, 68, 0.3); font-size: 0.78rem; padding: 6px 12px; cursor: not-allowed; border-r
      | adius: var(--radius-md); font-weight: 600; display: inline-flex; align-items: center; gap: 6px;"
0364:                                       {% if (req.borrower_unpaid_fines or 0) > 0 and req.available_copies_
      | count <= 0 %}
0365:                                           title="Cannot approve: Patron has unpaid fines of Rs. {{ '%.2f'|
      | format(req.borrower_unpaid_fines) }} AND 0 copies available in stock."
0366:                                       {% elif (req.borrower_unpaid_fines or 0) > 0 %}
0367:                                           title="Cannot approve: Patron has unpaid fines of Rs. {{ '%.2f'|
      | format(req.borrower_unpaid_fines) }}. They must clear overdue fines first."
0368:                                       {% elif req.available_copies_count <= 0 %}
0369:                                           title="Cannot approve: No physical copies available in inventory
      |  (0 Stock)."
0370:                                       {% elif (req.borrower_overdue_loans or 0) > 0 %}
0371:                                           title="Cannot approve: Patron has {{ req.borrower_overdue_loans
      | }} overdue loan(s). They must return overdue books first."
0372:                                       {% else %}
0373:                                           title="Cannot approve: Patron reached maximum threshold of 3 boo
      | ks. They must return or collect existing books first."
0374:                                       {% endif %}
0375:                                   >
0376:                                       <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="4.93" y1="4.93" x2=
      | "19.07" y2="19.07"></line></svg>
0377:                                       Blocked: {% if (req.borrower_unpaid_fines or 0) > 0 and req.availabl
      | e_copies_count <= 0 %}Fine / 0 Stock{% elif (req.borrower_unpaid_fines or 0) > 0 %}Fine{% elif req.ava
      | ilable_copies_count <= 0 %}0 Stock{% elif (req.borrower_overdue_loans or 0) > 0 %}Overdue{% else %}Quo
      | ta Full{% endif %}
0378:                                   </button>
0379:                               {% else %}
0380:                                   <!-- Approve & Reserve Copy Button -->
0381:                                   <form method="POST" action="{{ url_for('loan_requests') }}" style="displ
      | ay: inline;">
0382:                                       <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0383:                                       <input type="hidden" name="request_id" value="{{ req.request_id }}">
0384:                                       <input type="hidden" name="action" value="approve">
0385:                                       <button type="submit" class="btn-approve" title="Approve loan reques
      | t and reserve Copy">
0386:                                           <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stro
      | ke="currentColor" stroke-width="2.5"><polyline points="20 6 9 17 4 12"></polyline></svg>
0387:                                           Approve &amp;amp; Assign Copy
0388:                                       </button>
0389:                                   </form>
0390:                               {% endif %}
0391:
0392:                               <!-- Reject Button -->
0393:                               <form method="POST" action="{{ url_for('loan_requests') }}" style="display:
      | inline;" onsubmit="return confirm('Reject loan request #{{ req.request_id }} for {{ req.borrower_name
      | }}?');">
0394:                                   <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0395:                                   <input type="hidden" name="request_id" value="{{ req.request_id }}">
0396:                                   <input type="hidden" name="action" value="reject">
0397:                                   <button type="submit" class="btn-reject" title="Reject Request">
0398:                                       <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="
      | currentColor" stroke-width="2.5"><line x1="18" y1="6" x2="6" y2="18"></line><line x1="6" y1="6" x2="18
      | " y2="18"></line></svg>
0399:                                       Reject
0400:                                   </button>
0401:                               </form>
0402:                           </div>
0403:                       </td>
0404:                   </tr>
0405:                   {% endfor %}
0406:               {% else %}
0407:                   <tr>
0408:                       <td colspan="6" class="empty-state">
0409:                           <svg viewBox="0 0 24 24"><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path><po
      | lyline points="22 4 12 14.01 9 11.01"></polyline></svg>
0410:                           <div style="font-size: 1.05rem; font-weight: 700; color: var(--text-primary);">A
      | ll Clear! No Pending Requests</div>
0411:                           <p style="margin-top: 4px;">Patron loan permission requests will appear here whe
      | n submitted by Students or Staff.</p>
0412:                       </td>
0413:                   </tr>
0414:               {% endif %}
0415:           </tbody>
0416:       </table>
0417:     </div>
0418: </div>
```

```
0419: {% endblock %}
```

MODULE SOURCE: templates/graph_visualizer.html

Lines: 1151 | **Lang:** HTML5 / Jinja2 / Vis.js | **Category:** Knowledge Graph Visualizer

Core Role & Purpose: Interactive Vis.js knowledge graph visualizer featuring Barnes-Hut physics, dynamic state-aware Play/Pause controls, drawer details, and filter pills.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify that #physicsToggleBtn displays Pause icon (■■) when simulation is actively running, and Play icon (■■) when frozen.
- Verify that updatePhysicsBtnUI hides the button (display: none) when visible nodes count is 0.
- Verify that reStabilize() automatically restores isPhysicsActive = true and syncs the button icon.
- Verify that view switcher resets physics and loads correct subgraph.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Live Graph Explorer | VITCC LIBRARY Knowledge Graph{% endblock %}
0004: {% block breadcrumb %}Knowledge Graph Explorer{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     /* Full height Graph Visualizer Container */
0009:     .graph-page-container {
0010:         display: flex;
0011:         flex-direction: column;
0012:         gap: 16px;
0013:         height: calc(100vh - 120px);
0014:         min-height: 680px;
0015:     }
0016:
0017:     /* Top Control Header */
0018:     .graph-header-card {
0019:         background: var(--bg-surface);
0020:         border: 1px solid var(--border-subtle);
0021:         border-radius: var(--radius-lg);
0022:         padding: 16px 20px;
0023:         display: flex;
0024:         flex-direction: column;
0025:         gap: 14px;
0026:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0027:     }
0028:
0029:     .graph-title-row {
0030:         display: flex;
0031:         align-items: center;
0032:         justify-content: space-between;
0033:         flex-wrap: wrap;
0034:         gap: 12px;
0035:     }
0036:
0037:     .graph-title-group h1 {
0038:         font-size: 1.35rem;
0039:         font-weight: 800;
0040:         letter-spacing: -0.02em;
0041:         color: var(--text-primary);
0042:         display: flex;
0043:         align-items: center;
0044:         gap: 10px;
0045:     }
0046:
0047:     .graph-title-group p {
0048:         font-size: 0.82rem;
0049:         color: var(--text-secondary);
0050:         margin-top: 2px;
0051:     }
0052:
0053:     /* View Switcher Tabs */
0054:     .view-switcher {
0055:         display: inline-flex;
0056:         background: rgba(255, 255, 255, 0.04);
0057:         padding: 4px;
0058:         border-radius: var(--radius-full);
0059:         border: 1px solid var(--border-subtle);
0060:         gap: 4px;
0061:     }
0062:
0063:     .view-btn {
0064:         background: transparent;
0065:         border: none;
0066:         color: var(--text-secondary);
0067:         padding: 6px 14px;
0068:         font-size: 0.8rem;
0069:         font-weight: 600;
0070:         border-radius: var(--radius-full);
0071:         cursor: pointer;
0072:         transition: all 0.2s ease;
0073:         display: inline-flex;
0074:         align-items: center;
0075:         gap: 6px;
0076:     }
0077:
0078:     .view-btn.active {
0079:         background: var(--accent-primary);
0080:         color: #FFFFFF;
```

```
0081:         box-shadow: 0 2px 10px var(--accent-glow);
0082:     }
0083:
0084:     .view-btn:hover:not(.active) {
0085:         color: var(--text-primary);
0086:         background: rgba(255, 255, 255, 0.06);
0087:     }
0088:
0089:     /* Second Row: Filters, Search & Actions */
0090:     .graph-filter-row {
0091:         display: flex;
0092:         align-items: center;
0093:         justify-content: space-between;
0094:         flex-wrap: wrap;
0095:         gap: 12px;
0096:         padding-top: 10px;
0097:         border-top: 1px solid var(--border-subtle);
0098:     }
0099:
0100:     .filter-pills-group {
0101:         display: flex;
0102:         align-items: center;
0103:         flex-wrap: wrap;
0104:         gap: 6px;
0105:     }
0106:
0107:     .filter-pill {
0108:         display: inline-flex;
0109:         align-items: center;
0110:         gap: 6px;
0111:         padding: 4px 10px;
0112:         border-radius: var(--radius-full);
0113:         font-size: 0.75rem;
0114:         font-weight: 600;
0115:         cursor: pointer;
0116:         user-select: none;
0117:         border: 1px solid;
0118:         transition: all 0.15s ease;
0119:         opacity: 0.95;
0120:     }
0121:
0122:     .filter-pill.inactive {
0123:         opacity: 0.35;
0124:         filter: grayscale(80%);
0125:         border-color: rgba(255, 255, 255, 0.1) !important;
0126:         background: transparent !important;
0127:         color: var(--text-muted) !important;
0128:     }
0129:
0130:     .filter-pill.disabled {
0131:         opacity: 0.35 !important;
0132:         filter: grayscale(80%) !important;
0133:         cursor: not-allowed !important;
0134:         pointer-events: none !important;
0135:         border-color: rgba(255, 255, 255, 0.08) !important;
0136:         background: rgba(255, 255, 255, 0.02) !important;
0137:         color: var(--text-muted) !important;
0138:     }
0139:
0140:     .pill-count {
0141:         font-family: var(--font-mono);
0142:         font-size: 0.72rem;
0143:         opacity: 0.85;
0144:         margin-left: 2px;
0145:     }
0146:
0147:     .filter-pill .dot {
0148:         width: 8px;
0149:         height: 8px;
0150:         border-radius: 50%;
0151:     }
0152:
0153:     /* Colors for Filter Pills */
0154:     .pill-user       { background: rgba(59, 130, 246, 0.15); color: #60A5FA; border-color: rgba(59, 130, 246
      | , 0.35); }
0155:     .pill-user .dot  { background: #3B82F6; }
0156:     .pill-book       { background: rgba(16, 185, 129, 0.15); color: #34D399; border-color: rgba(16, 185, 129
      | , 0.35); }
0157:     .pill-book .dot  { background: #10B981; }
0158:     .pill-author     { background: rgba(139, 92, 246, 0.15); color: #C084FC; border-color: rgba(139, 92, 246
```

```
      | , 0.35); }
0159:     .pill-author .dot{ background: #8B5CF6; }
0160:     .pill-cat        { background: rgba(245, 158, 11, 0.15); color: #FBBF24; border-color: rgba(245, 158, 11
      | , 0.35); }
0161:     .pill-cat .dot   { background: #F59E0B; }
0162:     .pill-pub        { background: rgba(236, 72, 153, 0.15); color: #F472B6; border-color: rgba(236, 72, 153
      | , 0.35); }
0163:     .pill-pub .dot   { background: #EC4899; }
0164:     .pill-copy       { background: rgba(20, 184, 166, 0.15); color: #2DD4BF; border-color: rgba(20, 184, 166
      | , 0.35); }
0165:     .pill-copy .dot  { background: #14B8A6; }
0166:     .pill-loan       { background: rgba(6, 182, 212, 0.15); color: #22D3EE; border-color: rgba(6, 182, 212,
      | 0.35); }
0167:     .pill-loan .dot  { background: #06B6D4; }
0168:     .pill-fine       { background: rgba(239, 68, 68, 0.15); color: #F87171; border-color: rgba(239, 68, 68,
      | 0.35); }
0169:     .pill-fine .dot  { background: #EF4444; }
0170:     .pill-request    { background: rgba(249, 115, 22, 0.15); color: #FB923C; border-color: rgba(249, 115, 22
      | , 0.35); }
0171:     .pill-request .dot{ background: #F97316; }
0172:
0173:     /* Search & Canvas Tools */
0174:     .graph-tools-group {
0175:         display: flex;
0176:         align-items: center;
0177:         gap: 8px;
0178:     }
0179:
0180:     .graph-search-box {
0181:         position: relative;
0182:         display: flex;
0183:         align-items: center;
0184:     }
0185:
0186:     .graph-search-box input {
0187:         background: rgba(255, 255, 255, 0.05);
0188:         border: 1px solid var(--border-subtle);
0189:         border-radius: var(--radius-full);
0190:         padding: 6px 14px 6px 32px;
0191:         font-size: 0.8rem;
0192:         color: var(--text-primary);
0193:         outline: none;
0194:         width: 190px;
0195:         transition: all 0.2s ease;
0196:     }
0197:
0198:     .graph-search-box input:focus {
0199:         border-color: var(--accent-primary);
0200:         width: 240px;
0201:         background: rgba(255, 255, 255, 0.08);
0202:         box-shadow: 0 0 12px var(--accent-glow);
0203:     }
0204:
0205:     .graph-search-box svg {
0206:         position: absolute;
0207:         left: 10px;
0208:         width: 14px;
0209:         height: 14px;
0210:         color: var(--text-muted);
0211:         pointer-events: none;
0212:     }
0213:
0214:     .btn-icon-tool {
0215:         background: rgba(255, 255, 255, 0.05);
0216:         border: 1px solid var(--border-subtle);
0217:         color: var(--text-secondary);
0218:         width: 32px;
0219:         height: 32px;
0220:         border-radius: var(--radius-md);
0221:         display: flex;
0222:         align-items: center;
0223:         justify-content: center;
0224:         cursor: pointer;
0225:         transition: all 0.15s ease;
0226:     }
0227:
0228:     .btn-icon-tool:hover {
0229:         background: rgba(255, 255, 255, 0.1);
0230:         color: var(--text-primary);
0231:         border-color: var(--border-hover);
```

```
0232:     }
0233:
0234:     .btn-icon-tool.active {
0235:         background: var(--accent-primary);
0236:         color: #FFFFFF;
0237:         border-color: var(--accent-primary);
0238:     }
0239:
0240:     /* Main Canvas & Detail Sidebar Workspace */
0241:     .graph-workspace {
0242:         flex: 1;
0243:         position: relative;
0244:         background: radial-gradient(circle at 50% 50%, #111827 0%, #0B0F19 100%);
0245:         border: 1px solid var(--border-subtle);
0246:         border-radius: var(--radius-lg);
0247:         overflow: hidden;
0248:         display: flex;
0249:     }
0250:
0251:     #network-canvas {
0252:         width: 100%;
0253:         height: 100%;
0254:         cursor: grab;
0255:     }
0256:
0257:     #network-canvas:active {
0258:         cursor: grabbing;
0259:     }
0260:
0261:
0262:
0263:     /* Floating Stats Badge */
0264:     .graph-stats-badge {
0265:         position: absolute;
0266:         top: 16px;
0267:         left: 16px;
0268:         background: rgba(17, 24, 39, 0.85);
0269:         backdrop-filter: blur(12px);
0270:         border: 1px solid var(--border-subtle);
0271:         border-radius: var(--radius-md);
0272:         padding: 6px 12px;
0273:         font-size: 0.75rem;
0274:         color: var(--text-muted);
0275:         display: flex;
0276:         gap: 12px;
0277:         z-index: 10;
0278:         font-family: var(--font-mono);
0279:     }
0280:
0281:     .graph-stats-badge strong {
0282:         color: var(--accent-primary);
0283:     }
0284:
0285:     /* Side Detail Drawer (Collapsible) */
0286:     .node-detail-drawer {
0287:         width: 340px;
0288:         background: rgba(17, 24, 39, 0.95);
0289:         backdrop-filter: blur(16px);
0290:         border-left: 1px solid var(--border-subtle);
0291:         height: 100%;
0292:         position: absolute;
0293:         top: 0;
0294:         right: 0;
0295:         z-index: 20;
0296:         transform: translateX(100%);
0297:         transition: transform 0.3s cubic-bezier(0.4, 0, 0.2, 1);
0298:         display: flex;
0299:         flex-direction: column;
0300:         box-shadow: -4px 0 24px rgba(0, 0, 0, 0.4);
0301:     }
0302:
0303:     .node-detail-drawer.open {
0304:         transform: translateX(0);
0305:     }
0306:
0307:     .drawer-header {
0308:         padding: 16px 20px;
0309:         border-bottom: 1px solid var(--border-subtle);
0310:         display: flex;
0311:         align-items: center;
```

```
0312:         justify-content: space-between;
0313:     }
0314:
0315:     .drawer-header-info {
0316:         display: flex;
0317:         align-items: center;
0318:         gap: 10px;
0319:     }
0320:
0321:     .drawer-group-badge {
0322:         font-size: 0.7rem;
0323:         font-weight: 700;
0324:         text-transform: uppercase;
0325:         padding: 3px 8px;
0326:         border-radius: var(--radius-full);
0327:         letter-spacing: 0.05em;
0328:     }
0329:
0330:     .drawer-close-btn {
0331:         background: transparent;
0332:         border: none;
0333:         color: var(--text-muted);
0334:         cursor: pointer;
0335:         padding: 4px;
0336:         border-radius: var(--radius-sm);
0337:         display: flex;
0338:         align-items: center;
0339:         transition: color 0.15s;
0340:     }
0341:
0342:     .drawer-close-btn:hover {
0343:         color: var(--text-primary);
0344:     }
0345:
0346:     .drawer-body {
0347:         padding: 20px;
0348:         overflow-y: auto;
0349:         flex: 1;
0350:         display: flex;
0351:         flex-direction: column;
0352:         gap: 18px;
0353:     }
0354:
0355:     .drawer-node-title {
0356:         font-size: 1.15rem;
0357:         font-weight: 700;
0358:         color: var(--text-primary);
0359:         line-height: 1.3;
0360:     }
0361:
0362:     .drawer-section-title {
0363:         font-size: 0.72rem;
0364:         font-weight: 700;
0365:         text-transform: uppercase;
0366:         letter-spacing: 0.08em;
0367:         color: var(--text-muted);
0368:         margin-bottom: 8px;
0369:     }
0370:
0371:     .property-grid {
0372:         display: flex;
0373:         flex-direction: column;
0374:         gap: 8px;
0375:     }
0376:
0377:     .prop-row {
0378:         display: flex;
0379:         flex-direction: column;
0380:         padding: 8px 10px;
0381:         border-radius: var(--radius-sm);
0382:         background: rgba(255, 255, 255, 0.02);
0383:         border: 1px solid rgba(255, 255, 255, 0.04);
0384:     }
0385:
0386:     .prop-key {
0387:         font-size: 0.72rem;
0388:         color: var(--text-muted);
0389:         font-weight: 600;
0390:         text-transform: uppercase;
0391:         letter-spacing: 0.05em;
```

```
0392:     }
0393:
0394:     .prop-val {
0395:         font-size: 0.85rem;
0396:         color: var(--text-primary);
0397:         font-family: var(--font-mono);
0398:         word-break: break-word;
0399:         margin-top: 2px;
0400:     }
0401:
0402:     .connected-nodes-list {
0403:         display: flex;
0404:         flex-direction: column;
0405:         gap: 6px;
0406:     }
0407:
0408:     .connected-item {
0409:         display: flex;
0410:         align-items: center;
0411:         justify-content: space-between;
0412:         padding: 8px 10px;
0413:         border-radius: var(--radius-sm);
0414:         background: rgba(255, 255, 255, 0.02);
0415:         border: 1px solid var(--border-subtle);
0416:         cursor: pointer;
0417:         transition: all 0.15s;
0418:     }
0419:
0420:     .connected-item:hover {
0421:         background: rgba(255, 255, 255, 0.06);
0422:         border-color: var(--accent-primary);
0423:     }
0424:
0425:     .connected-item-label {
0426:         font-size: 0.8rem;
0427:         color: var(--text-primary);
0428:         font-weight: 500;
0429:         overflow: hidden;
0430:         text-overflow: ellipsis;
0431:         white-space: nowrap;
0432:         max-width: 200px;
0433:     }
0434:
0435:     .connected-item-rel {
0436:         font-size: 0.68rem;
0437:         color: var(--text-muted);
0438:         font-family: var(--font-mono);
0439:     }
0440:
0441:     /* Loading Overlay */
0442:     .graph-loading {
0443:         position: absolute;
0444:         inset: 0;
0445:         background: rgba(11, 15, 25, 0.85);
0446:         display: flex;
0447:         flex-direction: column;
0448:         align-items: center;
0449:         justify-content: center;
0450:         gap: 12px;
0451:         z-index: 30;
0452:         backdrop-filter: blur(8px);
0453:         transition: opacity 0.3s;
0454:     }
0455:
0456:     .graph-spinner {
0457:         width: 38px;
0458:         height: 38px;
0459:         border: 3px solid rgba(59, 130, 246, 0.2);
0460:         border-top-color: var(--accent-primary);
0461:         border-radius: 50%;
0462:         animation: spin 0.8s linear infinite;
0463:     }
0464:
0465:     @keyframes spin {
0466:         to { transform: rotate(360deg); }
0467:     }
0468: </style>
0469: {% endblock %}
0470:
0471: {% block content %}
```

```
0472: <div class="graph-page-container">
0473:     <!-- Top Control Bar Card -->
0474:     <div class="graph-header-card">
0475:         <!-- Title & View Switcher Row -->
0476:         <div class="graph-title-row">
0477:             <div class="graph-title-group">
0478:                 <h1>
0479:                     <svg viewBox="0 0 24 24" width="22" height="22" stroke="currentColor" fill="none" stroke
      | -width="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18"
      |  cy="19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6
      | .51" x2="8.59" y2="10.49"></line></svg>
0480:                     Live Graph Explorer
0481:                 </h1>
0482:                 <p>Interactive real-time traversal of library knowledge graph nodes, loans, and relationship
      | s.</p>
0483:             </div>
0484:
0485:             <!-- View Mode Switcher -->
0486:             <div class="view-switcher" id="viewSwitcher">
0487:                 {% if user_role in ['Admin', 'Librarian'] %}
0488:                 <button class="view-btn active" data-view="full" onclick="switchGraphView('full')">
0489:                     <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" fill="none" stroke
      | -width="2"><circle cx="12" cy="12" r="10"></circle><line x1="2" y1="12" x2="22" y2="12"></line><path d
      | ="M12 2a15.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-4 10 15.3 15.3 0 0 1-4-10 15.3 15.3 0 0 1 4-10z"></path><
      | /svg>
0490:                     Full System Graph
0491:                 </button>
0492:                 <button class="view-btn" data-view="catalog" onclick="switchGraphView('catalog')">
0493:                     <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" fill="none" stroke
      | -width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0494:                     Catalog Knowledge Graph
0495:                 </button>
0496:                 <button class="view-btn" data-view="personal" onclick="switchGraphView('personal')">
0497:                     <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" fill="none" stroke
      | -width="2"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><circle cx="12" cy="7" r="4"></c
      | ircle></svg>
0498:                     Global Circulation & Fines Subgraph
0499:                 </button>
0500:                 {% else %}
0501:                 <button class="view-btn active" data-view="personal" onclick="switchGraphView('personal')">
0502:                     <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" fill="none" stroke
      | -width="2"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><circle cx="12" cy="7" r="4"></c
      | ircle></svg>
0503:                     My Personal Borrowing Subgraph
0504:                 </button>
0505:                 <button class="view-btn" data-view="catalog" onclick="switchGraphView('catalog')">
0506:                     <svg viewBox="0 0 24 24" width="14" height="14" stroke="currentColor" fill="none" stroke
      | -width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0507:                     Full Catalog Knowledge Graph
0508:                 </button>
0509:                 {% endif %}
0510:             </div>
0511:         </div>
0512:
0513:         <!-- Filter Pills & Search Tools Row -->
0514:         <div class="graph-filter-row">
0515:             <!-- Node Type Toggle Pills -->
0516:             <div class="filter-pills-group" id="nodeFilters">
0517:                 <span class="filter-pill pill-user active" data-group="User" onclick="toggleGroupFilter('Use
      | r')">
0518:                     <span class="dot"></span> Users <span class="pill-count" id="count-User"></span>
0519:                 </span>
0520:                 <span class="filter-pill pill-book active" data-group="Book" onclick="toggleGroupFilter('Boo
      | k')">
0521:                     <span class="dot"></span> Books <span class="pill-count" id="count-Book"></span>
0522:                 </span>
0523:                 <span class="filter-pill pill-author active" data-group="Author" onclick="toggleGroupFilter(
      | 'Author')">
0524:                     <span class="dot"></span> Authors <span class="pill-count" id="count-Author"></span>
0525:                 </span>
0526:                 <span class="filter-pill pill-cat active" data-group="Category" onclick="toggleGroupFilter('
      | Category')">
0527:                     <span class="dot"></span> Categories <span class="pill-count" id="count-Category"></span
      | >
0528:                 </span>
0529:                 <span class="filter-pill pill-pub active" data-group="Publisher" onclick="toggleGroupFilter(
      | 'Publisher')">
0530:                     <span class="dot"></span> Publishers <span class="pill-count" id="count-Publisher"></spa
      | n>
0531:                 </span>
```

```
0532:                 <span class="filter-pill pill-copy active" data-group="Copy" onclick="toggleGroupFilter('Cop
      | y')">
0533:                     <span class="dot"></span> Copies <span class="pill-count" id="count-Copy"></span>
0534:                 </span>
0535:                 <span class="filter-pill pill-loan active" data-group="Loan" onclick="toggleGroupFilter('Loa
      | n')">
0536:                     <span class="dot"></span> Loans <span class="pill-count" id="count-Loan"></span>
0537:                 </span>
0538:                 <span class="filter-pill pill-fine active" data-group="Fine" onclick="toggleGroupFilter('Fin
      | e')">
0539:                     <span class="dot"></span> Fines <span class="pill-count" id="count-Fine"></span>
0540:                 </span>
0541:                 <span class="filter-pill pill-request active" data-group="LoanRequest" onclick="toggleGroupF
      | ilter('LoanRequest')">
0542:                     <span class="dot"></span> Requests <span class="pill-count" id="count-LoanRequest"></spa
      | n>
0543:                 </span>
0544:             </div>
0545:
0546:             <!-- Search & Canvas Actions -->
0547:             <div class="graph-tools-group">
0548:                 <div class="graph-search-box">
0549:                     <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="
      | 11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line></svg>
0550:                     <input type="text" id="nodeSearchInput" placeholder="Search node or title..." oninput="h
      | andleNodeSearch(this.value)">
0551:                 </div>
0552:                 <button class="btn-icon-tool active" id="physicsToggleBtn" title="Pause Physics Simulation (
      | Freeze Layout)" onclick="togglePhysics()" aria-label="Pause Physics Simulation">
0553:                     <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none" stroke
      | -width="2"><rect x="6" y="4" width="4" height="16"></rect><rect x="14" y="4" width="4" height="16"></r
      | ect></svg>
0554:                 </button>
0555:                 <button class="btn-icon-tool" title="Reset View / Fit Screen" onclick="fitGraphView()" aria-
      | label="Reset View">
0556:                     <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none" stroke
      | -width="2"><path d="M15 3h6v6"></path><path d="M9 21H3v-6"></path><path d="M21 3l-7 7"></path><path d=
      | "M3 21l7-7"></path></svg>
0557:                 </button>
0558:                 <button class="btn-icon-tool" id="reStabilizeBtn" title="Re-stabilize Layout" onclick="reSta
      | bilize()" aria-label="Re-stabilize Layout">
0559:                     <svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none" stroke
      | -width="2"><polyline points="23 4 23 10 17 10"></polyline><polyline points="1 20 1 14 7 14"></polyline
      | ><path d="M3.51 9a9 9 0 0 1 14.85-3.36L23 10M1 14l4.64 4.36A9 9 0 0 0 20.49 15"></path></svg>
0560:                 </button>
0561:             </div>
0562:         </div>
0563:     </div>
0564:
0565:     <!-- Main Network Canvas Area with Slide-out Drawer -->
0566:     <div class="graph-workspace">
0567:         <!-- Live Loading Spinner -->
0568:         <div class="graph-loading" id="graphLoading">
0569:             <div class="graph-spinner"></div>
0570:             <span style="font-size: 0.88rem; color: var(--text-secondary); font-weight: 500;">Traversing Aur
      | aDB Graph...</span>
0571:         </div>
0572:
0573:         <!-- Floating Stats Counter -->
0574:         <div class="graph-stats-badge" id="graphStatsBadge">
0575:             <span>Nodes: <strong id="statNodesCount">0</strong></span>
0576:             <span>Relationships: <strong id="statEdgesCount">0</strong></span>
0577:             <span>View: <strong id="statViewName">Loading...</strong></span>
0578:         </div>
0579:
0580:         <!-- Canvas Element -->
0581:         <div id="network-canvas"></div>
0582:
0583:
0584:         <!-- Node Property Detail Drawer (Collapsible) -->
0585:         <div class="node-detail-drawer" id="nodeDrawer">
0586:             <div class="drawer-header">
0587:                 <div class="drawer-header-info">
0588:                     <span class="drawer-group-badge" id="drawerBadge">NODE</span>
0589:                 </div>
0590:                 <button class="drawer-close-btn" onclick="closeDrawer()" title="Close details">
0591:                     <svg viewBox="0 0 24 24" width="18" height="18" stroke="currentColor" fill="none" stroke
      | -width="2"><line x1="18" y1="6" x2="6" y2="18"></line><line x1="6" y1="6" x2="18" y2="18"></line></svg
      | >
0592:                 </button>
```

```
0593:             </div>
0594:             <div class="drawer-body">
0595:                 <div>
0596:                     <h2 class="drawer-node-title" id="drawerNodeTitle">Select a Node</h2>
0597:                 </div>
0598:
0599:                 <div>
0600:                     <div class="drawer-section-title">Node Properties</div>
0601:                     <div class="property-grid" id="drawerProperties">
0602:                         <!-- Key-Value pairs populated dynamically -->
0603:                     </div>
0604:                 </div>
0605:
0606:                 <div>
0607:                     <div class="drawer-section-title">Connected Relationships (<span id="connectedCount">0</
      | span>)</div>
0608:                     <div class="connected-nodes-list" id="drawerConnections">
0609:                         <!-- Neighbor nodes populated dynamically -->
0610:                     </div>
0611:                 </div>
0612:             </div>
0613:         </div>
0614:     </div>
0615: </div>
0616: {% endblock %}
0617:
0618: {% block extra_js %}
0619: <!-- Vis.js Network Standalone UMD Bundle -->
0620: <script src="https://unpkg.com/vis-network/standalone/umd/vis-network.min.js"></script>
0621:
0622: <script>
0623:     // State management
0624:     let network = null;
0625:     let allNodes = [];
0626:     let allEdges = [];
0627:     let nodesDataSet = null;
0628:     let edgesDataSet = null;
0629:     let currentView = "{{ 'full' if user_role in ['Admin', 'Librarian'] else 'personal' }}";
0630:     let isPhysicsActive = true;
0631:     const PLAY_ICON_SVG = '<svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none"
      |  stroke-width="2"><polygon points="5 3 19 12 5 21 5 3"></polygon></svg>';
0632:     const PAUSE_ICON_SVG = '<svg viewBox="0 0 24 24" width="16" height="16" stroke="currentColor" fill="none
      | " stroke-width="2"><rect x="6" y="4" width="4" height="16"></rect><rect x="14" y="4" width="4" height=
      | "16"></rect></svg>';
0633:
0634:     function updatePhysicsBtnUI(hasNodes = true) {
0635:         const btn = document.getElementById("physicsToggleBtn");
0636:         const reBtn = document.getElementById("reStabilizeBtn");
0637:         if (!btn) return;
0638:
0639:         if (!hasNodes) {
0640:             // When there are no visible nodes, physics simulation has no effect,
0641:             // so we remove/hide the physics button and re-stabilize button
0642:             btn.style.display = "none";
0643:             if (reBtn) reBtn.style.display = "none";
0644:             return;
0645:         }
0646:
0647:         btn.style.display = "inline-flex";
0648:         if (reBtn) reBtn.style.display = "inline-flex";
0649:
0650:         btn.classList.toggle("active", isPhysicsActive);
0651:         if (isPhysicsActive) {
0652:             // Physics running: replace the play symbol with the pause symbol
0653:             btn.innerHTML = PAUSE_ICON_SVG;
0654:             btn.setAttribute("title", "Pause Physics Simulation (Freeze Layout)");
0655:             btn.setAttribute("aria-label", "Pause Physics Simulation");
0656:         } else {
0657:             // Physics paused/frozen: show play symbol to allow unfreezing/resuming
0658:             btn.innerHTML = PLAY_ICON_SVG;
0659:             btn.setAttribute("title", "Resume Physics Simulation (Unfreeze Layout)");
0660:             btn.setAttribute("aria-label", "Resume Physics Simulation");
0661:         }
0662:     }
0663:
0664:     let activeGroups = new Set(["User", "Book", "Author", "Category", "Publisher", "Copy", "Loan", "Fine", "
      | LoanRequest", "CurrentUser", "SharedBook", "RecommendedBook", "PeerUser", "Prerequisite", "StartBook",
      |  "TargetBook", "IntermediateBook"]);
0665:
0666:     // Vis-Network Group Color Definitions
```

```
0667:     const groupConfigurations = {
0668:         User: {
0669:             color: { background: '#1D4ED8', border: '#3B82F6', highlight: { background: '#2563EB', border: '
      | #60A5FA' } },
0670:             shape: 'dot',
0671:             size: 24,
0672:             font: { color: '#F9FAFB', face: 'Plus Jakarta Sans', size: 13 }
0673:         },
0674:         CurrentUser: {
0675:             color: { background: '#2563EB', border: '#60A5FA', highlight: { background: '#3B82F6', border: '
      | #93C5FD' } },
0676:             shape: 'star',
0677:             size: 28,
0678:             font: { color: '#F9FAFB', face: 'Plus Jakarta Sans', size: 14, strokeWidth: 2, strokeColor: '#00
      | 0000' }
0679:         },
0680:         Book: {
0681:             color: { background: '#047857', border: '#10B981', highlight: { background: '#059669', border: '
      | #34D399' } },
0682:             shape: 'dot',
0683:             size: 24,
0684:             font: { color: '#F9FAFB', face: 'Plus Jakarta Sans', size: 13 }
0685:         },
0686:         SharedBook: {
0687:             color: { background: '#059669', border: '#34D399', highlight: { background: '#10B981', border: '
      | #6EE7B7' } },
0688:             shape: 'dot',
0689:             size: 24,
0690:             font: { color: '#F9FAFB', face: 'Plus Jakarta Sans', size: 13 }
0691:         },
0692:         RecommendedBook: {
0693:             color: { background: '#B45309', border: '#F59E0B', highlight: { background: '#D97706', border: '
      | #FCD34D' } },
0694:             shape: 'star',
0695:             size: 28,
0696:             font: { color: '#FDE047', face: 'Plus Jakarta Sans', size: 14, strokeWidth: 2, strokeColor: '#00
      | 0000' }
0697:         },
0698:         Author: {
0699:             color: { background: '#6D28D9', border: '#8B5CF6', highlight: { background: '#7C3AED', border: '
      | #C4B5FD' } },
0700:             shape: 'dot',
0701:             size: 20,
0702:             font: { color: '#E9D5FF', face: 'Plus Jakarta Sans', size: 12 }
0703:         },
0704:         Category: {
0705:             color: { background: '#B45309', border: '#F59E0B', highlight: { background: '#D97706', border: '
      | #FCD34D' } },
0706:             shape: 'diamond',
0707:             size: 22,
0708:             font: { color: '#FEF3C7', face: 'Plus Jakarta Sans', size: 13 }
0709:         },
0710:         Publisher: {
0711:             color: { background: '#BE185D', border: '#EC4899', highlight: { background: '#DB2777', border: '
      | #F9A8D4' } },
0712:             shape: 'dot',
0713:             size: 19,
0714:             font: { color: '#FCE7F3', face: 'Plus Jakarta Sans', size: 12 }
0715:         },
0716:         Copy: {
0717:             color: { background: '#0F766E', border: '#14B8A6', highlight: { background: '#0D9488', border: '
      | #5EEAD4' } },
0718:             shape: 'square',
0719:             size: 16,
0720:             font: { color: '#CCFBF1', face: 'Plus Jakarta Sans', size: 11 }
0721:         },
0722:         Loan: {
0723:             color: { background: '#0E7490', border: '#06B6D4', highlight: { background: '#0891B2', border: '
      | #67E8F9' } },
0724:             shape: 'dot',
0725:             size: 18,
0726:             font: { color: '#CFFAFE', face: 'Plus Jakarta Sans', size: 11 }
0727:         },
0728:         Fine: {
0729:             color: { background: '#B91C1C', border: '#EF4444', highlight: { background: '#DC2626', border: '
      | #FCA5A5' } },
0730:             shape: 'triangle',
0731:             size: 20,
0732:             font: { color: '#FEE2E2', face: 'Plus Jakarta Sans', size: 12 }
0733:         },
```

```
0734:         LoanRequest: {
0735:             color: { background: '#C2410C', border: '#F97316', highlight: { background: '#EA580C', border: '
      | #FDBA74' } },
0736:             shape: 'hexagon',
0737:             size: 18,
0738:             font: { color: '#FFEDD5', face: 'Plus Jakarta Sans', size: 12 }
0739:         },
0740:         PeerUser: {
0741:             color: { background: '#4F46E5', border: '#6366F1', highlight: { background: '#4338CA', border: '
      | #A5B4FC' } },
0742:             shape: 'dot',
0743:             size: 22,
0744:             font: { color: '#E0E7FF', face: 'Plus Jakarta Sans', size: 13 }
0745:         }
0746:     };
0747: 
0748:     // Initialize Network on Page Load
0749:     document.addEventListener("DOMContentLoaded", () => {
0750:         loadGraphData(currentView);
0751:     });
0752: 
0753:     // Fetch data from /api/graph_data
0754:     function loadGraphData(viewName) {
0755:         currentView = viewName;
0756:         document.getElementById("graphLoading").style.display = "flex";
0757:         if (viewName === 'personal') {
0758:             document.getElementById("statViewName").textContent = ("{{ user_role }}" === "Admin" || "{{ user
      | _role }}" === "Librarian") ? "GLOBAL CIRCULATION & FINES" : "PERSONAL BORROWING";
0759:         } else {
0760:             document.getElementById("statViewName").textContent = viewName.toUpperCase();
0761:         }
0762: 
0763:         fetch(`/api/graph_data?view=${viewName}`)
0764:             .then(res => res.json())
0765:             .then(data => {
0766:                 if (data.error) {
0767:                     alert("Error loading graph data: " + data.error);
0768:                     return;
0769:                 }
0770: 
0771:                 allNodes = data.nodes || [];
0772:                 allEdges = data.edges || [];
0773: 
0774:                 document.getElementById("statNodesCount").textContent = allNodes.length;
0775:                 document.getElementById("statEdgesCount").textContent = allEdges.length;
0776: 
0777:                 // Live count calculation for the 9 filter chips
0778:                 const groupCounts = {
0779:                     User: 0,
0780:                     Book: 0,
0781:                     Author: 0,
0782:                     Category: 0,
0783:                     Publisher: 0,
0784:                     Copy: 0,
0785:                     Loan: 0,
0786:                     Fine: 0,
0787:                     LoanRequest: 0
0788:                 };
0789: 
0790:                 allNodes.forEach(node => {
0791:                     let g = node.group;
0792:                     if (g === "CurrentUser" || g === "PeerUser") g = "User";
0793:                     if (g === "SharedBook" || g === "RecommendedBook" || g === "Prerequisite" || g === "Star
      | tBook" || g === "TargetBook" || g === "IntermediateBook") g = "Book";
0794:                     if (groupCounts[g] !== undefined) {
0795:                         groupCounts[g]++;
0796:                     }
0797:                 });
0798: 
0799:                 // Update filter chips count and dim/disable chips with 0 nodes
0800:                 document.querySelectorAll("#nodeFilters .filter-pill").forEach(pill => {
0801:                     const group = pill.getAttribute("data-group");
0802:                     const count = groupCounts[group] || 0;
0803:                     const countElem = pill.querySelector(".pill-count");
0804:                     if (countElem) {
0805:                         countElem.textContent = `(${count})`;
0806:                     }
0807: 
0808:                     if (count === 0) {
0809:                         pill.classList.add("disabled");
```

```
                        pill.classList.remove("active");
                        pill.classList.remove("inactive");
                    } else {
                        pill.classList.remove("disabled");
                        if (activeGroups.has(group)) {
                            pill.classList.add("active");
                            pill.classList.remove("inactive");
                        } else {
                            pill.classList.remove("active");
                            pill.classList.add("inactive");
                        }
                    }
                });

                renderNetwork(allNodes, allEdges);
            })
            .catch(err => {
                console.error("Graph fetch failed:", err);
                alert("Failed to connect to graph API: " + err);
            })
            .finally(() => {
                setTimeout(() => {
                    const loader = document.getElementById("graphLoading");
                    if (loader) {
                        loader.style.opacity = "0";
                        setTimeout(() => loader.style.display = "none", 300);
                    }
                    if (network) {
                        network.fit({
                            animation: { duration: 400, easingFunction: 'easeInOutQuad' }
                        });
                    }
                }, 400);
            });
    }

    // Render Vis.js Network
    function renderNetwork(nodesList, edgesList) {
        const container = document.getElementById("network-canvas");

        // Filter based on active groups
        const filteredNodes = nodesList.filter(n => {
            if (activeGroups.has(n.group)) return true;
            if (n.group === "CurrentUser" && activeGroups.has("User")) return true;
            if (["SharedBook", "RecommendedBook", "Prerequisite", "StartBook", "TargetBook", "IntermediateBo
    | ok"].includes(n.group) && activeGroups.has("Book")) return true;
            return false;
        });
        const visibleNodeIds = new Set(filteredNodes.map(n => n.id));
        const filteredEdges = edgesList.filter(e => visibleNodeIds.has(e.from) && visibleNodeIds.has(e.to));

        const statNodes = document.getElementById("statNodesCount");
        const statEdges = document.getElementById("statEdgesCount");
        if (statNodes) statNodes.textContent = filteredNodes.length;
        if (statEdges) statEdges.textContent = filteredEdges.length;

        updatePhysicsBtnUI(filteredNodes.length > 0);

        nodesDataSet = new vis.DataSet(filteredNodes);
        edgesDataSet = new vis.DataSet(filteredEdges);

        const data = {
            nodes: nodesDataSet,
            edges: edgesDataSet
        };

        const options = {
            groups: groupConfigurations,
            nodes: {
                borderWidth: 2,
                shadow: {
                    enabled: true,
                    color: 'rgba(0,0,0,0.6)',
                    size: 8,
                    x: 2,
                    y: 2
                }
            },
            edges: {
                arrows: {
```

```
                    to: { enabled: true, scaleFactor: 0.85 }
                },
                color: {
                    color: 'rgba(156, 163, 175, 0.45)',
                    highlight: '#38BDF8',
                    hover: '#60A5FA'
                },
                font: {
                    size: 9,
                    align: 'top',
                    color: '#94a3b8',
                    strokeWidth: 3,
                    strokeColor: '#090d16'
                },
                smooth: {
                    type: 'curvedCW',
                    roundness: 0.15
                }
            },
            physics: {
                enabled: isPhysicsActive,
                solver: 'barnesHut',
                barnesHut: {
                    gravitationalConstant: -4500,
                    centralGravity: 0.25,
                    springLength: 165,
                    springConstant: 0.03,
                    damping: 0.15
                },
                stabilization: {
                    iterations: 150,
                    updateInterval: 25
                }
            },
            interaction: {
                hover: true,
                tooltipDelay: 150,
                navigationButtons: false,
                keyboard: true,
                zoomView: true,
                dragView: true
            }
        };

        if (network) {
            network.destroy();
        }

        network = new vis.Network(container, data, options);

        // Click event on node
        network.on("click", function(params) {
            if (params.nodes.length > 0) {
                const nodeId = params.nodes[0];
                const selectedNode = allNodes.find(n => n.id === nodeId);
                if (selectedNode) {
                    openDrawer(selectedNode);
                }
            } else {
                closeDrawer();
            }
        });

        // Double click to zoom and center
        network.on("doubleClick", function(params) {
            if (params.nodes.length > 0) {
                network.focus(params.nodes[0], {
                    scale: 1.4,
                    animation: { duration: 600, easingFunction: 'easeInOutQuad' }
                });
            }
        });
    }

    // Switch between graph views (Full / Catalog / Personal)
    function switchGraphView(viewName) {
        document.querySelectorAll("#viewSwitcher .view-btn").forEach(btn => {
            btn.classList.toggle("active", btn.getAttribute("data-view") === viewName);
        });
        closeDrawer();
```

```
0969:         isPhysicsActive = true;
0970:         updatePhysicsBtnUI(true);
0971:         activeGroups = new Set(["User", "Book", "Author", "Category", "Publisher", "Copy", "Loan", "Fine", "
      | LoanRequest", "CurrentUser", "SharedBook", "RecommendedBook", "PeerUser", "Prerequisite", "StartBook",
      |  "TargetBook", "IntermediateBook"]);
0972:         loadGraphData(viewName);
0973:     }
0974:
0975:     // Toggle specific node group filter on/off
0976:     function toggleGroupFilter(groupName) {
0977:         const pill = document.querySelector(`.filter-pill[data-group="${groupName}"]`);
0978:         if (pill && pill.classList.contains("disabled")) {
0979:             return;
0980:         }
0981:         if (activeGroups.has(groupName)) {
0982:             activeGroups.delete(groupName);
0983:             if (groupName === "User") {
0984:                 activeGroups.delete("CurrentUser");
0985:                 activeGroups.delete("PeerUser");
0986:             }
0987:             if (groupName === "Book") {
0988:                 activeGroups.delete("SharedBook");
0989:                 activeGroups.delete("RecommendedBook");
0990:                 activeGroups.delete("Prerequisite");
0991:                 activeGroups.delete("StartBook");
0992:                 activeGroups.delete("TargetBook");
0993:                 activeGroups.delete("IntermediateBook");
0994:             }
0995:             pill.classList.remove("active");
0996:             pill.classList.add("inactive");
0997:         } else {
0998:             activeGroups.add(groupName);
0999:             if (groupName === "User") {
1000:                 activeGroups.add("CurrentUser");
1001:                 activeGroups.add("PeerUser");
1002:             }
1003:             if (groupName === "Book") {
1004:                 activeGroups.add("SharedBook");
1005:                 activeGroups.add("RecommendedBook");
1006:                 activeGroups.add("Prerequisite");
1007:                 activeGroups.add("StartBook");
1008:                 activeGroups.add("TargetBook");
1009:                 activeGroups.add("IntermediateBook");
1010:             }
1011:             pill.classList.add("active");
1012:             pill.classList.remove("inactive");
1013:         }
1014:         renderNetwork(allNodes, allEdges);
1015:     }
1016:
1017:     // Search and focus node
1018:     function handleNodeSearch(query) {
1019:         if (!network || !query || query.trim() === "") return;
1020:         const q = query.toLowerCase().trim();
1021:         const currentNodes = nodesDataSet ? nodesDataSet.get() : allNodes;
1022:         const found = currentNodes.find(n => {
1023:             if (n.label && n.label.toLowerCase().includes(q)) return true;
1024:             if (n.properties) {
1025:                 return Object.values(n.properties).some(v => String(v).toLowerCase().includes(q));
1026:             }
1027:             return false;
1028:         });
1029:
1030:         if (found) {
1031:             network.selectNodes([found.id]);
1032:             network.focus(found.id, {
1033:                 scale: 1.3,
1034:                 animation: { duration: 500, easingFunction: 'easeInOutQuad' }
1035:             });
1036:             openDrawer(found);
1037:         }
1038:     }
1039:
1040:     // Toggle Physics engine
1041:     function togglePhysics() {
1042:         if (!network) return;
1043:         isPhysicsActive = !isPhysicsActive;
1044:         const hasNodes = nodesDataSet && nodesDataSet.length > 0;
1045:         updatePhysicsBtnUI(hasNodes);
1046:         network.setOptions({ physics: { enabled: isPhysicsActive } });
```

```
1047:     }
1048:
1049:     // Fit entire graph in view
1050:     function fitGraphView() {
1051:         if (network) {
1052:             network.fit({
1053:                 animation: { duration: 700, easingFunction: 'easeInOutQuad' }
1054:             });
1055:         }
1056:     }
1057:
1058:     // Re-stabilize physics
1059:     function reStabilize() {
1060:         if (!network) return;
1061:         // If physics was paused, re-enable it first because network.stabilize() requires physics to be acti
      | ve
1062:         if (!isPhysicsActive) {
1063:             isPhysicsActive = true;
1064:             const hasNodes = nodesDataSet && nodesDataSet.length > 0;
1065:             updatePhysicsBtnUI(hasNodes);
1066:             network.setOptions({ physics: { enabled: true } });
1067:         }
1068:         network.stabilize(100);
1069:     }
1070:
1071:     // Drawer management
1072:     function openDrawer(node) {
1073:         const drawer = document.getElementById("nodeDrawer");
1074:         const badge = document.getElementById("drawerBadge");
1075:         const title = document.getElementById("drawerNodeTitle");
1076:         const propContainer = document.getElementById("drawerProperties");
1077:         const connContainer = document.getElementById("drawerConnections");
1078:
1079:         badge.textContent = node.group;
1080:         const cfg = groupConfigurations[node.group] || { color: { border: '#3B82F6', background: '#1D4ED8' }
      |  };
1081:         badge.style.backgroundColor = cfg.color.background;
1082:         badge.style.color = '#FFFFFF';
1083:         badge.style.border = `1px solid ${cfg.color.border}`;
1084:
1085:         title.textContent = node.properties.title || node.properties.username || node.properties.name || nod
      | e.label;
1086:
1087:         // Render properties
1088:         propContainer.innerHTML = "";
1089:         const props = node.properties || {};
1090:         for (const [k, v] of Object.entries(props)) {
1091:             const row = document.createElement("div");
1092:             row.className = "prop-row";
1093:             const keySpan = document.createElement("span");
1094:             keySpan.className = "prop-key";
1095:             keySpan.textContent = k.replace(/_/g, ' ');
1096:             const valSpan = document.createElement("span");
1097:             valSpan.className = "prop-val";
1098:             valSpan.textContent = v !== null && v !== undefined ? String(v) : "";
1099:             row.appendChild(keySpan);
1100:             row.appendChild(valSpan);
1101:             propContainer.appendChild(row);
1102:         }
1103:
1104:         // Render connected relationships
1105:         connContainer.innerHTML = "";
1106:         const visibleNodeIds = new Set(nodesDataSet ? nodesDataSet.getIds() : allNodes.map(n => n.id));
1107:         const connectedEdges = allEdges.filter(e => {
1108:             const otherId = e.from === node.id ? e.to : (e.to === node.id ? e.from : null);
1109:             return otherId !== null && visibleNodeIds.has(otherId);
1110:         });
1111:         document.getElementById("connectedCount").textContent = connectedEdges.length;
1112:
1113:         connectedEdges.forEach(e => {
1114:             const isOutgoing = e.from === node.id;
1115:             const targetId = isOutgoing ? e.to : e.from;
1116:             const targetNode = allNodes.find(n => n.id === targetId);
1117:
1118:             if (targetNode && visibleNodeIds.has(targetNode.id)) {
1119:                 const item = document.createElement("div");
1120:                 item.className = "connected-item";
1121:                 const labelSpan = document.createElement("span");
1122:                 labelSpan.className = "connected-item-label";
1123:                 labelSpan.textContent = targetNode.label || "";
```

```
                const relSpan = document.createElement("span");
                relSpan.className = "connected-item-rel";
                relSpan.textContent = `${isOutgoing ? '→' : '←'} :${e.label || ''}`;
                item.appendChild(labelSpan);
                item.appendChild(relSpan);
                item.onclick = () => {
                    network.selectNodes([targetNode.id]);
                    network.focus(targetNode.id, {
                        scale: 1.3,
                        animation: { duration: 500, easingFunction: 'easeInOutQuad' }
                    });
                    openDrawer(targetNode);
                };
                connContainer.appendChild(item);
            }
        });

        drawer.classList.add("open");
    }

    function closeDrawer() {
        document.getElementById("nodeDrawer").classList.remove("open");
        if (network) {
            network.unselectAll();
        }
    }
</script>
{% endblock %}
```

MODULE SOURCE: templates/catalog.html

Lines: 417 | **Lang:** HTML5 / Jinja2 | **Category:** Catalog & Search

Core Role & Purpose: Master book catalog interface with live search and category browsing cards.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify category card subtitles are role-scoped: Admin/Librarian see 'inspect copy availability, and manage loans'; Staff/Student see 'check real-time copy availability, and request loans'.
• Verify '+ Register New Book' button renders a single clean plus sign.

```
{% extends "base.html" %}

{% block title %}Category Directory & Catalog | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Book Catalog{% endblock %}

{% block extra_css %}
<style>
    .hidden {
        display: none !important;
    }

    /* Page Header */
    .catalog-header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 24px;
        flex-wrap: wrap;
        gap: 16px;
    }

    .catalog-title {
        font-size: 1.65rem;
        font-weight: 800;
        letter-spacing: -0.03em;
        color: var(--text-primary);
        display: flex;
        align-items: center;
        gap: 10px;
    }

    .catalog-sub {
        font-size: 0.9rem;
        color: var(--text-secondary);
        margin-top: 4px;
    }

    /* Search Section */
    .search-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 18px 22px;
        margin-bottom: 28px;
        box-shadow: 0 4px 18px rgba(0, 0, 0, 0.2);
    }

    .search-wrapper {
        display: flex;
        align-items: center;
        justify-content: space-between;
        flex-wrap: wrap;
        gap: 16px;
    }

    .search-box {
        position: relative;
        flex: 1;
        min-width: 280px;
    }

    .search-box svg {
```

```
0063:         position: absolute;
0064:         left: 14px;
0065:         top: 50%;
0066:         transform: translateY(-50%);
0067:         width: 18px;
0068:         height: 18px;
0069:         stroke: var(--text-muted);
0070:         fill: none;
0071:         stroke-width: 2;
0072:         pointer-events: none;
0073:     }
0074:
0075:     .search-input {
0076:         width: 100%;
0077:         padding: 11px 40px 11px 44px;
0078:         background: var(--bg-surface-soft);
0079:         border: 1px solid var(--border-subtle);
0080:         border-radius: var(--radius-md);
```

```
0081:         color: var(--text-primary);
0082:         font-size: 0.92rem;
0083:         transition: all 0.2s ease;
0084:     }
0085:
0086:     .search-input:focus {
0087:         outline: none;
0088:         border-color: var(--accent-primary);
0089:         box-shadow: 0 0 0 3px var(--accent-glow);
0090:         background: rgba(30, 41, 59, 0.7);
0091:     }
0092:
0093:     .btn-clear-search {
0094:         position: absolute;
0095:         right: 12px;
0096:         top: 50%;
0097:         transform: translateY(-50%);
0098:         background: none;
0099:         border: none;
0100:         color: var(--text-muted);
0101:         font-size: 1rem;
0102:         cursor: pointer;
0103:         padding: 4px 6px;
0104:         border-radius: 4px;
0105:     }
0106:
0107:     .btn-clear-search:hover {
0108:         color: var(--text-primary);
0109:         background: rgba(255, 255, 255, 0.08);
0110:     }
0111:
0112:     .category-counter {
0113:         font-size: 0.88rem;
0114:         color: var(--text-muted);
0115:         white-space: nowrap;
0116:     }
0117:
0118:     .category-counter strong {
0119:         color: var(--accent-primary);
0120:     }
0121:
0122:     /* Category Cards Grid */
0123:     .category-grid {
0124:         display: grid;
0125:         grid-template-columns: repeat(auto-fill, minmax(290px, 1fr));
0126:         gap: 20px;
0127:     }
0128:
0129:     .category-card {
0130:         background: var(--bg-surface);
0131:         border: 1px solid var(--border-subtle);
0132:         border-radius: var(--radius-lg);
0133:         padding: 24px;
0134:         text-decoration: none;
0135:         color: inherit;
0136:         display: flex;
0137:         flex-direction: column;
0138:         justify-content: space-between;
0139:         min-height: 190px;
0140:         transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1);
0141:         position: relative;
0142:         overflow: hidden;
0143:     }
0144:
0145:     .category-card::before {
0146:         content: '';
0147:         position: absolute;
0148:         top: 0;
0149:         left: 0;
0150:         right: 0;
0151:         height: 3px;
0152:         background: linear-gradient(90deg, var(--accent-primary), #818CF8);
0153:         opacity: 0;
0154:         transition: opacity 0.25s ease;
0155:     }
0156:
0157:     .category-card:hover {
0158:         transform: translateY(-4px);
0159:         border-color: rgba(59, 130, 246, 0.5);
0160:         box-shadow: 0 12px 28px -6px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(59, 130, 246, 0.3);
```

```
0161:         background: rgba(30, 41, 59, 0.6);
0162:     }
0163:
0164:     .category-card:hover::before {
0165:         opacity: 1;
0166:     }
0167:
0168:     .cat-top {
0169:         display: flex;
0170:         align-items: center;
0171:         justify-content: space-between;
0172:         margin-bottom: 14px;
0173:     }
0174:
0175:     .cat-code-badge {
0176:         font-family: var(--font-mono);
0177:         font-size: 0.72rem;
0178:         font-weight: 700;
0179:         letter-spacing: 0.08em;
0180:         text-transform: uppercase;
0181:         color: #93C5FD;
0182:         background: rgba(59, 130, 246, 0.12);
0183:         border: 1px solid rgba(59, 130, 246, 0.3);
0184:         padding: 3px 9px;
0185:         border-radius: var(--radius-sm);
0186:     }
0187:
0188:     .cat-count-badge {
0189:         font-size: 0.78rem;
0190:         font-weight: 600;
0191:         color: var(--text-secondary);
0192:         background: rgba(255, 255, 255, 0.05);
0193:         border: 1px solid var(--border-subtle);
0194:         padding: 3px 10px;
0195:         border-radius: var(--radius-full);
0196:         display: flex;
0197:         align-items: center;
0198:         gap: 5px;
0199:     }
0200:
0201:     .cat-title {
0202:         font-size: 1.25rem;
0203:         font-weight: 700;
0204:         color: var(--text-primary);
0205:         letter-spacing: -0.01em;
0206:         margin-bottom: 8px;
0207:         line-height: 1.3;
0208:     }
0209:
0210:     .cat-desc {
0211:         font-size: 0.84rem;
0212:         color: var(--text-muted);
0213:         line-height: 1.5;
0214:         margin-bottom: 18px;
0215:     }
0216:
0217:     .cat-footer {
0218:         display: flex;
0219:         align-items: center;
0220:         justify-content: space-between;
0221:         padding-top: 14px;
0222:         border-top: 1px solid rgba(255, 255, 255, 0.05);
0223:         color: var(--accent-primary);
0224:         font-size: 0.84rem;
0225:         font-weight: 600;
0226:         transition: gap 0.2s ease;
0227:     }
0228:
0229:     .cat-footer svg {
0230:         transition: transform 0.2s ease;
0231:     }
0232:
0233:     .category-card:hover .cat-footer svg {
0234:         transform: translateX(4px);
0235:     }
0236:
0237:     /* Empty state */
0238:     .empty-category-match {
0239:         grid-column: 1 / -1;
0240:         background: var(--bg-surface);
```

```
0241:         border: 1px dashed var(--border-subtle);
0242:         border-radius: var(--radius-lg);
0243:         padding: 48px 24px;
0244:         text-align: center;
0245:     }
0246:
0247:     .empty-icon {
0248:         width: 56px;
0249:         height: 56px;
0250:         margin: 0 auto 16px auto;
0251:         border-radius: var(--radius-full);
0252:         background: rgba(59, 130, 246, 0.1);
0253:         display: flex;
0254:         align-items: center;
0255:         justify-content: center;
0256:         color: var(--accent-primary);
0257:     }
0258:
0259:     .empty-title {
0260:         font-size: 1.2rem;
0261:         font-weight: 700;
0262:         color: var(--text-primary);
0263:         margin-bottom: 6px;
0264:     }
0265:
0266:     .empty-sub {
0267:         font-size: 0.88rem;
0268:         color: var(--text-muted);
0269:         max-width: 440px;
0270:         margin: 0 auto 18px auto;
0271:     }
0272: </style>
0273: {% endblock %}
0274:
0275: {% block content %}
0276: <!-- Catalog Header -->
0277: <div class="catalog-header">
0278:     <div>
0279:         <h1 class="catalog-title">
0280:             <svg width="26" height="26" viewBox="0 0 24 24" fill="none" stroke="var(--accent-primary)" strok
      | e-width="2.2"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></path><path
      | d="M6 6h10"></path><path d="M6 10h10"></path></svg>
0281:             Library Catalog & Categories
0282:         </h1>
0283:         <p class="catalog-sub">Browse library collections by department category. Click any category to view
      |  and search its books.</p>
0284:     </div>
0285:
0286:     {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
0287:     <div>
0288:         <a href="{{ url_for('add_book') }}" class="btn btn-primary">
0289:             <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></line></svg>
0290:             Register New Book
0291:         </a>
0292:     </div>
0293:     {% endif %}
0294: </div>
0295:
0296: <!-- Search Categories Toolbar -->
0297: <div class="search-card">
0298:     <div class="search-wrapper">
0299:         <div class="search-box">
0300:             <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65"
      |  y2="16.65"></line></svg>
0301:             <input
0302:                 type="text"
0303:                 id="categorySearchInput"
0304:                 class="search-input"
0305:                 placeholder="Search categories by name or section code (e.g., Computer Science, CS-01)..."
0306:                 oninput="filterCategories()"
0307:                 autocomplete="off"
0308:             >
0309:             <button type="button" id="btnClearSearch" class="btn-clear-search hidden" onclick="clearCategory
      | Search()" title="Clear Search">X</button>
0310:         </div>
0311:         <div class="category-counter">
0312:             Showing <strong id="visibleCategoryCount">{{ categories|length }}</strong> of {{ categories|leng
      | th }} Categories
0313:         </div>
```

```
0314:     </div>
0315: </div>
0316:
0317: <!-- Categories Directory Grid -->
0318: <div class="category-grid" id="categoryGrid">
0319:     {% if categories %}
0320:         {% for cat in categories %}
0321:         <a
0322:             href="{{ url_for('category_books', category_id=cat.category_id) }}"
0323:             class="category-card"
0324:             id="catCard_{{ cat.category_id }}"
0325:             data-name="{{ cat.name|lower }}"
0326:             data-code="{{ (cat.section_code or '')|lower }}"
0327:         >
0328:             <div>
0329:                 <div class="cat-top">
0330:                     <span class="cat-code-badge">{{ cat.section_code or 'SEC' }}</span>
0331:                     <span class="cat-count-badge">
0332:                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
      | roke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0333:                         {{ cat.book_count or 0 }} Title{% if cat.book_count != 1 %}s{% endif %}
0334:                     </span>
0335:                 </div>
0336:                 <h3 class="cat-title">{{ cat.name }}</h3>
0337:                 {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Lib
      | rarian'] %}
0338:                 <p class="cat-desc">Click to browse titles, inspect copy availability, and manage loans in t
      | his category.</p>
0339:                 {% else %}
0340:                 <p class="cat-desc">Click to browse titles, check real-time copy availability, and request l
      | oans in this category.</p>
0341:                 {% endif %}
0342:             </div>
0343:             <div class="cat-footer">
0344:                 <span>Browse Collection</span>
0345:                 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2.5"><polyline points="9 18 15 12 9 6"></polyline></svg>
0346:             </div>
0347:         </a>
0348:         {% endfor %}
0349:     {% else %}
0350:         <div class="empty-category-match">
0351:             <div class="empty-icon">
0352:                 <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></path></svg>
0353:             </div>
0354:             <h3 class="empty-title">No Categories Found</h3>
0355:             <p class="empty-sub">No department categories currently exist in the database.</p>
0356:         </div>
0357:     {% endif %}
0358:
0359:     <!-- Real-time Filter No-Match State -->
0360:     <div class="empty-category-match hidden" id="noCategoryMatch">
0361:         <div class="empty-icon">
0362:             <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line></svg>
0363:         </div>
0364:         <h3 class="empty-title">No Matching Categories</h3>
0365:         <p class="empty-sub">We couldn't find any category matching your search query. Try another keyword o
      | r section code.</p>
0366:         <button type="button" class="btn btn-secondary" onclick="clearCategorySearch()">Clear Search Filter<
      | /button>
0367:     </div>
0368: </div>
0369: {% endblock %}
0370:
0371: {% block extra_js %}
0372: <script>
0373:     function filterCategories() {
0374:         const input = document.getElementById('categorySearchInput');
0375:         const query = (input ? input.value : '').trim().toLowerCase();
0376:         const clearBtn = document.getElementById('btnClearSearch');
0377:         if (clearBtn) {
0378:             clearBtn.classList.toggle('hidden', query.length === 0);
0379:         }
0380:
0381:         const cards = document.querySelectorAll('.category-card');
0382:         let visibleCount = 0;
0383:
0384:         cards.forEach(card => {
```

```
0385:             const name = card.getAttribute('data-name') || '';
0386:             const code = card.getAttribute('data-code') || '';
0387:             const matches = name.includes(query) || code.includes(query);
0388:
0389:             if (matches) {
0390:                 card.classList.remove('hidden');
0391:                 visibleCount++;
0392:             } else {
0393:                 card.classList.add('hidden');
0394:             }
0395:         });
0396:
0397:         const counter = document.getElementById('visibleCategoryCount');
0398:         if (counter) {
0399:             counter.textContent = visibleCount;
0400:         }
0401:
0402:         const noMatch = document.getElementById('noCategoryMatch');
0403:         if (noMatch) {
0404:             noMatch.classList.toggle('hidden', visibleCount > 0);
0405:         }
0406:     }
0407:
0408:     function clearCategorySearch() {
0409:         const input = document.getElementById('categorySearchInput');
0410:         if (input) {
0411:             input.value = '';
0412:             filterCategories();
0413:             input.focus();
0414:         }
0415:     }
0416: </script>
0417: {% endblock %}
```

MODULE SOURCE: templates/category_books.html

Lines: 1013 | **Lang:** HTML5 / Jinja2 | **Category:** Category Subgraph Browser

Core Role & Purpose: Category-filtered book list with real-time copy availability badges and role-based action buttons.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:

- Verify single plus sign on Register New Book button.
- Verify copy status badges (Available, Total) and loan request buttons for eligible members.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}{{ category.name }} Books | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}
0005:     <a href="{{ url_for('books') }}" style="color: var(--text-muted); text-decoration: none;">Catalog</a>
0006:     <span style="margin: 0 6px; color: var(--border-subtle);">/</span>
0007:     <span style="color: var(--text-primary);">{{ category.name }}</span>
0008: {% endblock %}
0009:
0010: {% block extra_css %}
0011: <style>
0012:     .hidden {
0013:         display: none !important;
0014:     }
0015:
0016:     /* Back Link & Header */
0017:     .category-page-header {
0018:         margin-bottom: 24px;
0019:     }
0020:
0021:     .back-nav-bar {
0022:         margin-bottom: 14px;
0023:     }
0024:
0025:     .back-nav-btn {
0026:         display: inline-flex;
0027:         align-items: center;
0028:         gap: 8px;
0029:         color: var(--text-secondary);
0030:         text-decoration: none;
0031:         font-size: 0.88rem;
0032:         font-weight: 600;
0033:         padding: 6px 12px;
0034:         background: var(--bg-surface);
0035:         border: 1px solid var(--border-subtle);
0036:         border-radius: var(--radius-sm);
0037:         transition: all 0.2s ease;
0038:     }
0039:
0040:     .back-nav-btn:hover {
0041:         color: var(--accent-primary);
0042:         border-color: var(--accent-primary);
0043:         background: rgba(59, 130, 246, 0.08);
0044:         transform: translateX(-2px);
0045:     }
0046:
0047:     .cat-banner {
0048:         background: var(--bg-surface);
0049:         border: 1px solid var(--border-subtle);
0050:         border-radius: var(--radius-lg);
0051:         padding: 24px 28px;
0052:         display: flex;
0053:         align-items: center;
0054:         justify-content: space-between;
0055:         flex-wrap: wrap;
0056:         gap: 20px;
0057:         position: relative;
0058:         overflow: hidden;
0059:     }
```

```
0060:
0061:     .cat-banner::before {
0062:         content: '';
0063:         position: absolute;
0064:         top: 0;
0065:         left: 0;
0066:         bottom: 0;
0067:         width: 4px;
0068:         background: linear-gradient(180deg, var(--accent-primary), #818CF8);
0069:     }
0070:
0071:     .cat-banner-info {
0072:         display: flex;
0073:         flex-direction: column;
0074:         gap: 6px;
0075:     }
0076:
0077:     .cat-banner-badges {
0078:         display: flex;
0079:         align-items: center;
0080:         gap: 8px;
```

```
0081:         flex-wrap: wrap;
0082:     }
0083:
0084:     .cat-code-pill {
0085:         font-family: var(--font-mono);
0086:         font-size: 0.75rem;
0087:         font-weight: 700;
0088:         letter-spacing: 0.08em;
0089:         text-transform: uppercase;
0090:         color: #93C5FD;
0091:         background: rgba(59, 130, 246, 0.15);
0092:         border: 1px solid rgba(59, 130, 246, 0.35);
0093:         padding: 3px 10px;
0094:         border-radius: var(--radius-sm);
0095:     }
0096:
0097:     .cat-total-pill {
0098:         font-size: 0.78rem;
0099:         font-weight: 600;
0100:         color: var(--text-secondary);
0101:         background: rgba(255, 255, 255, 0.06);
0102:         border: 1px solid var(--border-subtle);
0103:         padding: 3px 10px;
0104:         border-radius: var(--radius-full);
0105:     }
0106:
0107:     .cat-banner-title {
0108:         font-size: 1.8rem;
0109:         font-weight: 800;
0110:         letter-spacing: -0.03em;
0111:         color: var(--text-primary);
0112:         margin: 4px 0 0 0;
0113:     }
0114:
0115:     /* Clean Book Search Toolbar (No Filters) */
0116:     .search-toolbar {
0117:         background: var(--bg-surface);
0118:         border: 1px solid var(--border-subtle);
0119:         border-radius: var(--radius-lg);
0120:         padding: 16px 20px;
0121:         margin: 24px 0;
0122:         display: flex;
0123:         align-items: center;
0124:         justify-content: space-between;
0125:         flex-wrap: wrap;
0126:         gap: 16px;
0127:         box-shadow: 0 4px 18px rgba(0, 0, 0, 0.2);
0128:     }
0129:
0130:     .search-input-wrapper {
0131:         position: relative;
0132:         flex: 1;
0133:         min-width: 280px;
0134:     }
0135:
0136:     .search-input-wrapper svg {
0137:         position: absolute;
0138:         left: 14px;
0139:         top: 50%;
0140:         transform: translateY(-50%);
0141:         width: 18px;
0142:         height: 18px;
0143:         stroke: var(--text-muted);
0144:         fill: none;
0145:         stroke-width: 2;
0146:         pointer-events: none;
0147:     }
0148:
0149:     .search-input {
0150:         width: 100%;
0151:         padding: 11px 40px 11px 44px;
0152:         background: var(--bg-surface-soft);
0153:         border: 1px solid var(--border-subtle);
0154:         border-radius: var(--radius-md);
0155:         color: var(--text-primary);
0156:         font-size: 0.92rem;
0157:         transition: all 0.2s ease;
0158:     }
0159:
0160:     .search-input:focus {
```

```
0161:         outline: none;
0162:         border-color: var(--accent-primary);
0163:         box-shadow: 0 0 0 3px var(--accent-glow);
0164:         background: rgba(30, 41, 59, 0.7);
0165:     }
0166:
0167:     .btn-clear-search {
0168:         position: absolute;
0169:         right: 12px;
0170:         top: 50%;
0171:         transform: translateY(-50%);
0172:         background: none;
0173:         border: none;
0174:         color: var(--text-muted);
0175:         font-size: 1rem;
0176:         cursor: pointer;
0177:         padding: 4px 6px;
0178:         border-radius: 4px;
0179:     }
0180:
0181:     .btn-clear-search:hover {
0182:         color: var(--text-primary);
0183:         background: rgba(255, 255, 255, 0.08);
0184:     }
0185:
0186:     .results-counter {
0187:         font-size: 0.88rem;
0188:         color: var(--text-muted);
0189:         white-space: nowrap;
0190:     }
0191:
0192:     .results-counter strong {
0193:         color: var(--accent-primary);
0194:     }
0195:
0196:     /* Books Grid */
0197:     .books-grid {
0198:         display: grid;
0199:         grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
0200:         gap: 20px;
0201:     }
0202:
0203:     .book-card {
0204:         background: var(--bg-surface);
0205:         border: 1px solid var(--border-subtle);
0206:         border-radius: var(--radius-lg);
0207:         padding: 22px;
0208:         display: flex;
0209:         flex-direction: column;
0210:         justify-content: space-between;
0211:         transition: all 0.25s cubic-bezier(0.16, 1, 0.3, 1);
0212:         position: relative;
0213:     }
0214:
0215:     .book-card:hover {
0216:         transform: translateY(-3px);
0217:         border-color: rgba(59, 130, 246, 0.4);
0218:         box-shadow: 0 12px 28px -6px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(59, 130, 246, 0.2);
0219:     }
0220:
0221:     .book-meta-top {
0222:         display: flex;
0223:         align-items: center;
0224:         justify-content: space-between;
0225:         margin-bottom: 12px;
0226:         gap: 8px;
0227:     }
0228:
0229:     .badge-stock-avail {
0230:         background: rgba(16, 185, 129, 0.15);
0231:         color: #34D399;
0232:         border: 1px solid rgba(16, 185, 129, 0.35);
0233:         font-size: 0.72rem;
0234:         font-weight: 700;
0235:         padding: 3px 8px;
0236:         border-radius: var(--radius-full);
0237:     }
0238:
0239:     .badge-stock-out {
0240:         background: rgba(239, 68, 68, 0.15);
```

```
0241:         color: #F87171;
0242:         border: 1px solid rgba(239, 68, 68, 0.35);
0243:         font-size: 0.72rem;
0244:         font-weight: 700;
0245:         padding: 3px 8px;
0246:         border-radius: var(--radius-full);
0247:     }
0248:
0249:     .book-title {
0250:         font-size: 1.12rem;
0251:         font-weight: 700;
0252:         color: var(--text-primary);
0253:         letter-spacing: -0.01em;
0254:         line-height: 1.35;
0255:         margin-bottom: 8px;
0256:     }
0257:
0258:     .book-authors {
0259:         display: flex;
0260:         align-items: center;
0261:         gap: 6px;
0262:         font-size: 0.84rem;
0263:         color: var(--text-secondary);
0264:         margin-bottom: 16px;
0265:     }
0266:
0267:     .book-authors svg {
0268:         width: 14px;
0269:         height: 14px;
0270:         stroke: var(--text-muted);
0271:         fill: none;
0272:         flex-shrink: 0;
0273:     }
0274:
0275:     /* Frequently Borrowed Together Graph Pill */
0276:     .frequently-paired-badge {
0277:         display: flex;
0278:         align-items: flex-start;
0279:         gap: 7px;
0280:         background: linear-gradient(135deg, rgba(139, 92, 246, 0.12), rgba(59, 130, 246, 0.12));
0281:         border: 1px solid rgba(139, 92, 246, 0.35);
0282:         color: #C4B5FD;
0283:         padding: 6px 10px;
0284:         border-radius: var(--radius-md);
0285:         font-size: 0.75rem;
0286:         margin-bottom: 12px;
0287:         width: 100%;
0288:         box-sizing: border-box;
0289:     }
0290:
0291:     .frequently-paired-badge svg {
0292:         flex-shrink: 0;
0293:         margin-top: 2px;
0294:         color: #A78BFA;
0295:     }
0296:
0297:     .frequently-paired-text {
0298:         white-space: normal;
0299:         word-break: break-word;
0300:         overflow-wrap: break-word;
0301:         line-height: 1.4;
0302:         flex: 1;
0303:     }
0304:
0305:     .frequently-paired-badge strong {
0306:         color: #F8FAFC;
0307:         font-weight: 600;
0308:     }
0309:
0310:     .frequently-paired-count {
0311:         color: #A78BFA;
0312:         font-size: 0.7rem;
0313:         font-weight: 700;
0314:         white-space: nowrap;
0315:         background: rgba(139, 92, 246, 0.2);
0316:         padding: 2px 6px;
0317:         border-radius: var(--radius-sm);
0318:         margin-left: 4px;
0319:         flex-shrink: 0;
0320:         align-self: flex-start;
```

```
0321:         margin-top: 1px;
0322:     }
0323:
0324:     /* Prerequisite & Next-Step Graph Utility Badges */
0325:     .prereq-paired-badge {
0326:         display: flex;
0327:         align-items: flex-start;
0328:         gap: 7px;
0329:         background: linear-gradient(135deg, rgba(245, 158, 11, 0.12), rgba(217, 119, 6, 0.08));
0330:         border: 1px solid rgba(245, 158, 11, 0.35);
0331:         color: #FCD34D;
0332:         padding: 6px 10px;
0333:         border-radius: var(--radius-md);
0334:         font-size: 0.75rem;
0335:         margin-bottom: 8px;
0336:         width: 100%;
0337:         box-sizing: border-box;
0338:     }
0339:
0340:     .prereq-paired-badge svg {
0341:         flex-shrink: 0;
0342:         margin-top: 2px;
0343:         color: #F59E0B;
0344:     }
0345:
0346:     .next-step-paired-badge {
0347:         display: flex;
0348:         align-items: flex-start;
0349:         gap: 7px;
0350:         background: linear-gradient(135deg, rgba(99, 102, 241, 0.12), rgba(129, 140, 248, 0.08));
0351:         border: 1px solid rgba(99, 102, 241, 0.35);
0352:         color: #C7D2FE;
0353:         padding: 6px 10px;
0354:         border-radius: var(--radius-md);
0355:         font-size: 0.75rem;
0356:         margin-bottom: 8px;
0357:         width: 100%;
0358:         box-sizing: border-box;
0359:     }
0360:
0361:     .next-step-paired-badge svg {
0362:         flex-shrink: 0;
0363:         margin-top: 2px;
0364:         color: #818CF8;
0365:     }
0366:
0367:     .book-details-list {
0368:         background: var(--bg-surface-soft);
0369:         border: 1px solid rgba(255, 255, 255, 0.04);
0370:         border-radius: var(--radius-md);
0371:         padding: 12px 14px;
0372:         margin-bottom: 16px;
0373:         display: flex;
0374:         flex-direction: column;
0375:         gap: 6px;
0376:     }
0377:
0378:     .book-detail-row {
0379:         display: flex;
0380:         align-items: center;
0381:         justify-content: space-between;
0382:         font-size: 0.78rem;
0383:     }
0384:
0385:     .book-detail-row span {
0386:         color: var(--text-muted);
0387:     }
0388:
0389:     .book-detail-row strong {
0390:         color: var(--text-primary);
0391:         font-weight: 600;
0392:     }
0393:
0394:     /* Admin Inventory Adjuster with Numerical Quantity Input */
0395:     .inventory-adjust-box {
0396:         background: rgba(59, 130, 246, 0.06);
0397:         border: 1px solid rgba(59, 130, 246, 0.2);
0398:         border-radius: var(--radius-md);
0399:         padding: 12px 14px;
0400:         margin-top: 10px;
```

```
0401:     }
0402:
0403:     .inventory-adjust-top {
0404:         display: flex;
0405:         align-items: center;
0406:         justify-content: space-between;
0407:         margin-bottom: 8px;
0408:     }
0409:
0410:     .inventory-adjust-label {
0411:         font-size: 0.76rem;
0412:         font-weight: 700;
0413:         text-transform: uppercase;
0414:         letter-spacing: 0.05em;
0415:         color: #93C5FD;
0416:     }
0417:
0418:     .inventory-current-stock {
0419:         font-size: 0.75rem;
0420:         font-family: var(--font-mono);
0421:         color: var(--text-muted);
0422:     }
0423:
0424:     .inventory-type-controls {
0425:         display: flex;
0426:         align-items: center;
0427:         gap: 8px;
0428:     }
0429:
0430:     .qty-stepper-wrapper {
0431:         display: inline-flex;
0432:         align-items: center;
0433:         background: var(--bg-surface);
0434:         border: 1px solid var(--border-subtle);
0435:         border-radius: var(--radius-sm);
0436:         overflow: hidden;
0437:     }
0438:
0439:     .btn-stepper-step {
0440:         background: rgba(255, 255, 255, 0.04);
0441:         border: none;
0442:         color: var(--text-secondary);
0443:         font-size: 1rem;
0444:         font-weight: 700;
0445:         width: 28px;
0446:         height: 32px;
0447:         display: flex;
0448:         align-items: center;
0449:         justify-content: center;
0450:         cursor: pointer;
0451:         transition: all 0.15s ease;
0452:         user-select: none;
0453:     }
0454:
0455:     .btn-stepper-step:hover {
0456:         background: rgba(59, 130, 246, 0.2);
0457:         color: var(--accent-primary);
0458:     }
0459:
0460:     .btn-stepper-step:active {
0461:         background: rgba(59, 130, 246, 0.35);
0462:     }
0463:
0464:     .qty-type-input {
0465:         width: 48px;
0466:         height: 32px;
0467:         padding: 4px;
0468:         background: transparent;
0469:         border: none;
0470:         border-left: 1px solid var(--border-subtle);
0471:         border-right: 1px solid var(--border-subtle);
0472:         color: var(--text-primary);
0473:         font-family: var(--font-mono);
0474:         font-size: 0.9rem;
0475:         font-weight: 700;
0476:         text-align: center;
0477:         transition: all 0.2s ease;
0478:         appearance: textfield;
0479:         -moz-appearance: textfield;
0480:     }
```

```
0481:
0482:     .qty-type-input::-webkit-outer-spin-button,
0483:     .qty-type-input::-webkit-inner-spin-button {
0484:         -webkit-appearance: none;
0485:         margin: 0;
0486:     }
0487:
0488:     .qty-type-input:focus {
0489:         outline: none;
0490:         background: rgba(59, 130, 246, 0.08);
0491:     }
0492:
0493:     .btn-qty-action {
0494:         flex: 1;
0495:         display: inline-flex;
0496:         align-items: center;
0497:         justify-content: center;
0498:         gap: 5px;
0499:         padding: 7px 10px;
0500:         font-size: 0.76rem;
0501:         font-weight: 700;
0502:         border-radius: var(--radius-sm);
0503:         cursor: pointer;
0504:         transition: all 0.2s ease;
0505:     }
0506:
0507:     .btn-qty-decrease {
0508:         background: rgba(239, 68, 68, 0.12);
0509:         color: #F87171;
0510:         border: 1px solid rgba(239, 68, 68, 0.3);
0511:     }
0512:
0513:     .btn-qty-decrease:hover:not(:disabled) {
0514:         background: rgba(239, 68, 68, 0.25);
0515:         border-color: #EF4444;
0516:         color: #FFFFFF;
0517:     }
0518:
0519:     .btn-qty-increase {
0520:         background: rgba(59, 130, 246, 0.15);
0521:         color: #60A5FA;
0522:         border: 1px solid rgba(59, 130, 246, 0.35);
0523:     }
0524:
0525:     .btn-qty-increase:hover {
0526:         background: rgba(59, 130, 246, 0.3);
0527:         border-color: var(--accent-primary);
0528:         color: #FFFFFF;
0529:     }
0530:
0531:     /* Patron Request Action */
0532:     .patron-action-box {
0533:         margin-top: 14px;
0534:     }
0535:
0536:     .btn-request-loan {
0537:         width: 100%;
0538:         padding: 8px 12px;
0539:         background: rgba(59, 130, 246, 0.15);
0540:         color: #60A5FA;
0541:         border: 1px solid rgba(59, 130, 246, 0.35);
0542:         border-radius: var(--radius-md);
0543:         font-size: 0.82rem;
0544:         font-weight: 700;
0545:         cursor: pointer;
0546:         display: flex;
0547:         align-items: center;
0548:         justify-content: center;
0549:         gap: 8px;
0550:         transition: all 0.2s ease;
0551:     }
0552:
0553:     .btn-request-loan:hover {
0554:         background: var(--accent-primary);
0555:         color: #FFFFFF;
0556:         border-color: transparent;
0557:     }
0558:
0559:     /* Empty states */
0560:     .empty-state-panel {
```

```
0561:         grid-column: 1 / -1;
0562:         background: var(--bg-surface);
0563:         border: 1px dashed var(--border-subtle);
0564:         border-radius: var(--radius-lg);
0565:         padding: 48px 24px;
0566:         text-align: center;
0567:     }
0568:
0569:     .empty-state-icon {
0570:         width: 56px;
0571:         height: 56px;
0572:         margin: 0 auto 16px auto;
0573:         border-radius: var(--radius-full);
0574:         background: rgba(59, 130, 246, 0.1);
0575:         display: flex;
0576:         align-items: center;
0577:         justify-content: center;
0578:         color: var(--accent-primary);
0579:     }
0580:
0581:     .empty-state-title {
0582:         font-size: 1.2rem;
0583:         font-weight: 700;
0584:         color: var(--text-primary);
0585:         margin-bottom: 6px;
0586:     }
0587:
0588:     .empty-state-sub {
0589:         font-size: 0.88rem;
0590:         color: var(--text-muted);
0591:         max-width: 440px;
0592:         margin: 0 auto 18px auto;
0593:     }
0594: </style>
0595: {% endblock %}
0596:
0597: {% block content %}
0598: <!-- Back to All Categories Navigation -->
0599: <div class="back-nav-bar">
0600:     <a href="{{ url_for('books') }}" class="back-nav-btn">
0601:         <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"
      | ><line x1="19" y1="12" x2="5" y2="12"></line><polyline points="12 19 5 12 12 5"></polyline></svg>
0602:         Back to All Categories
0603:     </a>
0604: </div>
0605:
0606: <!-- Category Banner -->
0607: <div class="cat-banner">
0608:     <div class="cat-banner-info">
0609:         <div class="cat-banner-badges">
0610:             <span class="cat-code-pill">{{ category.section_code or 'SEC' }}</span>
0611:             {% set cat_count = category.total_books if category.total_books is defined else books|length %}
0612:             <span class="cat-total-pill" id="totalBooksInCatBadge">{{ cat_count }} Title{% if cat_count != 1
      |  %}s{% endif %} in Category</span>
0613:         </div>
0614:         <h1 class="cat-banner-title">{{ category.name }}</h1>
0615:     </div>
0616:
0617:     {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Librarian'] %}
0618:     <div>
0619:         <a href="{{ url_for('add_book') }}" class="btn btn-primary">
0620:             <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
      | 2"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></line></svg>
0621:             Register New Book
0622:         </a>
0623:     </div>
0624:     {% endif %}
0625: </div>
0626:
0627: {% if user_borrowing_blocked %}
0628: <div style="background: rgba(239, 68, 68, 0.12); border: 1px solid rgba(239, 68, 68, 0.35); border-radius: v
      | ar(--radius-md); padding: 14px 20px; margin-bottom: 24px; display: flex; align-items: center; gap: 14p
      | x;">
0629:     <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="#F87171" stroke-width="2" style="fle
      | x-shrink: 0;"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="8" x2="12" y2="12"></line><lin
      | e x1="12" y1="16" x2="12.01" y2="16"></line></svg>
0630:     <div>
0631:         <strong style="color: #F87171; font-size: 0.92rem;">Borrowing Blocked:</strong>
0632:         <span style="color: var(--text-secondary); margin-left: 6px; font-size: 0.88rem;">{{ block_reason }}
      | </span>
```

```
0633:     </div>
0634: </div>
0635: {% endif %}
0636:
0637: <!-- Clean Book Search Toolbar (Standard GET Form) -->
0638: <form method="GET" action="{{ url_for('category_books', category_id=category.category_id) }}" class="search-
      | toolbar">
0639:     <div class="search-input-wrapper">
0640:         <svg viewBox="0 0 24 24"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2=
      | "16.65"></line></svg>
0641:         <input
0642:             type="text"
0643:             name="q"
0644:             id="bookSearchInput"
0645:             class="search-input"
0646:             placeholder="Search books in {{ category.name }} by title, author, or ISBN..."
0647:             value="{{ q or '' }}"
0648:             autocomplete="off"
0649:         >
0650:         {% if q %}
0651:         <a href="{{ url_for('category_books', category_id=category.category_id) }}" class="btn-clear-search"
      |  title="Clear search" style="text-decoration: none; display: flex; align-items: center; justify-conten
      | t: center;">X</a>
0652:         {% endif %}
0653:     </div>
0654:     <button type="submit" class="btn btn-primary" style="padding: 10px 18px; border-radius: var(--radius-sm)
      | ; font-size: 0.88rem;">Search</button>
0655:     <div class="results-counter">
0656:         Showing <strong id="visibleBookCount">{{ books|length }}</strong> Books
0657:     </div>
0658: </form>
0659:
0660: <!-- Books Directory Grid -->
0661: <div class="books-grid" id="booksGrid">
0662:     {% if books %}
0663:         {% for book in books %}
0664:         <div
0665:             class="book-card"
0666:             id="bookCard_{{ book.book_id }}"
0667:             data-book-id="{{ book.book_id }}"
0668:             data-title="{{ book.title|lower }}"
0669:             data-author="{{ (book.authors or '')|lower }}"
0670:             data-isbn="{{ book.isbn|lower }}"
0671:             data-publisher="{{ (book.publisher_name or '')|lower }}"
0672:             data-year="{{ book.publication_year or '' }}"
0673:         >
0674:             <div>
0675:                 <!-- Card Header: Category & Stock Status -->
0676:                 <div class="book-meta-top">
0677:                     <span style="font-size: 0.72rem; font-weight: 700; color: var(--accent-primary); text-tr
      | ansform: uppercase;">
0678:                         {{ category.name }}
0679:                     </span>
0680:                     {% if (book.available_copies or 0) > 0 %}
0681:                         <span class="badge-stock-avail">{{ book.available_copies }} Available</span>
0682:                     {% else %}
0683:                         <span class="badge-stock-out">Out of Stock</span>
0684:                     {% endif %}
0685:                 </div>
0686:
0687:                 <!-- Book Title & Authors -->
0688:                 <h3 class="book-title">{{ book.title }}</h3>
0689:                 <div class="book-authors">
0690:                     <svg viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><cir
      | cle cx="12" cy="7" r="4"></circle></svg>
0691:                     <span>{{ book.authors or 'Author Unspecified' }}</span>
0692:                 </div>
0693:
0694:                 {% if book.companion %}
0695:                 <!-- Frequently Borrowed Together Graph Intelligence Pill -->
0696:                 <div class="frequently-paired-badge" title="Co-Borrowing Graph Traversal: Frequently borrowe
      | d together by {{ book.companion.count }} common patrons">
0697:                     <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2.2"><path d="M10 13a5 5 0 0 0 7.54.54l3-3a5 5 0 0 0-7.07-7.07l-1.72 1.71"></path><path d="M14
      |  11a5 5 0 0 0-7.54-.54l-3 3a5 5 0 0 0 7.07 7.07l1.71-1.71"></path></svg>
0698:                     <span class="frequently-paired-text">
0699:                         Frequently Paired With: <strong>{{ book.companion.title }}</strong>
0700:                     </span>
0701:                     <span class="frequently-paired-count">{{ book.companion.count }}&times;</span>
0702:                 </div>
```

```
0703:                 {% endif %}
0704:
0705:                 {% if book.prerequisites %}
0706:                 <!-- Prerequisite Requirement Badges -->
0707:                 {% for pre in book.prerequisites %}
0708:                 <div class="prereq-paired-badge" title="Academic Hierarchy: Prerequisite textbook recommende
      | d before studying this material">
0709:                     <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2.2"><polyline points="15 18 9 12 15 6"></polyline></svg>
0710:                     <span class="frequently-paired-text">
0711:                         Requires: <strong>{{ pre.title }}</strong>
0712:                     </span>
0713:                     {% if pre.completed %}
0714:                     <span class="badge" style="background: rgba(16, 185, 129, 0.25); color: #34D399; font-si
      | ze: 0.65rem; padding: 1px 6px; border-radius: var(--radius-full); white-space: nowrap; flex-shrink: 0;
      |  align-self: flex-start; margin-top: 1px;">✓ Completed</span>
0715:                     {% else %}
0716:                     <span class="badge" style="background: rgba(245, 158, 11, 0.25); color: #FDE047; font-si
      | ze: 0.65rem; padding: 1px 6px; border-radius: var(--radius-full); white-space: nowrap; flex-shrink: 0;
      |  align-self: flex-start; margin-top: 1px;">Prereq</span>
0717:                     {% endif %}
0718:                 </div>
0719:                 {% endfor %}
0720:                 {% endif %}
0721:
0722:                 {% if book.next_books %}
0723:                 <!-- Next Step Advancement Badges -->
0724:                 {% for nxt in book.next_books %}
0725:                 <div class="next-step-paired-badge" title="Academic Hierarchy: Next advanced textbook unlock
      | ed upon completing this book">
0726:                     <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2.2"><polyline points="9 18 15 12 9 6"></polyline></svg>
0727:                     <span class="frequently-paired-text">
0728:                         Unlocks: <strong>{{ nxt.title }}</strong>
0729:                     </span>
0730:                     <span class="badge" style="background: rgba(99, 102, 241, 0.25); color: #E0E7FF; font-si
      | ze: 0.65rem; padding: 1px 6px; border-radius: var(--radius-full); white-space: nowrap; flex-shrink: 0;
      |  align-self: flex-start; margin-top: 1px;">Next Step</span>
0731:                 </div>
0732:                 {% endfor %}
0733:                 {% endif %}
0734:             </div>
0735:
0736:             <div>
0737:                 <!-- Metadata Attributes -->
0738:                 <div class="book-details-list">
0739:                     <div class="book-detail-row">
0740:                         <span>ISBN:</span>
0741:                         <strong style="font-family: var(--font-mono); font-size: 0.76rem;">{{ book.isbn }}</
      | strong>
0742:                     </div>
0743:                     <div class="book-detail-row">
0744:                         <span>Publisher:</span>
0745:                         <strong>{{ book.publisher_name or 'N/A' }}</strong>
0746:                     </div>
0747:                     <div class="book-detail-row">
0748:                         <span>Publication Year:</span>
0749:                         <strong>{{ book.publication_year or 'Unknown' }}</strong>
0750:                     </div>
0751:                     <div class="book-detail-row">
0752:                         <span>Available / Total Copies:</span>
0753:                         <strong>{{ book.available_copies or 0 }} / {{ book.total_copies or 0 }}</strong>
0754:                     </div>
0755:                 </div>
0756:
0757:                 <!-- Admin & Librarian Edit Metadata Action -->
0758:                 {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Lib
      | rarian'] %}
0759:                 <div style="margin-top: 10px; margin-bottom: 10px;">
0760:                     <a href="{{ url_for('edit_book', book_id=book.book_id) }}" class="btn" style="width: 100
      | %; background: rgba(99, 102, 241, 0.12); color: #818CF8; border: 1px solid rgba(99, 102, 241, 0.3); pa
      | dding: 6px 12px; font-size: 0.78rem; font-weight: 700; border-radius: var(--radius-sm); text-decoratio
      | n: none; display: flex; align-items: center; justify-content: center; gap: 6px;">
0761:                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
      | roke-width="2"><path d="M12 20h9"></path><path d="M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3
      | .5z"></path></svg>
0762:                         Edit Metadata
0763:                     </a>
0764:                 </div>
0765:                 {% endif %}
```

```
0766:
0767:                 <!-- Admin & Librarian Physical Copy Inventory Adjuster with Numerical Quantity Input -->
0768:                 {% if session.get('role') in ['Admin', 'Librarian'] or session.get('Role') in ['Admin', 'Lib
      | rarian'] %}
0769:                 <div class="inventory-adjust-box">
0770:                     <div class="inventory-adjust-top">
0771:                         <span class="inventory-adjust-label">Modify Copies:</span>
0772:                         <span class="inventory-current-stock">{{ book.available_copies or 0 }} avail / {{ bo
      | ok.total_copies or 0 }} total</span>
0773:                     </div>
0774:                     <form method="POST" action="{{ url_for('adjust_inventory', book_id=book.book_id) }}" cla
      | ss="adjust-inventory-form" style="margin: 0;">
0775:                         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0776:                         <input type="hidden" name="category_id" value="{{ category.category_id }}">
0777:                         <div class="inventory-type-controls">
0778:                             <div class="qty-stepper-wrapper">
0779:                                 <button type="button" class="btn-stepper-step" onclick="stepQty(this, -1)" t
      | itle="Decrease quantity by 1 (Mouse)">–</button>
0780:                                 <input
0781:                                     type="number"
0782:                                     name="quantity"
0783:                                     class="qty-type-input"
0784:                                     value="1"
0785:                                     min="1"
0786:                                     max="100"
0787:                                     step="1"
0788:                                     placeholder="Qty"
0789:                                     inputmode="numeric"
0790:                                     pattern="[0-9]+"
0791:                                     oninput="this.value = this.value.replace(/[^0-9]/g, ''); if(this.value =
      | == '' || parseInt(this.value) < 1) this.value = '1'; if(parseInt(this.value) > 100) this.value = '100'
      | ;"
0792:                                     onkeydown="handleQtyKey(event, this)"
0793:                                     title="Enter number of books (Type with keyboard or click +/- with mouse
      | )"
0794:                                     required
0795:                                 >
0796:                                 <button type="button" class="btn-stepper-step" onclick="stepQty(this, 1)" ti
      | tle="Increase quantity by 1 (Mouse)">+</button>
0797:                             </div>
0798:                             <button
0799:                                 type="submit"
0800:                                 name="action"
0801:                                 value="decrease"
0802:                                 class="btn-qty-action btn-qty-decrease"
0803:                                 title="Decrease stock by entered quantity (Mouse or Keyboard)"
0804:                                 {% if (book.available_copies or 0) <= 0 %}disabled style="opacity: 0.35; cur
      | sor: not-allowed;"{% endif %}
0805:                                 onclick="return confirm('Decrease physical copies for \'{{ book.title|escape
      |  }}\' by ' + this.form.quantity.value + ' item(s)?');"
0806:                             >
0807:                                 <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentC
      | olor" stroke-width="2.5"><line x1="5" y1="12" x2="19" y2="12"></line></svg>
0808:                                 - Decrease
0809:                             </button>
0810:                             <button
0811:                                 type="submit"
0812:                                 name="action"
0813:                                 value="increase"
0814:                                 class="btn-qty-action btn-qty-increase"
0815:                                 title="Increase stock by entered quantity (Mouse or Keyboard)"
0816:                                 onclick="return confirm('Add ' + this.form.quantity.value + ' new physical c
      | opy/copies for \'{{ book.title|escape }}\'?');"
0817:                             >
0818:                                 <svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentC
      | olor" stroke-width="2.5"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="
      | 12"></line></svg>
0819:                                 + Increase
0820:                             </button>
0821:                         </div>
0822:                     </form>
0823:                 </div>
0824:                 {% endif %}
0825:
0826:                 <!-- Admin-Only Delete Book Action -->
0827:                 {% if session.get('role') == 'Admin' or session.get('Role') == 'Admin' %}
0828:                 <div style="margin-top: 10px;">
0829:                     <form method="POST" action="{{ url_for('delete_book', book_id=book.book_id) }}" onsubmit
      | ="return confirm('PERMANENT ACTION: Are you sure you want to completely delete and archive \'{{ book.t
      | itle|escape }}\' (#{{ book.book_id }}) from the catalog? This will delete all copies and history.');">
```

```
0830:                         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0831:                         <input type="hidden" name="category_id" value="{{ category.category_id }}">
0832:                         <button type="submit" class="btn" style="width: 100%; background: rgba(239, 68, 68,
      | 0.12); color: #F87171; border: 1px solid rgba(239, 68, 68, 0.3); padding: 6px 12px; font-size: 0.78rem
      | ; font-weight: 700; border-radius: var(--radius-sm); cursor: pointer; display: flex; align-items: cent
      | er; justify-content: center; gap: 6px;">
0833:                             <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor
      | " stroke-width="2"><polyline points="3 6 5 6 21 6"></polyline><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0
      | 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></path></svg>
0834:                             Delete Book
0835:                         </button>
0836:                     </form>
0837:                 </div>
0838:                 {% endif %}
0839:
0840:                 <!-- Student & Staff Borrowing Requests -->
0841:                 {% if session.get('role') in ['Student', 'Staff'] or session.get('Role') in ['Student', 'Sta
      | ff'] %}
0842:                 <div class="patron-action-box">
0843:                     {% if user_borrowing_blocked %}
0844:                         <div style="text-align: center; padding: 7px 10px; background: rgba(239, 68, 68, 0.1
      | 2); color: #FCA5A5; border: 1px solid rgba(239, 68, 68, 0.3); border-radius: var(--radius-md); font-si
      | ze: 0.78rem; font-weight: 600;" title="{{ block_reason }}">
0845:                             ■■ Borrowing Blocked
0846:                         </div>
0847:                     {% elif book.book_id in user_active_loans %}
0848:                         <div style="text-align: center; padding: 7px 10px; background: rgba(59, 130, 246, 0.
      | 12); color: #93C5FD; border: 1px solid rgba(59, 130, 246, 0.3); border-radius: var(--radius-md); font-
      | size: 0.78rem; font-weight: 600;">
0849:                             ■ Already Borrowed
0850:                         </div>
0851:                     {% elif user_awaiting_pickup and book.book_id in user_awaiting_pickup %}
0852:                         <div style="text-align: center; padding: 7px; background: rgba(59, 130, 246, 0.15);
      | color: #60A5FA; border: 1px solid rgba(59, 130, 246, 0.35); border-radius: var(--radius-md); font-size
      | : 0.8rem; font-weight: 700;">
0853:                             ✓ Approved &ndash; Ready for Desk Pickup
0854:                         </div>
0855:                     {% elif user_pending_requests and book.book_id in user_pending_requests %}
0856:                         <div style="text-align: center; padding: 7px; background: rgba(245, 158, 11, 0.12);
      | color: #FBBF24; border: 1px solid rgba(245, 158, 11, 0.3); border-radius: var(--radius-md); font-size:
      |  0.8rem; font-weight: 700;">
0857:                             ✓ Request Pending Approval
0858:                         </div>
0859:                     {% elif (not user_pending_requests and not user_awaiting_pickup) and book.book_id in use
      | r_requests %}
0860:                         <div style="text-align: center; padding: 7px; background: rgba(245, 158, 11, 0.12);
      | color: #FBBF24; border: 1px solid rgba(245, 158, 11, 0.3); border-radius: var(--radius-md); font-size:
      |  0.8rem; font-weight: 700;">
0861:                             ✓ Request Pending Approval
0862:                         </div>
0863:                     {% elif (book.available_copies or 0) > 0 %}
0864:                         <form method="POST" action="{{ url_for('request_loan', book_id=book.book_id) }}" sty
      | le="margin: 0;">
0865:                             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0866:                             <input type="hidden" name="category_id" value="{{ category.category_id }}">
0867:                             <button type="submit" class="btn-request-loan" onclick="return confirm('Submit l
      | oan request for \'{{ book.title|escape }}\'?');">
0868:                                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentC
      | olor" stroke-width="2.5"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></
      | path><path d="M6 6h10"></path></svg>
0869:                                 Request Physical Loan
0870:                             </button>
0871:                         </form>
0872:                     {% else %}
0873:                         <div style="text-align: center; padding: 7px; background: rgba(239, 68, 68, 0.1); co
      | lor: #F87171; border: 1px solid rgba(239, 68, 68, 0.25); border-radius: var(--radius-md); font-size: 0
      | .8rem; font-weight: 600;">
0874:                             Currently Out of Stock
0875:                         </div>
0876:                     {% endif %}
0877:                 </div>
0878:                 {% endif %}
0879:             </div>
0880:         </div>
0881:         {% endfor %}
0882:     {% else %}
0883:         {% if q %}
0884:         <div class="empty-state-panel" id="noBookMatch">
0885:             <div class="empty-state-icon">
0886:                 <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
```

```
      | th="2"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line></svg
      | >
0887:             </div>
0888:             <h3 class="empty-state-title">No Books Match Search</h3>
0889:             <p class="empty-state-sub">We couldn't find any books in this category matching "{{ q }}".</p>
0890:             <a href="{{ url_for('category_books', category_id=category.category_id) }}" class="btn btn-secon
      | dary">Clear Search</a>
0891:         </div>
0892:         {% else %}
0893:         <div class="empty-state-panel">
0894:             <div class="empty-state-icon">
0895:                 <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0896:             </div>
0897:             <h3 class="empty-state-title">No Books in this Category</h3>
0898:             <p class="empty-state-sub">There are currently no book titles registered under {{ category.name
      | }}.</p>
0899:             <a href="{{ url_for('books') }}" class="btn btn-secondary">Return to All Categories</a>
0900:         </div>
0901:         {% endif %}
0902:     {% endif %}
0903: </div>
0904: {% endblock %}
0905:
0906: {% block extra_js %}
0907: <script>
0908:     /* Mouse & Keyboard Inventory Quantity Helpers */
0909:     function stepQty(btn, delta) {
0910:         const wrapper = btn.closest('.qty-stepper-wrapper');
0911:         if (!wrapper) return;
0912:         const input = wrapper.querySelector('.qty-type-input');
0913:         if (input) {
0914:             let val = parseInt(input.value, 10);
0915:             if (isNaN(val) || val < 1) val = 1;
0916:             val = Math.max(1, Math.min(100, val + delta));
0917:             input.value = val;
0918:         }
0919:     }
0920:
0921:     function handleQtyKey(event, input) {
0922:         if (event.key === 'ArrowUp') {
0923:             event.preventDefault();
0924:             let val = parseInt(input.value, 10);
0925:             if (isNaN(val) || val < 1) val = 1;
0926:             input.value = Math.min(100, val + 1);
0927:         } else if (event.key === 'ArrowDown') {
0928:             event.preventDefault();
0929:             let val = parseInt(input.value, 10);
0930:             if (isNaN(val) || val < 1) val = 1;
0931:             input.value = Math.max(1, val - 1);
0932:         }
0933:     }
0934:
0935:     /* Instant Client-side Search Filtering for Category Books */
0936:     document.addEventListener('DOMContentLoaded', function() {
0937:         const searchInput = document.getElementById('bookSearchInput');
0938:         const booksGrid = document.getElementById('booksGrid');
0939:         const visibleCounter = document.getElementById('visibleBookCount');
0940:         if (!searchInput || !booksGrid) return;
0941:
0942:         const cards = Array.from(booksGrid.querySelectorAll('.book-card'));
0943:
0944:         // Client-side dynamic empty state panel
0945:         let clientEmpty = document.getElementById('clientNoMatch');
0946:         if (!clientEmpty) {
0947:             clientEmpty = document.createElement('div');
0948:             clientEmpty.id = 'clientNoMatch';
0949:             clientEmpty.className = 'empty-state-panel';
0950:             clientEmpty.style.display = 'none';
0951:             clientEmpty.style.marginTop = '24px';
0952:             clientEmpty.innerHTML = `
0953:                 <div class="empty-state-icon">
0954:                     <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line><
      | /svg>
0955:                 </div>
0956:                 <h3 class="empty-state-title">No Books Match Search</h3>
0957:                 <p class="empty-state-sub" id="clientEmptyText">No books found matching your search.</p>
0958:                 <button type="button" class="btn btn-secondary" onclick="document.getElementById('bookSearch
      | Input').value=''; document.getElementById('bookSearchInput').dispatchEvent(new Event('input'));">Clear
```

```
      |  Search</button>
0959:             `;
0960:             booksGrid.parentNode.insertBefore(clientEmpty, booksGrid.nextSibling);
0961:         }
0962:
0963:         function runLiveFilter() {
0964:             const raw = searchInput.value.trim().toLowerCase();
0965:             const terms = raw.split(/\s+/).filter(Boolean);
0966:             let visible = 0;
0967:
0968:             cards.forEach(card => {
0969:                 if (terms.length === 0) {
0970:                     card.style.display = '';
0971:                     visible++;
0972:                     return;
0973:                 }
0974:                 const title = card.getAttribute('data-title') || '';
0975:                 const author = card.getAttribute('data-author') || '';
0976:                 const isbn = card.getAttribute('data-isbn') || '';
0977:                 const cleanIsbn = isbn.replace(/[^a-z0-9]/gi, '');
0978:                 const publisher = card.getAttribute('data-publisher') || '';
0979:                 const year = card.getAttribute('data-year') || '';
0980:                 const fullText = `${title} ${author} ${isbn} ${cleanIsbn} ${publisher} ${year}`.toLowerCase(
      | );
0981:
0982:                 const matchesAll = terms.every(term => {
0983:                     const cleanTerm = term.replace(/[^a-z0-9]/gi, '');
0984:                     return fullText.includes(term) || (cleanTerm && fullText.includes(cleanTerm));
0985:                 });
0986:
0987:                 if (matchesAll) {
0988:                     card.style.display = '';
0989:                     visible++;
0990:                 } else {
0991:                     card.style.display = 'none';
0992:                 }
0993:             });
0994:
0995:             if (visibleCounter) {
0996:                 visibleCounter.textContent = visible;
0997:             }
0998:
0999:             if (clientEmpty) {
1000:                 if (visible === 0 && terms.length > 0) {
1001:                     clientEmpty.style.display = 'block';
1002:                     document.getElementById('clientEmptyText').textContent = `No books found in this categor
      | y matching "${searchInput.value.trim()}".`;
1003:                 } else {
1004:                     clientEmpty.style.display = 'none';
1005:                 }
1006:             }
1007:         }
1008:
1009:         searchInput.addEventListener('input', runLiveFilter);
1010:         searchInput.addEventListener('keyup', runLiveFilter);
1011:     });
1012: </script>
1013: {% endblock %}
```

MODULE SOURCE: templates/recommendations.html

Lines: 928 | **Lang:** HTML5 / Jinja2 | **Category:** AI Recommendation Engine

Core Role & Purpose: AI graph recommendation dashboard based on collaborative borrowing similarity and graph proximity.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify that raw debug label '(Jaccard Graph Similarity)' has been completely removed from recommendation cards.
• Verify clean rendering of recommendation reasons and borrow request triggers.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}AI / Graph Recommendations | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}AI Book Recommendations{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     .recs-container {
0009:         display: flex;
0010:         flex-direction: column;
0011:         gap: 28px;
0012:     }
0013:
0014:     /* Page Header */
0015:     .recs-header {
0016:         display: flex;
0017:         align-items: center;
0018:         justify-content: space-between;
0019:         flex-wrap: wrap;
0020:         gap: 16px;
0021:     }
0022:
0023:     .recs-title {
0024:         font-size: 1.6rem;
0025:         font-weight: 800;
0026:         letter-spacing: -0.03em;
0027:         color: var(--text-primary);
0028:         display: flex;
0029:         align-items: center;
0030:         gap: 10px;
0031:     }
0032:
0033:     .recs-sub {
0034:         font-size: 0.9rem;
```

```
0035:         color: var(--text-secondary);
0036:         margin-top: 4px;
0037:         max-width: 720px;
0038:     }
0039:
0040:     /* Collapsible Accordion Toggle Panel */
0041:     .graph-accordion-wrapper {
0042:         margin-top: 36px;
0043:         margin-bottom: 24px;
0044:     }
0045:
0046:     .graph-toggle-btn {
0047:         width: 100%;
0048:         background: var(--bg-surface);
0049:         border: 1px solid var(--border-subtle);
0050:         border-radius: var(--radius-lg);
0051:         padding: 16px 22px;
0052:         display: flex;
0053:         align-items: center;
0054:         justify-content: space-between;
0055:         cursor: pointer;
0056:         transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
0057:         text-align: left;
0058:     }
0059:
0060:     .graph-toggle-btn:hover {
0061:         background: rgba(255, 255, 255, 0.04);
0062:         border-color: rgba(99, 102, 241, 0.45);
0063:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);
0064:     }
0065:
0066:     .graph-toggle-left {
0067:         display: flex;
0068:         align-items: center;
0069:         gap: 12px;
0070:         flex-wrap: wrap;
0071:     }
0072:
0073:     .graph-toggle-icon {
0074:         font-size: 1.15rem;
0075:     }
0076:
0077:     .graph-toggle-title {
0078:         font-size: 0.98rem;
0079:         font-weight: 700;
0080:         color: var(--text-primary);
```

```
0081:         letter-spacing: -0.01em;
0082:     }
0083:
0084:     .graph-toggle-right {
0085:         display: flex;
0086:         align-items: center;
0087:         gap: 12px;
0088:     }
0089:
0090:     .graph-toggle-hint {
0091:         font-size: 0.78rem;
0092:         color: var(--text-muted);
0093:     }
0094:
0095:     .graph-chevron {
0096:         transition: transform 0.25s ease;
0097:         color: var(--text-muted);
0098:     }
0099:
0100:     .graph-collapse-panel {
0101:         animation: fadeInPanel 0.25s ease-out;
0102:     }
0103:
0104:     @keyframes fadeInPanel {
0105:         from { opacity: 0; transform: translateY(-6px); }
0106:         to { opacity: 1; transform: translateY(0); }
0107:     }
0108:
0109:     /* Mini Graph Relationship Canvas Card */
0110:     .graph-card {
0111:         background: var(--bg-surface);
0112:         border: 1px solid var(--border-subtle);
0113:         border-radius: var(--radius-lg);
0114:         overflow: hidden;
0115:         box-shadow: 0 4px 24px rgba(0, 0, 0, 0.3);
0116:         display: flex;
0117:         flex-direction: column;
0118:     }
0119:
0120:     .graph-card-header {
0121:         padding: 16px 24px;
0122:         background: rgba(255, 255, 255, 0.02);
0123:         border-bottom: 1px solid var(--border-subtle);
0124:         display: flex;
0125:         align-items: center;
0126:         justify-content: space-between;
0127:         flex-wrap: wrap;
0128:         gap: 12px;
0129:     }
0130:
0131:     .graph-card-title-group h2 {
0132:         font-size: 1.05rem;
0133:         font-weight: 700;
0134:         color: var(--text-primary);
0135:         display: flex;
0136:         align-items: center;
0137:         gap: 8px;
0138:     }
0139:
0140:     .graph-card-title-group p {
0141:         font-size: 0.78rem;
0142:         color: var(--text-muted);
0143:         margin-top: 2px;
0144:         font-family: var(--font-mono);
0145:     }
0146:
0147:     .graph-card-tools {
0148:         display: flex;
0149:         align-items: center;
0150:         gap: 8px;
0151:     }
0152:
0153:     .graph-btn-sm {
0154:         background: rgba(255, 255, 255, 0.05);
0155:         border: 1px solid var(--border-subtle);
0156:         color: var(--text-secondary);
0157:         padding: 6px 12px;
0158:         border-radius: var(--radius-md);
0159:         font-size: 0.78rem;
0160:         font-weight: 600;
```

```
0161:         cursor: pointer;
0162:         display: inline-flex;
0163:         align-items: center;
0164:         gap: 6px;
0165:         transition: all 0.15s ease;
0166:     }
0167: 
0168:     .graph-btn-sm:hover {
0169:         background: rgba(255, 255, 255, 0.1);
0170:         color: var(--text-primary);
0171:         border-color: var(--border-hover);
0172:     }
0173: 
0174:     /* Canvas Body */
0175:     .mini-canvas-body {
0176:         height: 420px;
0177:         min-height: 400px;
0178:         position: relative;
0179:         background: radial-gradient(circle at 50% 50%, #111827 0%, #0B0F19 100%);
0180:     }
0181: 
0182:     #rec-network-canvas {
0183:         width: 100%;
0184:         height: 100%;
0185:         cursor: grab;
0186:     }
0187: 
0188:     #rec-network-canvas:active {
0189:         cursor: grabbing;
0190:     }
0191: 
0192:     /* Graph Legend Footer */
0193:     .graph-card-footer {
0194:         padding: 12px 24px;
0195:         background: rgba(255, 255, 255, 0.02);
0196:         border-top: 1px solid var(--border-subtle);
0197:         display: flex;
0198:         align-items: center;
0199:         justify-content: space-between;
0200:         flex-wrap: wrap;
0201:         gap: 12px;
0202:         font-size: 0.76rem;
0203:         color: var(--text-secondary);
0204:     }
0205: 
0206:     .legend-list {
0207:         display: flex;
0208:         align-items: center;
0209:         flex-wrap: wrap;
0210:         gap: 16px;
0211:     }
0212: 
0213:     .legend-tag {
0214:         display: inline-flex;
0215:         align-items: center;
0216:         gap: 6px;
0217:     }
0218: 
0219:     .legend-tag-dot {
0220:         width: 9px;
0221:         height: 9px;
0222:         border-radius: 50%;
0223:     }
0224: 
0225:     /* Recommendation Cards Grid */
0226:     .recs-grid {
0227:         display: grid;
0228:         grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
0229:         gap: 20px;
0230:     }
0231: 
0232:     .rec-card {
0233:         background: var(--bg-surface);
0234:         border: 1px solid var(--border-subtle);
0235:         border-radius: var(--radius-lg);
0236:         display: flex;
0237:         flex-direction: column;
0238:         justify-content: space-between;
0239:         padding: 20px;
0240:         position: relative;
```

```
0241:         overflow: hidden;
0242:         transition: transform 0.2s ease, border-color 0.2s ease, box-shadow 0.2s ease;
0243:     }
0244:
0245:     .rec-card:hover {
0246:         transform: translateY(-3px);
0247:         border-color: rgba(245, 158, 11, 0.5);
0248:         box-shadow: 0 8px 24px rgba(245, 158, 11, 0.12);
0249:     }
0250:
0251:     .rec-card-top {
0252:         display: flex;
0253:         align-items: flex-start;
0254:         justify-content: space-between;
0255:         gap: 12px;
0256:         margin-bottom: 12px;
0257:     }
0258:
0259:     .rec-book-title {
0260:         font-size: 1.05rem;
0261:         font-weight: 700;
0262:         color: var(--text-primary);
0263:         line-height: 1.35;
0264:     }
0265:
0266:     .mutual-badge {
0267:         background: rgba(245, 158, 11, 0.15);
0268:         color: #FBBF24;
0269:         border: 1px solid rgba(245, 158, 11, 0.35);
0270:         padding: 4px 9px;
0271:         border-radius: var(--radius-full);
0272:         font-size: 0.72rem;
0273:         font-weight: 700;
0274:         display: inline-flex;
0275:         align-items: center;
0276:         gap: 5px;
0277:         white-space: nowrap;
0278:         flex-shrink: 0;
0279:     }
0280:
0281:     .rec-meta-list {
0282:         display: flex;
0283:         flex-direction: column;
0284:         gap: 6px;
0285:         margin-bottom: 18px;
0286:         font-size: 0.82rem;
0287:         color: var(--text-secondary);
0288:     }
0289:
0290:     .rec-meta-item {
0291:         display: flex;
0292:         align-items: center;
0293:         gap: 8px;
0294:     }
0295:
0296:     .rec-meta-item svg {
0297:         width: 14px;
0298:         height: 14px;
0299:         color: var(--text-muted);
0300:         flex-shrink: 0;
0301:     }
0302:
0303:     .rec-card-footer {
0304:         display: flex;
0305:         align-items: center;
0306:         justify-content: space-between;
0307:         padding-top: 14px;
0308:         border-top: 1px solid var(--border-subtle);
0309:     }
0310:
0311:     /* Jaccard Study Partners Card Grid */
0312:     .partners-section {
0313:         display: flex;
0314:         flex-direction: column;
0315:         gap: 16px;
0316:         margin-top: 10px;
0317:     }
0318:
0319:     .partners-header {
0320:         display: flex;
```

```
0321:         align-items: center;
0322:         justify-content: space-between;
0323:         flex-wrap: wrap;
0324:         gap: 12px;
0325:     }
0326:
0327:     .partners-title {
0328:         font-size: 1.15rem;
0329:         font-weight: 800;
0330:         letter-spacing: -0.02em;
0331:         color: var(--text-primary);
0332:         display: flex;
0333:         align-items: center;
0334:         gap: 9px;
0335:     }
0336:
0337:     .jaccard-formula-badge {
0338:         font-family: var(--font-mono);
0339:         font-size: 0.72rem;
0340:         color: #93C5FD;
0341:         background: rgba(59, 130, 246, 0.12);
0342:         border: 1px solid rgba(59, 130, 246, 0.3);
0343:         padding: 4px 10px;
0344:         border-radius: var(--radius-full);
0345:     }
0346:
0347:     .partners-grid {
0348:         display: grid;
0349:         grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
0350:         gap: 18px;
0351:     }
0352:
0353:     .partner-card {
0354:         background: var(--bg-surface);
0355:         border: 1px solid var(--border-subtle);
0356:         border-radius: var(--radius-lg);
0357:         padding: 18px 20px;
0358:         display: flex;
0359:         flex-direction: column;
0360:         justify-content: space-between;
0361:         gap: 14px;
0362:         position: relative;
0363:         overflow: hidden;
0364:         transition: transform 0.2s ease, border-color 0.2s ease, box-shadow 0.2s ease;
0365:     }
0366:
0367:     .partner-card:hover {
0368:         transform: translateY(-3px);
0369:         border-color: rgba(99, 102, 241, 0.45);
0370:         box-shadow: 0 8px 24px rgba(99, 102, 241, 0.15);
0371:     }
0372:
0373:     .partner-card-top {
0374:         display: flex;
0375:         align-items: center;
0376:         justify-content: space-between;
0377:         gap: 12px;
0378:     }
0379:
0380:     .partner-user-group {
0381:         display: flex;
0382:         align-items: center;
0383:         gap: 10px;
0384:     }
0385:
0386:     .partner-avatar {
0387:         width: 38px;
0388:         height: 38px;
0389:         border-radius: 50%;
0390:         background: linear-gradient(135deg, rgba(99, 102, 241, 0.25), rgba(168, 85, 247, 0.25));
0391:         border: 1px solid rgba(99, 102, 241, 0.4);
0392:         display: flex;
0393:         align-items: center;
0394:         justify-content: center;
0395:         font-weight: 800;
0396:         font-size: 0.95rem;
0397:         color: #C7D2FE;
0398:     }
0399:
0400:     .partner-username {
```

```
0401:         font-weight: 700;
0402:         font-size: 0.95rem;
0403:         color: var(--text-primary);
0404:     }
0405:
0406:     .partner-role-tag {
0407:         font-size: 0.72rem;
0408:         color: var(--text-muted);
0409:     }
0410:
0411:     .jaccard-match-pill {
0412:         background: linear-gradient(135deg, rgba(16, 185, 129, 0.18), rgba(59, 130, 246, 0.18));
0413:         color: #34D399;
0414:         border: 1px solid rgba(16, 185, 129, 0.35);
0415:         font-weight: 800;
0416:         font-size: 0.78rem;
0417:         padding: 4px 10px;
0418:         border-radius: var(--radius-full);
0419:         display: inline-flex;
0420:         align-items: center;
0421:         gap: 5px;
0422:         white-space: nowrap;
0423:     }
0424:
0425:     .partner-shared-box {
0426:         background: var(--bg-surface-soft);
0427:         border: 1px solid rgba(255, 255, 255, 0.04);
0428:         border-radius: var(--radius-md);
0429:         padding: 10px 12px;
0430:         display: flex;
0431:         flex-direction: column;
0432:         gap: 6px;
0433:     }
0434:
0435:     .partner-shared-title {
0436:         font-size: 0.72rem;
0437:         font-weight: 700;
0438:         color: var(--text-muted);
0439:         text-transform: uppercase;
0440:         letter-spacing: 0.05em;
0441:         display: flex;
0442:         align-items: center;
0443:         justify-content: space-between;
0444:     }
0445:
0446:     .partner-books-list {
0447:         display: flex;
0448:         flex-direction: column;
0449:         gap: 4px;
0450:         margin: 0;
0451:         padding: 0;
0452:         list-style: none;
0453:     }
0454:
0455:     .partner-book-item {
0456:         font-size: 0.78rem;
0457:         color: var(--text-secondary);
0458:         display: flex;
0459:         align-items: center;
0460:         gap: 6px;
0461:         white-space: nowrap;
0462:         overflow: hidden;
0463:         text-overflow: ellipsis;
0464:     }
0465:
0466:     .partner-book-item svg {
0467:         flex-shrink: 0;
0468:         color: #818CF8;
0469:     }
0470: </style>
0471: {% endblock %}
0472:
0473: {% block content %}
0474: <div class="recs-container">
0475:     <!-- Header -->
0476:     <div class="recs-header">
0477:         <div>
0478:             <h1 class="recs-title">
0479:                 <svg viewBox="0 0 24 24" width="26" height="26" fill="none" stroke="#F59E0B" stroke-width="2
      | "><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18 21.02 12 17.77 5.82 21.02 7 14.14 2 9.27 8.9
```

```
      | 1 8.26 12 2"></polygon></svg>
0480:                 AI & Graph Collaborative Recommendations
0481:             </h1>
0482:             <p class="recs-sub">
0483:                 {% if session.get('role') in ['Admin', 'Librarian'] or is_admin_view %}
0484:                 Personalized book suggestions based on patron borrowing history, shared peer reading interes
      | ts, and mutual topics across the library collection.
0485:                 {% else %}
0486:                 Personalized book suggestions based on your borrowing history, shared peer reading interests
      | , and mutual topics across the library collection.
0487:                 {% endif %}
0488:             </p>
0489:         </div>
0490:         <div>
0491:             <a href="{{ url_for('graph_visualizer') }}" class="btn btn-secondary">
0492:                 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18" cy=
      | "19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6.51"
      |  x2="8.59" y2="10.49"></line></svg>
0493:                 Open Full Graph Explorer
0494:             </a>
0495:         </div>
0496:     </div>
0497:
0498:     {% if session.get('role') in ['Admin', 'Librarian'] or is_admin_view %}
0499:     <!-- Admin Oversight Banner -->
0500:     <div class="admin-rec-banner" style="display: flex; align-items: center; justify-content: space-between;
      |  gap: 14px; padding: 14px 20px; background: linear-gradient(135deg, rgba(168, 85, 247, 0.15), rgba(59,
      |  130, 246, 0.15)); border: 1px solid rgba(168, 85, 247, 0.35); border-radius: var(--radius-lg); margin
      | -bottom: 20px; box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);">
0501:         <div style="display: flex; align-items: center; gap: 12px;">
0502:             <div style="width: 36px; height: 36px; border-radius: 50%; background: rgba(168, 85, 247, 0.2);
      | display: flex; align-items: center; justify-content: center; color: #C084FC;">
0503:                 <svg viewBox="0 0 24 24" width="20" height="20" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"></path></svg>
0504:             </div>
0505:             <div>
0506:                 <div style="font-weight: 700; font-size: 0.95rem; color: #E9D5FF; letter-spacing: -0.01em;">
0507:                     {{ session.get('role') }} View: Global Patron Collaborative Filtering Network
0508:                 </div>
0509:                 <div style="font-size: 0.8rem; color: var(--text-secondary); margin-top: 2px;">
0510:                     Administrative mode: Displaying global peer borrowing clusters across all student and st
      | aff readers instead of personal loans.
0511:                 </div>
0512:             </div>
0513:         </div>
0514:         <span class="badge" style="background: rgba(168, 85, 247, 0.25); color: #E9D5FF; border: 1px solid r
      | gba(168, 85, 247, 0.4); padding: 5px 12px; border-radius: var(--radius-full); font-size: 0.75rem; font
      | -weight: 600;">
0515:             {{ session.get('role') }} Oversight
0516:         </span>
0517:     </div>
0518:     {% endif %}
0519:
0520:     <!-- Recommendations Cards (Top of Page for Production UX) -->
0521:     {% if recommendations %}
0522:     <div>
0523:         <h2 class="section-title" style="margin-top: 5px;">
0524:             {% if session.get('role') in ['Admin', 'Librarian'] or is_admin_view %}
0525:             Recommended Books (Patron Cohort)
0526:             {% else %}
0527:             Recommended Books For You
0528:             {% endif %}
0529:         </h2>
0530:         <div class="recs-grid">
0531:             {% for book in recommendations %}
0532:             <div class="rec-card" id="book-card-{{ book.book_id }}">
0533:                 <div>
0534:                     <div class="rec-card-top">
0535:                         <h3 class="rec-book-title" title="{{ book.title }}">{{ book.title }}</h3>
0536:                         <span class="mutual-badge" title="Number of peer borrowers who share reading taste">
0537:                             <svg viewBox="0 0 24 24" width="12" height="12" fill="none" stroke="currentColor
      | " stroke-width="2"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r=
      | "4"></circle><path d="M23 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13a4 4 0 0 1 0 7.75"></path></
      | svg>
0538:                             {% if book.badge_label %}
0539:                                 {{ book.badge_label }}
0540:                             {% elif book.mutual_borrowers is not none and (book.mutual_borrowers > 0 or sess
      | ion.get('role') != 'Staff') %}
0541:                                 {{ book.mutual_borrowers }} {{ 'Reader' if book.mutual_borrowers == 1 else '
```

```
      | Readers' }}
0542:                             {% else %}
0543:                                 Curriculum Match
0544:                             {% endif %}
0545:                         </span>
0546:                     </div>
0547:
0548:                     <div class="rec-meta-list">
0549:                         <div class="rec-meta-item">
0550:                             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><pat
      | h d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path><circle cx="12" cy="7" r="4"></circle></svg>
0551:                             <span>{{ book.authors or 'Unknown Author' }}</span>
0552:                         </div>
0553:                         <div class="rec-meta-item">
0554:                             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><pat
      | h d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0555:                             <span>{{ book.category_name or 'General Category' }}</span>
0556:                         </div>
0557:                         <div class="rec-meta-item">
0558:                             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rec
      | t x="3" y="4" width="18" height="16" rx="2"></rect><line x1="7" y1="8" x2="7" y2="16"></line><line x1=
      | "11" y1="8" x2="11" y2="16"></line><line x1="15" y1="8" x2="15" y2="12"></line></svg>
0559:                             <span>ISBN: {{ book.isbn }}</span>
0560:                         </div>
0561:                     </div>
0562:                 </div>
0563:
0564:                 <div class="rec-card-footer">
0565:                     {% if book.available_copies > 0 %}
0566:                         <span class="badge badge-success">
0567:                             {{ book.available_copies }} Available
0568:                         </span>
0569:                         {% if session.get('role') in ['Student', 'Staff'] %}
0570:                         <form method="POST" action="{{ url_for('request_loan', book_id=book.book_id) }}" sty
      | le="margin: 0;">
0571:                             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
0572:                             <button type="submit" class="btn btn-primary" style="padding: 6px 14px; font-siz
      | e: 0.8rem;">
0573:                                 Request Loan
0574:                             </button>
0575:                         </form>
0576:                         {% endif %}
0577:                     {% else %}
0578:                         <span class="badge badge-warning">
0579:                             All Copies Checked Out
0580:                         </span>
0581:                     {% endif %}
0582:                 </div>
0583:             </div>
0584:             {% endfor %}
0585:         </div>
0586:     </div>
0587:
0588:     {% else %}
0589:     <div class="empty-state" style="background: var(--bg-surface); border: 1px solid var(--border-subtle); b
      | order-radius: var(--radius-lg); padding: 48px 24px;">
0590:         <svg viewBox="0 0 24 24"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></p
      | ath></svg>
0591:         <h3 style="font-size: 1.15rem; font-weight: 700; color: var(--text-primary); margin-bottom: 6px;">No
      |  Loan History Yet</h3>
0592:         <p style="font-size: 0.85rem; color: var(--text-muted); max-width: 480px; margin: 0 auto 20px auto;"
      | >
0593:             Borrow books from the catalog to activate personalized graph collaborative filtering tailored to
      |  your reading profile!
0594:         </p>
0595:         <a href="{{ url_for('books') }}" class="btn btn-primary">
0596:             Browse Catalog
0597:         </a>
0598:     </div>
0599:     {% endif %}
0600:
0601:     <!-- Role-Scoped Jaccard Study Partners Section -->
0602:     {% if session.get('role') != 'Staff' and study_partners %}
0603:     <div class="partners-section">
0604:         <div class="partners-header">
0605:             <h2 class="partners-title">
0606:                 <svg viewBox="0 0 24 24" width="22" height="22" fill="none" stroke="#6366F1" stroke-width="2
      | "><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></circle><pat
      | h d="M23 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
0607:                 {% if session.get('role') in ['Admin', 'Librarian'] %}
```

```
                    Student Cohort Study Pairs
                {% else %}
                    Top Study Partners
                {% endif %}
            </h2>
            <span class="jaccard-formula-badge" title="Jaccard Index = |A ∩ B| / |A ∪ B|">
                Jaccard Index: |A ∩ B| / |A ∪ B|
            </span>
        </div>

        <div class="partners-grid">
            {% for partner in study_partners %}
            <div class="partner-card">
                <div>
                    <div class="partner-card-top">
                        <div class="partner-user-group">
                            <div class="partner-avatar">
                                {% if '&' in partner.peer_name %}
                                    ■
                                {% else %}
                                    {{ partner.peer_name[:2].upper() }}
                                {% endif %}
                            </div>
                            <div>
                                <div class="partner-username">{{ partner.peer_name }}</div>
                                <div class="partner-role-tag">
                                    {% if session.get('role') in ['Admin', 'Librarian'] %}
                                        Student Pair
                                    {% else %}
                                        Matched Peer Student
                                    {% endif %}
                                </div>
                            </div>
                        </div>
                        <div class="jaccard-match-pill" title="Jaccard Reading Similarity Score">
                            ■ {{ partner.jaccard_pct }}% Match
                        </div>
                    </div>

                    <div class="partner-shared-box" style="margin-top: 14px;">
                        <div class="partner-shared-title">
                            <span>Shared Reading Overlap</span>
                            <span style="color: #38BDF8;">{{ partner.shared_count }} Book{% if partner.share
    | d_count != 1 %}s{% endif %}</span>
                        </div>
                        <ul class="partner-books-list">
                            {% for title in partner.shared_titles %}
                            <li class="partner-book-item" title="{{ title }}">
                                <svg viewBox="0 0 24 24" width="12" height="12" fill="none" stroke="currentC
    | olor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path>
    | </svg>
                                <span>{{ title }}</span>
                            </li>
                            {% endfor %}
                        </ul>
                    </div>
                </div>
            </div>
            {% endfor %}
        </div>
    </div>
    {% endif %}

    <!-- Collapsible Accordion: Multi-Hop Recommendation Path Vis.js Graph -->
    <div class="graph-accordion-wrapper">
        <button type="button" class="graph-toggle-btn" id="recGraphToggleBtn" onclick="toggleRecGraphAccordi
    | on()" aria-expanded="false">
            <div class="graph-toggle-left">
                <span class="graph-toggle-icon">■</span>
                <span class="graph-toggle-title">Explain Recommendations (View Neo4j Traversal Graph)</span>
                <span class="badge" style="background: rgba(99, 102, 241, 0.2); color: #A5B4FC; font-size: 0
    | .72rem; padding: 2px 8px; border-radius: var(--radius-full);">4-Column Hierarchy</span>
            </div>
            <div class="graph-toggle-right">
                <span class="graph-toggle-hint">Click to expand graph proof</span>
                <svg class="graph-chevron" id="recGraphChevron" viewBox="0 0 24 24" width="18" height="18" f
    | ill="none" stroke="currentColor" stroke-width="2.5"><polyline points="6 9 12 15 18 9"></polyline></svg
    | >
            </div>
        </button>
```

```
0681:
0682:         <div class="graph-collapse-panel" id="recGraphPanel" style="display: none;">
0683:             <div class="graph-card" style="margin-top: 14px;">
0684:                 <div class="graph-card-header">
0685:                     <div class="graph-card-title-group">
0686:                         <h2>
0687:                             <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor
      | " stroke-width="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle
      |  cx="18" cy="19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.4
      | 1" y1="6.51" x2="8.59" y2="10.49"></line></svg>
0688:                             Multi-Hop Recommendation Path
0689:                         </h2>
0690:                         {% if session.get('role') in ['Admin', 'Librarian'] or is_admin_view %}
0691:                         <p>(Active Patron) &rarr; [:borrowed] &rarr; (Shared Book / Category) &rarr; [:share
      | d] &rarr; (Peer Readers) &rarr; [:recommends] &rarr; (Recommended Book)</p>
0692:                         {% elif session.get('role') == 'Staff' %}
0693:                         <p>(You) &rarr; [:borrowed] &rarr; (Borrowed Book) &rarr; [:shared] &rarr; (Shared C
      | ategory/Author/Prerequisite) &rarr; [:recommends] &rarr; (Recommended Book)</p>
0694:                         {% else %}
0695:                         <p>(You) &rarr; [:borrowed] &rarr; (Shared Book / Category) &rarr; [:shared] &rarr;
      | (Peer Readers) &rarr; [:recommends] &rarr; (Recommended Book)</p>
0696:                         {% endif %}
0697:                     </div>
0698:                     <div class="graph-card-tools">
0699:                         <button class="graph-btn-sm" onclick="fitRecGraph()">
0700:                             <svg viewBox="0 0 24 24" width="13" height="13" stroke="currentColor" fill="none
      | " stroke-width="2"><path d="M15 3h6v6"></path><path d="M9 21H3v-6"></path><path d="M21 3l-7 7"></path>
      | <path d="M3 21l7-7"></path></svg>
0701:                             Reset View
0702:                         </button>
0703:                     </div>
0704:                 </div>
0705:
0706:                 <div class="mini-canvas-body">
0707:                     <div id="rec-network-canvas"></div>
0708:                 </div>
0709:
0710:                 <div class="graph-card-footer">
0711:                     <div class="legend-list">
0712:                         <span class="legend-tag"><span class="legend-tag-dot" style="background: #3B82F6;"><
      | /span> {% if session.get('role') in ['Admin', 'Librarian'] or is_admin_view %}■ Active Patron{% else %
      | }■ You{% endif %}</span>
0713:                         <span class="legend-tag"><span class="legend-tag-dot" style="background: #10B981;"><
      | /span> ■ {% if session.get('role') == 'Staff' %}Borrowed Book{% else %}Read Book{% endif %}</span>
0714:                         <span class="legend-tag"><span class="legend-tag-dot" style="background: #F59E0B;"><
      | /span> ■ Shared Category</span>
0715:                         <span class="legend-tag"><span class="legend-tag-dot" style="background: #8B5CF6;"><
      | /span> {% if session.get('role') == 'Staff' %}■ Prerequisite / Author{% else %}■ Peer Reader{% endif %
      | }</span>
0716:                         <span class="legend-tag"><span class="legend-tag-dot" style="background: #EAB308;"><
      | /span> ■ Recommended Book</span>
0717:                     </div>
0718:                     <div>
0719:                         <span style="font-family: var(--font-mono); color: var(--text-muted);">4-Column Mult
      | i-Hop Flow</span>
0720:                     </div>
0721:                 </div>
0722:             </div>
0723:         </div>
0724:     </div>
0725: </div>
0726: {% endblock %}
0727:
0728: {% block extra_js %}
0729: <!-- Vis.js Network CDN -->
0730: <script src="https://unpkg.com/vis-network/standalone/umd/vis-network.min.js"></script>
0731:
0732: <script>
0733:     let recNetwork = null;
0734:     let recGraphLoaded = false;
0735:
0736:     function toggleRecGraphAccordion() {
0737:         const panel = document.getElementById("recGraphPanel");
0738:         const chevron = document.getElementById("recGraphChevron");
0739:         const btn = document.getElementById("recGraphToggleBtn");
0740:
0741:         if (panel.style.display === "none" || panel.style.display === "") {
0742:             panel.style.display = "block";
0743:             btn.setAttribute("aria-expanded", "true");
0744:             chevron.style.transform = "rotate(180deg)";
```

```
            if (!recGraphLoaded) {
                loadRecGraph();
            } else if (recNetwork) {
                setTimeout(() => {
                    recNetwork.redraw();
                    recNetwork.fit({ padding: 35 });
                }, 80);
            }
        } else {
            panel.style.display = "none";
            btn.setAttribute("aria-expanded", "false");
            chevron.style.transform = "rotate(0deg)";
        }
    }

    function loadRecGraph() {
        fetch("/api/graph_data?view=recommendations")
            .then(res => res.json())
            .then(data => {
                const nodes = data.nodes || [];
                const edges = data.edges || [];
                renderRecNetwork(nodes, edges);
            })
            .catch(err => {
                console.error("Error loading recommendation graph:", err);
            });
    }

    function renderRecNetwork(nodesList, edgesList) {
        const container = document.getElementById("rec-network-canvas");
        if (!container) return;

        const data = {
            nodes: new vis.DataSet(nodesList),
            edges: new vis.DataSet(edgesList)
        };

        const options = {
            layout: {
                hierarchical: {
                    enabled: true,
                    direction: 'LR',
                    sortMethod: 'directed',
                    levelSeparation: 230,
                    nodeSpacing: 110,
                    treeSpacing: 130
                }
            },
            physics: {
                enabled: false
            },
            nodes: {
                shape: 'box',
                margin: 10,
                widthConstraint: { maximum: 140 },
                borderWidth: 2,
                font: {
                    multi: true,
                    size: 13,
                    face: 'Inter, sans-serif',
                    color: '#FFFFFF'
                },
                shadow: { enabled: true, color: 'rgba(0,0,0,0.5)', size: 6 }
            },
            groups: {
                CurrentUser: {
                    color: {
                        background: '#1E3A8A',
                        border: '#60A5FA',
                        highlight: { background: '#2563EB', border: '#93C5FD' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#DBEAFE' }
                },
                SharedBook: {
                    color: {
                        background: '#064E3B',
                        border: '#34D399',
                        highlight: { background: '#059669', border: '#6EE7B7' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#D1FAE5' }
```

```
                },
                Category: {
                    color: {
                        background: '#78350F',
                        border: '#F59E0B',
                        highlight: { background: '#B45309', border: '#FCD34D' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#FEF3C7' }
                },
                PeerUser: {
                    color: {
                        background: '#4C1D95',
                        border: '#A78BFA',
                        highlight: { background: '#6D28D9', border: '#C4B5FD' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#EDE9FE' }
                },
                RecommendedBook: {
                    color: {
                        background: '#713F12',
                        border: '#FBBF24',
                        highlight: { background: '#B45309', border: '#FDE047' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#FEF08A' }
                },
                Author: {
                    color: {
                        background: '#1E293B',
                        border: '#94A3B8',
                        highlight: { background: '#334155', border: '#CBD5E1' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#F1F5F9' }
                },
                Prerequisite: {
                    color: {
                        background: '#312E81',
                        border: '#818CF8',
                        highlight: { background: '#3730A3', border: '#A5B4FC' }
                    },
                    font: { multi: true, size: 13, face: 'Inter, sans-serif', color: '#E0E7FF' }
                }
            },
            edges: {
                arrows: { to: { enabled: true, scaleFactor: 0.8 } },
                color: { color: 'rgba(148, 163, 184, 0.45)', highlight: '#60A5FA' },
                font: {
                    size: 10,
                    align: 'middle',
                    background: '#0f172a',
                    color: '#94A3B8',
                    strokeWidth: 0
                },
                smooth: {
                    type: 'cubicBezier',
                    forceDirection: 'horizontal',
                    roundness: 0.25
                }
            },
            interaction: {
                hover: true,
                zoomView: true,
                dragView: true
            }
        };

        if (recNetwork) {
            recNetwork.destroy();
        }

        recNetwork = new vis.Network(container, data, options);

        // Auto-fit after drawing
        recNetwork.once("afterDrawing", function() {
            recNetwork.fit({ padding: 35 });
        });

        // Highlight card when clicking recommended book node
        recNetwork.on("click", function(params) {
            if (params.nodes.length > 0) {
                const nodeId = params.nodes[0];
```

```
                const clickedNode = nodesList.find(n => n.id === nodeId);
                if (clickedNode && clickedNode.properties && clickedNode.properties.book_id) {
                    const card = document.getElementById("book-card-" + clickedNode.properties.book_id);
                    if (card) {
                        card.scrollIntoView({ behavior: 'smooth', block: 'center' });
                        card.style.borderColor = '#F59E0B';
                        card.style.boxShadow = '0 0 20px rgba(245, 158, 11, 0.4)';
                        setTimeout(() => {
                            card.style.borderColor = '';
                            card.style.boxShadow = '';
                        }, 2500);
                    }
                }
            }
        });
    }

    function fitRecGraph() {
        if (recNetwork) {
            recNetwork.fit({ animation: { duration: 500, easingFunction: 'easeInOutQuad' }, padding: 35 });
        }
    }
</script>
{% endblock %}
```

MODULE SOURCE: templates/book_bridge.html

Lines: 671 | **Lang:** HTML5 / Jinja2 | **Category:** Peer Sharing Network

Core Role & Purpose: Peer-to-peer student and faculty book sharing and community borrowing exchange.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify listing submission form and peer borrow request workflow.
- Verify peer contact detail visibility and request status tracking.

```
{% extends "base.html" %}

{% block title %}Book-to-Book Bridge (Shortest Path) | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Book-to-Book Bridge{% endblock %}

{% block extra_css %}
<style>
    .bridge-container {
        display: flex;
        flex-direction: column;
        gap: 20px;
        padding-bottom: 30px;
    }

    /* Header & Control Card */
    .bridge-header-card {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 24px;
        display: flex;
        flex-direction: column;
        gap: 18px;
        box-shadow: 0 4px 24px rgba(0, 0, 0, 0.3);
    }

    .bridge-title-row {
        display: flex;
        align-items: center;
        justify-content: space-between;
        flex-wrap: wrap;
        gap: 16px;
    }

    .bridge-title-group h1 {
        font-size: 1.45rem;
        font-weight: 800;
        letter-spacing: -0.02em;
        color: var(--text-primary);
        display: flex;
        align-items: center;
        gap: 10px;
    }

    .bridge-title-group p {
        font-size: 0.85rem;
        color: var(--text-secondary);
        margin-top: 4px;
        max-width: 680px;
        line-height: 1.5;
    }

    /* Search & Selection Form */
    .bridge-form-row {
        display: grid;
        grid-template-columns: 1fr auto 1fr auto;
        align-items: flex-end;
        gap: 16px;
        padding-top: 12px;
        border-top: 1px solid var(--border-subtle);
    }

    @media (max-width: 900px) {
        .bridge-form-row {
            grid-template-columns: 1fr;
        }
    }

```

```
0069:     .select-group {
0070:         display: flex;
0071:         flex-direction: column;
0072:         gap: 6px;
0073:     }
0074:
0075:     .select-label {
0076:         font-size: 0.8rem;
0077:         font-weight: 700;
0078:         text-transform: uppercase;
0079:         letter-spacing: 0.05em;
0080:         display: flex;
```

```
0081:         align-items: center;
0082:         gap: 6px;
0083:     }
0084:
0085:     .select-label.origin { color: #60A5FA; }
0086:     .select-label.dest   { color: #34D399; }
0087:
0088:     .bridge-select {
0089:         background: rgba(17, 24, 39, 0.8);
0090:         border: 1px solid var(--border-subtle);
0091:         border-radius: var(--radius-md);
0092:         color: var(--text-primary);
0093:         padding: 10px 14px;
0094:         font-size: 0.85rem;
0095:         font-family: var(--font-sans);
0096:         outline: none;
0097:         transition: border-color 0.2s, box-shadow 0.2s;
0098:         width: 100%;
0099:     }
0100:
0101:     .bridge-select:focus {
0102:         border-color: var(--accent-primary);
0103:         box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.25);
0104:     }
0105:
0106:     .swap-btn {
0107:         background: rgba(255, 255, 255, 0.04);
0108:         border: 1px solid var(--border-subtle);
0109:         border-radius: var(--radius-md);
0110:         color: var(--text-muted);
0111:         width: 44px;
0112:         height: 42px;
0113:         display: flex;
0114:         align-items: center;
0115:         justify-content: center;
0116:         cursor: pointer;
0117:         transition: all 0.2s;
0118:         margin-bottom: 2px;
0119:     }
0120:
0121:     .swap-btn:hover {
0122:         color: #FFFFFF;
0123:         background: rgba(255, 255, 255, 0.08);
0124:         border-color: var(--accent-primary);
0125:         transform: rotate(180deg);
0126:     }
0127:
0128:     .btn-find-path {
0129:         background: linear-gradient(135deg, #2563EB, #1D4ED8);
0130:         color: #FFFFFF;
0131:         border: 1px solid rgba(59, 130, 246, 0.4);
0132:         padding: 11px 22px;
0133:         border-radius: var(--radius-md);
0134:         font-weight: 700;
0135:         font-size: 0.85rem;
0136:         cursor: pointer;
0137:         display: inline-flex;
0138:         align-items: center;
0139:         gap: 8px;
0140:         transition: all 0.2s;
0141:         box-shadow: 0 2px 10px rgba(37, 99, 235, 0.3);
0142:         margin-bottom: 2px;
0143:         height: 42px;
0144:     }
0145:
0146:     .btn-find-path:hover {
0147:         background: linear-gradient(135deg, #3B82F6, #2563EB);
0148:         box-shadow: 0 4px 16px rgba(59, 130, 246, 0.45);
0149:         transform: translateY(-1px);
0150:     }
0151:
0152:     /* Preset Shortcuts */
0153:     .presets-row {
0154:         display: flex;
0155:         align-items: center;
0156:         gap: 10px;
0157:         flex-wrap: wrap;
0158:         font-size: 0.78rem;
0159:         color: var(--text-muted);
0160:     }
```

```
0161:
0162:     .preset-pill {
0163:         background: rgba(255, 255, 255, 0.03);
0164:         border: 1px solid var(--border-subtle);
0165:         border-radius: var(--radius-full);
0166:         padding: 4px 12px;
0167:         color: var(--text-secondary);
0168:         cursor: pointer;
0169:         transition: all 0.15s;
0170:         text-decoration: none;
0171:     }
0172:
0173:     .preset-pill:hover {
0174:         background: rgba(59, 130, 246, 0.15);
0175:         border-color: rgba(59, 130, 246, 0.4);
0176:         color: #93C5FD;
0177:     }
0178:
0179:     /* Result Metric Banner */
0180:     .result-banner {
0181:         background: linear-gradient(135deg, rgba(16, 185, 129, 0.12), rgba(59, 130, 246, 0.12));
0182:         border: 1px solid rgba(16, 185, 129, 0.35);
0183:         border-radius: var(--radius-lg);
0184:         padding: 16px 22px;
0185:         display: flex;
0186:         align-items: center;
0187:         justify-content: space-between;
0188:         flex-wrap: wrap;
0189:         gap: 16px;
0190:     }
0191:
0192:     .result-metric-group {
0193:         display: flex;
0194:         align-items: center;
0195:         gap: 16px;
0196:     }
0197:
0198:     .degree-badge {
0199:         background: #10B981;
0200:         color: #FFFFFF;
0201:         font-size: 1.15rem;
0202:         font-weight: 800;
0203:         padding: 6px 14px;
0204:         border-radius: var(--radius-full);
0205:         box-shadow: 0 2px 10px rgba(16, 185, 129, 0.35);
0206:         display: inline-flex;
0207:         align-items: center;
0208:         gap: 6px;
0209:     }
0210:
0211:     .result-text-main {
0212:         font-size: 0.95rem;
0213:         font-weight: 700;
0214:         color: var(--text-primary);
0215:     }
0216:
0217:     .result-text-sub {
0218:         font-size: 0.8rem;
0219:         color: var(--text-secondary);
0220:         margin-top: 2px;
0221:     }
0222:
0223:     /* Visual Graph Canvas */
0224:     .bridge-canvas-card {
0225:         background: var(--bg-surface);
0226:         border: 1px solid var(--border-subtle);
0227:         border-radius: var(--radius-lg);
0228:         overflow: hidden;
0229:         position: relative;
0230:         box-shadow: 0 4px 24px rgba(0, 0, 0, 0.35);
0231:     }
0232:
0233:     .canvas-header {
0234:         padding: 14px 20px;
0235:         border-bottom: 1px solid var(--border-subtle);
0236:         display: flex;
0237:         align-items: center;
0238:         justify-content: space-between;
0239:         background: rgba(17, 24, 39, 0.6);
0240:     }
```

```
0241:
0242:     .canvas-title {
0243:         font-size: 0.88rem;
0244:         font-weight: 700;
0245:         color: var(--text-primary);
0246:         display: flex;
0247:         align-items: center;
0248:         gap: 8px;
0249:     }
0250:
0251:     #bridge-network-canvas {
0252:         height: 480px;
0253:         width: 100%;
0254:         background: radial-gradient(circle at center, rgba(30, 41, 59, 0.4) 0%, rgba(11, 15, 25, 0.95) 100%)
      | ;
0255:     }
0256:
0257:     .canvas-footer {
0258:         padding: 12px 20px;
0259:         border-top: 1px solid var(--border-subtle);
0260:         display: flex;
0261:         align-items: center;
0262:         justify-content: space-between;
0263:         flex-wrap: wrap;
0264:         gap: 12px;
0265:         font-size: 0.74rem;
0266:         color: var(--text-muted);
0267:         background: rgba(17, 24, 39, 0.6);
0268:     }
0269:
0270:     .legend-list {
0271:         display: flex;
0272:         align-items: center;
0273:         gap: 14px;
0274:         flex-wrap: wrap;
0275:     }
0276:
0277:     .legend-tag {
0278:         display: inline-flex;
0279:         align-items: center;
0280:         gap: 6px;
0281:     }
0282:
0283:     .legend-tag-dot {
0284:         width: 9px;
0285:         height: 9px;
0286:         border-radius: 50%;
0287:     }
0288:
0289:     /* Path Step Breadcrumbs */
0290:     .steps-card {
0291:         background: var(--bg-surface);
0292:         border: 1px solid var(--border-subtle);
0293:         border-radius: var(--radius-lg);
0294:         padding: 20px 24px;
0295:         display: flex;
0296:         flex-direction: column;
0297:         gap: 14px;
0298:     }
0299:
0300:     .steps-card h3 {
0301:         font-size: 0.95rem;
0302:         font-weight: 700;
0303:         color: var(--text-primary);
0304:         display: flex;
0305:         align-items: center;
0306:         gap: 8px;
0307:     }
0308:
0309:     .steps-timeline {
0310:         display: flex;
0311:         align-items: center;
0312:         gap: 12px;
0313:         overflow-x: auto;
0314:         padding-bottom: 8px;
0315:     }
0316:
0317:     .step-item {
0318:         background: rgba(255, 255, 255, 0.03);
0319:         border: 1px solid var(--border-subtle);
```

```
0320:         border-radius: var(--radius-md);
0321:         padding: 10px 14px;
0322:         min-width: 190px;
0323:         flex-shrink: 0;
0324:         display: flex;
0325:         flex-direction: column;
0326:         gap: 4px;
0327:         position: relative;
0328:     }
0329:
0330:     .step-item.origin       { border-color: rgba(59, 130, 246, 0.5); background: rgba(59, 130, 246, 0.08); }
0331:     .step-item.intermediate { border-color: rgba(100, 116, 139, 0.5); background: rgba(100, 116, 139, 0.12);
      |  }
0332:     .step-item.dest         { border-color: rgba(16, 185, 129, 0.5); background: rgba(16, 185, 129, 0.08); }
0333:
0334:     .step-number {
0335:         font-size: 0.68rem;
0336:         font-weight: 800;
0337:         text-transform: uppercase;
0338:         letter-spacing: 0.05em;
0339:         color: var(--text-muted);
0340:     }
0341:
0342:     .step-name {
0343:         font-size: 0.82rem;
0344:         font-weight: 700;
0345:         color: var(--text-primary);
0346:         white-space: nowrap;
0347:         overflow: hidden;
0348:         text-overflow: ellipsis;
0349:     }
0350:
0351:     .step-arrow {
0352:         color: var(--accent-primary);
0353:         font-weight: 800;
0354:         font-size: 1.1rem;
0355:         flex-shrink: 0;
0356:     }
0357:
0358:
0359: </style>
0360: {% endblock %}
0361:
0362: {% block content %}
0363: <div class="bridge-container">
0364:     <!-- Top Header & Selection Card -->
0365:     <div class="bridge-header-card">
0366:         <div class="bridge-title-row">
0367:             <div class="bridge-title-group">
0368:                 <h1>
0369:                     <svg viewBox="0 0 24 24" width="24" height="24" fill="none" stroke="#34D399" stroke-widt
      | h="2"><path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6"></path><polyline points="15 3
      |  21 3 21 9"></polyline><line x1="10" y1="14" x2="21" y2="3"></line></svg>
0370:                     The "Book-to-Book Bridge" Shortest Path Traversal
0371:                 </h1>
0372:                 <p style="font-size: 0.85rem; color: var(--text-secondary); margin-top: 4px;">
0373:                     Discover the direct connection between any two books through shared authors, categories,
      |  editions, and circulation history.
0374:                 </p>
0375:             </div>
0376:             <div>
0377:                 <a href="{{ url_for('graph_visualizer') }}" class="btn btn-secondary">
0378:                     <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18"
      |  cy="19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6
      | .51" x2="8.59" y2="10.49"></line></svg>
0379:                     Full Knowledge Graph
0380:                 </a>
0381:             </div>
0382:         </div>
0383:
0384:         <!-- Form for Book A & Book B -->
0385:         <form method="GET" action="{{ url_for('book_bridge') }}" id="bridgeForm" class="bridge-form-row">
0386:             <!-- Book A -->
0387:             <div class="select-group">
0388:                 <label class="select-label origin" for="bookASelect">
0389:                     <span>■ Origin Book (A)</span>
0390:                 </label>
0391:                 <select name="book_a" id="bookASelect" class="bridge-select" required>
0392:                     {% for b in all_books %}
```

```
                        <option value="{{ b.book_id }}" {% if b.book_id == book_a_id %}selected{% endif %}>
                            #{{ b.book_id }}: {{ b.title }} ({{ b.author_name or 'Author' }})
                        </option>
                    {% endfor %}
                </select>
            </div>

            <!-- Swap Button -->
            <div>
                <button type="button" class="swap-btn" title="Swap Origin and Destination" onclick="swapBook
    | s()">
                    <svg viewBox="0 0 24 24" width="18" height="18" stroke="currentColor" fill="none" stroke
    | -width="2"><polyline points="17 1 21 5 17 9"></polyline><path d="M3 11V9a4 4 0 0 1 4-4h14"></path><pol
    | yline points="7 23 3 19 7 15"></polyline><path d="M21 13v2a4 4 0 0 1-4 4H3"></path></svg>
                </button>
            </div>

            <!-- Book B -->
            <div class="select-group">
                <label class="select-label dest" for="bookBSelect">
                    <span>■ Destination Book (B)</span>
                </label>
                <select name="book_b" id="bookBSelect" class="bridge-select" required>
                    {% for b in all_books %}
                        <option value="{{ b.book_id }}" {% if b.book_id == book_b_id %}selected{% endif %}>
                            #{{ b.book_id }}: {{ b.title }} ({{ b.author_name or 'Author' }})
                        </option>
                    {% endfor %}
                </select>
            </div>

            <!-- Submit Button -->
            <div>
                <button type="submit" class="btn-find-path">
                    <svg viewBox="0 0 24 24" width="16" height="16" fill="none" stroke="currentColor" stroke
    | -width="2"><circle cx="11" cy="11" r="8"></circle><line x1="21" y1="21" x2="16.65" y2="16.65"></line><
    | /svg>
                    Calculate Bridge
                </button>
            </div>
        </form>

        <!-- Presets Row -->
        <div class="presets-row">
            <span>Quick Presets:</span>
            <a href="{{ url_for('book_bridge', book_a=1, book_b=5) }}" class="preset-pill">Database Category
    |  Bridge (2 Hops)</a>
            <a href="{{ url_for('book_bridge', book_a=2, book_b=4) }}" class="preset-pill">CS & AI Category
    | Bridge (2 Hops)</a>
            <a href="{{ url_for('book_bridge', book_a=1, book_b=4) }}" class="preset-pill">Cross-Domain Read
    | er Circulation Bridge (6 Hops)</a>
        </div>
    </div>

    <!-- Error or Notification -->
    {% if error_msg %}
    <div class="alert alert-danger" style="padding: 14px 20px; border-radius: var(--radius-md); font-size: 0
    | .85rem;">
        <strong>Notice:</strong> {{ error_msg }}
    </div>
    {% endif %}

    {% if path_found %}
    <!-- Degree & Result Banner -->
    <div class="result-banner">
        <div class="result-metric-group">
            <div class="degree-badge">
                <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor" stroke-wid
    | th="2.5"><polyline points="20 6 9 17 4 12"></polyline></svg>
                {{ degrees }} {% if degrees == 1 %}Degree{% else %}Degrees{% endif %}
            </div>
            <div>
                <div class="result-text-main">
                    Connected across {{ path_nodes|length }} nodes via {{ degrees }} relationship hops
                </div>
            </div>
        </div>
        <div style="font-family: var(--font-mono); font-size: 0.8rem; color: #6EE7B7;">
            MATCH p = shortestPath((b1:Book)-[:REL*]-(b2:Book))
        </div>
```

```
0463:     </div>
0464:
0465:     <!-- Step Timeline Breadcrumbs -->
0466:     <div class="steps-card">
0467:         <h3>
0468:             <svg viewBox="0 0 24 24" width="16" height="16" fill="none" stroke="currentColor" stroke-width="
      | 2"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyline></svg>
0469:             Sequential Graph Traversal Narrative
0470:         </h3>
0471:         <div class="steps-timeline">
0472:             {% for step in path_steps %}
0473:                 <div class="step-item {% if loop.first %}origin{% elif loop.last %}dest{% elif step.type ==
      | 'Intermediate Book' %}intermediate{% endif %}">
0474:                     <div class="step-number">Node {{ step.step }} &bull; {{ step.type }}</div>
0475:                     <div class="step-name" title="{{ step.name }}">{{ step.name }}</div>
0476:                 </div>
0477:                 {% if not loop.last %}
0478:                     <div class="step-arrow">&rarr;</div>
0479:                 {% endif %}
0480:             {% endfor %}
0481:         </div>
0482:     </div>
0483:
0484:     <!-- Interactive Vis.js Visual Network Canvas -->
0485:     <div class="bridge-canvas-card">
0486:         <div class="canvas-header">
0487:             <div class="canvas-title">
0488:                 <svg viewBox="0 0 24 24" width="16" height="16" stroke="#38BDF8" fill="none" stroke-width="2
      | "><circle cx="12" cy="12" r="10"></circle><line x1="2" y1="12" x2="22" y2="12"></line><path d="M12 2a1
      | 5.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-4 10 15.3 15.3 0 0 1-4-10 15.3 15.3 0 0 1 4-10z"></path></svg>
0489:                 Interactive Shortest Path Subgraph Visualization
0490:             </div>
0491:             <div>
0492:                 <button class="graph-btn-sm" onclick="fitBridgeGraph()" style="background: rgba(255,255,255,
      | 0.06); border: 1px solid var(--border-subtle); color: var(--text-secondary); padding: 4px 10px; border
      | -radius: var(--radius-sm); font-size: 0.75rem; cursor: pointer;">
0493:                     Reset View
0494:                 </button>
0495:             </div>
0496:         </div>
0497:
0498:         <div id="bridge-network-canvas"></div>
0499:
0500:         <div class="canvas-footer">
0501:             <div class="legend-list">
0502:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #2563EB;"></span> ■
      |  Origin Book</span>
0503:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #64748B;"></span> ■
      |  Intermediate Book</span>
0504:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #059669;"></span> ■
      |  Destination Book</span>
0505:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #F59E0B;"></span> C
      | ategory</span>
0506:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #8B5CF6;"></span> A
      | uthor</span>
0507:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #14B8A6;"></span> C
      | opy</span>
0508:                 <span class="legend-tag"><span class="legend-tag-dot" style="background: #06B6D4;"></span> L
      | oan / Reader</span>
0509:             </div>
0510:             <div>
0511:                 <span>Shortest Path Graph Engine</span>
0512:             </div>
0513:         </div>
0514:     </div>
0515:     {% endif %}
0516:
0517:
0518: </div>
0519:
0520: <!-- Vis.js Network Script -->
0521: <script src="https://unpkg.com/vis-network/standalone/umd/vis-network.min.js"></script>
0522: <script id="bridge-graph-data" type="application/json">
0523: {
0524:     "pathFound": {{ (path_found and path_nodes)|tojson }},
0525:     "nodes": {{ (path_nodes or [])|tojson }},
0526:     "edges": {{ (path_edges or [])|tojson }}
0527: }
0528: </script>
0529: <script>
```

```
0530:     function swapBooks() {
0531:         const selA = document.getElementById("bookASelect");
0532:         const selB = document.getElementById("bookBSelect");
0533:         const temp = selA.value;
0534:         selA.value = selB.value;
0535:         selB.value = temp;
0536:     }
0537:
0538:     let bridgeNetwork = null;
0539:
0540:     document.addEventListener("DOMContentLoaded", function() {
0541:         const dataEl = document.getElementById("bridge-graph-data");
0542:         if (dataEl) {
0543:             try {
0544:                 const payload = JSON.parse(dataEl.textContent);
0545:                 if (payload.pathFound && Array.isArray(payload.nodes) && payload.nodes.length > 0) {
0546:                     renderBridgeNetwork(payload.nodes, payload.edges || []);
0547:                 }
0548:             } catch (err) {
0549:                 console.error("Failed to parse bridge graph data:", err);
0550:             }
0551:         }
0552:     });
0553:
0554:     function fitBridgeGraph() {
0555:         if (bridgeNetwork) {
0556:             bridgeNetwork.fit({ animation: { duration: 600, easingFunction: 'easeInOutQuad' } });
0557:         }
0558:     }
0559:
0560:     function renderBridgeNetwork(nodesList, edgesList) {
0561:         const container = document.getElementById("bridge-network-canvas");
0562:         if (!container) return;
0563:
0564:         // Ensure any BELONGS_TO relationship points from Book to Category
0565:         edgesList.forEach(e => {
0566:             if (e.label && e.label.includes("BELONGS_TO")) {
0567:                 const fromNode = nodesList.find(n => n.id === e.from);
0568:                 const toNode = nodesList.find(n => n.id === e.to);
0569:                 if (fromNode && toNode) {
0570:                     const fromIsCat = fromNode.group === "Category" || (fromNode.properties && fromNode.prop
      | erties.category_id !== undefined);
0571:                     const toIsBook = ["StartBook", "TargetBook", "IntermediateBook", "Book"].includes(toNode
      | .group) || (toNode.properties && toNode.properties.book_id !== undefined);
0572:                     if (fromIsCat && toIsBook) {
0573:                         const tmp = e.from;
0574:                         e.from = e.to;
0575:                         e.to = tmp;
0576:                     }
0577:                 }
0578:             }
0579:         });
0580:
0581:         const data = {
0582:             nodes: new vis.DataSet(nodesList),
0583:             edges: new vis.DataSet(edgesList)
0584:         };
0585:
0586:         const options = {
0587:             groups: {
0588:                 StartBook: {
0589:                     color: { background: '#1D4ED8', border: '#60A5FA', highlight: { background: '#2563EB', b
      | order: '#93C5FD' } },
0590:                     shape: 'dot',
0591:                     size: 28,
0592:                     font: { color: '#FFFFFF', face: 'Inter, sans-serif', size: 13, bold: true }
0593:                 },
0594:                 IntermediateBook: {
0595:                     color: { background: '#475569', border: '#64748B', highlight: { background: '#64748B', b
      | order: '#94A3B8' } },
0596:                     shape: 'dot',
0597:                     size: 24,
0598:                     font: { color: '#F1F5F9', face: 'Inter, sans-serif', size: 12, bold: true }
0599:                 },
0600:                 Book: {
0601:                     color: { background: '#475569', border: '#64748B', highlight: { background: '#64748B', b
      | order: '#94A3B8' } },
0602:                     shape: 'dot',
0603:                     size: 24,
0604:                     font: { color: '#F1F5F9', face: 'Inter, sans-serif', size: 12, bold: true }
```

```
0605:                 },
0606:                 TargetBook: {
0607:                     color: { background: '#059669', border: '#34D399', highlight: { background: '#10B981', b
      | order: '#6EE7B7' } },
0608:                     shape: 'dot',
0609:                     size: 28,
0610:                     font: { color: '#FFFFFF', face: 'Inter, sans-serif', size: 13, bold: true }
0611:                 },
0612:                 Category: {
0613:                     color: { background: '#D97706', border: '#FBBF24', highlight: { background: '#F59E0B', b
      | order: '#FDE68A' } },
0614:                     shape: 'diamond',
0615:                     size: 24,
0616:                     font: { color: '#E5E7EB', face: 'Inter, sans-serif', size: 11 }
0617:                 },
0618:                 Author: {
0619:                     color: { background: '#7C3AED', border: '#C084FC', highlight: { background: '#8B5CF6', b
      | order: '#DDD6FE' } },
0620:                     shape: 'dot',
0621:                     size: 22,
0622:                     font: { color: '#E5E7EB', face: 'Inter, sans-serif', size: 11 }
0623:                 },
0624:                 Copy: {
0625:                     color: { background: '#0F766E', border: '#2DD4BF', highlight: { background: '#14B8A6', b
      | order: '#99F6E4' } },
0626:                     shape: 'box',
0627:                     font: { color: '#FFFFFF', face: 'monospace', size: 10 }
0628:                 },
0629:                 Loan: {
0630:                     color: { background: '#0891B2', border: '#22D3EE', highlight: { background: '#06B6D4', b
      | order: '#A5F3FC' } },
0631:                     shape: 'box',
0632:                     font: { color: '#FFFFFF', face: 'monospace', size: 10 }
0633:                 },
0634:                 User: {
0635:                     color: { background: '#4F46E5', border: '#818CF8', highlight: { background: '#6366F1', b
      | order: '#C7D2FE' } },
0636:                     shape: 'dot',
0637:                     size: 20,
0638:                     font: { color: '#E5E7EB', face: 'Inter, sans-serif', size: 11 }
0639:                 }
0640:             },
0641:             nodes: {
0642:                 borderWidth: 2,
0643:                 shadow: { enabled: true, color: 'rgba(0,0,0,0.5)', size: 8, x: 2, y: 3 }
0644:             },
0645:             edges: {
0646:                 width: 2.5,
0647:                 color: { color: '#60A5FA', highlight: '#93C5FD', opacity: 0.85 },
0648:                 arrows: { to: { enabled: true, scaleFactor: 0.7 } },
0649:                 smooth: { type: 'curvedCW', roundness: 0.2 },
0650:                 font: { align: 'top', size: 10, color: '#9CA3AF', strokeWidth: 0, background: 'rgba(17, 24,
      | 39, 0.7)' }
0651:             },
0652:             physics: {
0653:                 stabilization: { iterations: 120 },
0654:                 barnesHut: {
0655:                     gravitationalConstant: -3500,
0656:                     centralGravity: 0.2,
0657:                     springLength: 130,
0658:                     springConstant: 0.04
0659:                 }
0660:             },
0661:             interaction: {
0662:                 hover: true,
0663:                 navigationButtons: true,
0664:                 keyboard: false
0665:             }
0666:         };
0667:
0668:         bridgeNetwork = new vis.Network(container, data, options);
0669:     }
0670: </script>
0671: {% endblock %}
```

MODULE SOURCE: templates/login.html

Lines: 219 | **Lang:** HTML5 / Jinja2 | **Category:** Authentication

Core Role & Purpose: User login portal with CSRF protection and role-aware navigation redirection.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify CSRF token input field.
- Verify form submission targeting /login.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Sign In | VITCC LIBRARY{% endblock %}
0004:
0005: {% block extra_css %}
0006: <style>
0007:     .login-container {
0008:         display: flex;
0009:         align-items: center;
0010:         justify-content: center;
0011:         min-height: calc(100vh - 120px);
0012:         position: relative;
0013:     }
0014:
0015:     /* Ambient glow elements */
0016:     .ambient-glow-1 {
0017:         position: absolute;
0018:         width: 350px;
```

```
0019:         height: 350px;
0020:         background: radial-gradient(circle, rgba(59, 130, 246, 0.15) 0%, rgba(0, 0, 0, 0) 70%);
0021:         top: 20%;
0022:         left: 30%;
0023:         filter: blur(40px);
0024:         pointer-events: none;
0025:         z-index: 0;
0026:     }
0027:
0028:     .ambient-glow-2 {
0029:         position: absolute;
0030:         width: 300px;
0031:         height: 300px;
0032:         background: radial-gradient(circle, rgba(99, 102, 241, 0.12) 0%, rgba(0, 0, 0, 0) 70%);
0033:         bottom: 20%;
0034:         right: 30%;
0035:         filter: blur(40px);
0036:         pointer-events: none;
0037:         z-index: 0;
0038:     }
0039:
0040:     /* Glass Card */
0041:     .login-card {
0042:         width: 100%;
0043:         max-width: 440px;
0044:         background: rgba(17, 24, 39, 0.82);
0045:         backdrop-filter: blur(20px);
0046:         -webkit-backdrop-filter: blur(20px);
0047:         border: 1px solid var(--border-subtle);
0048:         border-radius: var(--radius-lg);
0049:         padding: 40px;
0050:         box-shadow: 0 20px 40px -15px rgba(0, 0, 0, 0.7);
0051:         position: relative;
0052:         z-index: 10;
0053:     }
0054:
0055:     .login-card:hover {
0056:         border-color: var(--border-hover);
0057:     }
0058:
0059:     .login-header {
0060:         text-align: center;
0061:         margin-bottom: 32px;
0062:     }
0063:
0064:     .login-badge {
0065:         display: inline-flex;
0066:         align-items: center;
0067:         gap: 6px;
0068:         font-size: 0.75rem;
0069:         font-weight: 600;
0070:         letter-spacing: 0.05em;
0071:         text-transform: uppercase;
0072:         color: var(--accent-primary);
0073:         background: rgba(59, 130, 246, 0.1);
0074:         padding: 4px 12px;
0075:         border-radius: var(--radius-full);
0076:         border: 1px solid rgba(59, 130, 246, 0.2);
0077:         margin-bottom: 12px;
0078:     }
0079:
0080:     .login-title {
```

```
0081:         font-size: 1.6rem;
0082:         font-weight: 800;
0083:         letter-spacing: -0.03em;
0084:         color: var(--text-primary);
0085:         margin-bottom: 6px;
0086:     }
0087:
0088:
0089:
0090:     /* Form Fields */
0091:     .form-group {
0092:         margin-bottom: 20px;
0093:     }
0094:
0095:     .form-label {
0096:         display: block;
0097:         font-size: 0.82rem;
0098:         font-weight: 600;
0099:         color: var(--text-secondary);
0100:         margin-bottom: 8px;
0101:         letter-spacing: 0.01em;
0102:     }
0103:
0104:     .input-wrapper {
0105:         position: relative;
0106:         display: flex;
0107:         align-items: center;
0108:     }
0109:
0110:     .input-icon {
0111:         position: absolute;
0112:         left: 14px;
0113:         width: 18px;
0114:         height: 18px;
0115:         stroke: var(--text-muted);
0116:         stroke-width: 2;
0117:         fill: none;
0118:         pointer-events: none;
0119:     }
0120:
0121:     .form-input {
0122:         width: 100%;
0123:         padding: 12px 14px 12px 42px;
0124:         background: rgba(11, 15, 25, 0.6);
0125:         border: 1px solid var(--border-subtle);
0126:         border-radius: var(--radius-md);
0127:         color: var(--text-primary);
0128:         font-size: 0.92rem;
0129:         font-family: inherit;
0130:         transition: all 0.15s ease-in-out;
0131:     }
0132:
0133:     .form-input:focus {
0134:         outline: none;
0135:         border-color: var(--accent-primary);
0136:         box-shadow: 0 0 0 3px var(--accent-glow);
0137:         background: rgba(11, 15, 25, 0.9);
0138:     }
0139:
0140:     .form-input::placeholder {
0141:         color: var(--text-muted);
0142:     }
0143:
0144:     .btn-submit {
0145:         width: 100%;
0146:         padding: 13px;
0147:         font-size: 0.92rem;
0148:         font-weight: 700;
0149:         margin-top: 10px;
0150:     }
0151:
0152:
0153: </style>
0154: {% endblock %}
0155:
0156: {% block content %}
0157: <div class="login-container">
0158:     <div class="ambient-glow-1"></div>
0159:     <div class="ambient-glow-2"></div>
0160:
```

```
0161:     <div class="login-card">
0162:         <div class="login-header">
0163:             <span class="login-badge">
0164:                 <svg width="8" height="8" viewBox="0 0 8 8" fill="currentColor"><circle cx="4" cy="4" r="3">
      | </circle></svg>
0165:                 Secure Portal
0166:             </span>
0167:             <h1 class="login-title">VITCC LIBRARY</h1>
0168:
0169:         </div>
0170:
0171:         <form method="POST" action="{{ url_for('login') }}">
0172:             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0173:             <div class="form-group">
0174:                 <label class="form-label" for="username">Username</label>
0175:                 <div class="input-wrapper">
0176:                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0
      | -4 4v2"></path><circle cx="12" cy="7" r="4"></circle></svg>
0177:                     <input
0178:                         type="text"
0179:                         id="username"
0180:                         name="username"
0181:                         class="form-input"
0182:                         placeholder="e.g. jdoe_admin"
0183:                         required
0184:                         autofocus
0185:                     >
0186:                 </div>
0187:             </div>
0188:
0189:             <div class="form-group">
0190:                 <label class="form-label" for="password">Password</label>
0191:                 <div class="input-wrapper">
0192:                     <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx
      | ="2" ry="2"></rect><path d="M7 11V7a5 5 0 0 1 10 0v4"></path></svg>
0193:                     <input
0194:                         type="password"
0195:                         id="password"
0196:                         name="password"
0197:                         class="form-input"
0198:                         placeholder="••••••••••••"
0199:                         required
0200:                     >
0201:                 </div>
0202:             </div>
0203:
0204:             <button type="submit" class="btn btn-primary btn-submit">
0205:                 Sign In to Dashboard &rarr;
0206:             </button>
0207:
0208:             <div style="margin-top: 18px; text-align: center; font-size: 0.85rem; color: var(--text-secondar
      | y);">
0209:                 Don't have an account?
0210:                 <a href="{{ url_for('register') }}" style="color: var(--accent-primary); font-weight: 600; t
      | ext-decoration: none;">
0211:                     Sign Up
0212:                 </a>
0213:             </div>
0214:         </form>
0215:
0216:
0217:     </div>
0218: </div>
0219: {% endblock %}
```

MODULE SOURCE: templates/register.html

Lines: 283 | **Lang:** HTML5 / Jinja2 | **Category:** User Registration

Core Role & Purpose: New member registration form with campus role selection and input validation.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify role selection (Student, Staff).
- Verify password confirmation and CSRF protection.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Sign Up | VITCC LIBRARY{% endblock %}
0004:
0005: {% block extra_css %}
0006: <style>
0007:     .register-container {
0008:         display: flex;
0009:         align-items: center;
0010:         justify-content: center;
0011:         min-height: calc(100vh - 120px);
0012:         position: relative;
0013:         padding: 30px 16px;
0014:     }
0015:
0016:     /* Ambient glow elements */
0017:     .ambient-glow-1 {
0018:         position: absolute;
0019:         width: 380px;
0020:         height: 380px;
0021:         background: radial-gradient(circle, rgba(59, 130, 246, 0.16) 0%, rgba(0, 0, 0, 0) 70%);
0022:         top: 15%;
0023:         left: 28%;
0024:         filter: blur(50px);
0025:         pointer-events: none;
0026:         z-index: 0;
0027:     }
0028:
```

```
0029:     .ambient-glow-2 {
0030:         position: absolute;
0031:         width: 320px;
0032:         height: 320px;
0033:         background: radial-gradient(circle, rgba(99, 102, 241, 0.14) 0%, rgba(0, 0, 0, 0) 70%);
0034:         bottom: 15%;
0035:         right: 28%;
0036:         filter: blur(50px);
0037:         pointer-events: none;
0038:         z-index: 0;
0039:     }
0040:
0041:     /* Glass Card */
0042:     .register-card {
0043:         width: 100%;
0044:         max-width: 480px;
0045:         background: rgba(17, 24, 39, 0.85);
0046:         backdrop-filter: blur(20px);
0047:         -webkit-backdrop-filter: blur(20px);
0048:         border: 1px solid var(--border-subtle);
0049:         border-radius: var(--radius-lg);
0050:         padding: 38px 40px;
0051:         box-shadow: 0 20px 45px -15px rgba(0, 0, 0, 0.75);
0052:         position: relative;
0053:         z-index: 10;
0054:         transition: border-color 0.2s ease;
0055:     }
0056:
0057:     .register-card:hover {
0058:         border-color: var(--border-hover);
0059:     }
0060:
0061:     .register-header {
0062:         text-align: center;
0063:         margin-bottom: 28px;
0064:     }
0065:
0066:     .register-badge {
0067:         display: inline-flex;
0068:         align-items: center;
0069:         gap: 6px;
0070:         font-size: 0.74rem;
0071:         font-weight: 600;
0072:         letter-spacing: 0.05em;
0073:         text-transform: uppercase;
0074:         color: var(--accent-primary);
0075:         background: rgba(59, 130, 246, 0.1);
0076:         padding: 4px 12px;
0077:         border-radius: var(--radius-full);
0078:         border: 1px solid rgba(59, 130, 246, 0.2);
0079:         margin-bottom: 12px;
0080:     }
```

```
0081:
0082:     .register-title {
0083:         font-size: 1.6rem;
0084:         font-weight: 800;
0085:         letter-spacing: -0.03em;
0086:         color: var(--text-primary);
0087:         margin-bottom: 6px;
0088:     }
0089:
0090:     .register-subtitle {
0091:         font-size: 0.88rem;
0092:         color: var(--text-secondary);
0093:     }
0094:
0095:     /* Form Fields */
0096:     .form-group {
0097:         margin-bottom: 18px;
0098:     }
0099:
0100:     .form-label {
0101:         display: block;
0102:         font-size: 0.82rem;
0103:         font-weight: 600;
0104:         color: var(--text-secondary);
0105:         margin-bottom: 8px;
0106:         letter-spacing: 0.01em;
0107:     }
0108:
0109:     .input-wrapper {
0110:         position: relative;
0111:         display: flex;
0112:         align-items: center;
0113:     }
0114:
0115:     .input-icon {
0116:         position: absolute;
0117:         left: 14px;
0118:         width: 18px;
0119:         height: 18px;
0120:         stroke: var(--text-muted);
0121:         stroke-width: 2;
0122:         fill: none;
0123:         pointer-events: none;
0124:     }
0125:
0126:     .form-input {
0127:         width: 100%;
0128:         padding: 12px 14px 12px 42px;
0129:         background: rgba(11, 15, 25, 0.65);
0130:         border: 1px solid var(--border-subtle);
0131:         border-radius: var(--radius-md);
0132:         color: var(--text-primary);
0133:         font-size: 0.92rem;
0134:         font-family: inherit;
0135:         transition: all 0.15s ease-in-out;
0136:     }
0137:
0138:     .form-input:focus {
0139:         outline: none;
0140:         border-color: var(--accent-primary);
0141:         box-shadow: 0 0 0 3px var(--accent-glow);
0142:         background: rgba(11, 15, 25, 0.95);
0143:     }
0144:
0145:     .form-input::placeholder {
0146:         color: var(--text-muted);
0147:     }
0148:
0149:     .btn-submit {
0150:         width: 100%;
0151:         padding: 13px;
0152:         font-size: 0.92rem;
0153:         font-weight: 700;
0154:         margin-top: 10px;
0155:     }
0156:
0157:     .login-redirect-link {
0158:         margin-top: 20px;
0159:         text-align: center;
0160:         font-size: 0.85rem;
```

```
0161:         color: var(--text-secondary);
0162:     }
0163:
0164:     .login-redirect-link a {
0165:         color: var(--accent-primary);
0166:         font-weight: 600;
0167:         text-decoration: none;
0168:         transition: color 0.15s;
0169:     }
0170:
0171:     .login-redirect-link a:hover {
0172:         text-decoration: underline;
0173:     }
0174: </style>
0175: {% endblock %}
0176:
0177: {% block content %}
0178: <div class="register-container">
0179:     <div class="ambient-glow-1"></div>
0180:     <div class="ambient-glow-2"></div>
0181:
0182:     <div class="register-card">
0183:         <div class="register-header">
0184:             <span class="register-badge">
0185:                 <svg width="8" height="8" viewBox="0 0 8 8" fill="currentColor"><circle cx="4" cy="4" r="3">
      | </circle></svg>
0186:                 Institutional Portal
0187:             </span>
0188:             <h1 class="register-title">Create Account</h1>
0189:             <p class="register-subtitle">Register to borrow books, browse stock, & access digital loans</p>
0190:         </div>
0191:
0192:         <form method="POST" action="{{ url_for('register') }}">
0193:             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0194:             <div class="form-group">
0195:                 <label class="form-label" for="username">Username *</label>
0196:                 <div class="input-wrapper">
0197:                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0
      | -4 4v2"></path><circle cx="12" cy="7" r="4"></circle></svg>
0198:                     <input
0199:                         type="text"
0200:                         id="username"
0201:                         name="username"
0202:                         class="form-input"
0203:                         placeholder="e.g. jsmith"
0204:                         required
0205:                         autofocus
0206:                     >
0207:                 </div>
0208:             </div>
0209:
0210:             <div class="form-group">
0211:                 <label class="form-label" for="email">Institutional Email *</label>
0212:                 <div class="input-wrapper">
0213:                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9
      | 2-2 2H4c-1.1 0-2-.9-2-2V6c0-1.1.9-2 2-2z"></path><polyline points="22,6 12,13 2,6"></polyline></svg>
0214:                     <input
0215:                         type="email"
0216:                         id="email"
0217:                         name="email"
0218:                         class="form-input"
0219:                         placeholder="jsmith@vit.edu"
0220:                         required
0221:                     >
0222:                 </div>
0223:                 <div style="font-size: 0.72rem; color: var(--text-muted); margin-top: 4px;">
0224:                     Must contain characters <strong>'@'</strong> and <strong>'.'</strong> (e.g. name@vit.edu
      | )
0225:                 </div>
0226:             </div>
0227:
0228:             <!-- Pre-Creation Password & Email Rules Notice -->
0229:             <div style="background: rgba(30, 41, 59, 0.7); border: 1px solid rgba(59, 130, 246, 0.35); borde
      | r-radius: var(--radius-md); padding: 12px 14px; margin-bottom: 18px; font-size: 0.77rem;">
0230:                 <div style="font-weight: 700; color: #93C5FD; display: flex; align-items: center; gap: 6px;
      | margin-bottom: 6px;">
0231:                     <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="16" x2="12" y2="12"></line><lin
      | e x1="12" y1="8" x2="12.01" y2="8"></line></svg>
0232:                     Credential & Password Requirements:
```

```
0233:                 </div>
0234:                 <ul style="margin: 0; padding-left: 18px; color: var(--text-secondary); line-height: 1.55;">
0235:                     <li>Email must compulsory contain <strong>'@'</strong> and <strong>'.'</strong></li>
0236:                     <li>Password length: <strong>8 to 64 characters</strong></li>
0237:                     <li>At least <strong>1 uppercase</strong> letter (A-Z)</li>
0238:                     <li>At least <strong>1 lowercase</strong> letter (a-z)</li>
0239:                     <li>At least <strong>1 numeric</strong> digit (0-9)</li>
0240:                     <li>At least <strong>1 special character</strong> (e.g. ! @ # $ % ^ & *)</li>
0241:                 </ul>
0242:             </div>
0243:
0244:             <div class="form-group">
0245:                 <label class="form-label" for="password">Password *</label>
0246:                 <div class="input-wrapper">
0247:                     <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx
      | ="2" ry="2"></rect><path d="M7 11V7a5 5 0 0 1 10 0v4"></path></svg>
0248:                     <input
0249:                         type="password"
0250:                         id="password"
0251:                         name="password"
0252:                         class="form-input"
0253:                         placeholder="••••••••••••"
0254:                         minlength="8"
0255:                         maxlength="64"
0256:                         required
0257:                     >
0258:                 </div>
0259:             </div>
0260:
0261:             <div class="form-group">
0262:                 <label class="form-label" for="role">Account Type (Role) *</label>
0263:                 <div class="input-wrapper">
0264:                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0
      | -4 4v2"></path><circle cx="9" cy="7" r="4"></circle><path d="M23 21v-2a4 4 0 0 0-3-3.87"></path><path
      | d="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
0265:                     <select id="role" name="role" class="form-input" style="cursor: pointer;" required>
0266:                         <option value="Student">Student (Academic Patron)</option>
0267:                         <option value="Staff">Staff (Faculty / Department)</option>
0268:                     </select>
0269:                 </div>
0270:             </div>
0271:
0272:             <button type="submit" class="btn btn-primary btn-submit">
0273:                 Complete Registration &rarr;
0274:             </button>
0275:
0276:             <div class="login-redirect-link">
0277:                 Already have an account?
0278:                 <a href="{{ url_for('login') }}">Sign In</a>
0279:             </div>
0280:         </form>
0281:     </div>
0282: </div>
0283: {% endblock %}
```

MODULE SOURCE: templates/change_password.html

Lines: 245 | **Lang:** HTML5 / Jinja2 | **Category:** Credentials Management

Core Role & Purpose: Self-service password modification interface enforcing current password verification.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:

- Verify current password validation field.
- Verify new password matching validation.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Change Password | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}Security & Credentials{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     .security-container {
0009:         display: flex;
0010:         justify-content: center;
0011:         align-items: flex-start;
0012:         padding: 40px 16px;
0013:     }
0014:
0015:     .security-card {
0016:         width: 100%;
0017:         max-width: 520px;
0018:         background: var(--bg-surface);
0019:         border: 1px solid var(--border-subtle);
0020:         border-radius: var(--radius-lg);
0021:         padding: 36px;
0022:         box-shadow: 0 20px 45px -15px rgba(0, 0, 0, 0.75);
0023:     }
0024:
0025:     .security-header {
0026:         text-align: center;
0027:         margin-bottom: 28px;
0028:     }
0029:
0030:     .security-icon {
0031:         width: 48px;
0032:         height: 48px;
0033:         border-radius: var(--radius-full);
0034:         background: rgba(59, 130, 246, 0.15);
0035:         color: var(--accent-primary);
0036:         display: flex;
0037:         align-items: center;
0038:         justify-content: center;
```

```
0039:         margin: 0 auto 14px auto;
0040:     }
0041:
0042:     .security-title {
0043:         font-size: 1.45rem;
0044:         font-weight: 800;
0045:         letter-spacing: -0.03em;
0046:         color: var(--text-primary);
0047:         margin-bottom: 6px;
0048:     }
0049:
0050:     .security-subtitle {
0051:         font-size: 0.85rem;
0052:         color: var(--text-secondary);
0053:         line-height: 1.5;
0054:     }
0055:
0056:     .rules-card {
0057:         background: rgba(30, 41, 59, 0.6);
0058:         border: 1px solid rgba(59, 130, 246, 0.3);
0059:         border-radius: var(--radius-md);
0060:         padding: 14px 16px;
0061:         margin-bottom: 24px;
0062:         font-size: 0.8rem;
0063:     }
0064:
0065:     .rules-header {
0066:         font-weight: 700;
0067:         color: #93C5FD;
0068:         display: flex;
0069:         align-items: center;
0070:         gap: 6px;
0071:         margin-bottom: 8px;
0072:     }
0073:
0074:     .rules-list {
0075:         margin: 0;
0076:         padding-left: 18px;
0077:         color: var(--text-secondary);
0078:         line-height: 1.6;
0079:     }
0080:
```

```
0081:     .rules-list strong {
0082:         color: var(--text-primary);
0083:     }
0084:
0085:     .form-group {
0086:         margin-bottom: 20px;
0087:     }
0088:
0089:     .form-label {
0090:         display: block;
0091:         font-size: 0.82rem;
0092:         font-weight: 600;
0093:         color: var(--text-secondary);
0094:         margin-bottom: 8px;
0095:     }
0096:
0097:     .input-wrapper {
0098:         position: relative;
0099:         display: flex;
0100:         align-items: center;
0101:     }
0102:
0103:     .input-icon {
0104:         position: absolute;
0105:         left: 14px;
0106:         width: 18px;
0107:         height: 18px;
0108:         stroke: var(--text-muted);
0109:         stroke-width: 2;
0110:         fill: none;
0111:         pointer-events: none;
0112:     }
0113:
0114:     .form-input {
0115:         width: 100%;
0116:         padding: 12px 14px 12px 42px;
0117:         background: rgba(11, 15, 25, 0.65);
0118:         border: 1px solid var(--border-subtle);
0119:         border-radius: var(--radius-md);
0120:         color: var(--text-primary);
0121:         font-size: 0.92rem;
0122:         font-family: inherit;
0123:         transition: all 0.15s ease-in-out;
0124:     }
0125:
0126:     .form-input:focus {
0127:         outline: none;
0128:         border-color: var(--accent-primary);
0129:         box-shadow: 0 0 0 3px var(--accent-glow);
0130:         background: rgba(11, 15, 25, 0.95);
0131:     }
0132:
0133:     .btn-submit {
0134:         width: 100%;
0135:         padding: 12px;
0136:         font-size: 0.92rem;
0137:         font-weight: 700;
0138:         margin-top: 6px;
0139:     }
0140:
0141:     .back-link {
0142:         display: block;
0143:         text-align: center;
0144:         margin-top: 18px;
0145:         font-size: 0.85rem;
0146:         color: var(--text-secondary);
0147:         text-decoration: none;
0148:     }
0149:
0150:     .back-link:hover {
0151:         color: var(--accent-primary);
0152:         text-decoration: underline;
0153:     }
0154: </style>
0155: {% endblock %}
0156:
0157: {% block content %}
0158: <div class="security-container">
0159:     <div class="security-card">
0160:         <div class="security-header">
```

```
0161:             <div class="security-icon">
0162:                 <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><rect x="3" y="11" width="18" height="11" rx="2" ry="2"></rect><path d="M7 11V7a5 5 0 0 1 10 0v
      | 4"></path></svg>
0163:             </div>
0164:             <h1 class="security-title">Change Password</h1>
0165:             <p class="security-subtitle">Update your account credentials to keep your VITCC LIBRARY profile
      | secure.</p>
0166:         </div>
0167:
0168:         <!-- Password Requirements Specifications -->
0169:         <div class="rules-card">
0170:             <div class="rules-header">
0171:                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2.5"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="16" x2="12" y2="12"></line><line x1
      | ="12" y1="8" x2="12.01" y2="8"></line></svg>
0172:                 Password Complexity Requirements:
0173:             </div>
0174:             <ul class="rules-list">
0175:                 <li>Password length: <strong>8 to 64 characters</strong></li>
0176:                 <li>At least <strong>1 uppercase</strong> letter (A-Z)</li>
0177:                 <li>At least <strong>1 lowercase</strong> letter (a-z)</li>
0178:                 <li>At least <strong>1 numeric</strong> digit (0-9)</li>
0179:                 <li>At least <strong>1 special character</strong> (e.g. ! @ # $ % ^ & * _ -)</li>
0180:             </ul>
0181:         </div>
0182:
0183:         <form method="POST" action="{{ url_for('change_password') }}">
0184:             <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0185:             <div class="form-group">
0186:                 <label class="form-label" for="current_password">Current Password *</label>
0187:                 <div class="input-wrapper">
0188:                     <svg class="input-icon" viewBox="0 0 24 24"><rect x="3" y="11" width="18" height="11" rx
      | ="2" ry="2"></rect><path d="M7 11V7a5 5 0 0 1 10 0v4"></path></svg>
0189:                     <input
0190:                         type="password"
0191:                         id="current_password"
0192:                         name="current_password"
0193:                         class="form-input"
0194:                         placeholder="Enter existing password"
0195:                         required
0196:                         autofocus
0197:                     >
0198:                 </div>
0199:             </div>
0200:
0201:             <div class="form-group">
0202:                 <label class="form-label" for="new_password">New Password *</label>
0203:                 <div class="input-wrapper">
0204:                     <svg class="input-icon" viewBox="0 0 24 24"><path d="M21 2l-2 2m-6 6l7-7M3 10a7 7 0 1 0
      | 14 0 7 7 0 1 0-14 0z"></path></svg>
0205:                     <input
0206:                         type="password"
0207:                         id="new_password"
0208:                         name="new_password"
0209:                         class="form-input"
0210:                         placeholder="Enter new strong password (8-64 chars)"
0211:                         minlength="8"
0212:                         maxlength="64"
0213:                         required
0214:                     >
0215:                 </div>
0216:             </div>
0217:
0218:             <div class="form-group">
0219:                 <label class="form-label" for="confirm_password">Confirm New Password *</label>
0220:                 <div class="input-wrapper">
0221:                     <svg class="input-icon" viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"></polyline
      | ></svg>
0222:                     <input
0223:                         type="password"
0224:                         id="confirm_password"
0225:                         name="confirm_password"
0226:                         class="form-input"
0227:                         placeholder="Re-enter new password"
0228:                         minlength="8"
0229:                         maxlength="64"
0230:                         required
0231:                     >
0232:                 </div>
```

```
            </div>

            <button type="submit" class="btn btn-primary btn-submit">
                Update Password &rarr;
            </button>

            <a href="{{ url_for('dashboard') }}" class="back-link">
                &larr; Return to Dashboard
            </a>
        </form>
    </div>
</div>
{% endblock %}
```

MODULE SOURCE: templates/my_fines.html

Lines: 64 | **Lang:** HTML5 / Jinja2 | **Category:** Fine Management

Core Role & Purpose: Member fine inspection and settlement history interface.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
- Verify fine status badge rendering (Unpaid, Paid, Waived).
- Verify settlement status reporting.

```
{% extends "base.html" %}

{% block title %}My Fines | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}My Fines & Penalties{% endblock %}

{% block content %}
<div style="margin-bottom: 28px;">
    <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">My Financial Statements</h1>
    <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">Review assessed overdue fi
| nes and payment statuses</p>
</div>

<div class="data-panel" style="background: var(--bg-surface); border: 1px solid var(--border-subtle); border
| -radius: var(--radius-lg); overflow: hidden;">
    <div style="padding: 18px 24px; border-bottom: 1px solid var(--border-subtle); display: flex; justify-co
| ntent: space-between; align-items: center;">
        <span style="font-weight: 700; font-size: 0.95rem;">Fine Ledger Records</span>
        <span class="badge badge-info">{{ fines|length if fines else 0 }} Incurred</span>
    </div>

    <div class="table-container" style="overflow-x: auto;">
        <table class="data-table" style="width: 100%; border-collapse: collapse; text-align: left; font-size
| : 0.88rem;">
            <thead>
                <tr style="background: rgba(255, 255, 255, 0.02); color: var(--text-muted); font-size: 0.72r
| em; text-transform: uppercase;">
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Fine ID</
| th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Book Titl
| e</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Loan Ref<
| /th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Due Date<
| /th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Return Da
| te</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Fine Amou
| nt</th>
                    <th style="padding: 14px 24px; border-bottom: 1px solid var(--border-subtle);">Status</t
| h>
                </tr>
            </thead>
            <tbody>
                {% if fines %}
                    {% for fine in fines %}
                    <tr style="border-bottom: 1px solid var(--border-subtle);">
                        <td style="padding: 16px 24px; font-family: var(--font-mono); font-size: 0.8rem;">#F
| N-{{ fine.fine_id }}</td>
                        <td style="padding: 16px 24px; color: var(--text-primary); font-weight: 600;">{{ fin
| e.book_title or fine.title }}</td>
                        <td style="padding: 16px 24px; font-family: var(--font-mono); font-size: 0.8rem;">#L
| N-{{ fine.loan_id }}</td>
                        <td style="padding: 16px 24px; color: var(--text-secondary);">{{ fine.due_date.strft
| ime('%b %d, %Y') if fine.due_date else 'N/A' }}</td>
                        <td style="padding: 16px 24px; color: var(--text-secondary);">{{ fine.return_date.st
| rftime('%b %d, %Y') if fine.return_date else 'Not Returned' }}</td>
                        <td style="padding: 16px 24px; font-weight: 700; color: var(--danger-text);">Rs. {{
| "%.2f"|format(fine.fine_amount) }}</td>
                        <td style="padding: 16px 24px;">
                            {% if fine.payment_status == 'Paid' %}
                                <span class="badge badge-success">Paid</span>
                            {% elif fine.payment_status == 'Waived' %}
                                <span class="badge badge-info">Waived</span>
                            {% else %}
                                <span class="badge badge-danger">Unpaid</span>
                            {% endif %}
                        </td>
                    </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="7" style="padding: 48px 24px; text-align: center; color: var(--text-mut
| ed);">
                            <div style="font-size: 1rem; font-weight: 600; color: var(--text-primary);">Zero
|  Fine</div>
                            <p style="margin-top: 4px;">You have no disciplinary fines recorded on your acco
| unt.</p>
                        </td>
                    </tr>
```

```
0059:                 {% endif %}
```

```
0060:             </tbody>
0061:         </table>
0062:     </div>
0063: </div>
0064: {% endblock %}
```

MODULE SOURCE: templates/add_book.html

Lines: 370 | **Lang:** HTML5 / Jinja2 |
Category: Catalog Management

Core Role & Purpose: New book registration form with automatic physical copy generation.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify author, publisher, category inputs and copy count creation.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Add New Book | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}Add Book{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     .form-section {
0009:         background: var(--bg-surface);
0010:         border: 1px solid var(--border-subtle);
0011:         border-radius: var(--radius-lg);
0012:         padding: 28px;
0013:         margin-bottom: 24px;
0014:     }
0015:
0016:     .section-headline {
0017:         font-size: 0.95rem;
0018:         font-weight: 700;
0019:         text-transform: uppercase;
0020:         letter-spacing: 0.06em;
0021:         color: var(--accent-primary);
0022:         margin-bottom: 18px;
0023:         display: flex;
0024:         align-items: center;
0025:         gap: 8px;
0026:     }
0027:
0028:     .form-group {
0029:         margin-bottom: 18px;
0030:     }
0031:
0032:     .form-label {
0033:         display: block;
0034:         font-size: 0.82rem;
0035:         font-weight: 600;
0036:         color: var(--text-secondary);
0037:         margin-bottom: 6px;
0038:     }
0039:
0040:     .form-input {
0041:         width: 100%;
0042:         padding: 10px 14px;
0043:         background: var(--bg-base);
0044:         border: 1px solid var(--border-subtle);
0045:         border-radius: var(--radius-md);
0046:         color: var(--text-primary);
0047:         font-size: 0.88rem;
0048:         font-family: inherit;
0049:         transition: all 0.15s ease;
0050:     }
0051:
0052:     .form-input:focus {
0053:         outline: none;
0054:         border-color: var(--accent-primary);
0055:         box-shadow: 0 0 0 3px var(--accent-glow);
0056:     }
0057:
0058:     .sub-box {
0059:         background: rgba(255, 255, 255, 0.02);
0060:         border: 1px dashed var(--border-subtle);
0061:         border-radius: var(--radius-md);
0062:         padding: 14px 16px;
0063:         margin-top: 10px;
0064:     }
0065:
0066:     .sub-box-title {
0067:         font-size: 0.76rem;
0068:         font-weight: 700;
0069:         color: var(--text-muted);
0070:         text-transform: uppercase;
0071:         letter-spacing: 0.05em;
0072:         margin-bottom: 10px;
0073:         display: flex;
0074:         align-items: center;
0075:         gap: 6px;
0076:     }
0077:
0078:     .hint-text {
0079:         font-size: 0.75rem;
0080:         color: var(--text-muted);
```

```
0081:         margin-top: 4px;
0082:     }
0083: </style>
0084: {% endblock %}
0085:
0086: {% block content %}
0087: <div style="max-width: 820px; margin: 0 auto;">
0088:     <div style="margin-bottom: 28px;">
0089:         <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">Add New Book Title</h1>
0090:         <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">
0091:             Catalog new titles with automated inventory copies, or dynamically create new publishers, catego
      | ries, and authors.
0092:         </p>
0093:     </div>
0094:
0095:     <form method="POST" action="{{ url_for('add_book') }}">
0096:         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0097:         <!-- 1. Primary Metadata -->
0098:         <div class="form-section">
0099:             <div class="section-headline">
0100:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></path></svg>
0101:                 Primary Book Information
0102:             </div>
0103:
0104:             <div style="display: grid; grid-template-columns: 1fr 2fr; gap: 16px;">
0105:                 <div class="form-group">
0106:                     <label class="form-label" for="isbn">ISBN (10 or 13) *</label>
0107:                     <input type="text" id="isbn" name="isbn" class="form-input" placeholder="e.g. 978-013468
      | 5991" required>
0108:                 </div>
0109:                 <div class="form-group">
0110:                     <label class="form-label" for="title">Book Title *</label>
0111:                     <input type="text" id="title" name="title" class="form-input" placeholder="e.g. Database
      |  System Concepts" required>
0112:                 </div>
0113:             </div>
0114:
0115:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 0;">
0116:                 <div class="form-group" style="margin-bottom: 0;">
0117:                     <label class="form-label" for="publication_year">Publication Year</label>
0118:                     <input type="number" id="publication_year" name="publication_year" class="form-input" pl
      | aceholder="2024" min="1800" max="2099">
0119:                 </div>
0120:                 <div class="form-group" style="margin-bottom: 0;">
0121:                     <label class="form-label" for="num_copies">Physical Copies to Generate</label>
0122:                     <input type="number" id="num_copies" name="num_copies" class="form-input" value="1" min=
      | "1" max="50">
0123:                     <div class="hint-text">Each copy will be assigned an automated inventory barcode.</div>
0124:                 </div>
0125:             </div>
0126:         </div>
0127:
0128:         <!-- 2. Classification & Publishing (With On-the-Fly Entry) -->
0129:         <div class="form-section">
0130:             <div class="section-headline">
0131:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><circle cx="12" cy="12" r="10"></circle><polygon points="12 6 12 12 14 14"></polygon></svg>
0132:                 Category & Publisher
0133:             </div>
0134:
0135:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
0136:                 <!-- Category Section -->
0137:                 <div>
0138:                     <div class="form-group">
0139:                         <label class="form-label" for="category_id">Select Existing Category</label>
0140:                         <select name="category_id" id="category_id" class="form-input" style="cursor: pointe
      | r;">
0141:                             <option value="">-- Choose Category --</option>
0142:                             {% for cat in categories %}
0143:                                 <option value="{{ cat.category_id }}">{{ cat.name }} ({{ cat.section_code or
      |  'General' }})</option>
0144:                             {% endfor %}
0145:                         </select>
0146:                     </div>
0147:
0148:                     <div class="sub-box">
0149:                         <div class="sub-box-title">
0150:                             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor
      | " stroke-width="2"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></
```

```
      | line></svg>
0151:                             Or Add New Category
0152:                         </div>
0153:                         <div style="display: flex; flex-direction: column; gap: 10px;">
0154:                             <input type="text" name="new_category_name" class="form-input" placeholder="New
      | category name (e.g. Distributed Systems)">
0155:                             <input type="text" name="new_category_code" class="form-input" placeholder="Sect
      | ion Code (e.g. CS-400)">
0156:                         </div>
0157:                     </div>
0158:                 </div>
0159:
0160:                 <!-- Publisher Section -->
0161:                 <div>
0162:                     <div class="form-group">
0163:                         <label class="form-label" for="publisher_id">Select Existing Publisher</label>
0164:                         <select name="publisher_id" id="publisher_id" class="form-input" style="cursor: poin
      | ter;">
0165:                             <option value="">-- Choose Publisher --</option>
0166:                             {% for pub in publishers %}
0167:                                 <option value="{{ pub.publisher_id }}">{{ pub.name }}</option>
0168:                             {% endfor %}
0169:                         </select>
0170:                     </div>
0171:
0172:                     <div class="sub-box">
0173:                         <div class="sub-box-title">
0174:                             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor
      | " stroke-width="2"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></
      | line></svg>
0175:                             Or Add New Publisher
0176:                         </div>
0177:                         <div style="display: flex; flex-direction: column; gap: 10px;">
0178:                             <input type="text" name="new_publisher_name" class="form-input" placeholder="New
      |  publisher name (e.g. O'Reilly Media)">
0179:                             <input type="email" name="new_publisher_email" class="form-input" placeholder="C
      | ontact email (optional)">
0180:                         </div>
0181:                     </div>
0182:                 </div>
0183:             </div>
0184:         </div>
0185:
0186:         <!-- 3. Author Attribution (With On-the-Fly Entry) -->
0187:         <div class="form-section">
0188:             <div class="section-headline">
0189:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></circle
      | ></svg>
0190:                 Author Attribution
0191:             </div>
0192:
0193:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
0194:                 <div>
0195:                     <div class="form-group">
0196:                         <label class="form-label" for="author_ids">Select Existing Authors (Hold Ctrl/Cmd to
      |  select multiple)</label>
0197:                         <select name="author_ids" id="author_ids" class="form-input" multiple style="height:
      |  125px; cursor: pointer;">
0198:                             {% for auth in authors %}
0199:                                 <option value="{{ auth.author_id }}">{{ auth.full_name }}</option>
0200:                             {% endfor %}
0201:                         </select>
0202:                     </div>
0203:                 </div>
0204:
0205:                 <div>
0206:                     <div class="sub-box" style="margin-top: 0; height: 100%; display: flex; flex-direction:
      | column; justify-content: center;">
0207:                         <div class="sub-box-title">
0208:                             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor
      | " stroke-width="2"><line x1="12" y1="5" x2="12" y2="19"></line><line x1="5" y1="12" x2="19" y2="12"></
      | line></svg>
0209:                             Or Add New Author
0210:                         </div>
0211:                         <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bottom:
      |  8px;">
0212:                             <input type="text" name="new_author_first" class="form-input" placeholder="First
      |  Name (e.g. Alan)">
0213:                             <input type="text" name="new_author_last" class="form-input" placeholder="Last N
```

```
      | ame (e.g. Turing)">
0214:                         </div>
0215:                         <div class="hint-text">If specified, the new author will be registered and linked au
      | tomatically.</div>
0216:                     </div>
0217:                 </div>
0218:             </div>
0219:         </div>
0220:
0221:         <!-- 4. Academic Curriculum & Prerequisite Links (Neo4j Graph) -->
0222:         <div class="form-section">
0223:             <div class="section-headline">
0224:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18" cy=
      | "19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6.51"
      |  x2="8.59" y2="10.49"></line></svg>
0225:                 Academic Curriculum & Prerequisite Links (Neo4j Graph)
0226:             </div>
0227:             <p style="font-size: 0.82rem; color: var(--text-secondary); margin-bottom: 20px; line-height: 1.
      | 5;">
0228:                 Connect this title into the academic progression roadmap. Linked books will automatically re
      | nder prerequisite alerts and "Next to Study" recommendations on the Patron Dashboard, Category Catalog
      | , and Graph Visualizer.
0229:             </p>
0230:
0231:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-bottom: 18px;">
0232:                 <!-- Prerequisite Select -->
0233:                 <div class="form-group">
0234:                     <label class="form-label" for="prereq_book_id">
0235:                         Prerequisite Book (Study Before This)
0236:                     </label>
0237:                     <select name="prereq_book_id" id="prereq_book_id" class="form-input" style="cursor: poin
      | ter;">
0238:                         <option value="">-- None (Foundational / Standalone Title) --</option>
0239:                         <option value="auto">■ Auto-Detect Prerequisite from Same Category</option>
0240:                         {% for b in all_books %}
0241:                             <option value="{{ b.book_id }}" data-category-id="{{ b.category_id }}">
0242:                                 [{{ b.category_name }} · Level {{ b.level }}] {{ b.title }}
0243:                             </option>
0244:                         {% endfor %}
0245:                     </select>
0246:                     <div class="hint-text">
0247:                         Patrons will be prompted to study this title first before tackling this material.
0248:                     </div>
0249:                 </div>
0250:
0251:                 <!-- Next-Step Select -->
0252:                 <div class="form-group">
0253:                     <label class="form-label" for="next_book_id">
0254:                         Next-Step Book (Study After This)
0255:                     </label>
0256:                     <select name="next_book_id" id="next_book_id" class="form-input" style="cursor: pointer;
      | ">
0257:                         <option value="">-- None (Advanced Leaf / Standalone Title) --</option>
0258:                         {% for b in all_books %}
0259:                             <option value="{{ b.book_id }}" data-category-id="{{ b.category_id }}">
0260:                                 [{{ b.category_name }} · Level {{ b.level }}] {{ b.title }}
0261:                             </option>
0262:                         {% endfor %}
0263:                     </select>
0264:                     <div class="hint-text">
0265:                         Unlocks upon borrowing; suggested as immediate next step in curriculum.
0266:                     </div>
0267:                 </div>
0268:             </div>
0269:
0270:             <div class="form-group" style="margin-bottom: 0;">
0271:                 <label class="form-label" for="curriculum_track">Curriculum Track Label (Optional)</label>
0272:                 <select name="curriculum_track" id="curriculum_track" class="form-input" style="cursor: poin
      | ter; max-width: 400px;">
0273:                     <option value="auto">Auto-Match Category</option>
0274:                     <option value="Software Engineering">Software Engineering</option>
0275:                     <option value="Artificial Intelligence">Artificial Intelligence</option>
0276:                     <option value="Data Engineering">Data Engineering</option>
0277:                     <option value="Systems & Infrastructure">Systems & Infrastructure</option>
0278:                     <option value="Theoretical CS">Theoretical CS</option>
0279:                 </select>
0280:                 <div class="hint-text">
0281:                     Assigns specialization track metadata for the Academic Learning Roadmaps view.
0282:                 </div>
```

```
            </div>
        </div>

        <div style="display: flex; gap: 14px; justify-content: flex-end; margin-bottom: 40px;">
            <a href="{{ url_for('books') }}" class="btn btn-secondary">Cancel</a>
            <button type="submit" class="btn btn-primary" style="padding: 12px 24px;">
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
    | th="2"><path d="M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2 2 0 0 1-2 2z"></path><polyline poin
    | ts="17 21 17 13 7 13 7 21"></polyline><polyline points="7 3 7 8 15 8"></polyline></svg>
                Save & Register Book
            </button>
        </div>
    </form>
</div>
{% endblock %}

{% block extra_js %}
<script>
document.addEventListener("DOMContentLoaded", function() {
    function reorderCategoryBooks(selectElem, currentCatId) {
        if (!selectElem) return;
        const currentVal = selectElem.value;
        const options = Array.from(selectElem.querySelectorAll("option"));
        if (options.length <= 2) return;

        const headerOpts = [];
        const matchingOpts = [];
        const otherOpts = [];

        options.forEach(opt => {
            const val = opt.value;
            if (val === "" || val === "auto") {
                headerOpts.push(opt);
            } else {
                const catId = opt.getAttribute("data-category-id");
                let baseTitle = opt.getAttribute("data-base-title");
                if (!baseTitle) {
                    baseTitle = opt.textContent.trim();
                    opt.setAttribute("data-base-title", baseTitle);
                }

                if (currentCatId && String(catId) === String(currentCatId)) {
                    opt.textContent = "■ " + baseTitle;
                    matchingOpts.push(opt);
                } else {
                    opt.textContent = baseTitle;
                    otherOpts.push(opt);
                }
            }
        });

        selectElem.innerHTML = "";
        headerOpts.forEach(opt => selectElem.appendChild(opt));

        if (matchingOpts.length > 0) {
            const grpMatch = document.createElement("optgroup");
            grpMatch.label = "■■ Titles in Selected Category ■■";
            matchingOpts.forEach(opt => grpMatch.appendChild(opt));
            selectElem.appendChild(grpMatch);
        }

        if (otherOpts.length > 0) {
            const grpOther = document.createElement("optgroup");
            grpOther.label = "■■ Other Curriculum Categories ■■";
            otherOpts.forEach(opt => grpOther.appendChild(opt));
            selectElem.appendChild(grpOther);
        }

        if (currentVal) {
            selectElem.value = currentVal;
        }
    }

    const catSelect = document.getElementById("category_id");
    const prereqSelect = document.getElementById("prereq_book_id");
    const nextSelect = document.getElementById("next_book_id");

    if (catSelect) {
        catSelect.addEventListener("change", function() {
            reorderCategoryBooks(prereqSelect, this.value);
```

```
            reorderCategoryBooks(nextSelect, this.value);
        });
        if (catSelect.value) {
            reorderCategoryBooks(prereqSelect, catSelect.value);
            reorderCategoryBooks(nextSelect, catSelect.value);
        }
    }
});
</script>
{% endblock %}
```

MODULE SOURCE: templates/edit_book.html

Lines: 325 | **Lang:** HTML5 / Jinja2 | **Category:** Catalog Editing

Core Role & Purpose: Book metadata editing interface for Librarians and Admins.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify title, ISBN, and publication year modification fields.

```
{% extends "base.html" %}

{% block title %}Edit Book & Academic Roadmap | VITCC LIBRARY{% endblock %}
{% block breadcrumb %}Edit Book #{{ book.book_id }}{% endblock %}

{% block extra_css %}
<style>
    .form-section {
        background: var(--bg-surface);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-lg);
        padding: 28px;
        margin-bottom: 24px;
    }

    .section-headline {
        font-size: 0.95rem;
        font-weight: 700;
        text-transform: uppercase;
        letter-spacing: 0.06em;
        color: var(--accent-primary);
        margin-bottom: 18px;
        display: flex;
        align-items: center;
        gap: 8px;
    }

    .form-group {
        margin-bottom: 18px;
    }

    .form-label {
        display: block;
        font-size: 0.82rem;
        font-weight: 600;
        color: var(--text-secondary);
        margin-bottom: 6px;
    }

    .form-input {
        width: 100%;
        padding: 10px 14px;
        background: var(--bg-base);
        border: 1px solid var(--border-subtle);
        border-radius: var(--radius-md);
        color: var(--text-primary);
        font-size: 0.88rem;
        font-family: inherit;
        transition: all 0.15s ease;
    }

    .form-input:focus {
        outline: none;
        border-color: var(--accent-primary);
        box-shadow: 0 0 0 3px var(--accent-glow);
    }

    .hint-text {
        font-size: 0.75rem;
        color: var(--text-muted);
        margin-top: 4px;
    }

    .current-graph-status {
        background: rgba(99, 102, 241, 0.08);
        border: 1px solid rgba(99, 102, 241, 0.25);
        border-radius: var(--radius-md);
        padding: 12px 16px;
        margin-bottom: 18px;
        display: flex;
        align-items: center;
        gap: 12px;
        font-size: 0.82rem;
        color: #C7D2FE;
    }
</style>
{% endblock %}

{% block content %}
<div style="max-width: 820px; margin: 0 auto;">
```

```
0081:     <div style="margin-bottom: 28px;">
0082:         <h1 style="font-size: 1.6rem; font-weight: 800; letter-spacing: -0.03em;">Edit Book &amp;amp; Academic L
      | earning Links</h1>
0083:         <p style="font-size: 0.88rem; color: var(--text-secondary); margin-top: 4px;">
0084:             Update catalog metadata and manage prerequisite/next-step hierarchy links for Book ID #{{ book.b
      | ook_id }}.
0085:         </p>
0086:     </div>
0087:
0088:     <form method="POST" action="{{ url_for('edit_book', book_id=book.book_id) }}">
0089:         <input type="hidden" name="csrf_token" value="{{ csrf_token() }}"/>
0090:
0091:         <!-- 1. Primary Metadata -->
0092:         <div class="form-section">
0093:             <div class="section-headline">
0094:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M4 19.5v-15A2.5 2.5 0 0 1 6.5 2H20v20H6.5a2.5 2.5 0 0 1-2.5-2.5Z"></path></svg>
0095:                 Book Information
0096:             </div>
0097:
0098:             <div style="display: grid; grid-template-columns: 2fr 1fr; gap: 16px; margin-bottom: 18px;">
0099:                 <div class="form-group" style="margin-bottom: 0;">
0100:                     <label class="form-label" for="title">Book Title *</label>
0101:                     <input type="text" id="title" name="title" class="form-input" value="{{ book.title }}" r
      | equired autofocus>
0102:                 </div>
0103:                 <div class="form-group" style="margin-bottom: 0;">
0104:                     <label class="form-label" for="isbn">ISBN (10 or 13 digits) *</label>
0105:                     <input type="text" id="isbn" name="isbn" class="form-input" value="{{ book.isbn }}" requ
      | ired>
0106:                 </div>
0107:             </div>
0108:
0109:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 18px;">
0110:                 <div class="form-group" style="margin-bottom: 0;">
0111:                     <label class="form-label" for="publication_year">Publication Year</label>
0112:                     <input type="number" id="publication_year" name="publication_year" class="form-input" va
      | lue="{{ book.publication_year or '' }}" min="1000" max="2099">
0113:                 </div>
0114:                 <div class="form-group" style="margin-bottom: 0;">
0115:                     <label class="form-label" for="category_id">Category</label>
0116:                     <select name="category_id" id="category_id" class="form-input" style="cursor: pointer;">
0117:                         <option value="">-- Choose Category --</option>
0118:                         {% for cat in categories %}
0119:                             <option value="{{ cat.category_id }}" {% if book.category_id == cat.category_id
      | %}selected{% endif %}>
0120:                                 {{ cat.name }} ({{ cat.section_code or 'General' }})
0121:                             </option>
0122:                         {% endfor %}
0123:                     </select>
0124:                 </div>
0125:             </div>
0126:
0127:             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-bottom: 0;">
0128:                 <div class="form-group" style="margin-bottom: 0;">
0129:                     <label class="form-label" for="publisher_id">Publisher</label>
0130:                     <select name="publisher_id" id="publisher_id" class="form-input" style="cursor: pointer;
      | ">
0131:                         <option value="">-- Choose Publisher --</option>
0132:                         {% for pub in publishers %}
0133:                             <option value="{{ pub.publisher_id }}" {% if book.publisher_id == pub.publisher_
      | id %}selected{% endif %}>
0134:                                 {{ pub.name }}
0135:                             </option>
0136:                         {% endfor %}
0137:                     </select>
0138:                 </div>
0139:                 <div class="form-group" style="margin-bottom: 0;">
0140:                     <label class="form-label" for="author_ids">Authors (Hold Ctrl/Cmd to select multiple)</l
      | abel>
0141:                     <select name="author_ids" id="author_ids" class="form-input" multiple style="height: 105
      | px; cursor: pointer;">
0142:                         {% for auth in authors %}
0143:                             <option value="{{ auth.author_id }}" {% if book.current_author_ids and auth.auth
      | or_id in book.current_author_ids %}selected{% endif %}>
0144:                                 {{ auth.full_name }}
0145:                             </option>
0146:                         {% endfor %}
0147:                     </select>
0148:                 </div>
```

```
            </div>
        </div>

        <!-- 2. Academic Curriculum & Prerequisite Links (Neo4j Graph) -->
        <div class="form-section">
            <div class="section-headline">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
    | th="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18" cy=
    | "19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6.51"
    |  x2="8.59" y2="10.49"></line></svg>
                Academic Curriculum &amp;amp; Prerequisite Links (Neo4j Graph)
            </div>

            {% if book.current_prereq_id or book.current_next_id %}
            <div class="current-graph-status">
                <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="#818CF8" stroke-width="2
    | "><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path><polyline points="22 4 12 14.01 9 11.01"></polyl
    | ine></svg>
                <div>
                    <strong>Active Graph Links:</strong>
                    {% if book.current_prereq_id %}
                        Predecessor: <span style="color: #A5B4FC; font-weight: 600;">Book #{{ book.current_p
    | rereq_id }}</span>
                    {% endif %}
                    {% if book.current_next_id %}
                        {% if book.current_prereq_id %} • {% endif %}
                        Successor: <span style="color: #A5B4FC; font-weight: 600;">Book #{{ book.current_nex
    | t_id }}</span>
                    {% endif %}
                    {% if book.current_track %}
                        • Track: <span style="color: #34D399; font-weight: 600;">{{ book.current_track }}</s
    | pan>
                    {% endif %}
                </div>
            </div>
            {% endif %}

            <p style="font-size: 0.82rem; color: var(--text-secondary); margin-bottom: 20px; line-height: 1.
    | 5;">
                Assign or update academic progression links for this book. Updating these selections re-wire
    | s [:PREREQUISITE_FOR] edges in Neo4j AuraDB and synchronizes the Academic Learning Roadmaps tree.
            </p>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-bottom: 18px;">
                <!-- Prerequisite Select -->
                <div class="form-group">
                    <label class="form-label" for="prereq_book_id">
                        Prerequisite Book (Study Before This)
                    </label>
                    <select name="prereq_book_id" id="prereq_book_id" class="form-input" style="cursor: poin
    | ter;">
                        <option value="">-- None (Foundational / Standalone Title) --</option>
                        <option value="auto">■ Auto-Detect Prerequisite from Same Category</option>
                        {% for b in all_books %}
                            {% if b.book_id != book.book_id %}
                            <option value="{{ b.book_id }}" data-category-id="{{ b.category_id }}" {% if boo
    | k.current_prereq_id == b.book_id %}selected{% endif %}>
                                [{{ b.category_name }} · Level {{ b.level }}] {{ b.title }}
                            </option>
                            {% endif %}
                        {% endfor %}
                    </select>
                    <div class="hint-text">
                        Patrons will be prompted to study this title first before tackling this material.
                    </div>
                </div>

                <!-- Next-Step Select -->
                <div class="form-group">
                    <label class="form-label" for="next_book_id">
                        Next-Step Book (Study After This)
                    </label>
                    <select name="next_book_id" id="next_book_id" class="form-input" style="cursor: pointer;
    | ">
                        <option value="">-- None (Advanced Leaf / Standalone Title) --</option>
                        {% for b in all_books %}
                            {% if b.book_id != book.book_id %}
                            <option value="{{ b.book_id }}" data-category-id="{{ b.category_id }}" {% if boo
    | k.current_next_id == b.book_id %}selected{% endif %}>
                                [{{ b.category_name }} · Level {{ b.level }}] {{ b.title }}
```

```
                            </option>
                            {% endif %}
                        {% endfor %}
                    </select>
                    <div class="hint-text">
                        Unlocks upon borrowing; suggested as immediate next step in curriculum.
                    </div>
                </div>
            </div>

            <div class="form-group" style="margin-bottom: 0;">
                <label class="form-label" for="curriculum_track">Curriculum Track Label (Optional)</label>
                <select name="curriculum_track" id="curriculum_track" class="form-input" style="cursor: poin
    | ter; max-width: 400px;">
                    <option value="auto" {% if not book.current_track or book.current_track == 'auto' %}sele
    | cted{% endif %}>Auto-Match Category</option>
                    <option value="Software Engineering" {% if book.current_track == 'Software Engineering'
    | %}selected{% endif %}>Software Engineering</option>
                    <option value="Artificial Intelligence" {% if book.current_track == 'Artificial Intellig
    | ence' %}selected{% endif %}>Artificial Intelligence</option>
                    <option value="Data Engineering" {% if book.current_track == 'Data Engineering' %}select
    | ed{% endif %}>Data Engineering</option>
                    <option value="Systems & Infrastructure" {% if book.current_track == 'Systems & Infrastr
    | ucture' %}selected{% endif %}>Systems & Infrastructure</option>
                    <option value="Theoretical CS" {% if book.current_track == 'Theoretical CS' %}selected{%
    |  endif %}>Theoretical CS</option>
                </select>
                <div class="hint-text">
                    Assigns specialization track metadata for the Academic Learning Roadmaps view.
                </div>
            </div>
        </div>

        <div style="display: flex; gap: 14px; justify-content: flex-end; margin-bottom: 40px;">
            <a href="{{ url_for('category_books', category_id=book.category_id) }}" class="btn btn-secondary
    | ">Cancel</a>
            <button type="submit" class="btn btn-primary" style="padding: 12px 24px;">
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
    | th="2"><path d="M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2 2 0 0 1-2 2z"></path><polyline poin
    | ts="17 21 17 13 7 13 7 21"></polyline><polyline points="7 3 7 8 15 8"></polyline></svg>
                Save Changes &amp;amp; Update Graph
            </button>
        </div>
    </form>
</div>
{% endblock %}

{% block extra_js %}
<script>
document.addEventListener("DOMContentLoaded", function() {
    function reorderCategoryBooks(selectElem, currentCatId) {
        if (!selectElem) return;
        const currentVal = selectElem.value;
        const options = Array.from(selectElem.querySelectorAll("option"));
        if (options.length <= 2) return;

        const headerOpts = [];
        const matchingOpts = [];
        const otherOpts = [];

        options.forEach(opt => {
            const val = opt.value;
            if (val === "" || val === "auto") {
                headerOpts.push(opt);
            } else {
                const catId = opt.getAttribute("data-category-id");
                let baseTitle = opt.getAttribute("data-base-title");
                if (!baseTitle) {
                    baseTitle = opt.textContent.trim();
                    opt.setAttribute("data-base-title", baseTitle);
                }

                if (currentCatId && String(catId) === String(currentCatId)) {
                    opt.textContent = "■ " + baseTitle;
                    matchingOpts.push(opt);
                } else {
                    opt.textContent = baseTitle;
                    otherOpts.push(opt);
                }
            }
```

```
0285:         });
0286:
0287:         selectElem.innerHTML = "";
0288:         headerOpts.forEach(opt => selectElem.appendChild(opt));
0289:
0290:         if (matchingOpts.length > 0) {
0291:             const grpMatch = document.createElement("optgroup");
0292:             grpMatch.label = "■■ Titles in Selected Category ■■";
0293:             matchingOpts.forEach(opt => grpMatch.appendChild(opt));
0294:             selectElem.appendChild(grpMatch);
0295:         }
0296:
0297:         if (otherOpts.length > 0) {
0298:             const grpOther = document.createElement("optgroup");
0299:             grpOther.label = "■■ Other Curriculum Categories ■■";
0300:             otherOpts.forEach(opt => grpOther.appendChild(opt));
0301:             selectElem.appendChild(grpOther);
0302:         }
0303:
0304:         if (currentVal) {
0305:             selectElem.value = currentVal;
0306:         }
0307:     }
0308:
0309:     const catSelect = document.getElementById("category_id");
0310:     const prereqSelect = document.getElementById("prereq_book_id");
0311:     const nextSelect = document.getElementById("next_book_id");
0312:
0313:     if (catSelect) {
0314:         catSelect.addEventListener("change", function() {
0315:             reorderCategoryBooks(prereqSelect, this.value);
0316:             reorderCategoryBooks(nextSelect, this.value);
0317:         });
0318:         if (catSelect.value) {
0319:             reorderCategoryBooks(prereqSelect, catSelect.value);
0320:             reorderCategoryBooks(nextSelect, catSelect.value);
0321:         }
0322:     }
0323: });
0324: </script>
0325: {% endblock %}
```

MODULE SOURCE: templates/learning_paths.html

Lines: 896 | **Lang:** HTML5 / Jinja2 | **Category:** Academic Pathways

Core Role & Purpose: Curated academic book tracks and sequential learning paths.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify milestone progression and prerequisite reading trail links.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Academic Learning Roadmaps | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}Academic Learning Roadmaps{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     .roadmap-container {
0009:         display: flex;
0010:         flex-direction: column;
0011:         gap: 22px;
0012:         padding-bottom: 35px;
0013:     }
0014:
0015:     /* Executive Header */
0016:     .roadmap-header-card {
0017:         background: var(--bg-surface);
0018:         border: 1px solid var(--border-subtle);
0019:         border-radius: var(--radius-lg);
0020:         padding: 24px;
0021:         display: flex;
0022:         align-items: center;
0023:         justify-content: space-between;
0024:         flex-wrap: wrap;
0025:         gap: 16px;
0026:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0027:     }
0028:
0029:     .roadmap-header-info h1 {
0030:         font-size: 1.45rem;
0031:         font-weight: 800;
0032:         letter-spacing: -0.02em;
0033:         color: var(--text-primary);
0034:         display: flex;
0035:         align-items: center;
0036:         gap: 10px;
0037:     }
0038:
0039:     .roadmap-header-info p {
0040:         font-size: 0.85rem;
0041:         color: var(--text-secondary);
0042:         margin-top: 4px;
0043:         max-width: 720px;
0044:         line-height: 1.5;
0045:     }
0046:
0047:     /* Metric Grid */
0048:     .roadmap-stats-grid {
0049:         display: grid;
0050:         grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
0051:         gap: 16px;
0052:     }
```

```
0053:
0054:     .roadmap-stat-card {
0055:         background: var(--bg-surface);
0056:         border: 1px solid var(--border-subtle);
0057:         border-radius: var(--radius-lg);
0058:         padding: 16px 20px;
0059:         display: flex;
0060:         flex-direction: column;
0061:         gap: 4px;
0062:         transition: transform 0.2s, border-color 0.2s;
0063:     }
0064:
0065:     .roadmap-stat-card:hover {
0066:         transform: translateY(-2px);
0067:         border-color: rgba(99, 102, 241, 0.4);
0068:     }
0069:
0070:     .roadmap-stat-label {
0071:         font-size: 0.74rem;
0072:         font-weight: 700;
0073:         text-transform: uppercase;
0074:         letter-spacing: 0.05em;
0075:         color: var(--text-muted);
0076:         display: flex;
0077:         align-items: center;
0078:         gap: 6px;
0079:     }
0080:
```

```
0081:     .roadmap-stat-val {
0082:         font-size: 1.65rem;
0083:         font-weight: 800;
0084:         letter-spacing: -0.02em;
0085:         color: var(--text-primary);
0086:     }
0087:
0088:     .roadmap-stat-sub {
0089:         font-size: 0.72rem;
0090:         color: var(--text-secondary);
0091:     }
0092:
0093:     /* Main Workspace: Vis.js Canvas + Node Inspector Drawer */
0094:     .roadmap-workspace {
0095:         display: grid;
0096:         grid-template-columns: 1fr 340px;
0097:         gap: 20px;
0098:         align-items: start;
0099:     }
0100:
0101:     @media (max-width: 1080px) {
0102:         .roadmap-workspace {
0103:             grid-template-columns: 1fr;
0104:         }
0105:     }
0106:
0107:     .roadmap-canvas-card {
0108:         background: var(--bg-surface);
0109:         border: 1px solid var(--border-subtle);
0110:         border-radius: var(--radius-lg);
0111:         overflow: hidden;
0112:         box-shadow: 0 4px 24px rgba(0, 0, 0, 0.35);
0113:         display: flex;
0114:         flex-direction: column;
0115:     }
0116:
0117:     .canvas-toolbar {
0118:         padding: 14px 20px;
0119:         background: rgba(17, 24, 39, 0.75);
0120:         border-bottom: 1px solid var(--border-subtle);
0121:         display: flex;
0122:         align-items: center;
0123:         justify-content: space-between;
0124:         flex-wrap: wrap;
0125:         gap: 12px;
0126:     }
0127:
0128:     .track-pills {
0129:         display: flex;
0130:         align-items: center;
0131:         gap: 8px;
0132:         flex-wrap: wrap;
0133:     }
0134:
0135:     .track-pill-btn {
0136:         background: rgba(255, 255, 255, 0.05);
0137:         border: 1px solid var(--border-subtle);
0138:         color: var(--text-secondary);
0139:         padding: 5px 12px;
0140:         border-radius: var(--radius-full);
0141:         font-size: 0.75rem;
0142:         font-weight: 600;
0143:         cursor: pointer;
0144:         transition: all 0.15s ease;
0145:     }
0146:
0147:     .track-pill-btn:hover {
0148:         background: rgba(255, 255, 255, 0.1);
0149:         color: var(--text-primary);
0150:     }
0151:
0152:     .track-pill-btn.active {
0153:         background: var(--accent-primary);
0154:         border-color: var(--accent-primary);
0155:         color: #FFFFFF;
0156:         box-shadow: 0 0 12px rgba(99, 102, 241, 0.4);
0157:     }
0158:
0159:     .canvas-actions {
0160:         display: flex;
```

```
0161:         align-items: center;
0162:         gap: 8px;
0163:     }
0164:
0165:     .tool-btn {
0166:         background: rgba(255, 255, 255, 0.05);
0167:         border: 1px solid var(--border-subtle);
0168:         color: var(--text-secondary);
0169:         padding: 5px 10px;
0170:         border-radius: var(--radius-md);
0171:         font-size: 0.75rem;
0172:         font-weight: 600;
0173:         cursor: pointer;
0174:         display: inline-flex;
0175:         align-items: center;
0176:         gap: 5px;
0177:         transition: all 0.15s ease;
0178:     }
0179:
0180:     .tool-btn:hover {
0181:         background: rgba(255, 255, 255, 0.1);
0182:         color: var(--text-primary);
0183:     }
0184:
0185:     #roadmap-canvas {
0186:         height: 540px;
0187:         width: 100%;
0188:         background: radial-gradient(circle at 50% 50%, #111827 0%, #090D16 100%);
0189:         cursor: grab;
0190:     }
0191:
0192:     #roadmap-canvas:active {
0193:         cursor: grabbing;
0194:     }
0195:
0196:     .canvas-legend-footer {
0197:         padding: 12px 20px;
0198:         background: rgba(17, 24, 39, 0.7);
0199:         border-top: 1px solid var(--border-subtle);
0200:         display: flex;
0201:         align-items: center;
0202:         justify-content: space-between;
0203:         flex-wrap: wrap;
0204:         gap: 12px;
0205:         font-size: 0.75rem;
0206:         color: var(--text-secondary);
0207:     }
0208:
0209:     .legend-stages {
0210:         display: flex;
0211:         align-items: center;
0212:         flex-wrap: wrap;
0213:         gap: 14px;
0214:     }
0215:
0216:     .stage-item {
0217:         display: inline-flex;
0218:         align-items: center;
0219:         gap: 6px;
0220:     }
0221:
0222:     .stage-dot {
0223:         width: 8px;
0224:         height: 8px;
0225:         border-radius: 50%;
0226:     }
0227:
0228:     /* Node Inspector Drawer */
0229:     .node-inspector-card {
0230:         background: var(--bg-surface);
0231:         border: 1px solid var(--border-subtle);
0232:         border-radius: var(--radius-lg);
0233:         padding: 20px;
0234:         display: flex;
0235:         flex-direction: column;
0236:         gap: 16px;
0237:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0238:     }
0239:
0240:     .inspector-header {
```

```
0241:         display: flex;
0242:         align-items: center;
0243:         justify-content: space-between;
0244:         padding-bottom: 12px;
0245:         border-bottom: 1px solid var(--border-subtle);
0246:     }
0247:
0248:     .inspector-header h3 {
0249:         font-size: 0.95rem;
0250:         font-weight: 700;
0251:         color: var(--text-primary);
0252:         display: flex;
0253:         align-items: center;
0254:         gap: 8px;
0255:     }
0256:
0257:     .inspector-body {
0258:         display: flex;
0259:         flex-direction: column;
0260:         gap: 14px;
0261:     }
0262:
0263:     .inspector-book-title {
0264:         font-size: 1.1rem;
0265:         font-weight: 800;
0266:         color: var(--text-primary);
0267:         line-height: 1.35;
0268:     }
0269:
0270:     .stock-badge-live {
0271:         display: inline-flex;
0272:         align-items: center;
0273:         gap: 6px;
0274:         font-size: 0.75rem;
0275:         font-weight: 700;
0276:         padding: 4px 10px;
0277:         border-radius: var(--radius-full);
0278:     }
0279:
0280:     .stock-badge-live.avail {
0281:         background: rgba(16, 185, 129, 0.15);
0282:         color: #34D399;
0283:         border: 1px solid rgba(16, 185, 129, 0.35);
0284:     }
0285:
0286:     .stock-badge-live.out {
0287:         background: rgba(239, 68, 68, 0.15);
0288:         color: #F87171;
0289:         border: 1px solid rgba(239, 68, 68, 0.35);
0290:     }
0291:
0292:     .inspector-meta-row {
0293:         display: flex;
0294:         justify-content: space-between;
0295:         align-items: center;
0296:         font-size: 0.78rem;
0297:         padding: 4px 0;
0298:         border-bottom: 1px solid rgba(255, 255, 255, 0.03);
0299:     }
0300:
0301:     .inspector-meta-label {
0302:         color: var(--text-muted);
0303:     }
0304:
0305:     .inspector-meta-val {
0306:         color: var(--text-primary);
0307:         font-weight: 600;
0308:     }
0309:
0310:     .relation-section-title {
0311:         font-size: 0.72rem;
0312:         font-weight: 700;
0313:         text-transform: uppercase;
0314:         letter-spacing: 0.05em;
0315:         color: var(--text-muted);
0316:         margin-top: 4px;
0317:         margin-bottom: 6px;
0318:     }
0319:
0320:     .relation-pill-list {
```

```
0321:         display: flex;
0322:         flex-direction: column;
0323:         gap: 6px;
0324:     }
0325:
0326:     .relation-pill-item {
0327:         background: var(--bg-surface-soft);
0328:         border: 1px solid rgba(255, 255, 255, 0.04);
0329:         border-radius: var(--radius-sm);
0330:         padding: 6px 10px;
0331:         font-size: 0.76rem;
0332:         color: var(--text-secondary);
0333:         display: flex;
0334:         align-items: center;
0335:         justify-content: space-between;
0336:         cursor: pointer;
0337:         transition: border-color 0.15s, background 0.15s;
0338:     }
0339:
0340:     .relation-pill-item:hover {
0341:         border-color: var(--accent-primary);
0342:         background: rgba(99, 102, 241, 0.1);
0343:         color: #FFFFFF;
0344:     }
0345:
0346:     /* Sequential Paths Table/Accordion */
0347:     .paths-card {
0348:         background: var(--bg-surface);
0349:         border: 1px solid var(--border-subtle);
0350:         border-radius: var(--radius-lg);
0351:         padding: 22px;
0352:         display: flex;
0353:         flex-direction: column;
0354:         gap: 16px;
0355:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0356:     }
0357:
0358:     .paths-header h2 {
0359:         font-size: 1.05rem;
0360:         font-weight: 800;
0361:         color: var(--text-primary);
0362:         display: flex;
0363:         align-items: center;
0364:         gap: 8px;
0365:     }
0366:
0367:     .path-row-item {
0368:         background: var(--bg-surface-soft);
0369:         border: 1px solid rgba(255, 255, 255, 0.04);
0370:         border-radius: var(--radius-md);
0371:         padding: 12px 16px;
0372:         display: flex;
0373:         align-items: center;
0374:         justify-content: space-between;
0375:         flex-wrap: wrap;
0376:         gap: 12px;
0377:         transition: border-color 0.2s;
0378:     }
0379:
0380:     .path-row-item:hover {
0381:         border-color: rgba(99, 102, 241, 0.35);
0382:     }
0383:
0384:     .path-nodes-chain {
0385:         display: flex;
0386:         align-items: center;
0387:         flex-wrap: wrap;
0388:         gap: 8px;
0389:     }
0390:
0391:     .chain-book-pill {
0392:         background: rgba(255, 255, 255, 0.05);
0393:         border: 1px solid var(--border-subtle);
0394:         padding: 4px 10px;
0395:         border-radius: var(--radius-sm);
0396:         font-size: 0.78rem;
0397:         font-weight: 600;
0398:         color: var(--text-primary);
0399:     }
0400:
```

```
0401:     .chain-arrow {
0402:         color: var(--accent-primary);
0403:         font-weight: 800;
0404:         font-size: 0.85rem;
0405:     }
0406:
0407:     .chain-depth-badge {
0408:         font-size: 0.72rem;
0409:         font-weight: 700;
0410:         color: #93C5FD;
0411:         background: rgba(59, 130, 246, 0.15);
0412:         border: 1px solid rgba(59, 130, 246, 0.3);
0413:         padding: 3px 8px;
0414:         border-radius: var(--radius-full);
0415:     }
0416:
0417:     .track-specialization-badge {
0418:         font-size: 0.72rem;
0419:         font-weight: 700;
0420:         text-transform: uppercase;
0421:         letter-spacing: 0.04em;
0422:         padding: 3px 9px;
0423:         border-radius: var(--radius-full);
0424:         display: inline-flex;
0425:         align-items: center;
0426:         gap: 5px;
0427:     }
0428: </style>
0429: {% endblock %}
0430:
0431: {% block content %}
0432: <div class="roadmap-container">
0433:     <!-- Executive Top Header -->
0434:     <div class="roadmap-header-card">
0435:         <div class="roadmap-header-info">
0436:             <h1>
0437:                 <svg viewBox="0 0 24 24" width="24" height="24" fill="none" stroke="#818CF8" stroke-width="2
      | "><polygon points="12 2 2 7 12 12 22 7 12 2"></polygon><polyline points="2 17 12 22 22 17"></polyline>
      | <polyline points="2 12 12 17 22 12"></polyline></svg>
0438:                 Academic Learning Roadmaps &amp;amp; Prerequisite Hierarchy
0439:             </h1>
0440:             <p style="font-size: 0.85rem; color: var(--text-secondary); margin-top: 4px;">
0441:                 {% if session.get('role') in ['Admin', 'Librarian'] %}
0442:                 Structured reading pathways from foundational textbooks to advanced specialized topics with
      | real-time copy availability.
0443:                 {% else %}
0444:                 Structured reading pathways guiding you from foundational textbooks to advanced specialized
      | topics with real-time copy availability.
0445:                 {% endif %}
0446:             </p>
0447:         </div>
0448:         <div style="display: flex; gap: 10px; align-items: center;">
0449:             <a href="{{ url_for('book_bridge') }}" class="btn btn-secondary">
0450:                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6"></path><polyline points="15
      | 3 21 3 21 9"></polyline><line x1="10" y1="14" x2="21" y2="3"></line></svg>
0451:                 Book Bridge
0452:             </a>
0453:             <a href="{{ url_for('books') }}" class="btn btn-primary">
0454:                 Browse Full Catalog
0455:             </a>
0456:         </div>
0457:     </div>
0458:
0459:     <!-- Live Curriculum KPI Grid -->
0460:     <div class="roadmap-stats-grid">
0461:         <div class="roadmap-stat-card">
0462:             <span class="roadmap-stat-label">
0463:                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0464:                 Curriculum Volumes
0465:             </span>
0466:             <span class="roadmap-stat-val">{{ stats.total_volumes }}</span>
0467:             <span class="roadmap-stat-sub">Core academic textbooks linked</span>
0468:         </div>
0469:         <div class="roadmap-stat-card">
0470:             <span class="roadmap-stat-label">
0471:                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="#34D399" stroke-width="2
      | "><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyline></svg>
0472:                 Real-Time Available
```

```
0473:             </span>
0474:             <span class="roadmap-stat-val" style="color: #34D399;">{{ stats.available_volumes }} / {{ stats.
      | total_volumes }}</span>
0475:             <span class="roadmap-stat-sub">Titles with copies on shelf</span>
0476:         </div>
0477:         <div class="roadmap-stat-card">
0478:             <span class="roadmap-stat-label">
0479:                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="#818CF8" stroke-width="2
      | "><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18 21.02 12 17.77 5.82 21.02 7 14.14 2 9.27 8.9
      | 1 8.26 12 2"></polygon></svg>
0480:                 Learning Pathways
0481:             </span>
0482:             <span class="roadmap-stat-val" style="color: #818CF8;">{{ stats.total_paths }}</span>
0483:             <span class="roadmap-stat-sub">Traversable curriculum sequences</span>
0484:         </div>
0485:         <div class="roadmap-stat-card">
0486:             <span class="roadmap-stat-label">
0487:                 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="#F59E0B" stroke-width="2
      | "><line x1="18" y1="20" x2="18" y2="10"></line><line x1="12" y1="20" x2="12" y2="4"></line><line x1="6
      | " y1="20" x2="6" y2="14"></line></svg>
0488:                 Max Academic Depth
0489:             </span>
0490:             <span class="roadmap-stat-val" style="color: #F59E0B;">{{ stats.max_depth }} Levels</span>
0491:             <span class="roadmap-stat-sub">Topological hierarchy depth</span>
0492:         </div>
0493:     </div>
0494:
0495:     <!-- Main Workspace (Interactive Top-to-Bottom Tree Canvas + Node Inspector) -->
0496:     <div class="roadmap-workspace">
0497:         <!-- Vis.js Hierarchical Graph Canvas Card -->
0498:         <div class="roadmap-canvas-card">
0499:             <div class="canvas-toolbar">
0500:                 <div class="track-pills">
0501:                     <button class="track-pill-btn active" onclick="filterTrack('all', this)">All Curriculum
      | Tracks</button>
0502:                     <button class="track-pill-btn" onclick="filterTrack('software_ai', this)">Software &amp;
      |  AI</button>
0503:                     <button class="track-pill-btn" onclick="filterTrack('data', this)">Data &amp; Databases<
      | /button>
0504:                     <button class="track-pill-btn" onclick="filterTrack('systems', this)">Systems &amp; Netw
      | orks</button>
0505:                 </div>
0506:                 <div class="canvas-actions">
0507:                     <button class="tool-btn" onclick="fitRoadmapGraph()" title="Recenter and fit roadmap tre
      | e">
0508:                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
      | roke-width="2"><path d="M15 3h6v6"></path><path d="M9 21H3v-6"></path><path d="M21 3l-7 7"></path><pat
      | h d="M3 21l7-7"></path></svg>
0509:                         Fit View
0510:                     </button>
0511:                     <button class="tool-btn" onclick="toggleLayoutDirection()" id="btn-layout-toggle" title=
      | "Toggle between Hierarchical and Free Physics layout">
0512:                         <svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColor" st
      | roke-width="2"><polyline points="7 13 12 18 17 13"></polyline><polyline points="7 6 12 11 17 6"></poly
      | line></svg>
0513:                         Hierarchical (UD)
0514:                     </button>
0515:                 </div>
0516:             </div>
0517:
0518:             <div id="roadmap-canvas"></div>
0519:
0520:             <div class="canvas-legend-footer">
0521:                 <div class="legend-stages">
0522:                     <span class="stage-item"><span class="stage-dot" style="background: #3B82F6;"></span> Le
      | vel 1 (Foundational)</span>
0523:                     <span class="stage-item"><span class="stage-dot" style="background: #8B5CF6;"></span> Le
      | vel 2 (Core CS)</span>
0524:                     <span class="stage-item"><span class="stage-dot" style="background: #EC4899;"></span> Le
      | vel 3 (Specialization)</span>
0525:                     <span class="stage-item"><span class="stage-dot" style="background: #F59E0B;"></span> Le
      | vel 4 (Advanced Mastery)</span>
0526:                     <span class="stage-item"><span class="stage-dot" style="background: #10B981;"></span> ✓
      | Mastered</span>
0527:                     <span class="stage-item"><span class="stage-dot" style="background: #38BDF8;"></span> ■
      | In Progress</span>
0528:                     <span class="stage-item"><span class="stage-dot" style="background: #34D399;"></span> Av
      | ailable Stock</span>
0529:                     <span class="stage-item"><span class="stage-dot" style="background: #EF4444;"></span> Ou
      | t of Stock</span>
```

```
0530:                 </div>
0531:                 <div>
0532:                     <span style="font-family: var(--font-mono); color: var(--text-muted);">Curriculum Graph
      | Engine</span>
0533:                 </div>
0534:             </div>
0535:         </div>
0536:
0537:         <!-- Node Inspector Drawer -->
0538:         <div class="node-inspector-card" id="node-inspector">
0539:             <div class="inspector-header">
0540:                 <h3>
0541:                     <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="#818CF8" stroke-widt
      | h="2"><circle cx="12" cy="12" r="10"></circle><line x1="12" y1="16" x2="12" y2="12"></line><line x1="1
      | 2" y1="8" x2="12.01" y2="8"></line></svg>
0542:                     Textbook Detail &amp;amp; Prerequisites
0543:                 </h3>
0544:             </div>
0545:
0546:             <div class="inspector-body" id="inspector-content">
0547:                 <div style="text-align: center; color: var(--text-muted); font-size: 0.85rem; padding: 30px
      | 10px;">
0548:                     <svg width="32" height="32" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="1.5" style="margin-bottom: 8px; opacity: 0.6;"><circle cx="12" cy="12" r="10"></circle><polyli
      | ne points="12 6 12 12 14 14"></polyline></svg>
0549:                     <p>Click any node in the roadmap canvas to inspect its prerequisites, unlocked volumes,
      | and physical stock availability.</p>
0550:                 </div>
0551:             </div>
0552:         </div>
0553:     </div>
0554:
0555:     <!-- Discovered Learning Pathways Sequential List -->
0556:     <div class="paths-card">
0557:         <div class="paths-header">
0558:             <h2>
0559:                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="#818CF8" stroke-width="2
      | "><polyline points="22 12 18 12 15 21 9 3 6 12 2 12"></polyline></svg>
0560:                 Traversable Academic Learning Sequences (Variable-Length Paths)
0561:             </h2>
0562:         </div>
0563:
0564:         <div style="display: flex; flex-direction: column; gap: 10px;">
0565:             {% for p in paths %}
0566:             <div class="path-row-item">
0567:                 <div style="display: flex; flex-direction: column; gap: 8px; flex: 1;">
0568:                     <div style="display: flex; align-items: center; gap: 8px; flex-wrap: wrap;">
0569:                         <span class="track-specialization-badge" style="background: rgba(255, 255, 255, 0.04
      | ); color: {{ p.track_color or '#818CF8' }}; border: 1px solid {{ p.track_color or '#818CF8' }}55;">
0570:                             {{ p.track_badge or 'Academic Track' }}
0571:                         </span>
0572:                         <span class="chain-depth-badge">{{ p.depth }} Hop{{ 's' if p.depth > 1 else '' }} Pr
      | ogression</span>
0573:                     </div>
0574:                     <div class="path-nodes-chain">
0575:                         {% for n in p.nodes_in_path %}
0576:                             <span class="chain-book-pill" onclick="focusBookNode({{ n.id }})" style="cursor:
      |  pointer;" title="Click to highlight on graph">{{ n.title }}</span>
0577:                             {% if not loop.last %}
0578:                                 <span class="chain-arrow">&rarr;</span>
0579:                             {% endif %}
0580:                         {% endfor %}
0581:                     </div>
0582:                 </div>
0583:             </div>
0584:             {% endfor %}
0585:         </div>
0586:     </div>
0587: </div>
0588:
0589: <!-- Vis.js Network Script -->
0590: <script src="https://unpkg.com/vis-network/standalone/umd/vis-network.min.js"></script>
0591: <script>
0592:     const rawNodes = {{ vis_nodes|tojson }};
0593:     const rawEdges = {{ vis_edges|tojson }};
0594:     const bookNodesMap = {};
0595:     rawNodes.forEach(n => {
0596:         bookNodesMap[n.id] = n.properties;
0597:     });
0598:
```

```
    let network = null;
    let isHierarchical = true;

    document.addEventListener("DOMContentLoaded", function() {
        initRoadmapNetwork(rawNodes, rawEdges);
        // Default select the first foundational book (Clean Code or DB Concepts)
        const rootNode = rawNodes.find(n => n.level === 1);
        if (rootNode) {
            inspectNode(rootNode.id);
        }
    });

    function initRoadmapNetwork(nodesList, edgesList) {
        const container = document.getElementById("roadmap-canvas");
        if (!container) return;

        // Safely destroy existing network instance to avoid memory leak and stale canvases
        if (network) {
            network.destroy();
            network = null;
        }

        const data = {
            nodes: new vis.DataSet(nodesList),
            edges: new vis.DataSet(edgesList)
        };

        const options = {
            layout: {
                hierarchical: {
                    enabled: true,
                    direction: "UD", // Top-to-Bottom
                    sortMethod: "directed",
                    levelSeparation: 130,
                    nodeSpacing: 220,
                    treeSpacing: 240
                }
            },
            physics: {
                hierarchicalRepulsion: {
                    nodeDistance: 220
                }
            },
            nodes: {
                font: {
                    size: 15,
                    face: "Inter, sans-serif",
                    color: "#F8FAFC"
                }
            },
            interaction: {
                hover: true,
                navigationButtons: false,
                keyboard: false,
                zoomView: true,
                dragView: true
            }
        };

        network = new vis.Network(container, data, options);

        network.once("afterDrawing", function() {
            network.fit({
                padding: 40,
                animation: { duration: 600, easingFunction: 'easeInOutQuad' }
            });
        });

        network.on("click", function(params) {
            if (params.nodes.length > 0) {
                const selectedId = params.nodes[0];
                inspectNode(selectedId);
            }
        });
    }

    // Responsive auto-fit on window resize
    window.addEventListener("resize", function() {
        if (network) {
            network.fit({ padding: 40 });
```

```
0679:         }
0680:     });
0681: 
0682:     function fitRoadmapGraph() {
0683:         if (network) {
0684:             network.fit({
0685:                 padding: 40,
0686:                 animation: { duration: 600, easingFunction: 'easeInOutQuad' }
0687:             });
0688:         }
0689:     }
0690: 
0691:     function toggleLayoutDirection() {
0692:         if (!network) return;
0693:         isHierarchical = !isHierarchical;
0694:         const btn = document.getElementById("btn-layout-toggle");
0695: 
0696:         if (isHierarchical) {
0697:             network.setOptions({
0698:                 layout: {
0699:                     hierarchical: {
0700:                         enabled: true,
0701:                         direction: "UD",
0702:                         sortMethod: "directed",
0703:                         levelSeparation: 130,
0704:                         nodeSpacing: 220
0705:                     }
0706:                 }
0707:             });
0708:             btn.innerHTML = `<svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColo
      | r" stroke-width="2"><polyline points="7 13 12 18 17 13"></polyline><polyline points="7 6 12 11 17 6"><
      | /polyline></svg> Hierarchical (UD)`;
0709:         } else {
0710:             network.setOptions({
0711:                 layout: {
0712:                     hierarchical: { enabled: false }
0713:                 },
0714:                 physics: {
0715:                     solver: 'forceAtlas2Based',
0716:                     forceAtlas2Based: {
0717:                         gravitationalConstant: -70,
0718:                         centralGravity: 0.015,
0719:                         springLength: 160
0720:                     }
0721:                 }
0722:             });
0723:             btn.innerHTML = `<svg width="13" height="13" viewBox="0 0 24 24" fill="none" stroke="currentColo
      | r" stroke-width="2"><circle cx="12" cy="12" r="10"></circle><line x1="2" y1="12" x2="22" y2="12"></lin
      | e><path d="M12 2a15.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-4 10 15.3 15.3 0 0 1-4-10 15.3 15.3 0 0 1 4-10z"
      | ></path></svg> Free Graph`;
0724:         }
0725:         setTimeout(fitRoadmapGraph, 300);
0726:     }
0727: 
0728:     function filterTrack(trackKey, btnElement) {
0729:         document.querySelectorAll(".track-pill-btn").forEach(b => b.classList.remove("active"));
0730:         if (btnElement) btnElement.classList.add("active");
0731: 
0732:         if (trackKey === "all") {
0733:             initRoadmapNetwork(rawNodes, rawEdges);
0734:             const rootNode = rawNodes.find(n => n.level === 1);
0735:             if (rootNode) inspectNode(rootNode.id);
0736:             return;
0737:         }
0738: 
0739:         // Filter edges matching track key
0740:         const matchingEdges = rawEdges.filter(e => {
0741:             const tr = (e.properties && e.properties.track ? e.properties.track : "").toLowerCase();
0742:             if (trackKey === "software_ai") {
0743:                 return tr.includes("software") || tr.includes("intelligence") || tr.includes("theoretical");
0744:             } else if (trackKey === "data") {
0745:                 return tr.includes("data");
0746:             } else if (trackKey === "systems") {
0747:                 return tr.includes("systems") || tr.includes("infrastructure");
0748:             }
0749:             return tr.includes(trackKey.toLowerCase());
0750:         });
0751: 
0752:         // Collect matching nodes
0753:         const activeNodeIds = new Set();
```

```
0754:         matchingEdges.forEach(e => {
0755:             activeNodeIds.add(e.from);
0756:             activeNodeIds.add(e.to);
0757:         });
0758:
0759:         const filteredNodes = rawNodes.filter(n => activeNodeIds.has(n.id));
0760:         initRoadmapNetwork(filteredNodes, matchingEdges);
0761:
0762:         // Select the earliest level node of this track for inspector
0763:         if (filteredNodes.length > 0) {
0764:             const sortedNodes = [...filteredNodes].sort((a, b) => a.level - b.level);
0765:             inspectNode(sortedNodes[0].id);
0766:         }
0767:         setTimeout(fitRoadmapGraph, 250);
0768:     }
0769:
0770:     function focusBookNode(bookId) {
0771:         if (!network) return;
0772:         network.selectNodes([bookId]);
0773:         network.focus(bookId, {
0774:             scale: 1.15,
0775:             animation: { duration: 600, easingFunction: 'easeInOutQuad' }
0776:         });
0777:         inspectNode(bookId);
0778:     }
0779:
0780:     function inspectNode(bookId) {
0781:         const book = bookNodesMap[bookId];
0782:         const container = document.getElementById("inspector-content");
0783:         if (!book || !container) return;
0784:
0785:         const isAvail = book.is_available;
0786:         const stockClass = isAvail ? 'avail' : 'out';
0787:         const stockText = isAvail
0788:             ? `✓ ${book.available_copies} of ${book.total_copies} Copies Available`
0789:             : `✗ Out of Stock (${book.total_copies} Checked Out)`;
0790:
0791:         let userStatusHtml = '';
0792:         if (book.user_status === 'completed') {
0793:             userStatusHtml = `
0794:                 <div style="background: rgba(16, 185, 129, 0.12); border: 1px solid rgba(16, 185, 129, 0.35)
      | ; border-radius: var(--radius-sm); padding: 8px 12px; display: flex; align-items: center; gap: 8px; ma
      | rgin-top: 8px;">
0795:                     <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="#34D399" stroke-widt
      | h="2.5"><polyline points="20 6 9 17 4 12"></polyline></svg>
0796:                     <span style="font-size: 0.78rem; font-weight: 700; color: #34D399;">You have read &amp;
      | mastered this volume</span>
0797:                 </div>
0798:             `;
0799:         } else if (book.user_status === 'reading') {
0800:             userStatusHtml = `
0801:                 <div style="background: rgba(56, 189, 248, 0.12); border: 1px solid rgba(56, 189, 248, 0.35)
      | ; border-radius: var(--radius-sm); padding: 8px 12px; display: flex; align-items: center; gap: 8px; ma
      | rgin-top: 8px;">
0802:                     <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="#38BDF8" stroke-widt
      | h="2"><circle cx="12" cy="12" r="10"></circle><polyline points="12 6 12 12 14 14"></polyline></svg>
0803:                     <span style="font-size: 0.78rem; font-weight: 700; color: #38BDF8;">Currently checked ou
      | t on active loan to you</span>
0804:                 </div>
0805:             `;
0806:         }
0807:
0808:         let prereqsHtml = '<div style="color: var(--text-muted); font-size: 0.75rem;">None (Foundational Ent
      | ry Point)</div>';
0809:         if (book.prereqs && book.prereqs.length > 0) {
0810:             prereqsHtml = book.prereqs.map(pid => {
0811:                 const pBook = bookNodesMap[pid];
0812:                 const pTitle = pBook ? pBook.title : `Book #${pid}`;
0813:                 const pCompleted = pBook && pBook.user_status === 'completed';
0814:                 const doneBadge = pCompleted
0815:                     ? '<span style="font-size: 0.68rem; font-weight: 700; color: #34D399; background: rgba(1
      | 6, 185, 129, 0.15); border: 1px solid rgba(16, 185, 129, 0.3); padding: 2px 6px; border-radius: 4px;">
      | ✓ Mastered</span>'
0816:                     : '<span style="font-size: 0.68rem; color: var(--text-muted);">Prerequisite</span>';
0817:                 return `
0818:                     <div class="relation-pill-item" onclick="focusBookNode(${pid})" title="Focus on prerequi
      | site">
0819:                         <span style="font-weight: 600;">${escapeHtml(pTitle)}</span>
0820:                         ${doneBadge}
0821:                     </div>
```

```
0822:                 `;
0823:             }).join("");
0824:         }
0825:
0826:         let unlocksHtml = '<div style="color: var(--text-muted); font-size: 0.75rem;">Terminal Advanced Topi
      | c</div>';
0827:         if (book.unlocks && book.unlocks.length > 0) {
0828:             unlocksHtml = book.unlocks.map(uid => {
0829:                 const uBook = bookNodesMap[uid];
0830:                 const uTitle = uBook ? uBook.title : `Book #${uid}`;
0831:                 return `
0832:                     <div class="relation-pill-item" onclick="focusBookNode(${uid})" title="Focus on unlocked
      |  topic">
0833:                         <span style="font-weight: 600;">${escapeHtml(uTitle)}</span>
0834:                         <span style="font-size: 0.68rem; color: #818CF8;">Next &rarr;</span>
0835:                     </div>
0836:                 `;
0837:             }).join("");
0838:         }
0839:
0840:         const targetUrl = book.category_id
0841:             ? `/category/${book.category_id}?q=${encodeURIComponent(book.isbn)}`
0842:             : `{{ url_for('books') }}`;
0843:
0844:         container.innerHTML = `
0845:             <div>
0846:                 <span class="stock-badge-live ${stockClass}">${stockText}</span>
0847:                 ${userStatusHtml}
0848:                 <h4 class="inspector-book-title" style="margin-top: 10px;">${escapeHtml(book.title)}</h4>
0849:                 <div style="font-size: 0.8rem; color: var(--text-secondary); margin-top: 4px;">by ${escapeHt
      | ml(book.authors || 'Unknown')}</div>
0850:             </div>
0851:
0852:             <div style="display: flex; flex-direction: column; gap: 4px; padding-top: 6px;">
0853:                 <div class="inspector-meta-row">
0854:                     <span class="inspector-meta-label">Curriculum Level:</span>
0855:                     <span class="inspector-meta-val" style="color: #93C5FD;">${escapeHtml(book.stage_title)}
      | </span>
0856:                 </div>
0857:                 <div class="inspector-meta-row">
0858:                     <span class="inspector-meta-label">Subject Category:</span>
0859:                     <span class="inspector-meta-val">${escapeHtml(book.category_name)}</span>
0860:                 </div>
0861:                 <div class="inspector-meta-row">
0862:                     <span class="inspector-meta-label">ISBN:</span>
0863:                     <span class="inspector-meta-val" style="font-family: var(--font-mono); font-size: 0.74re
      | m;">${escapeHtml(book.isbn)}</span>
0864:                 </div>
0865:             </div>
0866:
0867:             <div>
0868:                 <div class="relation-section-title">Requires Prerequisite Knowledge:</div>
0869:                 <div class="relation-pill-list">${prereqsHtml}</div>
0870:             </div>
0871:
0872:             <div>
0873:                 <div class="relation-section-title">Unlocks Subsequent Topics:</div>
0874:                 <div class="relation-pill-list">${unlocksHtml}</div>
0875:             </div>
0876:
0877:             <div style="padding-top: 10px;">
0878:                 <a href="${targetUrl}" class="btn btn-primary" style="width: 100%; display: flex; align-item
      | s: center; justify-content: center; gap: 6px;">
0879:                     <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke
      | -width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0880:                     View in Category / Borrow
0881:                 </a>
0882:             </div>
0883:         `;
0884:     }
0885:
0886:     function escapeHtml(text) {
0887:         if (!text) return "";
0888:         return text
0889:             .replace(/&/g, "&amp;")
0890:             .replace(/</g, "&lt;")
0891:             .replace(/>/g, "&gt;")
0892:             .replace(/"/g, "&quot;")
0893:             .replace(/'/g, "&#039;");
0894:     }
```

```
0895: </script>
0896: {% endblock %}
```

MODULE SOURCE: templates/library_analytics.html

Lines: 462 | **Lang:** HTML5 / Jinja2 |
Category: System Analytics

Core Role & Purpose: Circulation analytics and category distribution reporting interface.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify borrowing trend charts and category KPI statistics.

```
0001: {% extends "base.html" %}
0002:
0003: {% block title %}Library Centrality & Influencer Analytics | VITCC LIBRARY{% endblock %}
0004: {% block breadcrumb %}Library Centrality Analytics{% endblock %}
0005:
0006: {% block extra_css %}
0007: <style>
0008:     .analytics-container {
0009:         display: flex;
0010:         flex-direction: column;
0011:         gap: 22px;
0012:         padding-bottom: 30px;
0013:     }
0014:
0015:     /* Top Executive Header */
0016:     .analytics-header {
0017:         background: var(--bg-surface);
0018:         border: 1px solid var(--border-subtle);
0019:         border-radius: var(--radius-lg);
0020:         padding: 24px;
0021:         display: flex;
0022:         align-items: center;
0023:         justify-content: space-between;
0024:         flex-wrap: wrap;
0025:         gap: 16px;
0026:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0027:     }
0028:
0029:     .analytics-header-info h1 {
0030:         font-size: 1.45rem;
0031:         font-weight: 800;
0032:         letter-spacing: -0.02em;
0033:         color: var(--text-primary);
0034:         display: flex;
0035:         align-items: center;
0036:         gap: 10px;
0037:     }
0038:
0039:     .analytics-header-info p {
0040:         font-size: 0.85rem;
0041:         color: var(--text-secondary);
0042:         margin-top: 4px;
0043:         max-width: 680px;
0044:         line-height: 1.5;
0045:     }
0046:
0047:     /* Metric Grid */
0048:     .metric-grid {
0049:         display: grid;
0050:         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
0051:         gap: 16px;
0052:     }
0053:
0054:     .metric-card {
0055:         background: var(--bg-surface);
0056:         border: 1px solid var(--border-subtle);
0057:         border-radius: var(--radius-lg);
0058:         padding: 18px 20px;
0059:         display: flex;
0060:         flex-direction: column;
0061:         gap: 8px;
0062:         position: relative;
0063:         overflow: hidden;
0064:         transition: transform 0.2s, border-color 0.2s;
0065:     }
0066:
0067:     .metric-card:hover {
0068:         transform: translateY(-2px);
0069:         border-color: rgba(59, 130, 246, 0.4);
0070:     }
0071:
0072:     .metric-top {
0073:         display: flex;
0074:         align-items: center;
0075:         justify-content: space-between;
0076:     }
0077:
0078:     .metric-label {
0079:         font-size: 0.76rem;
0080:         font-weight: 700;
```

```
0081:         text-transform: uppercase;
0082:         letter-spacing: 0.05em;
0083:         color: var(--text-muted);
0084:     }
0085:
0086:     .metric-icon {
0087:         width: 32px;
0088:         height: 32px;
0089:         border-radius: 8px;
0090:         display: flex;
0091:         align-items: center;
0092:         justify-content: center;
0093:     }
0094:
0095:     .metric-value {
0096:         font-size: 1.85rem;
0097:         font-weight: 800;
0098:         color: var(--text-primary);
0099:         letter-spacing: -0.02em;
0100:         font-family: var(--font-sans);
0101:     }
0102:
0103:     .metric-sub {
0104:         font-size: 0.78rem;
0105:         color: var(--text-secondary);
0106:     }
0107:
0108:     /* Section Cards */
0109:     .analytics-section {
0110:         background: var(--bg-surface);
0111:         border: 1px solid var(--border-subtle);
0112:         border-radius: var(--radius-lg);
0113:         padding: 22px;
0114:         display: flex;
0115:         flex-direction: column;
0116:         gap: 16px;
0117:         box-shadow: 0 4px 20px rgba(0, 0, 0, 0.25);
0118:     }
0119:
0120:     .section-header {
0121:         display: flex;
0122:         align-items: center;
0123:         justify-content: space-between;
0124:         flex-wrap: wrap;
0125:         gap: 10px;
0126:         border-bottom: 1px solid var(--border-subtle);
0127:         padding-bottom: 12px;
0128:     }
0129:
0130:     .section-header h2 {
0131:         font-size: 1.05rem;
0132:         font-weight: 800;
0133:         color: var(--text-primary);
0134:         display: flex;
0135:         align-items: center;
0136:         gap: 8px;
0137:     }
0138:
0139:     .section-header p {
0140:         font-size: 0.8rem;
0141:         color: var(--text-muted);
0142:     }
0143:
0144:     /* Custom Table Styling */
0145:     .analytics-table {
0146:         width: 100%;
0147:         border-collapse: collapse;
0148:         font-size: 0.84rem;
0149:     }
0150:
0151:     .analytics-table th {
0152:         text-align: left;
0153:         padding: 10px 14px;
0154:         font-size: 0.72rem;
0155:         font-weight: 700;
0156:         text-transform: uppercase;
0157:         letter-spacing: 0.05em;
0158:         color: var(--text-muted);
0159:         border-bottom: 1px solid var(--border-subtle);
0160:         background: rgba(255, 255, 255, 0.02);
```

```
0161:     }
0162:
0163:     .analytics-table td {
0164:         padding: 12px 14px;
0165:         border-bottom: 1px solid rgba(255, 255, 255, 0.04);
0166:         color: var(--text-secondary);
0167:         vertical-align: middle;
0168:     }
0169:
0170:     .analytics-table tr:hover td {
0171:         background: rgba(255, 255, 255, 0.02);
0172:         color: var(--text-primary);
0173:     }
0174:
0175:     .rank-badge {
0176:         width: 26px;
0177:         height: 26px;
0178:         border-radius: 50%;
0179:         display: inline-flex;
0180:         align-items: center;
0181:         justify-content: center;
0182:         font-size: 0.75rem;
0183:         font-weight: 800;
0184:     }
0185:
0186:     .rank-1 { background: rgba(245, 158, 11, 0.2); color: #F59E0B; border: 1px solid #F59E0B; }
0187:     .rank-2 { background: rgba(156, 163, 175, 0.2); color: #9CA3AF; border: 1px solid #9CA3AF; }
0188:     .rank-3 { background: rgba(180, 83, 9, 0.2); color: #B45309; border: 1px solid #B45309; }
0189:     .rank-other { background: rgba(255, 255, 255, 0.05); color: var(--text-muted); }
0190:
0191:     /* Density Bar */
0192:     .density-bar-wrap {
0193:         width: 100%;
0194:         max-width: 180px;
0195:         background: rgba(255, 255, 255, 0.06);
0196:         border-radius: var(--radius-full);
0197:         height: 8px;
0198:         overflow: hidden;
0199:         margin-top: 4px;
0200:     }
0201:
0202:     .density-bar-fill {
0203:         height: 100%;
0204:         width: 0%;
0205:         border-radius: var(--radius-full);
0206:         background: linear-gradient(90deg, #3B82F6, #10B981);
0207:         transition: width 0.6s cubic-bezier(0.4, 0, 0.2, 1);
0208:     }
0209:
0210:     /* Grid of Two Columns */
0211:     .analytics-grid-two {
0212:         display: grid;
0213:         grid-template-columns: 1fr 1fr;
0214:         gap: 20px;
0215:     }
0216:
0217:     @media (max-width: 960px) {
0218:         .analytics-grid-two {
0219:             grid-template-columns: 1fr;
0220:         }
0221:     }
0222:
0223:
0224: </style>
0225: {% endblock %}
0226:
0227: {% block content %}
0228: <div class="analytics-container">
0229:     <!-- Header Card -->
0230:     <div class="analytics-header">
0231:         <div class="analytics-header-info">
0232:             <h1>
0233:                 <svg viewBox="0 0 24 24" width="24" height="24" fill="none" stroke="#FBBF24" stroke-width="2
      | "><line x1="18" y1="20" x2="18" y2="10"></line><line x1="12" y1="20" x2="12" y2="4"></line><line x1="6
      | " y1="20" x2="6" y2="14"></line></svg>
0234:                 Author Centrality & Graph Influencer Analytics
0235:             </h1>
0236:             <p>
0237:                 Real-time topological link density, degree centrality, and multi-hop circulation statistics
      | computed directly over the Neo4j Knowledge Graph.
```

```
0238:             </p>
0239:         </div>
0240:         <div style="display: flex; gap: 10px;">
0241:             <a href="{{ url_for('book_bridge') }}" class="btn btn-secondary">
0242:                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6"></path><polyline points="15
      | 3 21 3 21 9"></polyline><line x1="10" y1="14" x2="21" y2="3"></line></svg>
0243:                 Book Bridge
0244:             </a>
0245:             <a href="{{ url_for('graph_visualizer') }}" class="btn btn-secondary">
0246:                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
      | th="2"><circle cx="18" cy="5" r="3"></circle><circle cx="6" cy="12" r="3"></circle><circle cx="18" cy=
      | "19" r="3"></circle><line x1="8.59" y1="13.51" x2="15.42" y2="17.49"></line><line x1="15.41" y1="6.51"
      |  x2="8.59" y2="10.49"></line></svg>
0247:                 Live Graph
0248:             </a>
0249:         </div>
0250:     </div>
0251:
0252:     <!-- Top Metric Grid -->
0253:     <div class="metric-grid">
0254:         <div class="metric-card">
0255:             <div class="metric-top">
0256:                 <span class="metric-label">Knowledge Nodes</span>
0257:                 <div class="metric-icon" style="background: rgba(59, 130, 246, 0.15); color: #60A5FA;">
0258:                     <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor" stroke
      | -width="2"><circle cx="12" cy="12" r="10"></circle><circle cx="12" cy="12" r="4"></circle></svg>
0259:                 </div>
0260:             </div>
0261:             <div class="metric-value">{{ metrics.nodes_count }}</div>
0262:             <div class="metric-sub">Entities in AuraDB catalog</div>
0263:         </div>
0264:
0265:         <div class="metric-card">
0266:             <div class="metric-top">
0267:                 <span class="metric-label">Relationships</span>
0268:                 <div class="metric-icon" style="background: rgba(16, 185, 129, 0.15); color: #34D399;">
0269:                     <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor" stroke
      | -width="2"><line x1="6" y1="3" x2="6" y2="15"></line><circle cx="18" cy="6" r="3"></circle><circle cx=
      | "6" cy="18" r="3"></circle><path d="M18 9a9 9 0 0 1-9 9"></path></svg>
0270:                 </div>
0271:             </div>
0272:             <div class="metric-value">{{ metrics.rels_count }}</div>
0273:             <div class="metric-sub">Edges across 10 relationship types</div>
0274:         </div>
0275:
0276:         <div class="metric-card">
0277:             <div class="metric-top">
0278:                 <span class="metric-label">Average Degree</span>
0279:                 <div class="metric-icon" style="background: rgba(245, 158, 11, 0.15); color: #FBBF24;">
0280:                     <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor" stroke
      | -width="2"><path d="M16.5 9.4 7.55 4.24a1.78 1.78 0 0 0-2.5 1.55v12.42a1.78 1.78 0 0 0 2.5 1.55L16.5 1
      | 4.6a1.78 1.78 0 0 0 0-3.2z"></path></svg>
0281:                 </div>
0282:             </div>
0283:             <div class="metric-value">{{ metrics.avg_degree }}</div>
0284:             <div class="metric-sub">Connections per entity node</div>
0285:         </div>
0286:
0287:         <div class="metric-card">
0288:             <div class="metric-top">
0289:                 <span class="metric-label">Active Borrowers</span>
0290:                 <div class="metric-icon" style="background: rgba(168, 85, 247, 0.15); color: #C084FC;">
0291:                     <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor" stroke
      | -width="2"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></ci
      | rcle><path d="M23 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
0292:                 </div>
0293:             </div>
0294:             <div class="metric-value">{{ metrics.active_borrowers }}</div>
0295:             <div class="metric-sub">Across {{ metrics.active_loans }} active circulation loans</div>
0296:         </div>
0297:     </div>
0298:
0299:     <!-- Section 1: Author Borrowing Centrality (Primary Task Query) -->
0300:     <div class="analytics-section">
0301:         <div class="section-header">
0302:             <div>
0303:                 <h2>
0304:                     <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="#F59E0B" stroke-widt
      | h="2"><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18 21.02 12 17.77 5.82 21.02 7 14.14 2 9.27
```

```
      |  8.91 8.26 12 2"></polygon></svg>
0305:                     Top Authors by Graph Degree Centrality & Borrowing Density
0306:                 </h2>
0307:                 <p>Computed via <code>(:Author)-[:WROTE]-&gt;(:Book)-[:HAS_COPY]-&gt;(:Copy)&lt;-[:FOR_COPY]
      | -(:Loan)</code></p>
0308:             </div>
0309:             <span class="badge badge-info">Top 5 Influencers</span>
0310:         </div>
0311:
0312:         <div style="overflow-x: auto;">
0313:             <table class="analytics-table">
0314:                 <thead>
0315:                     <tr>
0316:                         <th style="width: 50px;">Rank</th>
0317:                         <th>Author Name</th>
0318:                         <th>Total Borrowings</th>
0319:                         <th>Authored Titles</th>
0320:                         <th>Active Copies</th>
0321:                         <th>Borrowing Density Share</th>
0322:                     </tr>
0323:                 </thead>
0324:                 <tbody>
0325:                     {% for auth in top_authors %}
0326:                     <tr>
0327:                         <td>
0328:                             <span class="rank-badge {% if loop.index == 1 %}rank-1{% elif loop.index == 2 %}
      | rank-2{% elif loop.index == 3 %}rank-3{% else %}rank-other{% endif %}">
0329:                                 {{ loop.index }}
0330:                             </span>
0331:                         </td>
0332:                         <td>
0333:                             <strong style="color: var(--text-primary); font-size: 0.9rem;">{{ auth.author_na
      | me }}</strong>
0334:                         </td>
0335:                         <td>
0336:                             <span style="font-family: var(--font-mono); font-weight: 700; color: #60A5FA; fo
      | nt-size: 0.95rem;">
0337:                                 {{ auth.total_borrowings }} {{ 'Loan' if auth.total_borrowings == 1 else 'Lo
      | ans' }}
0338:                             </span>
0339:                         </td>
0340:                         <td>{{ auth.authored_books }} {{ 'Book' if auth.authored_books == 1 else 'Books' }}<
      | /td>
0341:                         <td>{{ auth.active_copies }} {{ 'Copy' if auth.active_copies == 1 else 'Copies' }}</
      | td>
0342:                         <td>
0343:                             <div style="display: flex; align-items: center; gap: 8px;">
0344:                                 <div class="density-bar-wrap">
0345:                                     <div class="density-bar-fill" data-pct="{{ auth.bar_width }}"></div>
0346:                                 </div>
0347:                                 <span style="font-family: var(--font-mono); font-size: 0.72rem; color: var(-
      | -text-muted);">
0348:                                     {{ auth.share_pct }}%
0349:                                 </span>
0350:                             </div>
0351:                         </td>
0352:                     </tr>
0353:                     {% endfor %}
0354:                 </tbody>
0355:             </table>
0356:         </div>
0357:     </div>
0358:
0359:     <!-- Section 2: Two Column Grid (Top Books Centrality & Category Density) -->
0360:     <div class="analytics-grid-two">
0361:         <!-- Top Influential Books -->
0362:         <div class="analytics-section">
0363:             <div class="section-header">
0364:                 <div>
0365:                     <h2>
0366:                         <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="#38BDF8" stroke-
      | width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20V3H6.5A2.5 2.5 0 0 0 4 5.5v14z"></path></svg>
0367:                         Most Connected Books (Degree Centrality)
0368:                     </h2>
0369:                     <p>Composite score: Copies &times; 2 + Loans &times; 3 + Requests</p>
0370:                 </div>
0371:             </div>
0372:
0373:             <div style="overflow-x: auto;">
0374:                 <table class="analytics-table">
```

```
0375:                     <thead>
0376:                         <tr>
0377:                             <th>Book Title</th>
0378:                             <th>Category</th>
0379:                             <th>Degree Score</th>
0380:                             <th>Loans</th>
0381:                         </tr>
0382:                     </thead>
0383:                     <tbody>
0384:                         {% for bk in top_books %}
0385:                         <tr>
0386:                             <td>
0387:                                 <strong style="color: var(--text-primary);">{{ bk.title }}</strong>
0388:                                 <div style="font-size: 0.72rem; color: var(--text-muted);">{{ bk.author_name
      |  or 'Author' }}</div>
0389:                             </td>
0390:                             <td>
0391:                                 <span class="badge" style="background: rgba(245, 158, 11, 0.12); color: #FBB
      | F24; font-size: 0.7rem;">
0392:                                     {{ bk.category_name }}
0393:                                 </span>
0394:                             </td>
0395:                             <td>
0396:                                 <span style="font-family: var(--font-mono); font-weight: 800; color: #34D399
      | ;">
0397:                                     {{ bk.degree_score }}
0398:                                 </span>
0399:                             </td>
0400:                             <td style="font-family: var(--font-mono);">{{ bk.loans_count }}</td>
0401:                         </tr>
0402:                         {% endfor %}
0403:                     </tbody>
0404:                 </table>
0405:             </div>
0406:         </div>
0407:
0408:         <!-- Category Circulation Distribution -->
0409:         <div class="analytics-section">
0410:             <div class="section-header">
0411:                 <div>
0412:                     <h2>
0413:                         <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="#A78BFA" stroke-
      | width="2"><circle cx="12" cy="12" r="10"></circle><path d="M12 2a15.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-
      | 4 10 15.3 15.3 0 0 1-4-10 15.3 15.3 0 0 1 4-10z"></path></svg>
0414:                         Topic Circulation & Category Density
0415:                     </h2>
0416:                     <p>Cluster closeness by borrowing volume</p>
0417:                 </div>
0418:             </div>
0419:
0420:             <div style="overflow-x: auto;">
0421:                 <table class="analytics-table">
0422:                     <thead>
0423:                         <tr>
0424:                             <th>Category Domain</th>
0425:                             <th>Code</th>
0426:                             <th>Catalog Titles</th>
0427:                             <th>Total Loans</th>
0428:                         </tr>
0429:                     </thead>
0430:                     <tbody>
0431:                         {% for cat in categories %}
0432:                         <tr>
0433:                             <td>
0434:                                 <strong style="color: var(--text-primary);">{{ cat.category_name }}</strong>
0435:                             </td>
0436:                             <td><span style="font-family: var(--font-mono); color: var(--accent-primary);">{
      | { cat.section_code }}</span></td>
0437:                             <td>{{ cat.books_count }} {{ 'Title' if cat.books_count == 1 else 'Titles' }}</t
      | d>
0438:                             <td>
0439:                                 <span style="font-family: var(--font-mono); font-weight: 700; color: #F59E0B
      | ;">
0440:                                     {{ cat.total_loans }} {{ 'Loan' if cat.total_loans == 1 else 'Loans' }}
0441:                                 </span>
0442:                             </td>
0443:                         </tr>
0444:                         {% endfor %}
0445:                     </tbody>
0446:                 </table>
```

```
            </div>
        </div>
    </div>


</div>

<script>
    document.addEventListener("DOMContentLoaded", function() {
        document.querySelectorAll(".density-bar-fill[data-pct]").forEach(function(bar) {
            const pct = bar.getAttribute("data-pct") || "0";
            bar.style.width = pct + "%";
        });
    });
</script>
{% endblock %}
```

MODULE SOURCE: templates/unauthorized.html

Lines: 18 | **Lang:** HTML5 / Jinja2 | **Category:** Security & RBAC

Core Role & Purpose: 403 Forbidden security error page shown when role boundaries are violated.

REVIEWING LLM VERIFICATION CHECKLIST FOR THIS FILE:
• Verify clear denial message and redirect link to dashboard.

```
{% extends "base.html" %}

{% block title %}403 Access Denied | VITCC LIBRARY{% endblock %}

{% block content %}
<div style="display: flex; align-items: center; justify-content: center; min-height: 60vh;">
    <div style="text-align: center; max-width: 480px; padding: 40px; background: var(--bg-surface); border:
| 1px solid var(--border-subtle); border-radius: var(--radius-lg);">
        <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--danger-bg); color: var(
| --danger-text); display: flex; align-items: center; justify-content: center; margin: 0 auto 20px auto;
| ">
            <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="
| 2"><circle cx="12" cy="12" r="10"></circle><line x1="4.93" y1="4.93" x2="19.07" y2="19.07"></line></sv
| g>
        </div>
        <h1 style="font-size: 1.5rem; font-weight: 800; margin-bottom: 8px;">Access Denied (403)</h1>
        <p style="color: var(--text-secondary); font-size: 0.9rem; margin-bottom: 24px;">
            Your current account role (<strong>{{ user_role }}</strong>) does not have sufficient administra
| tive privileges to view or perform this action.
        </p>
        <a href="{{ url_for('dashboard') }}" class="btn btn-primary">Return to Dashboard</a>
    </div>
</div>
{% endblock %}
```